
EMERGENCE OF INTELLIGENT SOFTWARE AGENTS: FROM SEMANTIC REASONING VIA REAL-WORLD DATASETS TOWARDS GENERAL ARTIFICIAL INTELLIGENCE

Tapio Pitkäranta
Aalto University
`tapio.pitkaranta@iki.fi`

Helsinki, March 24, 2026

Doctors Thesis submitted in partial fulfillment of the requirements for the degree of Doctor of Science in Technology.

Supervisor:	Professor Petri Vuorimaa (2023-2025) Professor Eljas Soisalon-Soininen (2004 - 2014)
Instructor:	Senior Research Scientist (Dr) Timo Laakko (2001 - 2005), Senior Research Scientist Santtu Toivonen (2001 - 2005)

Abstract

AALTO UNIVERSITY

ABSTRACT OF THE DOCTOR'S THESIS

Author:	Tapio Pitkäranta		
Name of the thesis:	PhD Thesis		
Date:	August, 2025	Number of pages:	192 (?)
Department:	School of Science	Professorship:	Computer Science
Spanning 2001-2025, this thesis synthesises research on Artificial Intelligence (AI) with a sustained focus on software agents and structures it into three phases.			
<i>Context.</i> In Phase I, work began in the Semantic Web era, where agents were expected to interpret machine-readable content and coordinate via shared ontologies and protocol specifications. In Phase II, the emphasis shifted to applied machine learning and information retrieval over large, coded datasets in healthcare and retail. In Phase III, advances in Generative AI (GenAI) and Large Language Models (LLMs) reframed agents as conversational, tool-using systems capable of natural interaction and value-aligned behaviour.			
<i>Approach.</i> The research follows a Design Science orientation across three phases: (i) modelling and prototyping ontology-driven agents and interaction protocols on Web standards (Phase I); (ii) designing vector-space and structure-aware retrieval pipelines that exploit existing domain taxonomies and sparse codes without requiring complete redesign (Phase II); and (iii) engineering LLM agent patterns, AI alignment and governance mechanisms, and evaluation artefacts that emphasise transparency, auditability, and human-in-command oversight (Phase III). Evidence is drawn from implemented prototypes, case studies, and public releases, including benchmarks and documentation artefacts intended for reuse.			
<i>Findings.</i> Phase I shows that, despite technical promise, practical convergence between Semantic Web formalisms and multi-agent protocols was limited: heterogeneity of real-world ecosystems, modelling overhead, and brittle runtime assumptions constrained adoption. Phase II demonstrates that domain-specific encodings (e.g., clinical codes, retail hierarchies) combined with vector representations enable efficient analysis and retrieval over sparse, large-scale corpora, improving utility without changing established taxonomies. Phase III shows LLMs shift design toward language-native planning and tool use, while scale is constrained more by engineering maturity and leadership than theory. The thesis releases a public <i>NordDRG AI Benchmark</i> for hospital-funding logic, proposes a <i>reference development process</i> for human-AI rule alignment and maintenance, and introduces a protocol-agnostic multi-agent architecture layering stakeholder agents over LLMs and legacy logic to translate objectives into auditable, vendor-neutral controls, demonstrated on retail-bank credit scoring—collectively positioning alignment as an engineering discipline requiring instrumentation, versioning, and verifiable governance rather than an unsolved scientific mystery.			
<i>Summary.</i> Taken together, the results trace the emergence of intelligent software agents, from semantic reasoning through real-world data pipelines toward general AI, advocating systems that pair LLM adaptability with explicit governance and audit trails. After the hype cycles, agents become practically useful with LLMs. This work identified the shift early, redirected research, and delivers artifacts for reproducible, safer deployment.			
Contributions have been published as follows:			
Phase I — 2001–2006: Licentiate Thesis and [1, 2, 3, 4, 5, 6, 7, 8]			
Phase II — 2007–2013: Publications [9, 10, 11, 12, 13, 14, 15, 16, 17]			
Phase III — 2023–2025: Publications [18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29]			
Keywords: Artificial Intelligence (AI), Generative AI (GenAI), Large Language Models (LLM), Machine Learning (ML), Multi-Agent Systems (MAS) Natural Language Processing (NLP), Semantic Web, Software Agents			
Language: English			

Contents

Abbreviations and Key Terms	7
List of Figures	11
List of Tables	15
PREFACE	16
1 INTRODUCTION	17
1.1 Research Method	17
1.1.1 Research Process	17
1.1.2 Research Phases, Contributions, and Communication	18
1.2 Phase I Problem and Motivation: Early Web Evolution - From Information Space to Service Platform	19
1.2.1 Interaction Protocol Based Service Adaptation	20
1.2.2 Ontology-Driven Software Agents for B2B Purchasing Automation	22
1.2.3 Matching Service Descriptions in Context-Aware Environment	23
1.2.4 Phase I from a 2025 perspective	24
1.3 Phase II Problem and Motivation: Software Agents and ML Healthcare Item Sets	25
1.3.1 Assisting Clinical Decision Support with ML and Software Agents	25
1.3.2 Phase II from a 2025 perspective	27
1.4 Phase III Problem and Motivation: Software Agents in Generative AI Era	27
1.4.1 HADA: Human-AI Agent Decision Alignment Architecture	28
1.4.2 NordDRG AI Benchmark: Teaching LLMs the Nuances of Hospital Funding Instruments	29
1.4.3 CCL: How Humans and AI Agents Align on Decision-Making Rules	30
1.5 Main Research Questions	32
1.6 PhD Artefacts	33
1.6.1 Main Artefacts	33
1.6.2 Phase I Artefacts	35
1.6.3 Phase II artefacts	35
1.6.4 Phase III artefacts	35
1.7 Summary: Problem Identification and Objectives	37
1.8 The Structure of the Thesis	37
2 BACKGROUND	38
2.1 Short history of the AI: AI Summers and Winters	38
2.1.1 The Hype Cycle	40
2.1.2 Review and Synthesis of AI Winters and AI Summers	41
2.1.3 Summary of AI Illustrations	42
2.2 History of the Web	42
2.2.1 The Hypermedia Concept (1945)	42
2.2.2 Prototypes and Concept Development (1960s)	43
2.2.3 Hypermedia Research (1980s)	43

2.2.4	The Invention of the World Wide Web (1989)	44
2.2.5	Research Boost (1990–1994)	46
2.2.6	Public Growth	46
2.2.7	The Web: From Document Collection to Application Platform	46
2.3	The Semantic Web and Its Vision of AI Software Agents	47
2.3.1	Software Agents Before the Semantic Web	47
2.3.2	The Vision of the Semantic Web and Intelligent Agents	48
2.3.3	Key Enabling Technologies and the Semantic Web Stack	48
2.3.4	Semantic Web Influence on Modern AI/ML Data Integration	51
2.3.5	Could Semantic Web Principles Complement LLMs?	51
2.3.6	How AI Has Shaped Real-World Semantic Web Applications	51
2.4	Software Agents	52
2.4.1	Agents and Distributed Artificial Intelligence (DAI)	52
2.4.2	Hype Cycles: The Ever Promising Agent Technology	53
2.4.3	Definitions	53
2.4.4	Single Agent Architectures	56
2.4.5	Multi Agent Architectures	57
2.4.6	Multi Agent Communication	58
2.4.7	Multi Agent Communication Languages (ACL)	58
2.4.8	Interaction Protocols	60
2.4.9	Content Languages	60
2.5	LLM Agents	62
2.5.1	Emerging field of LLM Agents	62
2.5.2	Single LLM Agent	63
2.5.3	Emerging LLM Agent Protocols	65
2.5.4	Multi-Agent Conversation Patterns in LLM Era	67
2.5.5	Prominent LLM Agent Frameworks and Libraries	67
2.6	LLM and DRG Related Studies	68
2.6.1	LLMs	68
2.6.2	AI alignment and safety	68
2.6.3	LLM Benchmarks and Leaderboards	69
2.6.4	Version Control Systems for Governed Change	69
2.6.5	MLOps: operating ML under software discipline	70
2.6.6	NordDRG CaseMix System	70
2.6.7	Related Work: DRGs and LLMs	71
2.6.8	DRG Grouper Software	71
2.7	Chapter Summary: Background	71
3	FORMAL DEFINITIONS	73
3.1	Deep Learning Analytical Framework and Core Definitions	73

3.1.1	Core concepts and notation	73
3.1.2	Definition: Transformer	75
3.2	LLM Agents	79
4	PHASE I: DESIGN & DEVELOPMENT AND DEMONSTRATIONS	80
4.1	Interaction Protocol Based Service Adaptation	80
4.1.1	Design and Development	80
4.1.2	Demonstration	86
4.2	Ontology-Driven Software Agents for B2B Purchasing Automation	89
4.2.1	Design and Development	90
4.2.2	Demonstration	92
4.3	Matching Service Descriptions in Context-Aware Environment	96
4.3.1	Design and Development	96
4.3.2	Demonstration	99
5	PHASE II: DESIGN & DEVELOPMENT AND DEMONSTRATIONS	104
5.1	Assisting Clinical Decision Support with ML and Software Agents	104
5.1.1	Design and Development	104
5.1.2	Demonstration and Results	110
6	PHASE III: DESIGN & DEVELOPMENT AND DEMONSTRATIONS	114
6.1	HADA: Human-AI Agent Decision Alignment Architecture	114
6.1.1	Design & Development	115
6.1.2	Demonstration	121
6.2	NordDRG AI Benchmark for LLMs: Teaching LLMs the Nuances of Hospital Funding Instruments	128
6.2.1	Design & Development: NordDRG AI Benchmark for LLMs	128
6.2.2	Demonstration and Results	136
6.3	CCL: How Humans and AI Agents Align on Decision-Making Rules	138
6.3.1	Design & Development: Collaborative Curated Learning (CCL)	140
6.3.2	Demonstration: Collaborative Curated Learning	151
7	EVALUATION AND DISCUSSION	163
7.1	Phase I: Interaction Protocol Adaptation with Software Agents)	163
7.2	Phase I: Ontology-Driven Software Agents for B2B Purchasing Automation	165
7.3	Phase I: Matching Service Descriptions in Context-Aware Environment	166
7.4	Phase II: Assisting Clinical Decision Support with ML and Software Agents	168
7.5	Phase III: Human-AI Decision Alignment (HADA)	170
7.6	Phase III: Evaluation of the NordDRG AI Benchmark	172
7.7	Phase III: Evaluation of the CCL	174
7.7.1	Evaluation of the NORDDRG Decision-Tree Abstraction	174
7.7.2	Evaluation of Preparing NordDRG for CCL	175
7.7.3	Evaluation of the NordDRG Contributor and Reviewer Agents	176
7.7.4	Discussion	177

8	CONCLUSIONS	179
8.1	Phase I: The History of Semantic Web and Software Agents	179
8.2	Phase I: Software Agents and Semantic Web Integration	179
8.2.1	Phase I: Runtime Protocol Adaptation	180
8.2.2	Phase I: B2B Negotiation Systems.	181
8.2.3	Phase I: Matching Service Descriptions in Context-Aware Environment	182
8.2.4	Phase I: Irreducible Integration Gaps	183
8.2.5	Phase I Transition: Toward Data-Driven Approaches	184
8.3	Phase II: Assisting Clinical Decision Support with ML and Software Agents	185
8.4	Phase III: Human-AI Agent Decision Alignment (HADA) Architecture	186
8.5	Phase III: NordDRG AI Benchmark	189
8.6	Phase III: Collaborative Curated Learning (CCL)	190
8.7	Summary	192
9	EPILOGUE	195
9.1	Agents are (finally) useful	195

Abbreviations

Organizations

AAAI	American Association for Artificial Intelligence
AAMAS	International Conference on Autonomous Agents and Multiagent Systems
ACM	Association for Computing Machinery
CERN	Conseil Européen pour la Recherche Nucleaire
CHIRA	International Conference on Computer-Human Interaction Research and Applications
DARPA	Defense Advanced Research Projects Agency
FIPA	Foundation for Intelligent Physical Agents
IEEE	Institute of Electrical and Electronics Engineers
IETF	Internet Engineering Task Force
ISO	International Standardization Organization
NCSA	National Center for Supercomputing Applications
NOMESCO	Nordic Medico-Statistical Committee
OECD	Organisation for Economic Co-operation and Development
PCSI	Patient Classification Systems International
SRI	Stanford Research Institute
VTT	Valtion Teknillinen Tutkimuskeskus (Technical Research Centre of Finland)
W3C	World Wide Web Consortium

Technical

A2A	Agent2Agent
ACL	Agent Communication Language
ADK	Agent Development Kit
ADSL	Asymmetric Digital Subscriber Line
AID	Agent Identifier
AI	Artificial Intelligence
AIGC	AI-Generated Content
AJAX	Asynchronous JavaScript and XML
AMS	Agent Messaging System
AOSE	Agent Oriented Software Engineering
AP	Agent Platform
API	Application Programming Interface
AR-DRG	Australian Refined Diagnosis-Related Group
B2B	Business-to-Business
B2C	Business-to-Consumer
BBS	Boneh–Boyen–Shacham signature scheme
BDI	Belief, Desire, Intention
BERT	Bidirectional Encoder Representations from Transformers
BIG-bench	Beyond the Imitation Game Benchmark
BPEL4WS	Business Process Execution Language for Web Services
C2C	Consumer-to-Consumer
CAC	Computer-Assisted Coding
CAD	Cosine Angle Distance
CaseMix	Case Mix (hospital patient classification)
CC	Complication or Comorbidity
ChatGPT	Chat Generative Pre-trained Transformer
CI/CD	Continuous Integration / Continuous Delivery
CORBA	Common Object Request Broker Architecture
CoT	Chain of Thought
CRM	Customer Relationship Management
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
DAI	Distributed Artificial Intelligence

DAML	DARPA Agent Markup Language
DAML+OIL	DARPA Agent Markup Language + Ontology Inference Layer
DAML-S	DARPA Agent Markup Language for Services
DCOM	Distributed Component Object Model
DF	Directory Facilitator
DID	Decentralized Identifier
DNS	Domain Name System
DRG	Diagnosis-Related Group
DSL	Digital Subscriber Line
DSR	Design Science Research
DSRM	Design Science Research Methodology
DSRP	Design Science Research Process
DTD	Document Type Definition
EDI	Electronic Data Interchange
ERP	Enterprise Resource Planning
EUD	Euclidean Distance
FIPA-ACL	FIPA Agent Communication Language
FIPA-SL	FIPA Semantic Language
FSCM	Financial Supply Chain Management
FSM	Finite State Machine
FTP	File Transfer Protocol
GAN	Generative Adversarial Network
GB	Gigabyte
GDPR	General Data Protection Regulation
Gemini	Google’s large language model
GenAI	Generative AI; foundation-model-based systems (e.g., LLMs) that generate novel content
GLUE	General Language Understanding Evaluation
GPRS	General Packet Radio Service
GPT	Generative Pre-trained Transformer
gRPC	Remote Procedure Call framework
GPU	Graphics Processing Unit
GPS	Global Positioning System
GUI	Graphical User Interface
HADA	Human-AI Agent Decision Alignment
HELM	Holistic Evaluation of Language Models
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
ICD	International Classification of Diseases
ICD-10	International Classification of Diseases, 10th Revision
ICD-11	International Classification of Diseases, 11th Revision
IDE	Integrated Development Environment
IDF	Inverse Document Frequency
IP	Internet Protocol
IPPS	Inpatient Prospective Payment System
IR	Information Retrieval
JADE	Java Agent Development Platform
JDK	Java Development Kit
JSON	JavaScript Object Notation
JSON-LD	JSON for Linking Data
KIF	Knowledge Interchange Format
KPI	Key Performance Indicator
KQML	Knowledge Query and Manipulation Language
kNN	k-Nearest Neighbors
KR	Knowledge Representation
LangChain	Large language model framework
LangGraph	Graph-based language model framework
LEAP	Lightweight Extensible Agent Platform
LLaMA	Large Language Model Meta AI
LlamaIndex	Data framework for LLMs

LLM	Large Language Model
LMSys	Large Model Systems
MRQ	Main Research Question
MAS	Multi-Agent System
MCC	Major Complication or Comorbidity
MCP	Model Context Protocol
MDC	Major Diagnostic Category
MB	Megabyte
ML	Machine Learning
MMLU	Massive Multitask Language Understanding
MS-DRG	Medicare Severity Diagnosis-Related Group
MTS	Message Transport System
NCSP	NOMESCO Classification of Surgical Procedures
NL	Natural Language
NLG	Natural Language Generation
NLP	Natural Language Processing
NLU	Natural Language Understanding
NordDRG	Nordic Diagnosis-Related Groups
OCI	Open Container Initiative
OKR	Objectives and Key Results
OLAP	Online Analytical Processing
OpenAPI	OpenAPI Specification
OPT	Open Pre-trained Transformer
ORD	Execution order (priority) in <code>drg_logic</code>
OWL	Web Ontology Language
OWL-S	Web Ontology Language for Services
PaLM	Pathways Language Model
PDF	Portable Document Format
PROCPRO	Procedure property table in NordDRG
PWA	Progressive Web Application
RACI	Responsible, Accountable, Consulted, Informed
RAG	Retrieval Augmented Generation
RDF	Resource Description Framework
RDFC-1.0	RDF Dataset Canonicalisation and Hash 1.0
RDFS	RDF Schema
RDQL	RDF Data Query Language
REST	Representational State Transfer
RFQ	Request for Quote
RIF	Rule Interchange Format
RL	Reinforcement Learning
RMI	Remote Method Invocation
RPC	Remote Procedure Call
RuleML	Rule Markup Language
RAM	Random Access Memory
SaaS	Software as a Service
SCM	Supply Chain Management
SDK	Software Development Kit
SEO	Search Engine Optimization
SGML	Standard Generalized Markup Language
SHACL	Shapes Constraint Language
SL	Semantic Language (FIPA)
SMS	Short Message Service
SOAP	Simple Object Access Protocol
SOM	Self-Organizing Map
SOP	Standard Operating Procedure
SPARQL	SPARQL Protocol and RDF Query Language
SQL	Structured Query Language
SSE	Server-Sent Events
SSO	Single Sign-On

SuperGLUE	Super General Language Understanding Evaluation
SRQ	Supporting Research Question
SVG	Scalable Vector Graphics
SVM	Support Vector Machine
SWE-DRG	Swedish Diagnosis-Related Group
SWRL	Semantic Web Rule Language
T5	Text-to-Text Transfer Transformer
TCP	Transmission Control Protocol
TF-IDF	Term Frequency-Inverse Document Frequency
ToT	Tree of Thoughts
UDDI	Universal Description, Discovery and Integration
UML	Unified Modeling Language
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
WAP	Wireless Application Protocol
WLAN	Wireless Local Area Network
WSDL	Web Services Description Language
WSFL	Web Services Flow Language
WWW	World Wide Web
WYSIWYG	What You See Is What You Get
XLANG	Extension to WSDL
XLSX	Office Open XML Spreadsheet
XLTX	Excel Open XML Template
XML	eXtensible Markup Language
XP	Extreme Programming
ZIP	Zone Improvement Plan (postal codes)

List of Figures

1	Design Science Research Process (DSRP) [33] model depicting a nominal six-step flow: problem identification and motivation (define and justify the problem), objectives of a solution (what an improved artefact should achieve), design and development (build the artefact), demonstration (apply the artefact in an appropriate context), evaluation (assess effectiveness and iterate back to design), and communication (report results to scholarly and professional audiences). The figure further highlights that studies may start from different entry points, including problem-, objective-, design/development-, or observation-centred approaches.	17
2	Structure and contributions of the Thesis.	19
3	Phase I: Semantic Web Stack [39] and Intelligent Web Services [40].	20
4	Phase II: Analyzing Item Sets with ML and Software Agents.	26
5	Phase III: Software Agents in Generative AI Era.	28
6	Categorization issue: Semantic Web research vision had strong AI focus but it is categorized under WWW branch.	39
7	Hype Cycle: Hype Level, Engineering or Business Maturity, and Combined Hype Cycle [62]	40
8	History of AI as illustrated by previous studies. Period of active Semantic Web development is identified typically as AI Winter period.	41
9	From Hypertext to the Killer App: The Web’s Evolution.	42
10	Web architecture by Tim Berners-Lee in 1990 [85].	45
11	Internet Domain Survey Host Count [93].	46
12	Semantic Web Stack by Tim Berners-Lee 2006 [122].	49
13	The Semantic Web Wave 2003 by Tim Berners-Lee [57].	50
14	ACM Computing Classification System of Computer Science [137].	52
15	Agents and their environments [151].	54
16	Agent characteristics [150].	55
17	Agent Management Reference Model [157].	58
18	Relationships between communicative acts defined by FIPA [152].	59
19	The FIPA Contract Net Interaction Protocol [187].	60
20	Single LLM agent core components.	63
21	Comparison of single-path vs. multi-path reasoning (redrawn from [215]).	64
22	Comparison between Reason Only, Act Only, Reason and Act, and Reflexion framework (redrawn from [217] [218]).	65
23	A glance at the development of agent protocols (redrawn from [209]).	66
24	MCP General architecture [221].	66
25	Example library (AutoGen) agents are conversable, customizable, and can be based on LLMs, tools, humans, or even a combination of them (redrawn from [224]).	67
26	LLM Leaderboard illustration: Vellum leaderboard [263]	69
27	Deep learning core concepts and their relationships. Interactive illustration with links to definitions.	73
28	Simple illustration of <i>Optimization Problems: Supervised Learning, Unsupervised Learning and Reinforcement Learning</i>	74
29	Transformer ecosystem concepts and their relationships (interactive links to definitions).	75

30	Left: Block-level view of a <i>GPT</i> that is a <i>Transformer Mathematical Model</i> with L layers from input to output. Dataflow: input tokens are embedded, combined with positional encodings, passed through a stack of L decoder Transformer Blocks, layer-normalized, and mapped by a final linear+softmax head to token probabilities. Right: Internal structure of a single Block_ℓ with nodes drawn from <i>Transformer Mathematical Functions</i> : masked multi-head self-attention f_{MHA} with H heads and a causal mask, followed by a position-wise feed-forward network f_{FFN} (GeLU). Both sublayers use residual connections ($\oplus$), LayerNorm f_{LN} , and dropout.	78
31	Progression of the expediting process from the contractor's point of view.	81
32	Core concepts of the interaction protocol ontology [293].	83
33	The protocol state machine.	85
34	A tool for communicating with the software agents.	87
35	A tool for monitoring the state and information the agent receives and sends.	88
36	Agents and messages.	89
37	The process flow.	90
38	Overall RFQ process with negotiation and decision outcomes.	91
39	Proposal Ontology.	92
40	Product ontology.	93
41	Requesting for quotes.	94
42	Proposing a lower price.	94
43	GUI: products, tables.	95
44	Hybrid approach for service provision [300].	96
45	User Profile and User Context.	97
46	Overall System Architecture.	98
47	DYNAMOS Event Distribution Server.	99
48	DYNAMOS System Overview.	100
49	DYNAMOS Client Screenshots.	101
50	The hierarchy of classification.	105
51	Primary and secondary business processes and data warehousing.	106
52	A sample coded data set network interlinked by a coding scheme.	108
53	Recommendation accuracy using topN as variable (kNN=5).	111
54	Recommendation accuracy using kNN as a variable (topN=1).	111
55	Recommendation accuracy using topN as variable with low quality data (kNN=5).	112
56	Query execution time (seconds) when scaling the database size (kNN=5, topN=5).	113
57	Bank strategy process: time horizons and stakeholder coverage.	115
58	AI Tool <code>getLoanDecision</code> based on Kaggle dataset depicting the decision tree and OpenAPI specification.	118
59	HADA High Level Architecture.	119
60	HADA: Screenshot from the prototype system.	122
61	HADA prototype implementation showing the supervisor-worker agent pattern	123
62	Dialogue: Business Manager (BM) Shifting the AI Tool Objective to Minimizing Credit Risk	124
63	Dialogue: Data Scientist Delivering a New AI Tool Version for Business Approval	125

64	Dialogue: Business Manager Approves Version 1.1 for Production	125
65	Dialogue: Customer Applying for a Personal Loan	126
66	Dialogue: Customer Questioning the Use of ZIP Codes in Lending	126
67	Dialogue: Value and Ethics Manager (DVEM) Evaluating ZIP Code	127
68	Materials used: NordDRG definition tables [328].	130
69	Materials used: NordDRG definition table expert documentation [328]	131
70	Learning paradigms create a decision model $\hat{f}_\theta$ from data. Supervised, unsupervised, reinforcement, and transfer learning differ only in the form and timing of the signals they exploit.	141
71	Example CCL process slice. A directed, possibly cyclic AOV graph where activities (vertices) are typed by participants $\tau(v) \in \{\text{Agent}, \text{Human}, \text{Procedure}\}$. The slice illustrates agentic ($\text{Agent} \rightarrow \text{Agent}$ and $\text{Agent} \rightarrow \text{Procedure}$), hybrid ($\text{Agent} \rightarrow \text{Human}$ and $\text{Human} \rightarrow \text{Procedure}$), and human ($\text{Human} \rightarrow \text{Human}$) workflows.	142
72	Participants and workflow types. Top: representative AOV snippets for agentic, hybrid, human, and procedural flows. Bottom: the participant set $\mathcal{P} = \{\text{Agent}, \text{Human}, \text{Procedure}\}$	142
73	CCL PR state machine. Lifecycle from <i>Draft</i> $\rightarrow$ <i>Review</i> $\rightarrow$ <i>Approve</i> $\rightarrow$ <i>Merge</i> $\rightarrow$ <i>Release</i> , with gated iteration (<i>Changes Requested</i>), explicit human <i>Rejected/Withdrawn/Superseded</i> exits, and <i>Rollback</i> from <i>Release</i> on monitoring alerts; merges require mandated human sign-off.	144
74	Governed Version Control history for CCL. A feature branch (<code>feat-ccl-edit</code>) opens a PR with Impact Brief and CI (tier R2), iterates agentially, then merges <i>no-ff</i> into <i>main</i> after required human approvals; an unrelated hotfix and tagged releases (<code>v1.4</code> , <code>v1.5</code>) are shown. A second branch (<code>feat-rejected</code>) illustrates a PR that is rejected (no merge). Only humans may merge to <i>main</i>	145
75	CCL PR workflow. A feature branch applies atomic diffs and opens a PR with an Impact Brief and declared risk tier. An agentic pre-review (Contributor $\leftrightarrow$ Reviewer) precedes four CI gates executed in parallel (G1–G4). A join confirms that all configured gates pass before <i>human review</i> , which can request changes, reject, or approve. Approved PRs are merged (<code>-no-ff</code>), released, and written to the audit ledger; monitoring may trigger rollback to a new branch. All merges require mandated human sign-off.	146
76	Roles and human–AI hand-offs across the PR timeline. Agents collaborate during an initial <i>agentic review</i> phase; escalation to human sign-off preserves the human-in-command invariant.	147
77	Gate policy map rooted in maintenance-time alignment: objective fidelity and locality; oversee signals for drift/leakage; provenance for auditability; adversarial/shift stress tests with fail-closed policies. Risk tier τ scales required evidence and human sign-offs.	148
78	Illustrative fragment of the first-layer decision logic for DRG 373X. Top: excerpt of the custom NORDDRG format where each row expands to a path condition. Bottom: the corresponding path-wise view as a canonical decision tree compiled for analysis.	149
79	High-level architecture of the CCL Platform. The browser-based front end communicates with the FastAPI back end over REST. The back end integrates with PostgreSQL for persistence, a local git repository for version control, and multiple LLM providers for agent capabilities.	153
80	The Overview page of the CCL Platform. The top section displays live system metrics (pull requests, pending reviews, specification tables, decision rules). Below, the quality-gate pipeline (G1–G4) and decision-model summary provide at-a-glance situational awareness. The five-step workflow diagram illustrates the change lifecycle, and navigation cards link to all platform sections.	155
81	The Agentic Workflow Configurator. Administrators define multi-step pipelines specifying which agents participate, the iteration limit, and optional test and verification procedures. The lower panel lists existing workflow configurations with their status and execution counts.	156

82	The Agent Configurator tab within Agentic Workflows. Each agent configuration specifies the role (Contributor, Reviewer, or Verifier), the LLM provider and model, a system prompt, and behavioural parameters. Configurations can be selected per workflow step and per Agent Chat session, allowing different tasks to use different models.	157
83	The Decision Tree page in SVG Graph mode, showing NordDRG decision logic as a flowchart. Blue nodes represent decision points on input features (diagnosis codes, procedures, age, sex), and green leaf nodes represent DRG group outputs. The toolbar provides view-mode switching, DRG code filtering, layout orientation controls, and SVG export.	158
84	NORDDRG as decision trees: illustrative slices with 10, 25, and 100 root→leaf paths. Only small portions of the full structure are readable as static graphics. In practice: <i>10 paths</i> are fully legible at print scale and useful for explaining predicate structure and local flow; <i>25 paths</i> remain readable on screen with moderate zoom and are borderline for print without enlarging; <i>100 paths</i> are overcrowded in static form and are only useful with interactive panning/zoom, making them unsuitable for print. The full specification contains nearly 9,000 paths. The full 9000 path DAG can be rendered in browser but is difficult to navigate.	159
85	Preparing the NordDRG specification for CCL: (left) the CSV tables that constitute the agent-editable decision model; (middle/right) the two PDF guides from the NordDRG Forum used to ground agents in the custom notation and change process.	160
86	Contributor Agent processing a ticket from the NordDRG Forum. The agent proposes a change to the NordDRG specification, including atomic diffs and a rationale. The user can then review and export the suggestions for further processing.	161
87	NordDRG Review Agent taking in implementation by NordDRG Contributor Agent together with the original ticket specification. The Review Agent proposes changes to the implementation to align with the NordDRG decision model coding standards and guidelines.	162
88	Combined results. Percentage accuracy on the <i>NordDRG-AI-Benchmark Logic</i> (rule-level) and <i>Grouper</i> (full emulation) suites. Logic saturates for top models, while exact-match emulation—requiring the correct <code>drg_nat</code> and triggering <code>drg_logic.id</code> —remains challenging; only a few endpoints exceed 40% on Grouper.	173
89	How the agent plugs into CCL. The Contributor NORDDRG Agent ingests NordDRG Forum tickets (via pasted text), grounds them in the official specification and guidelines, and emits atomic diffs with an explanation-rich specification; it then opens a pull request against the model-of-record, where CI runs the CCL gates (G1–G4) and risk-tiered human approvals are collected before a maintainer performs a merge, tags a release, and writes an immutable audit trail to the ledger with canary/rollback safeguards. This shows the agent acting as a first-line contributor while “human-in-command” governance ensures auditability and accountability end-to-end.	176
90	Human in control or agentic workflow	195

List of Tables

3	DSRP process sequence (Figure 1) linked to Phases and Structure of this Thesis.	18
4	Main Research Questions by phase.	33
5	Main artefacts created across the three research phases.	34
6	Phase I individual artefacts and their mapping to the main artefacts (Table 5).	35
7	Phase II individual artefacts and their mapping to the main artefacts (Table 5).	35
8	Phase III individual artefacts and their mapping to the main artefacts (Table 5).	36
9	Comparison of Prominent LLM Agent Frameworks.	68
10	Core symbols and typical values in Transformer architectures.	76
11	Key matrices and intermediate representations in a Transformer block (row = token, column = feature).	76
12	User Content Server (calls/second).	101
13	Service Content Server (calls/second).	102
14	Materials used in scalability experiment.	112
15	Stakeholder map for the banking use-case <code>getLoanDecision</code>	117
16	Stakeholder (see Table 15) responsibilities as RACI matrix for the <code>getLoanDecision</code> AI Tool RACI keys: R = Responsible, A = Accountable, C = Consulted, I = Informed. Role abbreviations: CCO = Chief Credit Officer, BM = Business Manager, DS = Data Scientist, Audit = AI Audit Manager, DVEM = Value & Ethics Manager, HADA = The Human Agent Decision Alignment System	119
17	Stakeholder agents (see Table 15), capabilities and A2A contracts for the <code>getLoanDecision</code> pilot.	121
18	Publicly released NordDRG benchmark resources.	129
19	CaseMix questions with automatically verifiable answers (Logic-1–Logic-13).	132
20	Reference answers for each CaseMix question (Logic-1–Logic-13).	133
21	CaseMix questions for emulating the official NordDRG grouper (Grouper-1–Grouper-13). . .	134
22	Reference answers for each CaseMix grouper question (Grouper-1–Grouper-13).	134
23	Configuration types for the NordDRG benchmark datasets.	135
24	Accuracy on the 13 <i>quantitative</i> NordDRG Logic tasks (Logic-1–Logic-13). All runs used the Definition Tables + PDF Instructions bundle. (✓=correct, ✗=incorrect)	137
25	DRG grouper emulation results on thirteen case-level tasks (Grouper-1–Grouper-13). All runs used the Definition Tables + PDF Instructions bundle and the structured test cases. (✓=correct; exact match on <i>DRG</i> and <i>drg_logic.id</i> ; ✗=incorrect)	138
26	Quantitative metrics of the CCL Platform implementation.	153
27	Scale and structural statistics for the NORDDRg decision model. The “first-layer” tree refers to the top-level grouping layer.	155
28	Outcome of each ticket under two workflow modes across three LLM endpoints. A check mark (✓) indicates that the generated pull request passed manual review and CI gates; a cross (✗) indicates rejection.	177

PREFACE

Agents are (finally) here

During the course of this thesis research, intelligent software agents have evolved from theoretical Distributed Artificial Intelligence (DAI) concepts into integral components of practical, everyday applications through the emergence of *LLM Agents*. This advancement is exemplified by the grammar review process for this 150+ page document, which was completed within a few hours using a small Multi-Agent System (MAS) consisting of two agents and one human—myself. Setting up such agents can now be accomplished in seconds with modern tools. More importantly, this foregrounds the question of what role humans should assume in these workflows. Should the entire grammar perfection task be delegated to *LLM Agents*, and what role should remain for the human? See Section 9.1 for how I chose to allocate the responsibilities in the case of this document.

My experience leaves two lingering questions: why have these agents reached practical utility only now, and why did I not adopt this application earlier?

About the research period of this Thesis

Spanning over two decades, my career in software development has been intrinsically linked with the practical application of AI and Machine Learning (ML). This journey commenced in the early 2000s amidst the burgeoning integration of advanced mathematical principles into software research and development. This period, characterized by transformative advancements and the cyclical nature of technological hype, has provided a rich backdrop for my professional trajectory. A substantial portion of my career has been dedicated to deploying AI and ML algorithms in diverse industrial settings, including healthcare, retail, international tax, and human resources, fostering a deep understanding of real-world applications. This industry experience has significantly shaped the research direction of my Doctoral Thesis, which transitioned from the complexities of Web infrastructure and semantics to the tangible analysis of real-world datasets.

The field of AI has witnessed remarkable advancements, resulting in transformative applications across various domains. These developments include the creation of highly accurate, large-scale search engines, the development of neural network-based algorithms achieving superhuman performance in complex games, and the recent emergence of LLMs that convincingly simulate human conversation, demonstrating a significant leap in Natural Language Processing (NLP). Notably, the accessibility of these powerful AI tools has broadened considerably. Even young children, regardless of literacy, can now utilize current AI applications to generate creative content, such as bedtime stories, highlighting the democratization of advanced technology. Recent reports [30] indicate that some LLMs have reportedly passed the Turing test, further underscoring their advanced capabilities in mimicking human-like communication.

The contemporary discourse surrounding AI and ML has consequently shifted towards profound, almost philosophical, inquiries. These include examining the alignment of human values and objectives with the decisions made by algorithms [31] and contemplating the potential for AI to surpass human capabilities [32]. This thesis compiles my research papers and contributions from these areas, spanning a period of two decades. My own public research trajectory commenced with investigations into software agents and the Semantic Web, progressing towards the development of flexible ML, NLP and Information Retrieval (IR) algorithms, and more recently, exploring the integration of software agents with LLMs to enable more autonomous and intelligent systems to align human and AI decisions.

Acknowledgements

Add acknowledgements for the people who have provided help enable this Thesis.

1 INTRODUCTION

This chapter traces a two-decade research journey investigating how software agents can enable automated, auditable interaction across heterogeneous systems and domains. The research unfolds across three distinct phases that mirror the evolution of AI technology: from Semantic Web services (2001-2006), through ML and data integration (2007-2013), to the current era of generative AI and LLMs (2023-2025). Throughout these phases, software agents provide the architectural through-line, while the underlying representational substrates evolve from ontologies and protocols to statistical models and now foundation models. The chapter first presents the research methodology and process that guided this longitudinal study, then details each phase’s context, problems, and contributions. Only after establishing this comprehensive background are the formal research questions and specific artifacts introduced, allowing readers to understand not just what was investigated but why these particular challenges emerged and persisted across technological generations.

1.1 Research Method

1.1.1 Research Process

This dissertation employs a *modified Design Science Research (DSR) methodology* (DSRM) and process (DSRP) illustrated in Figure 1 that adheres to the canonical six-step process of Peffers et al. [33] while tailoring each stage to the practical realities of more than twenty individual studies conducted over two decades. Guided by the relevance-rigour balance articulated in Hevner et al.’s [34] DSR guidelines, every study consistently executed the downstream phases of *artefact design, demonstration, evaluation, and communication*. By contrast, the upstream *entry points*—whether problem-, objective-, or design-centred—varied from study to study, showcasing the methodological flexibility that Peffers et al. explicitly allow.

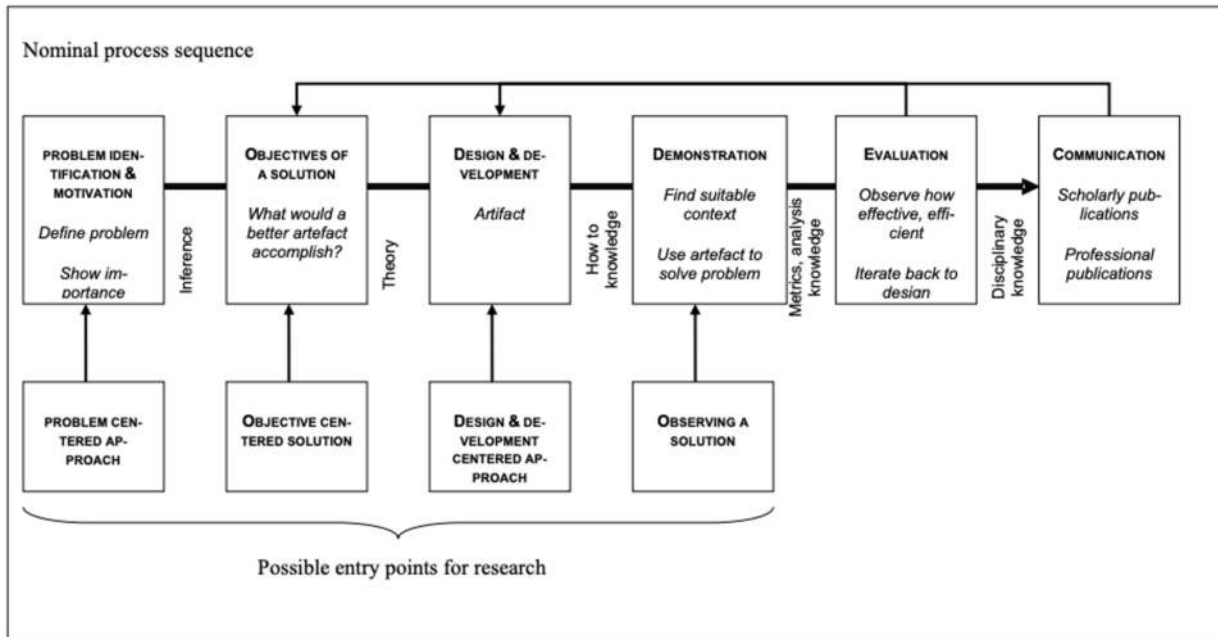


Figure 1: Design Science Research Process (DSRP) [33] model depicting a nominal six-step flow: problem identification and motivation (define and justify the problem), objectives of a solution (what an improved artefact should achieve), design and development (build the artefact), demonstration (apply the artefact in an appropriate context), evaluation (assess effectiveness and iterate back to design), and communication (report results to scholarly and professional audiences). The figure further highlights that studies may start from different entry points, including problem-, objective-, design/development-, or observation-centred approaches.

Inspired by Gregor and Jones’s [35] anatomy of *mid-range design theory*, the programme sought not merely to deliver standalone technical artefacts but to encapsulate their *constructs, design principles, and justificatory knowledge*. Each iteration therefore began with a clearly defined problem motivation or design opportunity,

proceeded to explicit objectives that set success criteria, and advanced through incremental development cycles. Formative evaluations—both analytical and prototypical—were used to refine successive iterations in accordance with Hevner et al.’s [34] recommendations.

Because large-scale field deployments were beyond the scope, the resulting artifacts were disseminated primarily as *proof-of-concept*, *conceptual demonstrations*. These took the form of scenario walk-throughs and storyboarded simulations built from representative data. Rather than relying on internal expert panels, each artefact underwent the double-blind peer-review process of specialist conferences in its domain (e.g., health-informatics venues for hospital decision support, enterprise-systems tracks for SaaS artefacts). Such external vetting—aligned with DSR’s emphasis on independent evaluation [34, 33]—provided methodological soundness, domain relevance, and scholarly rigour despite the absence of full production roll-outs.

Interim results were also consolidated into two milestone monographs: a *Master’s thesis* that captured the earliest design cycles and a subsequent *Licentiate thesis* that documented an expanded artefact portfolio and its preliminary evaluations. Both monographs served as formal checkpoints, ensuring that cumulative knowledge was systematically archived and critiqued before the research progressed to later phases.

Finally, mature findings were published in peer-reviewed conference proceedings aimed at both academic and practitioner audiences, cementing the practical utility and scholarly contribution of the work. In sum, this research operationalizes the Peffers et al. process model [33], embodies Hevner et al.’s [34] design guidelines, and advances Gregor and Jones’s [35] theory-building perspective, collectively yielding a coherent contribution that furthers both practice and mid-range theory in AI-enabled decision systems.

1.1.2 Research Phases, Contributions, and Communication

Figure 2 and Table 3 trace the dissertation’s three phases, which mirror both the wider evolution of AI technology and the author’s own professional arc from Web-centric infrastructure to modern generative models. In every phase, *software agents* provide the architectural through-line; what changes is the representational substrate over which the agents reason and act.

Table 3: DSRP process sequence (Figure 1) linked to Phases and Structure of this Thesis.

DSRP	Time Frame	1, 2	3, 4	5	6
DSRP Labels		Problem Identification, Objectives	Design & Development, Demonstration	Evaluation	Communication
Thesis Phase I	2001-2006	Section 1.2	Chapter 4: Section 4.1, Section 4.2, Section 4.3	Chapter 7: Section 7.1, Section 7.2, Section 7.3	Licentiate Thesis and [1, 2, 3, 4, 5, 6, 7, 8]
Thesis Phase II	2008-2013	Section 1.3	Chapter 5: Section 5.1	Chapter 7: Section 7.4	Publications: [9, 10, 11, 12, 13, 14, 15, 16, 17]
Thesis Phase III	2023-2025	Section 1.4	Chapter 6: Section 6.1, Section 6.2, Section 6.3	Chapter 7: Section 7.5, Section 7.6, Section 7.7	Publications: [18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29]

Phase I—2001 – 2006. The work begins with agent-mediated *Semantic Web* services: ontological descriptions, agent communication languages, and MAS coordination for intelligent service matching. Findings from this formative stage were consolidated into a Master’s thesis and expanded in a Licentiate thesis, alongside a cluster of early conference papers. Together they establish the semantic foundations and messaging patterns that later phases reuse.

Phase II—2007 – 2013. Confronting the brittleness of purely symbolic approaches, the research pivots to *data-centric ML*. Kernel methods, large item-set mining, and OLAP analytics are layered beneath the existing agent framework, allowing agents to orchestrate decisions over high-dimensional coded datasets from healthcare and retail. This hybrid symbolic–statistical stance is documented in a suite of peer-reviewed healthcare-informatics and retail-analytics publications.

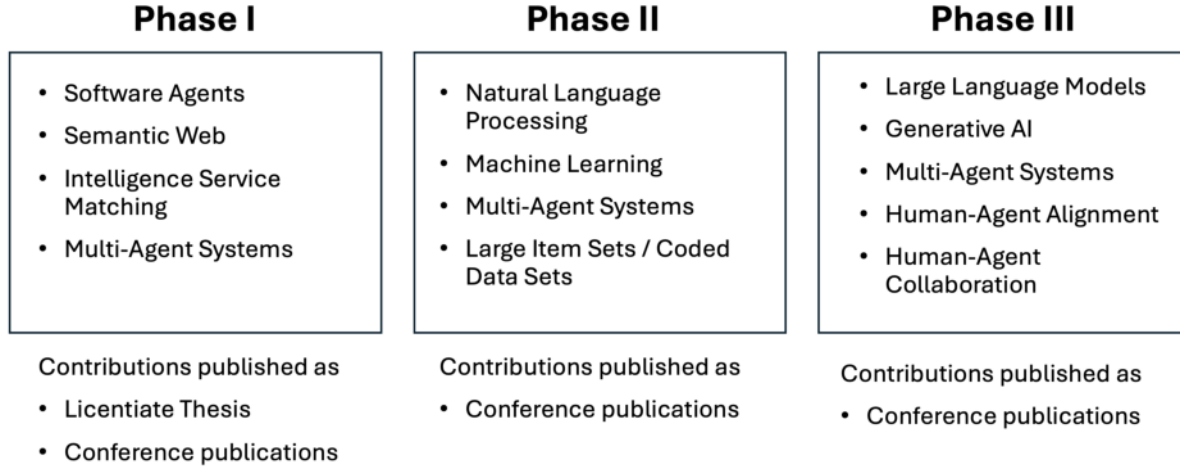


Figure 2: Structure and contributions of the Thesis.

Phase III—2023–2025. The final phase integrates GenAI; foundation-model-based systems, e.g., LLMs, that generate content from prompts into multi-agent architectures, enabling agents to offload portions of planning, explanation, and natural-language interaction to foundation models. Contributions include the human-AI agent alignment framework and the *NordDRG AI Benchmark for LLMs*, each vetted through specialist conference venues. These artefacts demonstrate how agent governance, statistical learning, and LLM-based reasoning can be fused into auditable, human-aligned decision pipelines.

Across the three phases, communication outlets evolve from theses and early Semantic-Web conferences to domain-specific ML venues and, most recently, generative AI workshops—yet the methodological spine of DSR remains constant, ensuring that each contribution is both rigorously evaluated and practically grounded.

1.2 Phase I Problem and Motivation: Early Web Evolution - From Information Space to Service Platform

Research Context.

The World Wide Web, conceived in 1989 [36] and popularised during 1991–1995 [37, 38], was initially a decentralised, hyperlinked information space. In the early 2000s (circa 2000–2005)—which coincides with the initial phase of this study—the Web evolved from a predominantly human-readable document repository toward a platform for distributed services, providing programmatic access ¹ alongside traditional browser-based interaction. This period also saw mobile phones emerge as significant endpoints for Web access (feature-phone Internet usage circa 2003–2006; smartphone adoption 2007–2010), and context-aware information (user location, device capabilities, and preferences) gained importance for tailoring services and interactions.

Web Services vs. Semantic Web (synthesis). By the early 2000s, industry-grade Web services delivered pragmatic, platform-neutral interfaces for Remote Procedure Call (RPC) -style integration, whereas the Semantic Web vision [41] aimed for machine-interpretable data and agent-level reasoning over shared ontologies. Figure 3 situates both: Web services operationalised interoperability at the *message* layer; the Semantic Web targeted *meaning*-level interoperability and autonomous discovery/coordination by software agents. This thesis treats them as complementary layers—transport/API standardisation below, semantics and agent protocols above—asking where they converge in practice and where irreducible gaps remain.

Main RQ I.1 (Table 4, page 33): *What are the historical origins of the Web and of software agents?*

Rationale. Understanding these origins was essential for Phase I. Both paradigms emerged from parallel efforts to enable autonomous yet interoperable systems on the Internet. Clarifying their shared lineage

¹e.g., SOAP (Simple Object Access Protocol) and REST (Representational State Transfer) APIs (Application Programming Interfaces)

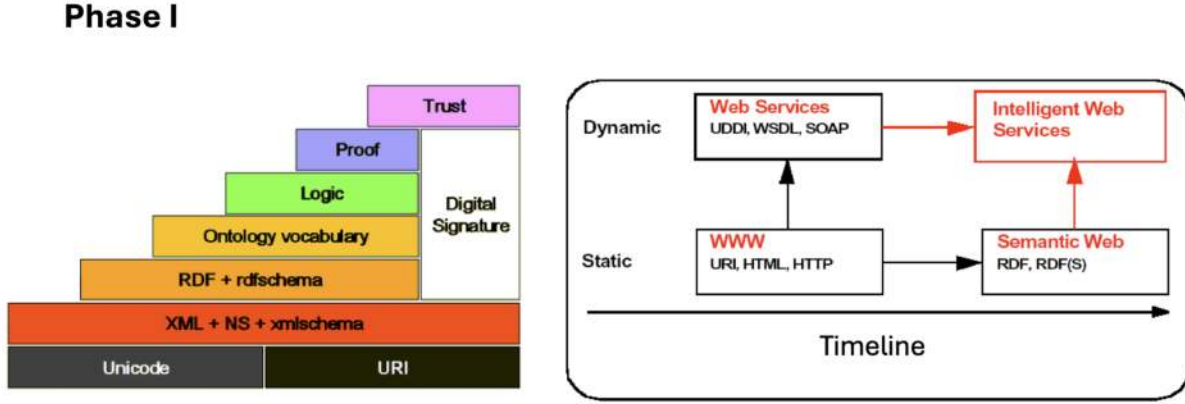


Figure 3: Phase I: Semantic Web Stack [39] and Intelligent Web Services [40].

explains why the Semantic Web vision and agent architectures were repeatedly intertwined, and why their integration remains relevant for runtime interoperability and adaptation today.

Operationalisation via three scenarios. We examine this complementarity by implementing three “Intelligent Web Services” scenarios that bind semantic descriptions and agent interaction protocols to concrete tasks:

- Interaction-protocol adaptation (construction expediting).** At runtime, agents ingest Resource Description Framework (RDF) -serialised agent protocol descriptions and execute auditable conversations across partners to reduce status latency and reconcile reports (see Section 1.2.1, Section 2.4).
- Ontology-driven purchasing.** Buyer/supplier agents use shared domain ontologies to automate Request for Quote (RFQ) dissemination, normalise heterogeneous quotes, and produce first-round triage prior to human decision in Business-to-Business (B2B) contexts (see Section 1.2.2).
- Context-aware service matching (mobile/sailing).** A hybrid model aligns multi-modal context (time, activity, location) with Business-to-Consumer (B2C) service descriptions and Consumer-to-Consumer (C2C) annotations to return only “here-and-now” relevant results (see Section 1.2.3).

1.2.1 Interaction Protocol Based Service Adaptation

1.2.1.1 Problem Identification and Motivation

Business context. Large general contractors coordinate multi-tier supply networks of subcontractors, suppliers, and carriers. To control schedule and cost risk, they run an *expediting process*: soliciting progress reports, validating them with third parties, and acting on discrepancies. In practice, this coordination still occurs over ad-hoc channels (phone and e-mail), which introduce latency and poor auditability—conditions that fuel disputes in construction projects [42].

Potential solution. A run-time-adaptive, agent-mediated solution that interoperates with heterogeneous partner protocols could (i) shorten the confirm-deny feedback loop, (ii) reduce manual effort, and (iii) yield a machine-traceable audit trail—benefits that scale with project size and complexity.

Assumptions. The following are assumed: (i) shared formal protocol descriptions, (ii) run-time ingestion/-compilation into executable conversations, and (iii) auditable execution on an agent platform, in the presence of realistic heterogeneity (partners, payloads, and protocol variants).

1.2.1.2 Research gap

There are standards for agent interaction patterns and RDF provides a machine-readable substrate, yet it remains unclear how far these can be *practically* combined for *run-time* adaptation (protocol variants published at, or close to, execution time) without hard-coded flows. The construction expediting scenario—where partner-specific protocol variants appear late and differ subtly—offers a concrete probe of both possibilities and limits of semantic-agent integration.

1.2.1.3 Research Questions

Main RQ.I.2 (Table 4, page 33): To what extent can Semantic Web formalisms and software-agent protocols be integrated to deliver runtime interoperability and adaptation on the Web, and what irreducible gaps prevent practical convergence?

Through the lens of a construction expediting scenario, what does the contractor agent’s ability to parse RDF-serialized protocols reveal about the extent and limits of integrating Semantic Web formalisms with software-agent protocols for runtime adaptation? The following Supporting Research Questions (SRQ) examine specific integration points and the gaps they expose:

SRQ.I.1.1 Surface integration versus semantic understanding. While RDF successfully serializes Foundation for Intelligent Physical Agents (FIPA) protocols for runtime parsing, how does the reliance on pre-programmed state machines rather than semantic reasoning reveal a fundamental gap between syntactic data exchange and the Semantic Web vision of machine understanding?

SRQ.I.1.2 Platform-specific versus Web-scale interoperability. The agent platform enables agent message exchange in a controlled Java environment, but what does this platform dependency reveal about the irreducible gap between agent-platform protocols and true Web-based interoperability?

SRQ.I.1.3 Constrained adaptation versus open-world flexibility. The expeditor adapts to conforming protocols within a fixed ontology, but how does this limited variability expose the practical convergence barrier when facing the Web’s heterogeneous services and evolving semantics?

SRQ.I.1.4 Data extraction versus semantic integration. Basic report parsing and parameter passing work within the scenario, but where does the absence of inference and context reasoning demonstrate that current integration remains far from autonomous semantic service composition?

1.2.1.4 Objectives of the Solution

O.I.1.1 Protocol parsing from RDF serialization. Demonstrate that the expeditor agent can parse RDF-serialized FIPA protocol descriptions at runtime using a pre-programmed state machine that recognizes the interaction protocol ontology structure (cf. Section 4.1.1.2, Section 4.1.1.4).

O.I.1.2 Message exchange on agent platform. Successfully exchange FIPA Agent Communication Language (FIPA-ACL) messages between contractor, subcontractor, supplier, and carrier agents on the agent platform, demonstrating basic message routing and visualization through agent platform debugging tools (cf. Section 4.1.2.2).

O.I.1.3 Scenario-specific protocol handling. Enable the expeditor to interact with supplier (FIPA Query) and carrier (similar protocol) agents in the construction scenario without hard-coding these specific interactions, though limited to protocols conforming to the predefined ontology (cf. Section 4.1.1.3).

O.I.1.4 Report extraction and comparison. Extract order identifiers from RDF-formatted status reports and use them to query related information from supplier and carrier agents, enabling basic cross-validation of delivery information (cf. Section 4.1.1.3).

1.2.1.5 Design, Demonstration and Evaluation

Design of the artefacts and demonstration can be found in Section 4.1. Discussion and evaluation can be found in Section 7.1. This study was originally published and communicated in [1, 2, 7]

1.2.2 Ontology-Driven Software Agents for B2B Purchasing Automation

1.2.2.1 Problem Identification and Motivation

Context This second demonstration focuses on application-domain ontologies and negotiation processes within B2B purchasing scenarios. In this approach, intelligent services are implemented as software agents representing participants across various business sectors, communicating according to predefined interaction protocols.

The scenario centers on purchasing processes where a buyer requires products or services and must negotiate with multiple potential suppliers. While many B2B processes have already been automated using communication networks, and the Web’s growth has encouraged companies to adopt open architectures for process automation, current purchasing processes remain time-consuming and resource-heavy for all participating companies. The technological objective is to create services that perform routine tasks on behalf of company personnel by autonomously locating other services and conducting background negotiations, thereby demonstrating significant potential for process optimization through intelligent service automation.

Business pain-point. Even though electronic data-interchange and Web-based portals have streamlined many B2B activities, the *purchasing process* itself remains labour-intensive and slow. A buying firm must (i) locate suitable suppliers, (ii) exchange multiple rounds of RFQ messages, and (iii) manually compare heterogeneous quotes—often via e-mail or proprietary portals that provide little semantic structure. These repetitive tasks tie up highly paid personnel, delay production schedules, and raise transaction costs.

Root causes. **1. Syntactic rather than semantic integration.** Current Web-service integration exchanges messages that machines can route but not *understand*; product descriptions and commercial terms are still interpreted by humans. **2. Static bindings.** Service endpoints are hard-coded through bilateral contracts, and dynamic discovery of alternative suppliers is rare, limiting competition and resilience. **3. Lack of autonomous negotiation.** No common negotiation protocol or decision logic exists that would let software agents filter, rank, or counter-propose quotes without human intervention.

Motivation for a design artefact. If routine steps—supplier discovery, RFQ dissemination, first-round quote evaluation, and simple counter-offers—could be delegated to ontology-driven software agents, buyer personnel would engage only at value-adding decision points. This would shorten purchasing lead-time, widen the supplier pool, and reduce clerical errors. Moreover, by grounding agent communication in shared domain ontologies and agent interaction protocols, the resulting solution could generalise across industries and interoperate with existing enterprise IT system back-ends.

1.2.2.2 Research Questions

The following SQRs guided the investigation of ontology-driven software agents for B2B purchasing automation:

SRQ.I.2.1 How can product and process knowledge in B2B procurement be formalized using domain ontologies to enable machine-interpretable purchasing requests and offers?

SRQ.I.2.2 Which agent communication and coordination mechanisms best support autonomous matching and negotiation based on shared ontological vocabularies?

SRQ.I.2.3 How does ontology-based reasoning contribute to interoperability and automation across heterogeneous enterprise systems?

1.2.2.3 Objectives of the Solution

O.I.2.1 Automate routine purchasing tasks. Enable services that can autonomously discover potential suppliers and conduct preliminary negotiations in the background.

O.I.2.2 Realise automation through software agents. Use software agents that represent buyer and supplier organisations and interact via predefined FIPA-compliant protocols.

O.I.2.3 Maintain human oversight. Let agents handle routine interactions while critical pricing and contractual decisions are escalated to company personnel.

O.I.2.4 Leverage application-domain ontologies. Provide shared semantics for product descriptions and negotiation messages throughout the purchasing process.

1.2.2.4 Design, Demonstration and Evaluation

Design of the artefacts and demonstration can be found in Section 4.2. Discussion and evaluation can be found in Section 7.2. This study was originally published and communicated in [1, 2]

1.2.3 Matching Service Descriptions in Context-Aware Environment

1.2.3.1 Problem Identification and Motivation

Context. Mobile users—such as recreational boaters—frequently switch location, activity, and social setting. The DYNAMOS ² initiative therefore targets *context-aware, hybrid service provision* that integrates: (i) third-party B2C service descriptions, and (ii) C2C user annotations and notes, all expressed through shared ontologies. Traditional Web / service-discovery mechanisms are designed for stationary desktop use and therefore *fail to surface only the services that matter "here & now"*. Consequently, end-users must either (i) sift through irrelevant results or (ii) miss safety- and mission-critical information (e.g., submerged rocks, guest-harbour availability, race checkpoints).

1.2.3.2 Research Questions

Main RQ.I.3 (Table 4): Which end-to-end architecture best supports an intelligent model that fuses context-aware personalization with community-shared annotations of services, so it can operate on resource-limited smartphones and still provide proactive recommendations?

To operationalise the MRQ in these settings, we decompose it into the following focused SQRs, informed by our performance measurements and field-trial feedback:

SRQ.I.3.1 End-to-end bottlenecks. Which pipeline stages dominate latency in practice, and do the results require special technology choices?

SRQ.I.3.2 Field usability and resilience. In high-workload, adverse conditions, which UI and interaction choices preserve proactive value while minimizing attention demand, crashes, and battery drain?

SRQ.I.3.3 Lightweight context ontology. What is the minimal ontology (profiles + time/location/activity) that preserves recommendation relevance while remaining feasible on smartphones and constrained links?

SRQ.I.3.4 First-class collaborative annotations. When user notes/ratings are modeled and matched like services, which matching/indexing and event-distribution choices keep real-time delivery and throughput acceptable?

1.2.3.3 Research gap.

Prior context-aware prototypes focus almost exclusively on *location* and treat service provision as a *Business-to-Consumer* (B2C) pipeline. They ignore two dimensions that have become pivotal:

G.I.3.1 Rich, multi-modal context (time, activity, social setting, sensor feeds) that fluctuates on a minute-by-minute scale; and

G.I.3.2 C2C knowledge — ratings, warnings, and free-form notes — which today is the primary driver of trust and adoption in mobile scenarios.

1.2.3.4 Objectives of the Solution

Guided by the identified gap, this study pursues the following design objectives:

O.I.3.1 Context Modelling – Create an ontology-based model that unifies relatively static *user profiles* with highly dynamic *user context* (time, location, activity) while remaining lightweight enough for mobile devices.

O.I.3.2 Matching Mechanism – Design and implement an automated matcher that links available service descriptions and user-generated annotations to the current context model, delivering only the most relevant items to the user in real time *while minimising user attention and energy overhead on smartphones*.

²Research project name

O.I.3.3 Collaborative Annotation Support – Enable end-users to contribute, rate, and share annotations and free-form notes, and ensure that these contributions are themselves treated as first-class, context-matched artefacts.

Motivation for a design artefact. A *hybrid, ontology-driven service-matching mechanism* that blends B2C service descriptions with crowd-sourced C2C annotations *and* aligns them automatically with the user’s real-time context would

- increase personal safety and decision speed for sailors and other mobile communities;
- expand the commercial reach of micro-service providers without spamming uninterested users;
- supply researchers with a reusable test-bed for evaluating context models, matching algorithms, and event-based distribution at scale.

In short, the problem is how to deliver the *right* professional and peer-generated information to a mobile user at the *right* moment, and the motivation is the substantial practical and scholarly value unlocked by solving this challenge.

1.2.3.5 Design, Demonstration and Evaluation

Design of the artefacts and demonstration can be found in Section 4.3. Discussion and evaluation can be found in Section 7.3. This study was originally published and communicated in [1, 2, 6, 8, 3, 4].

1.2.4 Phase I from a 2025 perspective

While the Phase I research and demonstrations (2001-2006) (Section 1.1.2) predate the current GenAI revolution by nearly two decades, they identified three fundamental challenges that remain central to today’s LLM agent deployments:

Runtime protocol adaptation (Section 1.2.1). The construction expediting scenario required agents to dynamically interpret and execute partner-specific interaction protocols without hard-coding. This exact problem resurfaces in 2025 as LLM agents struggle to reliably invoke tools through OpenAPI specifications, Model Context Protocol (MCP), and Google’s Agent2Agent (A2A) protocol (see Section 2.5). Where Phase I agents parsed RDF-serialized FIPA protocols, today’s agents must interpret JSON schemas and function signatures. The core challenge—enabling runtime adaptation to heterogeneous service interfaces—remains unchanged. Current solutions like function-calling in GPT models and tool-use in Claude face the same semantic gap between syntactic interface descriptions and genuine understanding of service semantics that limited Phase I agents.

Ontology-driven service discovery and negotiation (Section 1.2.2). The B2B purchasing scenario’s reliance on shared ontologies for automated supplier discovery and quote evaluation directly parallels how modern Retrieval-Augmented Generation (RAG) systems and agent frameworks ground LLM outputs in structured knowledge bases. Today’s vector databases and embedding-based retrieval are sophisticated descendants of the semantic matching attempted in Phase I. The fundamental tension between maintaining semantic precision (through explicit ontologies) and achieving practical scalability (through statistical methods) that constrained Phase I systems manifests today in hybrid neuro-symbolic approaches and the ongoing challenge of preventing LLM hallucinations when interfacing with structured business logic.

Context-aware service matching (Section 1.2.3). The DYNAMOS system’s ambition to deliver location-, time-, and activity-aware services to mobile users prefigures current challenges in personalizing LLM agent behavior based on multimodal context. Modern agents must similarly integrate user preferences, real-time sensor data, and situational awareness to provide relevant responses. The Phase I insight that context requires both static profiles and dynamic state remains foundational to today’s personalization pipelines, though now implemented through prompt engineering and RAG rather than rule-based matching.

Why these problems persist. The persistence of these challenges across technological generations suggests they represent invariant aspects of the agent-service integration problem rather than mere implementation difficulties. The Phase I work’s value lies not in its technical solutions—which were constrained by era-appropriate technology—but in its precise identification of where human semantics, machine reasoning, and

distributed coordination must interface. These friction points remain active sites of research and engineering effort in 2025, validating the problem framing even as solution approaches have evolved from symbolic to statistical to generative paradigms.

1.3 Phase II Problem and Motivation: Software Agents and ML Healthcare Item Sets

Research Context. Healthcare organizations face a critical knowledge fragmentation problem that undermines clinical decision-making effectiveness and patient safety. This problem manifests through three interconnected dimensions that collectively prevent optimal utilization of accumulated clinical experience and patient data.

Problem Dimension 1: Information System Fragmentation. Modern hospitals operate complex ecosystems of legacy information systems—ranging from dozens to hundreds of specialized platforms—each designed for specific clinical, administrative, or laboratory functions. This architectural fragmentation creates data silos where patient information becomes distributed across heterogeneous databases with incompatible interfaces and terminologies. The resulting information architecture forces clinicians to sequentially interrogate multiple systems to construct comprehensive patient views, introducing delays, cognitive overhead, and potential for incomplete information retrieval during critical decision points.

Problem Dimension 2: Limited Real-time Clinical Intelligence. Current healthcare information systems provide limited capability for real-time pattern recognition and case-based recommendation generation. Despite hospitals accumulating vast repositories of clinical cases and outcomes, this institutional knowledge remains largely inaccessible for immediate decision support. Clinicians cannot efficiently leverage similar historical cases to inform current treatment decisions, resulting in missed opportunities for evidence-based care optimization and potential repetition of suboptimal treatment patterns.

Problem Dimension 3: Scalability Constraints for Intelligent Systems. While software agent architectures and machine learning techniques demonstrate theoretical promise for healthcare data integration and intelligent recommendation systems, their deployment in production hospital environments remains limited. This implementation gap stems from challenges in achieving real-time performance with large-scale healthcare datasets while maintaining acceptable accuracy levels and operating within the hardware constraints typical of clinical environments.

Theoretical Foundation and Research Gap. The confluence of these problem dimensions creates a significant gap between the potential value of accumulated clinical data and its practical utilization for decision support. Existing approaches typically address individual dimensions in isolation—data integration solutions focus on technical interoperability without intelligent analysis capabilities, while clinical decision support systems operate on limited data sources without comprehensive patient context.

This research identifies a critical need for an integrated artifact that simultaneously addresses: **Real-time heterogeneous data integration** across multiple clinical systems. **Intelligent pattern recognition** for case-based recommendation generation. **Scalable performance** suitable for production healthcare environments.

1.3.1 Assisting Clinical Decision Support with ML and Software Agents

1.3.1.1 Problem Identification and Motivation

Given the critical importance of timely, comprehensive clinical decision support and the demonstrated limitations of existing approaches, this research addresses the following design problem: How can an artifact be designed and implemented that provides clinicians with unified, real-time access to comprehensive patient data across heterogeneous information systems while delivering intelligent, case-based treatment recommendations that scale to institutional and national-level data volumes? This problem statement encompasses both the technical challenge of real-time heterogeneous data integration and the analytical challenge of generating actionable clinical intelligence from large-scale healthcare datasets, while maintaining the practical constraints of production healthcare environments.

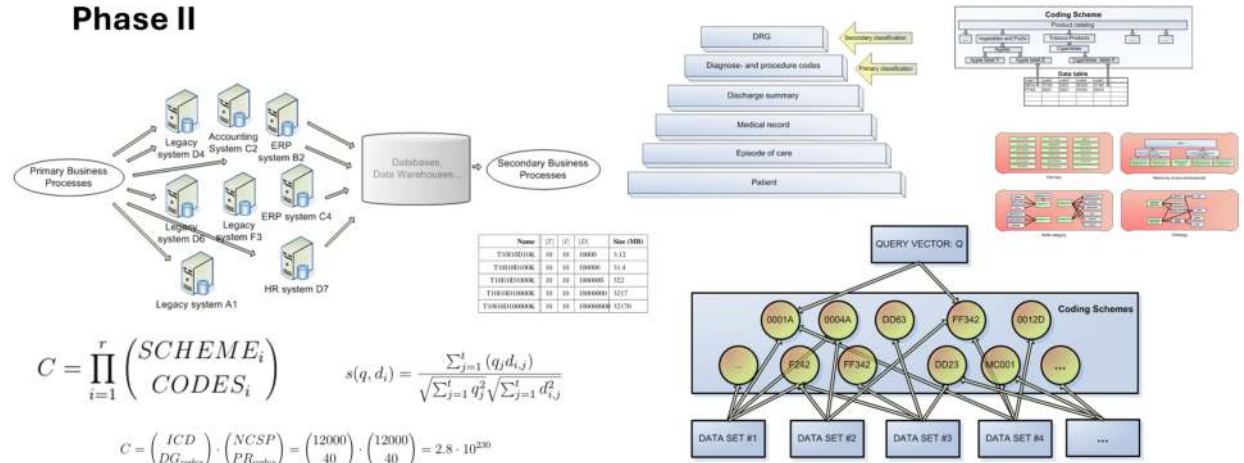


Figure 4: Phase II: Analyzing Item Sets with ML and Software Agents.

1.3.1.2 Research Questions

Main RQ.II.1 (Table 4): How can software agents perform ad-hoc, flexible analysis over large healthcare item sets on commodity hardware under real-world data-quality defects while meeting explicit bounds on latency, memory, and accuracy to support clinical decision-making and hospital funding decisions (DRG coding)?

To operationalise the MRQ in these settings, we decompose it into the following focused SQRs, informed by our performance measurements and field-trial feedback:

- SRQ.II.1.1 Robust ML architectures.** What ML/IR model characteristics are essential for agent-based systems operating on large, imperfect healthcare datasets with limited hardware resources?
- SRQ.II.1.2 Recommendation accuracy thresholds.** What recommendation accuracy levels are achievable with real healthcare data using vector space models in multi-agent architectures?
- SRQ.II.1.3 Robustness to data-quality defects.** How do realistic documentation defects affect recommendation accuracy, and which retrieval constraints mitigate degradation?
- SRQ.II.1.4 Real-time scalability on commodity hardware.** What maximum healthcare data volumes can MAS architecture handle while maintaining sub-second query response times on standard hardware?

1.3.1.3 Objectives of the Solution

This primary objective is operationalized through the following specific solution objectives:

- O.II.1.1 Robust ML/IR Architecture (Arch)** - Design vector space model implementations and kernel-based similarity methods that efficiently handle high-dimensional, sparse healthcare data spaces while leveraging semantic relationships inherent in clinical classification systems (ICD-10, NCSP).
- O.II.1.2 Clinically Viable Recommendation Accuracy (Acc)** - Achieve recommendation systems delivering >50% first-proposal accuracy and >90% top-10 accuracy for clinical coding suggestions, with flexible fetching strategies that maintain quality under varying data conditions.
- O.II.1.3 Robustness to Data-Quality Defects (Rob)** - Implement flexible retrieval strategies that maintain recommendation quality when confronted with incomplete, inconsistent, or erroneous healthcare documentation, demonstrating graceful degradation under realistic data defects.
- O.II.1.4 Real-time Scalability on Commodity Hardware (Scale)** - Develop distributed MAS architecture capable of processing queries against datasets containing up to 10^8 patient episodes within sub-second response times on standard hardware, with linear scaling as data volume and system diversity increase.

These objectives collectively aim to demonstrate that software agent technology can provide a practical, scalable solution for healthcare decision support that bridges the gap between theoretical ML capabilities and real-world clinical information system requirements.

1.3.1.4 Design, Demonstration and Evaluation

Design of the artefacts and demonstration can be found in Section 5.1. Discussion and evaluation can be found in Section 7.4. This study was originally published and communicated in [14].

1.3.2 Phase II from a 2025 perspective

Examining this 2008–2013 research (Section 1.1.2) from a 2025 perspective reveals both prescient insights and fundamental shifts in healthcare AI adoption.

During 2009–2010, I experimented and demonstrated ontology-aware query-projection methods to enhance healthcare NLP/IR agents. The approach preserved minimal datasets in orthogonal, high-dimensional sparse bases and projected queries through clinical taxonomies (e.g., ICD-10, NCSP, DRG) to improve recall while trying to maintain precision. These results were implemented as demonstration prototypes and results were not published in peer-reviewed conferences at the time. Notably, this work has similarities and predates subsequent formal publications on distributional representations such as word2vec (published in 2013) [43] and dual-encoder ("two-tower") retrieval architectures—e.g., DSSM (2013) and large-scale recommendation two-tower models (2016)—in which separate encoders map queries and items into a shared embedding space for efficient matching [44, 45].

By 2025, AI has moved beyond experimental implementations to become embedded in clinical workflows, with applications ranging from real-time diagnostic support to workflow optimization. The original research’s focus on addressing healthcare data fragmentation and enabling real-time clinical intelligence anticipated core challenges that remain central to contemporary AI implementations.

The MAS architecture proposed in this Phase II work demonstrated remarkable foresight regarding distributed healthcare AI systems. Current 2025 developments explicitly recognize AI agents as "significant advancements in AI because they work independently to carry out tasks on behalf of a user," with applications spanning from clinical notetaking to disease detection. However, the technological foundation has shifted dramatically—while the original research relied on Java Agent Development Platform (JADE) platform for agent coordination, modern implementations leverage cloud-native architectures, LLMs, and advanced ML frameworks that were not available during the original research period.

The research’s mathematical analysis demonstrating the infeasibility of neural networks for sparse healthcare data spaces has been superseded by technological advances. Contemporary AI-driven clinical decision support systems successfully employ deep learning architectures, including convolutional neural networks, transformers, and attention mechanisms specifically designed for high-dimensional healthcare data. The 2.8×10^{230} combinatorial complexity that originally justified information retrieval approaches over neural networks is now addressed through sophisticated embedding techniques, transfer learning, and large-scale pre-trained models that can effectively handle sparse, high-dimensional clinical data.

Yet the fundamental problems identified in Phase II persist with striking relevance. Healthcare organizations in 2025 continue to struggle with fragmented information systems, data quality defects, and the need for real-time integration across heterogeneous platforms. The original research’s emphasis on performance bounds (sub-250ms response times, commodity hardware constraints) aligns closely with contemporary requirements for healthcare AI systems to demonstrate clear ROI, operational efficiency improvements, and practical deployability in resource-constrained clinical environments. The robustness to data quality defects, remains highly relevant as modern AI-CDS systems continue to grapple with incomplete documentation, algorithmic bias, and the need for explainable recommendations in clinical settings.

1.4 Phase III Problem and Motivation: Software Agents in Generative AI Era

Research Context. The third and most recent phase of this research revisits the concept of software agents within a radically transformed technological landscape, marked by the emergence of LLMs and GenAI. Whereas the earlier phases of the project concentrated on infrastructural concerns and domain-specific data analytics, Phase III shifts the focus toward reimagining agent-based systems through the capabilities offered by LLM-powered multi-agent architectures.

Phase III

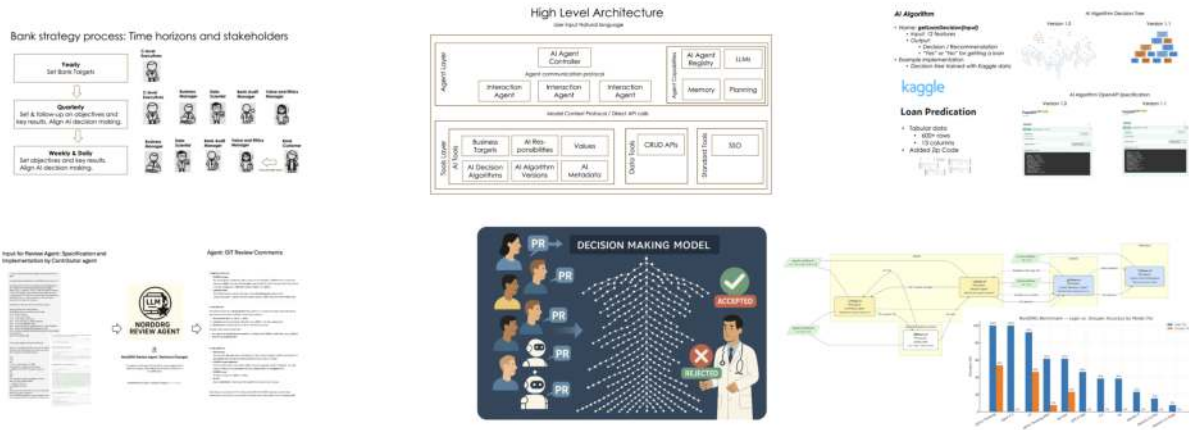


Figure 5: Phase III: Software Agents in Generative AI Era.

1.4.1 HADA: Human-AI Agent Decision Alignment Architecture

1.4.1.1 Problem Identification and Motivation

Despite the explosive adoption of LLM software agents, most organisations still lack a rigorous, end-to-end mechanism for ensuring that these autonomous components — together with the legacy decision algorithms they increasingly wrap — remain aligned with measurable business targets, codified organisational values, and evolving regulation.

1.4.1.2 Research Questions

Main RQ.III.1 (Table 4): How could we leverage LLM agents to facilitate human-AI decision alignment in large corporate settings?

Building on this **MRQ** this study asks the following **SRQs**:

SRQ.III.3.1 What LLM agent protocol-level gaps hinder human-AI decision alignment in enterprise settings?

SRQ.III.3.2 What integration issues does large-scale LLM-agent integration reveal for human-AI decision alignment?

SRQ.III.3.3 What human-in-the-loop control mechanisms are required in practice to keep decision responsibilities clear?

1.4.1.3 Research gap

Three persistent gaps make this deficit salient and practically urgent:

G.III.1.1 Alignment Gap — No systematic way exists to translate high-level objectives and ethical principles into the reward functions or policies that drive individual agents and algorithms.

G.III.1.2 Containment Gap — Enterprises lack dependable circuit-breakers and “kill switches” to restrain agents that deviate, self-modify or hallucinate in production environments.

G.III.1.3 Integration Gap — Practitioners have few vendor-agnostic blueprints for embedding heterogeneous decision algorithms into multi-agent, LLM-powered workflows while preserving end-to-end auditability.

The confluence of (i) accelerating regulatory pressure (e.g., EU AI Act), (ii) reputational risks from misaligned decisions, and (iii) the strategic value of autonomous, conversational systems motivates the search for a design solution that can close these gaps at scale.

1.4.1.4 Objectives of the Solution

Guided by DSRM, the research sets six design objectives that together define the required capabilities of a Human-AI agent Decision Alignment (HADA) architecture solution:

- O.III.1.1 Natural-language interaction across planning horizons** — Enable every stakeholder (C-suite, risk officers, data scientists, auditors, customers) to steer goals and examine decisions through conversational interfaces, bridging annual strategy and daily operations.
- O.III.1.2 Multi-dimensional alignment (targets & values)** — Bind each algorithm’s optimisation logic simultaneously to quantitative *Objectives & Key Results* (OKRs) and to machine-readable organisational values, maintaining a verifiable audit chain from principles to actions.
- O.III.1.3 Reference architecture for hierarchical, hybrid AI systems** — Provide a modular, protocol-agnostic stack that cleanly separates strategy, coordination and execution layers, allowing LLM agents and classical algorithms to coexist under common governance.
- O.III.1.4 Stakeholder alignment across time-scales** — Offer repeatable methods for eliciting, reconciling and propagating stakeholder objectives at yearly, quarterly and operational cadences.
- O.III.1.5 Scalability, auditability and near real-time propagation** — Support thousands of agents and algorithms, propagate objective or policy changes within hours, and preserve immutable, explainable decision logs suitable for external audit.
- O.III.1.6 Framework-agnostic, policy-driven design** — Remain independent of any single LLM, agent library or orchestration tool by exposing open standards so that strategic edits flow unimpeded to heterogeneous runtimes.

1.4.1.5 Design, Demonstration and Evaluation

Design of the artefacts and demonstration can be found in Section 6.1. Discussion and evaluation can be found in Section 7.5. Conclusions can be found in Section 8.4. This study was published and communicated in [27], building on earlier work in [25] and related publications [18, 19] where I served as a contributing author.

1.4.2 NordDRG AI Benchmark: Teaching LLMs the Nuances of Hospital Funding Instruments

1.4.2.1 Problem identification and motivation

LLMs are increasingly piloted for clinical coding and decision support, yet almost no open evaluation targets the *hospital-funding* layer where Diagnosis-Related Groups (DRGs) determine reimbursement. The stakes are macroeconomic: global health spending is on the order of US\$10 trillion annually, with hospitals absorbing the largest share; in DRG-based systems a sizeable portion of these flows is routed through governed grouper software. Transparency and auditability are therefore first-order requirements, not implementation details.

In most OECD settings, a DRG grouper is a governed decision model that maps a patient episode’s minimum dataset to a single payment group via an ordered rule graph with explicit inclusions, exclusions, and national activation flags. NordDRG makes this logic uniquely inspectable through ~20 inter-linked definition tables released annually alongside expert manuals and change templates. For non-specialists, the rule graph remains opaque; for experts, verifying end-to-end control flow and its governance trace is laborious.

1.4.2.2 Research Questions

Main RQ.III.2 (Table 4:) To what extent can LLMs emulate, explain, and apply DRG-based hospital payment rules?

The following questions support RQ★ and align one-to-one with the objectives of the solution:

- SRQ.III.2.1 Artefact design.** What *minimal, versioned* artefact bundle and schema are necessary and sufficient to encode the complete NordDRG rule graph and its governance semantics in a public, machine-readable form?
- SRQ.III.2.2 Benchmark operationalisation.** How should tasks, inputs, and exact-match scoring be structured so that (i) the *Logic* suite measures rule-level reasoning over the definition tables and

manuals, and (ii) the *Grouper* suite enforces *full emulation* of the official control flow—returning both `drg_nat` and the triggering `drg_logic.id`?

SRQ.III.2.3 Baseline capability and failure modes. When constrained to the released artefacts (no external web access), how do representative LLM endpoints perform on the *Logic* and *Grouper* suites, and which systematic errors account for misses (e.g., cross-table joins, ORD priority, CC/MCC exclusions, national activation)?

1.4.2.3 Research gap

Generic LLM leaderboards neither encode this rule graph nor enforce the audit trail that payers require. This yields two persistent gaps:

G.III.2.1 Scope gap — the absence of a public, *rule-complete*, machine-readable artefact that bundles the full NordDRG specification (tables *and* governance documents) with stable identifiers.

G.III.2.2 Benchmark gap — the absence of a standardised, *governance-grade* benchmark that tests both rule-level CaseMix reasoning and faithful emulation of the grouper’s control flow with strict, auditable outputs (DRG and triggering `drg_logic.id`).

As a result, the community lacks a reproducible yardstick for measuring whether LLMs can read the DRG rule graph, apply governed control flow, and return decisions traceable to specific rule rows—capabilities that are essential for regulator-compliant automation in hospital funding.

1.4.2.4 Objectives of the Solution

Guided by the DSRM, this work translates the above gaps into a rule-complete artefact and two evaluation suites with exact-match scoring.

O.III.2.1 Artefact construction (*addresses the Scope Gap; DSRM: Design & Development*). Release an open, machine-readable, versioned benchmark (*NordDRG-AI-Benchmark*) that captures the *complete* NordDRG rule graph—definition tables, diagnosis/procedure property tables, age/sex splits, national activation flags, and governance manuals—with schema-stable identifiers and modular packaging for reuse and extension (see Section 6.2.1).

O.III.2.2 Benchmark operationalisation (*bridges both gaps; DSRM: Demonstration*). Instantiate two complementary test suites with exact-match scoring: (i) the *NordDRG Logic Benchmark*—13 rule-level questions reflecting coder, clinician, and policy-analyst workflows, with public answer keys for quantitative tasks and rubric guides for qualitative ones (Section 6.2.1.3); and (ii) the *NordDRG Grouper Benchmark*—13 case-level tasks that require faithful execution of the specification’s control flow and return both `drg_nat` and the triggering `drg_logic.id` (Section 6.2.1.4).

O.III.2.3 Baseline evaluation (*targets the Benchmark Gap; DSRM: Evaluation*). Provide first-cut, fully reproducible results for representative state-of-the-art LLM endpoints across providers, including prompts, context bundles and model outputs — establishing the initial yardstick for head-to-head comparisons and longitudinal tracking in hospital-funding contexts (Section 6.2.2).

1.4.2.5 Design, Demonstration and Evaluation

Design of the artefacts and demonstration can be found in Section 6.2. Discussion and evaluation can be found in Section 7.6. Conclusions can be found in Section 8.5. This study was published and communicated in [23], with earlier work appearing in [20, 21].

1.4.3 CCL: How Humans and AI Agents Align on Decision-Making Rules

1.4.3.1 Problem Identification and Motivation

Context. In safety-critical domains (healthcare, finance, public policy), the decision function $f : \mathcal{X} \rightarrow \mathcal{Y}$ is often implemented as an *explicit, human-auditable* artefact—rule bases, scorecards, or multi-objective algorithms—so that every transformation step can be inspected and justified. NORDDRG, with millions of hand-curated decision points, exemplifies this approach. The very safeguards that make such logic trustworthy—line-by-line review, change control, traceable audit trails, and formal sign-off—also slow release cadence to a handful of updates per year, while coding standards, clinical practice, and policy evolve continuously. The result is policy–practice drift and accumulating maintenance debt.

Problem statement.

When AI agents are introduced into regulated decision systems, there is no governed, scalable workflow that allows those agents to propose fine-grained edits for expert curation and ratification—end to end with an auditable trail—while preserving the accountability required by governance frameworks.

Scope. This problem statement applies to AI-augmented workflows; it does not claim that governed human-only workflows are unavailable.

Illustrative challenges when considering introducing AI agents into the NordDRG development cycle

- 1 **Release cadence constraints.** Annual release trains remain the norm; under current QA and sign-off, continuous delivery of AI-assisted micro-edits is difficult in practice. This study acknowledges that many processes—such as cost-weight calculation and budgetary allocation—are not amenable to continuous delivery, but here this study focuses on the decision logic itself.
- 2 **Scale and cognitive load.** As the rule base expands, this study observes challenges with conflict resolution, and onboarding remains challenging because of substantial context dependencies.
- 3 **Limited agent–expert loop.** In the tools and processes this study had access to, no production-grade, bidirectional workflow where AI agents propose well-typed diffs and certified experts curate and ratify them with guardrails and auditability was observed. This point concerns AI-augmented loops and does not speak to human-only governance processes.
- 4 **Opacity of AI suggestions.** Off-the-shelf ML/LLM outputs this study examined or piloted tended to lack line-level provenance, counterfactual evidence, and legally admissible documentation typically required for acceptance.

Scope and basis of observations. These points pertain only to AI-augmented workflows. They synthesise the authors’ limited interactions with NORDDRG stakeholders across selected release cycles (e.g., participation in some specification review discussions and a small set of change-ticket exchanges), complemented by analysis of definition tables and publicly available change artefacts, and several semi-structured conversations. They are offered with respect for maintainers’ expertise and constraints, as indicative observations rather than comprehensive claims about all NORDDRG practices.

Motivation for research and practice. Addressing these challenges would (i) operationalise the Collaborative Curated Learning (CCL) paradigm as a governed development process for decision logic, (ii) combine the agility of continuous ML adaptation with the transparency of expert-curated rules, and (iii) yield a transferable template for other safety-critical domains where algorithmic decisions must remain both explainable and current. This motivates the research questions in Section 1.4.3.2 and informs the design objectives in Section 1.4.3.4.

1.4.3.2 Research Questions

Main RQ.III.3 (Table 4): What end-to-end governance requirements and workflow properties enable AI agents to contribute to decision-model development without eroding human expert accountability?

Building on this **MRQ**, the research questions of this study are articulated at the *process* level. They derive from the gap analysis (Section 1.4.3.3) and map directly onto the design objectives (Section 1.4.3.4).

This study asks:

- SRQ.III.3.1** How should a million-rule decision model like NordDRG best be represented for AI agents to support safe, atomic diffs?
- SRQ.III.3.2** Under which constraints do LLM agents produce reviewer-acceptable change proposals for decision models at $\approx 10^7$ -point scale?

In particular, first is intentionally framed to test *any* governed end-to-end workflow that enables agent contributions while preserving expert accountability; the CCL process introduced later is evaluated as one candidate instantiation rather than presupposed by the question. The latter two then specialise this framing to representation choices and agent reliability at the $\approx 10^7$ -scale characteristic of systems like NORDDRG.

1.4.3.3 Research Gap

Despite rapid advances in machine learning and software-engineering toolchains, literature review of XAI, MLOps, and health-informatics literature finds *no governance-grade, end-to-end framework* that allows AI agents and certified experts to co-maintain a safety-critical decision model at the scale of NORDDRG. Four gaps motivate this study:

- G.III.3.1 Governed co-development loop is missing.** Prior work typically positions human validation as a terminal “approval step” [46], rather than embedding experts in an *iterative* proposer–reviewer cycle where agents submit fine-grained diffs and experts curate them under explicit roles and gates.
- G.III.3.2 Change-level auditability is insufficient.** Popular explainability methods justify a *static* model post hoc [47, 48], but rarely capture the lineage, test evidence, and reviewer sign-offs needed to defend each micro-update to the decision logic.
- G.III.3.3 Software-engineering mechanics are underused in AI governance.** Version control, pull requests, and automated test suites are standard in code development, yet are seldom operationalised in human–AI governance of decision rules [49].
- G.III.3.4 Empirical evidence at operational scale is scarce.** Published case studies do not reach NORDDRG scale spans $\approx 10^7$ decision points, outscaling the largest documented human-in-the-loop experiments by orders of magnitude and leaving scalability claims untested.

These gaps lead directly to the research questions (Section 1.4.3.2) and shape the design objectives (Section 1.4.3.4). This study addresses the gap by specifying a governed co-maintenance process (CCL; Section 6.3.1.3), providing an agent-operable model-of-record for NORDDRG (Section 6.3.1.4), and demonstrating agent roles within that process (Section 6.3.2), with evaluation in Section 7.7.

1.4.3.4 Objectives of the Solution

To answer research questions (Section 1.4.3.2), this study defines three objectives for the solution. Each objective notes where it is specified, instantiated, and evaluated.

- O.III.3.1 Governed workflow.** Operationalise maintenance-time alignment by specifying the *governance requirements and workflow properties* (roles, risk tiers, CI gates, immutable ledger, human-in-command) that enable agent contributions without eroding expert accountability.
- O.III.3.2 Agent-operable model-of-record.** Provide a line-addressable representation for decision models on the order of $\approx 10^7$ *decision points*, enabling safe, atomic diffs and reviewer-legible slices.
- O.III.3.3 LLM agents under constraints.** Implement Contributor/Reviewer agents under explicit *constraints* (grounding, role restrictions, gate policy) and measure reviewer acceptance, CI pass rates, error severity, and reviewer effort.

1.4.3.5 Design, Demonstration and Evaluation

Design of the artefacts and demonstration can be found in Section 6.3. The CCL Platform, a full-stack web application implementing these artefacts, is described in Section 6.3.2.1. Discussion and evaluation can be found in Section 7.7. Conclusions can be found in Section 8.6. This study was published and communicated in [28].

1.5 Main Research Questions

Main research questions and objectives of the Thesis are summarized in Table 4. The research is divided into three phases, each corresponding to a distinct period, with specific research questions for each phase. The first phase focuses on the evolution of the Web and the role of software agents, while the second phase delves into the analysis of item sets using ML and software agents. The third phase explores the integration of software agents with LLMs and their implications for algorithmic decision-making and human alignment.

Table 4: Main Research Questions by phase.

Phase	MRQ #	Main Research Question
I	I.1	What are the historical origins of the Web and of software agents?
	I.2	To what extent can Semantic Web formalisms and software-agent protocols be integrated to deliver runtime interoperability and adaptation on the Web, and what irreducible gaps prevent practical convergence?
	I.3	Which end-to-end architecture best supports an intelligent model that fuses context-aware personalization with community-shared annotations of services, so it can operate on resource-limited smartphones and still provide proactive recommendations?
II	II.1	How can software agents perform ad-hoc, flexible analysis over large healthcare item sets on commodity hardware under real-world data-quality defects while meeting explicit bounds on latency, memory, and accuracy to support clinical decision-making and hospital funding decisions (DRG coding)?
III	III.1	How could we leverage LLM agents to facilitate human–AI decision alignment in large corporate settings?
	III.2	To what extent can LLMs emulate, explain, and apply DRG-based hospital payment rules?
	III.3	What end-to-end governance requirements and workflow properties enable AI agents to contribute to decision-model development without eroding human expert accountability?

1.6 PhD Artefacts

1.6.1 Main Artefacts

As summarised in Table 5, this dissertation produced a portfolio of seven main design artefacts that map directly to the three research phases. Phase I delivers the agent-level innovations—run-time protocol adaptation, ontology-driven negotiation, and context-aware mobile service matching—that probe the limits of early Semantic-Web tooling. Phase II contributes a large-scale, machine-learning-enabled clinical recommendation system, demonstrating that statistical retrieval can extend symbolic hierarchies without redesigning them. Phase III culminates in three complementary contributions: the HADA alignment framework for translating organizational values into auditable multi-agent controls, the NordDRG AI Benchmark establishing the first public evaluation suite for hospital funding logic, and the CCL process enabling governed human-AI co-maintenance of safety-critical decision models. Together, these Phase III artifacts show how value-aligned, auditable agent cohorts can operate on contemporary LLM substrates while maintaining the governance requirements essential for regulated domains. Taken as a whole, the artefact set illustrates the thesis’s longitudinal claim: agent-oriented design remains viable across successive AI paradigms while progressively tightening the link between transparency, governance, and real-world deployment.

Table 5: Main artefacts created across the three research phases.

Phase	Years	No.	Main Artefact	Purpose / Contribution
I	2001–2006	1	Run-time interaction-protocol adaptation engine.	Dynamically retrieves RDF/agent protocol specifications and drives interactions, dispensing with hard-coded workflows. Problem: Section 1.2.1, Design and Demonstration: Section 4.1, Artefacts: Section 4.1.1
		2	Ontology-driven B2B negotiation prototype.	Contract-Net multi-agent demonstrator that automates RFQ, quote revision, and contract closure in supply-chain scenarios. Problem: Section 1.2.2, Design and Demonstration: Section 4.2, Artefacts: Section 4.2.1
		3	<i>DYNAMOS</i> context-aware service matcher.	Intelligent mobile service matching that fuses B2C descriptions with C2C annotations via shared ontologies and real-time context. Problem: Section 1.2.3, Design and Demonstration: Section 4.3, Artefacts: Section 4.3.1
II	2007–2013	4	ML-enabled clinical recommendation MAS.	Crawler–integrator–aggregator pipeline; IR vector engine imputes missing medical codes over $> 10^8$ patient episodes on commodity hardware. Problem: Section 1.3, Design and Demonstration: Section 5.1, Artefacts: Section 5.1.1
III	2023–2025	5	<i>HADA</i> architecture.	A protocol-agnostic, multi-agent reference architecture that translates organizational OKRs/values into auditable, vendor-neutral controls by orchestrating stakeholder agents over LLMs and legacy decision logic (demonstrated on retail-bank credit scoring). Problem: Section 1.4.1, Design and Demonstration: Section 6.1, Artefacts: Section 6.1.1
		6	<i>NordDRG AI Benchmark</i> .	First public open hospital funding-logic AI benchmark for LLMs uniting Nordic DRG definition tables, and 14 CaseMix logic and 14 CaseMix grouping tasks. Problem: Section 1.4.2, Design and Demonstration: Section 6.2, Artefacts: Section 6.2.1
		7	<i>CCL</i> process.	Reference development process for automated decision-making logic enabling human-AI alignment and collaboration in maintenance of decision rules. Problem: Section 1.4.3, Design and Demonstration: Section 6.3, Artefacts: Section 6.3.1

1.6.2 Phase I Artefacts

Table 6: Phase I individual artefacts and their mapping to the main artefacts (Table 5).

ID	Artefact	Main Artefact	Source section
A.I.1.1	Process Flow.	Run-time interaction-protocol adaptation engine	Section 4.1.1
A.I.1.2	Interaction Protocol Ontology.	Run-time interaction-protocol adaptation engine	Section 4.1.1
A.I.1.3	Interaction Protocol Serializations.	Run-time interaction-protocol adaptation engine	Section 4.1.1
A.I.1.4	Adapting to Interaction Protocol Descriptions.	Run-time interaction-protocol adaptation engine	Section 4.1.1
A.I.2.1	Purchase Process Flow.	Ontology-driven B2B negotiation prototype	Section 4.2.1
A.I.2.2	Decision Making.	Ontology-driven B2B negotiation prototype	Section 4.2.1
A.I.2.3	Data Models and Ontologies.	Ontology-driven B2B negotiation prototype	Section 4.2.1
A.I.2.4	Locating Services / Registry.	Ontology-driven B2B negotiation prototype	Section 4.2.1
A.I.3.1	Context-Profile-Content Ontologies	DYNAMOS context-aware service matcher	Section 4.3.1
A.I.3.2	Matching Engine & System Architecture	DYNAMOS context-aware service matcher	Section 4.3.1
A.I.3.3	Event-Driven Distribution Layer	DYNAMOS context-aware service matcher	Section 4.3.1

1.6.3 Phase II artefacts

Table 7: Phase II individual artefacts and their mapping to the main artefacts (Table 5).

ID	Artefact	Main Artefact	Source section
A.II.1.1	Process Flow.	ML-enabled clinical recommendation MAS	Section 5.1.1
A.II.1.2	Data Space.	ML-enabled clinical recommendation MAS	Section 5.1.1
A.II.1.3	ML/IR Model.	ML-enabled clinical recommendation MAS	Section 5.1.1
A.II.1.4	MAS Architecture.	ML-enabled clinical recommendation MAS	Section 5.1.1

1.6.4 Phase III artefacts

Table 8: Phase III individual artefacts and their mapping to the main artefacts (Table 5).

ID	Artefact	Main Artefact	Source section
A.III.1.1	Strategy-Process Blueprint	HADA architecture	Section 6.1.1
A.III.1.2	Stakeholder & User-Story Specification	HADA architecture	Section 6.1.1
A.III.1.3	Prototype AI Tool <code>getLoanDecision</code>	HADA architecture	Section 6.1.1
A.III.1.4	AI-Responsibility Tool (RACI Matrix)	HADA architecture	Section 6.1.1
A.III.1.5	Generic HADA Architecture	HADA architecture	Section 6.1.1
A.III.1.6	Agent Modelling	HADA architecture	Section 6.1.1
A.III.2.1	NordDRG Full Specification	NordDRG AI Benchmark	Section 6.2.1
A.III.2.2	Expert documentation	NordDRG AI Benchmark	Section 6.2.1
A.III.2.3	NordDRG Logic Benchmark	NordDRG AI Benchmark	Section 6.2.1
A.III.2.4	NordDRG Grouper Benchmark	NordDRG AI Benchmark	Section 6.2.1
A.III.2.5	Curated table excerpts (FI / FI-EN)	NordDRG AI Benchmark	Section 6.2.1
A.III.2.6	Lightweight reference agents.	NordDRG AI Benchmark	Section 6.2.1
A.III.3.1	Decision-making model	CCL process	Section 6.3.1
A.III.3.2	Participants and workflows	CCL process	Section 6.3.1
A.III.3.3	CCL governance process	CCL process	Section 6.3.1
A.III.3.4	NordDRG decision model representation	CCL process	Section 6.3.1
A.III.3.5	LLM-based software agents	CCL process	Section 6.3.1

1.7 Summary: Problem Identification and Objectives

Context. This chapter situates the work within the evolution of Web technologies and intelligent agents, clarifying why runtime interoperability and adaptation are central to the thesis.

Problem. The core problem concerns how heterogeneous protocols, data models, and organizational constraints hinder automated, auditable interaction across systems and domains.

Research questions. The chapter defines the guiding questions and their linkage to the thesis-level inquiry (see Table 4), specifying what must be demonstrated empirically and through artefact design.

Objectives. The problem is decomposed into concrete objectives, each mapped to an evaluation method and evidence type (e.g., experiments, case studies, benchmarks), establishing clear acceptance criteria.

Scope and assumptions. The solution space is bounded (technical scope, datasets, platforms), and key assumptions and constraints are recorded to clarify feasibility and external validity.

Transition. The next Chapter **Background**, provides the theoretical and technical foundations that underpin these objectives and inform the design and evaluation choices made later.

See Table 3 for quick navigation to the DSRP process sequence linked to Phases and Structure of this Thesis.

1.8 The Structure of the Thesis

The rest of this Thesis is organized as follows:

Chapter 2 – Provides background information relevant to the Thesis and the upcoming chapters.

Chapter 4 – Describes the design, development, and demonstrations of Phase I.

Chapter 5 – Describes the design, development, and demonstrations of Phase II.

Chapter 6 – Describes the design, development, and demonstrations of Phase III.

Chapter 7 – Discusses and evaluates the design, development, and demonstrations from Phases I, II, and III.

Chapter 8 – Summarizes the main findings and presents the overall conclusions of the Thesis.

See Table 3 for quick navigation to the DSRP process sequence linked to Phases and Structure of this Thesis.

2 BACKGROUND

This chapter provides essential background for the subsequent chapters of this thesis. History and evolution of AI, Software Agents, the World Wide Web, and the Semantic Web are presented and discussed.

This chapter is structured as follows:

- Section 2.1 presents an overview of the technological hype cycles, highlighting the recurring patterns of "AI summers" and "AI winters". Parts of this section have been published in [24].
- Section 2.2 outlines the foundational concepts and historical development of the Web. Parts of this section have been published in [1, 2].
- Section 2.3 explores the concept of the Semantic Web, its original vision, and its relationship with AI-driven Software Agents. Parts of this section have been published in [24].
- Section 2.4 provides an overview of Software Agents and MAS. Parts of this section have been published in [1, 2].
- Section 2.5 introduces emerging technologies related to LLM-based agents. Parts of this section have been published in [23, 24, 25, 26, 27, 28].

2.1 Short history of the AI: AI Summers and Winters

The notion of AI as a distinct discipline within Computer Science was introduced in 1956 at a workshop at Dartmouth College by early pioneers such as John McCarthy, Marvin Minsky, and others [50]. Since then, the history of AI has experienced recurring cycles of enthusiasm and subsequent disillusionment, commonly referred to as “AI summers” and “AI winters” [51]. An AI winter refers to a period of reduced funding, interest, and progress in AI research, typically following unmet expectations or technological limitations. In contrast, an AI summer denotes a phase of heightened enthusiasm, investment, and innovation, often driven by breakthroughs in algorithms, computing power, or applications. These cycles are associated with two main approaches to creating AI: symbolic (“white box”) approaches, based on explicit knowledge representation, logic, reasoning, and search, and sub-symbolic (“black box”) approaches, based on neural networks, ML, and statistical models.

Following the first AI summer in the 1960s, the first AI winter occurred in the 1970s after it was argued in [52] that sub-symbolic neural networks (then based on single-layer perceptron) would never be useful for solving real-world tasks. As a response, the symbolic approach, including knowledge-based expert systems and logic programming, fuelled the next AI summer in the 1980s. However, challenges in, e.g., knowledge representation and acquisition [53] soon led to another AI winter. Research on multi-layer sub-symbolic neural models and learning systems continued despite—or perhaps because of—the shortcomings of the knowledge-based approach. By expanding ML models through deep learning, employing new training algorithms such as backpropagation, introducing novel neural architectures and non-linear activation functions, leveraging Big Data from the Web and available databases for training, and using new efficient AI chips for parallel neural computations, AI has entered an unprecedented period of advancement in the 2020s [54]. A prominent research topic today, aimed at addressing challenges in neural systems such as hallucinations, is the combination of symbolic and sub-symbolic approaches in neuro-symbolic (hybrid) AI systems [55, 56].

The early 2000s are often labelled as an AI winter, with little recognition of the relevant AI research conducted during that time. However, this narrative overlooks significant developments from that era, including the emergence of the Semantic Web, which gained momentum in 2001 [41, 57]. Although rooted in knowledge representation, logic, reasoning, and ontologies—core subjects of traditional symbolic AI—this branch of research is typically not classified as part of AI. For example, in the authoritative ACM Computing Classification System³ of Computer Science, AI is a major first-level category under Computing Methodologies (see Figure 6).

```
Computing Methodologies
>Artificial Intelligence,
```

while Semantic Web is only mentioned under the minor category under Information Systems:

³ACM Computing Classification System: <https://dl.acm.org/ccs>

```
Information Systems
  >World Wide Web
    >Web data description languages
    >Semantic web description languages
```

Figure 6: Categorization issue: Semantic Web research vision had strong AI focus but it is categorized under WWW branch.

According to the original Semantic Web vision outlined in 2001 [41], the Semantic Web is centered around intelligent agents on the Web. The category of Intelligent Agents can be found as a minor subcategory under AI. This categorization issue is also illustrated in Figure 6.

```
Computing Methodologies
  >Artificial Intelligence,
    > Distributed computing
      > Multi-agent systems (MAS)
      > Intelligent agents
      > Mobile agents
    ...
```

Even review papers on the Semantic Web do not focus on the AI or Software Agent research conducted during that period within the Semantic Web domain. For example, Hitzler [58] provides a comprehensive review of the Semantic Web, detailing its evolution, core principles, and ongoing challenges. The paper highlights how the Semantic Web extends the traditional Web by incorporating structured and linked data to enhance machine interoperability. Key technologies, such as RDF, OWL (Web Ontology Language), and Protocol and RDF Query Language (SPARQL), are discussed, emphasizing their roles in enabling semantic interoperability. In addition, the paper explores real-world applications in domains such as AI, knowledge graphs, and data integration. Hitzler also addresses challenges, such as scalability, reasoning complexity, and data quality, underscoring the need for continued research and development to fully realize the Semantic Web’s potential.

The rapid advancement of GenAI and the widespread adoption of LLMs were catalyzed by the introduction of the Transformer architecture, which enabled more efficient training and superior contextual understanding in deep learning models [59, 54]. This breakthrough laid the foundation for applications, such as ChatGPT, which demonstrated the potential of LLMs in generating human-like text, leading to a surge in research and commercial interest. More recently, the focus has expanded beyond standalone LLMs to the integration of Software Agents and agentic capabilities, where models exhibit autonomy, goal-directed behaviour, and enhanced reasoning abilities [60, 61]. These agentic systems, which incorporate planning, memory, and interaction mechanisms, are now being extensively studied to augment the functionality of LLMs, enabling them to operate dynamically in complex environments, solve multi-step problems, and interact more effectively with users and digital ecosystems.

The general hype cycle model will be discussed in Section 2.1.1, a framework used to describe the waves of enthusiasm followed by subsequent disappointment that technological fields often experience. This pattern, commonly referred to as the “hype cycle,” has been observed in various industries and technologies.

Section 2.1.2 reviews and synthesizes illustrations of AI history, exploring the alternating periods of “AI winters” and “AI summers” and placing them on a normalized scale. These phases highlight the cycles of overenthusiasm, disillusionment, and eventual productivity, from the early promises of symbolic AI to the breakthroughs of the deep learning revolution.

2.1.1 The Hype Cycle

The hype cycle model, introduced by Gartner Inc. in 1995, [62] illustrates the evolution of technological expectations over time. As shown in Figure 7 [62], the model is formed by merging two distinct curves: a human-centric expectation curve, represented by the dashed blue line, and a classical technology maturity S-curve, represented by the dotted orange line. The first equation describes the hype level, which follows a bell-shaped curve driven by excitement, social contagion, and heuristic decision making. This curve captures the tendency of people to overestimate the potential of a technology, leading to an initial peak of inflated expectations, followed by a sharp decline when real-world limitations become evident.

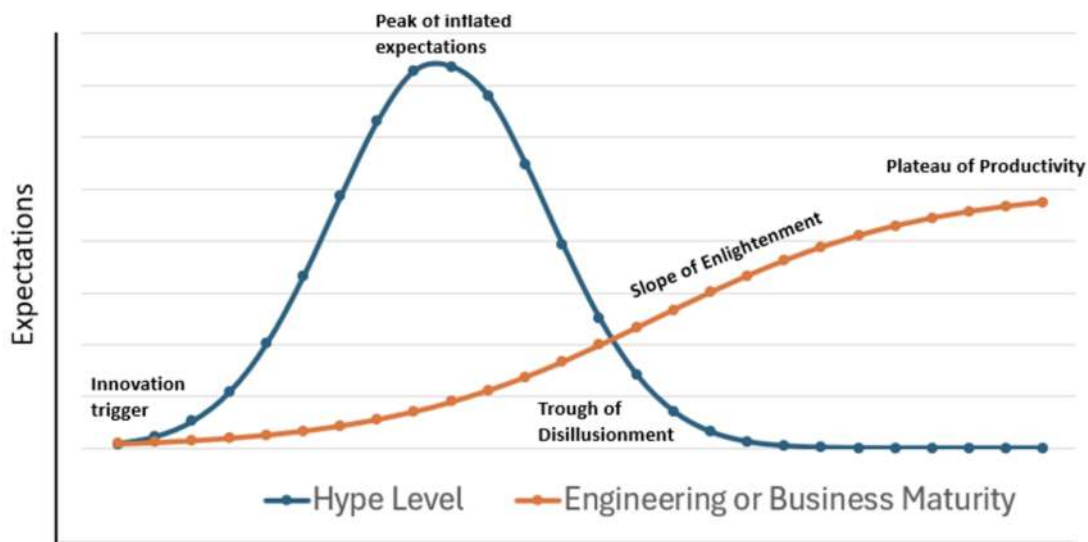


Figure 7: Hype Cycle: Hype Level, Engineering or Business Maturity, and Combined Hype Cycle [62]

The second equation, the S-curve of technological maturity, represents the gradual advancement of a technology over time. Initially, progress is slow due to limited understanding and minimal performance gains from early investments. However, as knowledge accumulates and improvements accelerate, the technology experiences rapid development until it eventually stabilizes at a plateau defined by its inherent limitations. Together, these two curves form the hype cycle, which models the typical trajectory of emerging technologies, highlighting both the initial overhype and the eventual stabilization at a realistic level of maturity. The stages outlined in the hype cycle depicted in Figure 7 are [62]:

1. **Innovation Trigger:** A new technology or concept emerges, generating early excitement and media interest, although practical applications are not yet fully understood.
2. **Peak of Inflated Expectations:** Media hype and early success stories fuel high expectations, leading to overenthusiasm and unrealistic predictions about the technology’s potential, with many projects launched but most failing to deliver.
3. **Trough of Disillusionment:** Initial excitement fades as challenges and limitations become evident, causing skepticism due to failed projects and unmet expectations, leading to a decline in interest and reduced investments.

4. **Slope of Enlightenment:** A gradual understanding of the technology’s real potential and limitations develops, leading to practical applications and increasing adoption as early adopters recognize value in realistic use cases.
5. **Plateau of Productivity:** The technology becomes mainstream and widely adopted, with stable, practical use cases delivering real benefits, achieving sustainable growth and a proven track record.

Although the hype-cycle is typically illustrated as a single expectation curve, it actually overlays the S-curve of technology diffusion; how these two curves interact determines both the tempo and the amplitude of successive boom-and-bust phases, yet this relationship remains only sketchily articulated in the literature. Consequently, existing research offers limited guidance on when firms and investors should initiate, scale, or exit their involvement in order to capture upside while containing downside risk. Closing this gap will require analytical frameworks that fuse expectation dynamics with adoption kinetics and strategic positioning along the hype-cycle.

2.1.2 Review and Synthesis of AI Winters and AI Summers

This section synthesises evidence from peer-reviewed papers, technical blogs, and public presentations to trace the recurring boom-and-bust pattern (AI Summers and AI Winters) that has characterised AI research and commercialisation. All available AI history illustrations were analyzed, as listed below: [63, 50, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77].

To quantify “summer” and “winter”, a grading scale from +10 to -10 was used, normalizing each illustration to this same scale. This means that all figures receive a maximum value of +10 when they visually reach their peak and -10 when they visually reach their lowest point. This grading serves as an indication of “temperature,” verbally expressed as “AI summer” (hot) or “AI winter” (cold). The aggregated illustration is presented in Figure 8. This figure also indicates that the period of active Semantic Web research is generally considered an AI winter. Furthermore, none of the studies mention either the topic *Semantic Web* or *Software Agents* as representing any form of AI boom in historical AI illustrations.

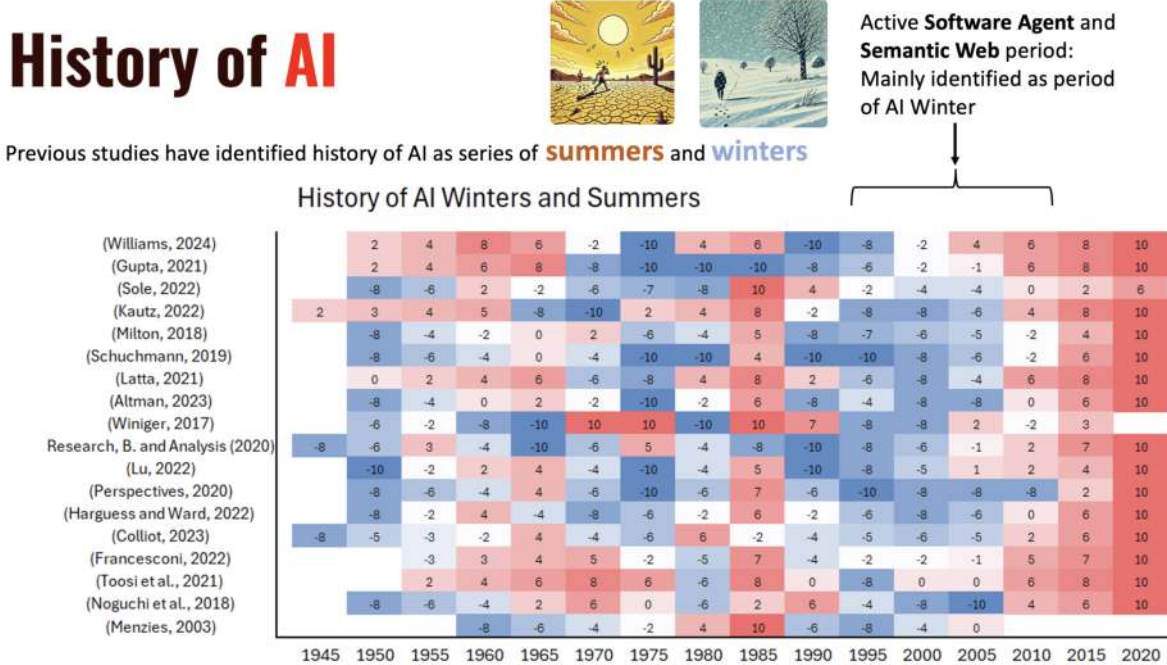


Figure 8: History of AI as illustrated by previous studies. Period of active Semantic Web development is identified typically as AI Winter period.

It is important to note that AI is not a single invention, but a shifting portfolio of techniques that is constantly redefined over time — from symbolic reasoning and expert systems to ML, deep learning, and today’s

large-scale generative models. Each element in this portfolio follows its own trajectory, so the broad history of AI is best understood as the superposition of many overlapping hype cycles rather than one monolithic wave.

These technology-specific curves explain why the field repeatedly climbs to a *Peak of Inflated Expectations*, descends into a *Trough of Disillusionment*, and, where genuine utility is present, climbs the *Slope of Enlightenment* to a durable *Plateau of Productivity*. Recognising AI as a constellation of evolving technologies helps clarify why optimism and disappointment can coexist—and why practical value often emerges only after multiple passes through the cycle.

Take *neural networks* and *software agents* as illustrative examples. For a long time, neural networks were not considered mainstream AI [78]; however, especially during the past 15 years, that has changed. Similarly, introduced in the early 1990s, agent technology ascended to widespread media enthusiasm, encountered technical and market barriers, resurfaced with the rise of web services, and is now experiencing renewed interest through autonomous AI assistants. In other words, the same underlying concept has already traversed the Gartner hype cycle several times.

2.1.3 Summary of AI Illustrations

AI history illustrations known to the authors of this study depict the period from 1995 to 2010 as an AI winter rather than a time of substantial research advancements. These accounts consistently emphasize stagnation and minimal innovation, portraying this period as one characterized by diminished funding, waning public interest, and a general lack of transformative breakthroughs in AI. While some research activities continued during this time, the prevailing perception is that AI did not experience any major surge or revolution.

Furthermore, none of the AI history illustrations reviewed in this study explicitly mention Software Agents or the Semantic Web. This absence suggests that these topics were not widely recognized or highlighted as pivotal elements within the broader narrative of AI development during this period. Their exclusion reinforces the notion that this era was marked more by stagnation than by the emergence of significant new paradigms or technologies.

2.2 History of the Web

This section presents a short history of the Web. Read through the thesis Phase I lens (Section 1.1.2); this section establishes the historical and architectural baseline for the Web that motivates the Phase I problem framing in Section 1.2. The evolution of how a hyperlinked document system became the general application platform—and later a data and service substrate—that acted as the Internet’s “killer application” is traced in this section. This baseline anchors RQ I.1 (Table 4): What are the historical origins of the Web and of software agents? Consistent with this scope, the focus is placed on Web-level evolution and governance rather than detailing lower-layer network and protocol mechanisms. For the overall sequence of phases and thesis structure, see Section 1.1.2.

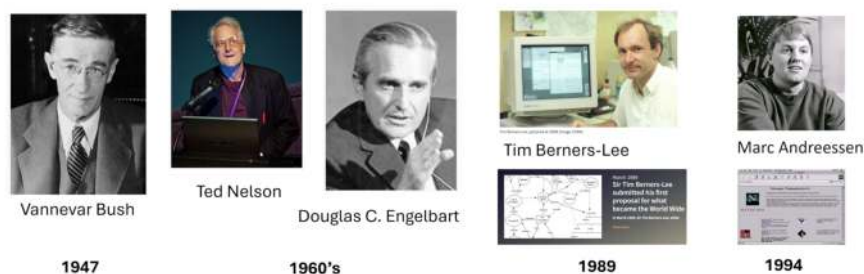


Figure 9: From Hypertext to the Killer App: The Web’s Evolution.

2.2.1 The Hypermedia Concept (1945)

Vannevar Bush is widely acknowledged to be the inventor of hypermedia. As early as in 1945 he wrote a famous article called *As We May Think* in the *Atlantic Monthly* [79], where he broadly discussed the

problems of information management in the research world of those days. He described the latest technical achievements and ended up proposing a photo-electrical-mechanical device called *memex* ("memory extension") for personal information storage and management.

The idea behind memex was not to structure information as it was typically organized then, *i.e.*, in hierarchical, tree-like data structures, but to draw inspiration from the associative nature of the human mind:

"The human mind does not work that way [organize data as trees]. It operates by association. With one item in its grasp, it snaps instantly to the next that is suggested by the association of thoughts, in accordance with some intricate web of trails carried by the cells of the brain. (...) Man cannot hope fully to duplicate this mental process artificially, but he certainly ought to be able to learn from it. (...) The first idea, however, to be drawn from the analogy concerns selection. Selection by association, rather than indexing, may yet be mechanized. One cannot hope thus to equal the speed and flexibility with which the mind follows an associative trail, but it should be possible to beat the mind decisively in regard to the permanence and clarity of the items resurrected from storage. (...) It affords an immediate step, however, to associative indexing, the basic idea of which is a provision whereby any item may be caused at will to select immediately and automatically another. This is the essential feature of the memex. The process of tying two items together is the important thing." [79]

Thus, Bush proposed that users could not only import information into a memex, but also create links between documents, mirroring the associative nature of human thought. He envisioned these documents as microfilms, which, in 1945, represented a cutting-edge technology for high-density information storage. While Bush could not anticipate the digital revolution, his paper fundamentally introduced the concept of hypermedia.

The memex was a groundbreaking concept, representing an unprecedented application of electronics [80]. Its vision profoundly influenced subsequent generations of scientists and engineers, including Douglas Engelbart, whose work laid the groundwork for the Web and the Internet.

2.2.2 Prototypes and Concept Development (1960s)

Ted Nelson coined the term *hypertext* in his 1965 paper "A File Structure for the Complex, the Changing, and the Indeterminate" [81], and further elaborated on the concept in his 1967 article "Getting it out of our system" [82]. Nelson was a visionary, not a technician, and even today, the technology to implement a worldwide *Xanadu* system, as originally envisioned, does not exist [83]. *Xanadu* shared some characteristics with the Web, but it was far more complex. Despite over three decades spent trying to implement his grand vision, Nelson has not yet produced a fully working *Xanadu* system.

Douglas C. Engelbart and a group of researchers working with him at the Augmentation Research Center at SRI were the first to actually build a working hypertext system in 1968 [84]. It was called "The Mother of All Demos" ⁴, since it presented, for the first time, a computer mouse, hypertext, object addressing and dynamic file linking, as well as shared-screen collaboration involving two persons at different sites communicating over a network with an audio and video interface.

In the demonstration ⁵, Engelbart showed how the computer could be used to deal with everyday tasks. Most of the presentation centered around him using the computer to plan a set of tasks he had to do later. All of this information was contained in a simple hypertext system that incorporated many different methods of organization, each appropriate to the task at hand.

2.2.3 Hypermedia Research (1980s)

Despite the research efforts of scientists following the initial prototypes, the 1980s were still characterized by incompatible networks, disk formats, data formats, and character-encoding schemes [85]. This incompatibility made attempts to transfer information between systems daunting and impractical. The situation was frustrating because computer usage was increasing, and a large amount of important information was already

⁴The name was given by Andy van Dam,
URL: <http://www.cs.brown.edu/stc/resea/telecollaboration/engelbart.html>

⁵The original video of the demonstration is available at
URL: <https://www.dougenelbart.org/mousesite/1968Demo.html>

stored on computers, many of which were networked. However, the systems in use remained fundamentally incompatible.

2.2.4 The Invention of the World Wide Web (1989)

The development of the World Wide Web (WWW) began with a famous project proposal written in 1989 by Tim Berners-Lee [36]. In the proposal, Berners-Lee discussed how many information management systems organized information hierarchically, as trees, although a graph structure would be more suitable. To illustrate the limitations of tree-like data structures, Berners-Lee used newsgroup discussions as an example. Newsgroups are typically hierarchical discussion systems. Discussions within a newsgroup often diverge from the original topic, fragmenting into subtopics and entirely different subjects. At that point, part of the discussion should logically reside in a different part of the tree. Even splitting the tree into multiple trees, or dividing discussions into different newsgroups based on these topics, would not fully resolve the issue: a tree-like structure lacks the necessary flexibility for this purpose.

Berners-Lee argued that CERN, at the time, was losing information due to the inflexibility of its information systems. He proposed that systems should be organized to allow direct links between documents. This structure would be more dynamic and more natural for users who prefer to find information related to a specific topic rather than navigating hierarchical file systems. It is important to note that the problem was precisely the same one that Bush had addressed 44 years earlier. All the necessary concepts⁶ and data structures had been invented long before 1989. Berners-Lee simply applied hypertext to this new environment in a straightforward manner. As Berners-Lee himself stated:

"I happened to come along with time, and the right interest and inclination, after hypertext and the Internet had come of age. The task left to me was to marry them together." [38]

Nevertheless, this invention has had a profound impact on society, and Sir Berners-Lee has received numerous awards, such as the Millennium Technology Prize⁷.

2.2.4.1 The Architecture of the Web

The Web's design was based on few criteria [85]:

- An information system must be able to record random associations between arbitrary objects, unlike most database systems then.
- Making a link from one system to another should be an incremental effort, not requiring unscalable operations such as the merging of link databases.
- Attempts to limit users to particular languages or operating systems are doomed to failure.
- Information must be available on all platforms.
- Attempts to force users to deal with information in the same way computers deal with information are doomed to failure.
- To insure accuracy, information must be easy to enter and correct.

Figure 10 depicts the Web's architecture, which was designed to meet the aforementioned criteria and adhere to established principles of software design adapted to the network environment [85]. Flexibility was clearly a key goal. There should be as few specifications as possible, because every specification needed to ensure interoperability simultaneously constrained the Web's implementation. An important principle of Berners-Lee's proposal was that the client software program's user interface should be consistent across all types of computer platforms, allowing users to access information from a wide variety of computers.

Three new technologies were created in the project that emerged from the proposal. Briefly, these were the HyperText Markup Language (HTML), used to create Web documents; the HyperText Transfer Protocol

⁶Such as the Internet (TCP/IP, DNS), hypermedia, various lower-level protocols, and operating systems

⁷The Finnish Technology Award Foundation named Berners-Lee its first recipient of the Millennium Technology Prize in 2004, selecting him from a field of 78 technologists whose work "promotes people's quality of life, is based on humane values, and encourages sustainable economic development." The prize was one million euros.

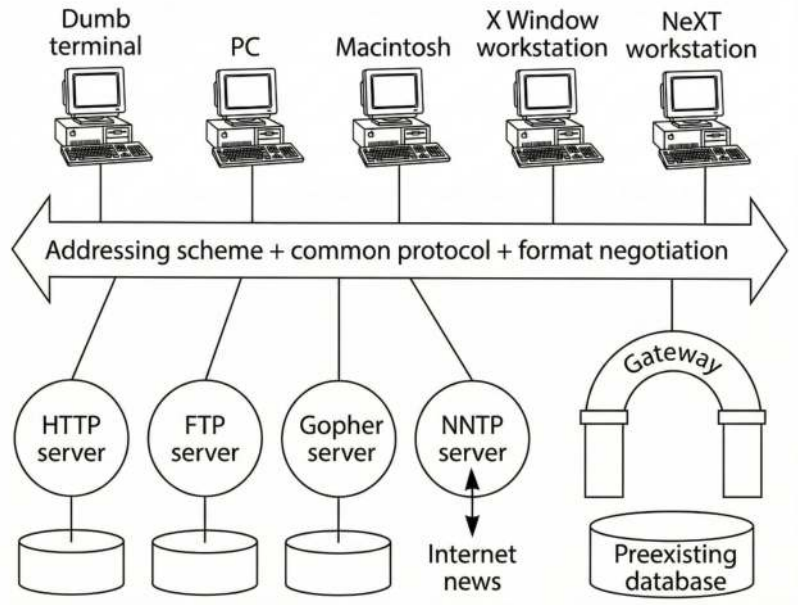


Figure 10: Web architecture by Tim Berners-Lee in 1990 [85].

(HTTP), used to transmit the pages; and the Universal Resource Identifier (URI) ⁸, used for addressing. These technologies will now be briefly examined before continuing with the historical overview.

2.2.4.2 Universal Resource Identifier (URI)

In the 1980s, hypertext systems were typically built around a database of links [85]. Because these systems were centralized, they became unmanageable when scaled up. However, they guaranteed that links would be consistent and removed links to documents that had been deleted. The removal of this guarantee was the principal compromise made in the Web architecture. Allowing people to create a reference without consulting the reference's destination enabled the scalability that the Web subsequently exploited.

The power of a link on the Web lies in its ability to point to any document or other resource within the universe of information. This capability requires a global space of identifiers. These universal resource identifiers (URIs) are fundamental elements of the Web's architecture.

The URI space [88, 87] is universal in the sense that any new space with an identifying, naming, or addressing syntax can be mapped into a printable syntax, given a prefix, and then incorporated into the URI space [85]. This well-known structure begins with a prefix, such as *http:*, which indicates the specific space to which the rest of the string refers.

2.2.4.3 HTTP

The HTTP [89] was developed because File Transfer Protocol (FTP) was insufficiently feature-rich and too slow to be optimal for the Web [85]. HTTP URIs are resolved into the addressed document by splitting them into two parts. The first part is used to query the Domain Name Service (DNS) to discover a suitable server, and the second part is passed to the server to identify the specific resource.

2.2.4.4 HTML

The HTML [90] was designed for the interchange of hypertext, serving as a data format to be transmitted over the network. HTML resembled some SGML-based systems and was thus quickly adopted by both the

⁸Originally named Universal Document Identifier (UDI) by Tim Berners-Lee, the term evolved within the IETF to Uniform Resource Locator (URL) and later to Uniform Resource Identifier [38, 86, 87]. In practice, the terms "uniform" and "universal" are often used interchangeably, and URI and URL are treated as synonyms.

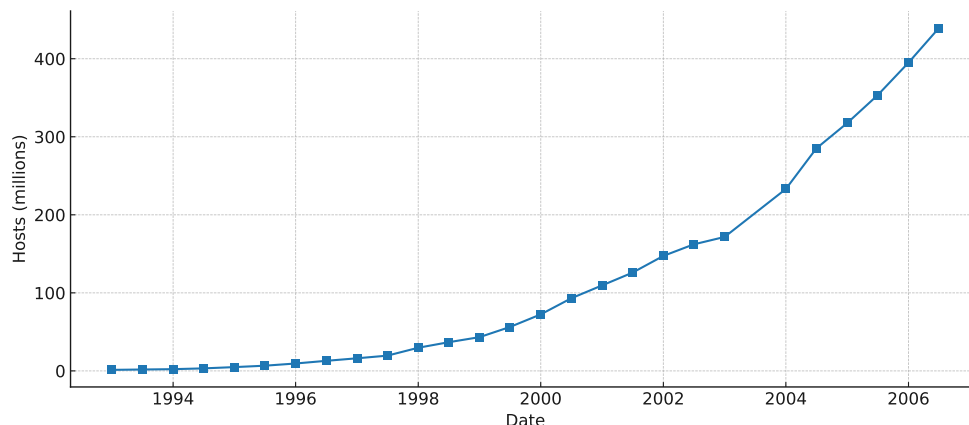


Figure 11: Internet Domain Survey Host Count [93].

documentation community, which favored the SGML⁹ syntax, and the hypertext community, in which only SGML-based standards had been proposed [38]. The adoption of SGML encouraged these communities to embrace the Web more readily, but SGML proved to have a very complex and poorly defined syntax. The attempt to achieve both full SGML compatibility and HTML's ease of use challenged experts for a considerable time [85].

2.2.5 Research Boost (1990–1994)

In September 1990, Berners-Lee received permission to prototype a global hypertext system [38]. By December, the first Web server, a combined “what you see is what you get” (WYSIWYG) browser/editor (WorldWideWeb), and the first HTML pages were running and demonstrated [38, 85]. Over the next two years, several browsers appeared in European universities, and by late 1992 there were at least 26 reasonably reliable Web servers worldwide [91]. Meanwhile, researchers were still exploring practical uses for hypermedia [92].

In February 1993, NCSA released the first version of *Mosaic for X*. In 1994, Marc Andreessen and colleagues left NCSA to found *Mosaic Communications Corp.*, later Netscape. Traffic grew quickly: the first server logged about 100 requests per week in July 1991 and roughly 100 000 per week by April 1994 [91]. In October 1994, the World Wide Web Consortium (W3C) was founded [38, 91].

2.2.6 Public Growth

It is not easy to obtain exact figures on the growth of the Web [94], nor to pinpoint the precise moment and key factors when the “snowball effect” truly began. An illustration of the growth rate can be seen in Figure 11, which presents one of the many host count statistics, all of which differ considerably. The variation in these statistics is natural, given that the Web's nature is inherently unmanageable: it is constantly changing, and capturing precise snapshots is exceedingly difficult.

2.2.7 The Web: From Document Collection to Application Platform

The Web is not just about information; it is also about services – about performing tasks. Many of these services can be accessed in various ways. This means that not only can users access services through regular web browsers or other interfaces, but other computer programs can also access the same services. There are banking services, travel and leisure services, investment and financing services, news services, shopping services, weather forecasts, games, museums, education services, science and engineering resources, libraries, and the list continues. Services are expanding into new areas, and simultaneously, the distinction between what resides on the Web and what lies beyond it on the Internet has begun to blur. This blurring has created

⁹Standard Generalized Markup Language (SGML), ISO 8879, URL: <http://www.iso.ch/cate/d16387.html>

new dimensions for Web-based services and sites, such as marketing, user experience, and online communities. For further, primarily historical, perspectives on different services, see [95].

Early visions of the Web as a hypertext library have evolved into today’s reality of the browser as a general-purpose runtime: the advent of asynchronous JavaScript and XML (AJAX) first enabled desktop-like responsiveness in the mid-2000s [96], the “Web 2.0” paradigm framed the browser as a software platform rather than a document viewer [97], and subsequent standardisation—HTML5 APIs, Service Workers, and WebAssembly—has delivered offline execution, background sync, and near-native performance [98, 99]. These capabilities underpin the Progressive-Web-Application (PWA) model adopted by products such as Google Docs and Figma, illustrating how the Web now functions as a cross-device application substrate that abstracts away operating-system heterogeneity while retaining the universal reach of URLs.

2.3 The Semantic Web and Its Vision of AI Software Agents

This section is read through the Phase I lens—how the Web’s future was envisioned with Semantic-Web-native Software Agents—and it provides the conceptual baseline used Section 1.2. The focal inquiry is RQ I.2 (Table 4): to what extent Semantic Web formalisms and agent protocols can be combined to deliver runtime interoperability and adaptation on the open Web, and where irreducible gaps prevent practical convergence. This section examines the integration surface between Web Knowledge Representation (KR) standards (RDF/OWL, rules, service ontologies) and FIPA-ACL, anticipating which promises proved workable and which remained aspirational. See also the Phases and thesis structure overview in Section 1.1.2.

2.3.1 Software Agents Before the Semantic Web

Long before the Web era, the conceptual foundations for software agents was established by research in *Distributed AI* (DAI) and the *Actor model* [100]. DAI cast intelligence as an *emergent* phenomenon arising from multiple, loosely-coupled problem solvers that cooperate across a network, thereby elevating autonomy and negotiation to primary design concerns [101, 102]. In parallel, the Actor model supplied a concrete computational metaphor of independent, concurrently executing entities that communicate solely through asynchronous message passing. Together these two traditions contributed the twin conceptual pillars of **distribution** and **concurrency**, which later researchers distilled into the now-standard agent properties of social ability and proactive-reactive behaviour [103].

The 1990s witnessed the emergence of software agents as a concept in computer science, driven by the need for autonomous and intelligent systems in increasingly complex environments. This decade saw the consolidation of core agent characteristics, including autonomy, proactivity, reactivity, and social ability [104, 105]. The rapid expansion of the World Wide Web around 1994 underscored the potential for agents to operate within this new digital space [106]. Foundational research explored diverse aspects of agent technology, from theoretical underpinnings and formal methods to practical applications for information management and personalized assistance [105, 106].

The development of various agent architectures, such as reactive, deliberative, and hybrid models, further illustrated the field’s commitment to creating sophisticated autonomous entities [107]. Researchers also focused on defining the boundaries of agency, distinguishing agents from conventional programs through taxonomies and formal definitions [108]. The intellectual roots of software agents were traced back to DAI and Carl Hewitt’s Actor model from 1977, highlighting the long-standing interest in distributed and interacting intelligent systems [104, 100, 103].

From Definitions to Standards. The explicit taxonomies and formal definitions produced during this period played a pivotal role in *standardising* the concept of agency. By establishing a common vocabulary, they enabled the FIPA to publish its initial specifications—most notably the FIPA-ACL and a reference architecture that has since underpinned almost every industrial-grade multi-agent platform [109]. These efforts, together with subsequent W3C initiatives (e.g., the *Agent Markup Language* submission), ensured that later Semantic-Web agents could interoperate across organisational and domain boundaries on the basis of well-defined semantics rather than ad-hoc conventions.

2.3.1.1 Impact of the World Wide Web’s Expansion on Software Agents

The exponential expansion of the World Wide Web in the mid-1990s profoundly altered the landscape in which software agents were conceived and applied. Originally developed in the context of distributed AI

and local computation environments, software agents found new significance as the Web became a vast, decentralized, and heterogeneous information space. The growing complexity of navigating, searching, and processing this information catalyzed the demand for autonomous systems capable of filtering, retrieving, and reasoning over web-based content on behalf of users.

Seminal work by Pattie Maes and others [106] recognized this shift early, introducing agents as personalized assistants that could reduce information overload in the emerging Web era. This period also saw the rise of "information agents"—autonomous programs designed specifically to operate over Web protocols, harvest content, and interact with web services. The scalability and openness of the Web provided a compelling environment for agents to demonstrate their core properties: autonomy, proactivity, and social interaction, especially as HTML and HTTP standards facilitated agent-based crawling and retrieval.

Moreover, the ubiquity of browsers and online services created a new user expectation: software should act intelligently in navigating and managing web interactions. This sociotechnical transformation fueled both academic research and industrial exploration into agent-based architectures, eventually motivating the integration of agents into the foundational vision of the Semantic Web [41]. Thus, the Web's growth did not merely provide a backdrop—it actively shaped the functionality, relevance, and perceived utility of software agents during this foundational decade.

2.3.2 The Vision of the Semantic Web and Intelligent Agents

Tim Berners-Lee, James Hendler, and Ora Lassila, in their seminal 2001 paper "The Semantic Web" [41], envisioned an evolution of the World Wide Web into a more intelligent and interconnected system. This Semantic Web would be an infrastructure where information has explicitly defined meaning, facilitating human-computer cooperation and enabling machines to understand and process web data through a universal framework of metadata and ontologies. This involved enriching web resources with semantic markup using languages like RDF and OWL, making content machine-processable [110]. The goal was a transition from unstructured documents to interconnected data, enhancing knowledge sharing, reuse, and integration [111], aligning directly with AI's goals of machine understanding and reasoning [41]. As of the time of writing, this highly influential paper has garnered over 30 000 citations.

Intelligent agents were conceived as crucial actors within this vision, able to automatically access, interpret, and reason about the semantically marked-up information [112]. These agents would act on behalf of users, performing tasks such as information retrieval, service discovery, automated decision-making (e.g., scheduling meetings or making purchases), and seamless interaction across different platforms [113]. Personal assistants, guided by rules, ontologies, and user profiles, were highlighted as prime examples of potential applications [114].

Realising this vision depends on the *ability of autonomous agents to converse*. The FIPA ACL specification meets this requirement by supplying a performative-based message envelope whose semantics are defined in a modal logic of communicative acts [115]. When the `content` field of such messages is expressed in RDF or OWL, the resulting utterances are simultaneously interpretable at the *data* and the *speech-act* levels, thereby enabling richly contextualised dialogues among heterogeneous Web agents [116]. Accordingly, ACLs form the critical bridge between the Semantic Web's data layer and the multi-agent community's interaction layer.

The paper's emphasis on AI Software Agents, and the backgrounds of Hendler and Lassila in intelligent agents and knowledge representation, underscore the Semantic Web's connection to advancing autonomous systems capable of reasoning, decision-making, and communication [41]. By leveraging standardized semantics, agents could interpret and act upon information with greater sophistication. This vision linked the future of the Semantic Web to developments in AI and positioned Software Agents as a critical component in realizing the Web's full potential. While the full vision remains unrealized, it remains a cornerstone in the intersection of Semantic Web technologies and AI Software Agent development.

2.3.3 Key Enabling Technologies and the Semantic Web Stack

Several technologies were crucial for realizing the vision of the Semantic Web and its integration with intelligent agents. The RDF provided a standard model for data interchange on the web, enabling the expression of relationships between resources [117]. The OWL, built upon RDF, was designed to represent rich and complex knowledge about entities, their classifications, and relationships [110, 118]. Ontologies, formal and explicit specifications of shared conceptualizations, provided a common vocabulary and understanding of specific domains, which was crucial for enabling agents to interpret information consistently [119].

Agent Communication Languages (ACLs), such as the FIPA ACL [120], were recognized as essential for facilitating interaction and information exchange between autonomous agents. Furthermore, the *DAML-S*¹⁰ ontology—renamed *OWL-S* after the release of OWL—[121] was created to add a machine-interpretable semantic layer to conventional Web-service descriptions. It exposes three mutually supportive components: a *Service Profile* that advertises what the service does, a *Process Model* that specifies how the service can be executed, and a *Grounding* that binds abstract inputs and outputs to concrete message exchanges. Together these components allow an intelligent agent to match its goals to advertised profiles and thus discover relevant services, to use the process model and grounding information to construct and send the correct messages for direct invocation, and to chain multiple process models into coherent workflows for automatic composition of complex tasks. The development of DAML-S/OWL-S therefore represents a focused effort to supply the semantic infrastructure—rooted in ontologies, RDF, and OWL—along with the inter-agent communication mechanisms required for truly autonomous agents on the Semantic Web.

Figure 12 illustrates the "Semantic Web Stack," a layered architecture proposed by Tim Berners-Lee for organizing the technologies of machine-readable web data [122]. URIs and Unicode provide the foundation for identification and character encoding. XML offers a syntax for structured documents, while RDF and RDF Schema (RDFS) define a standardized way to describe resources and their relationships [123]. OWL builds on this to enable the creation of complex ontologies. Rule languages like RIF (Rule Interchange Format) and SWRL (Semantic Web Rule Language), along with the Simple Protocol and SPARQL, were intended to support inference and querying over this semantically rich data [124].

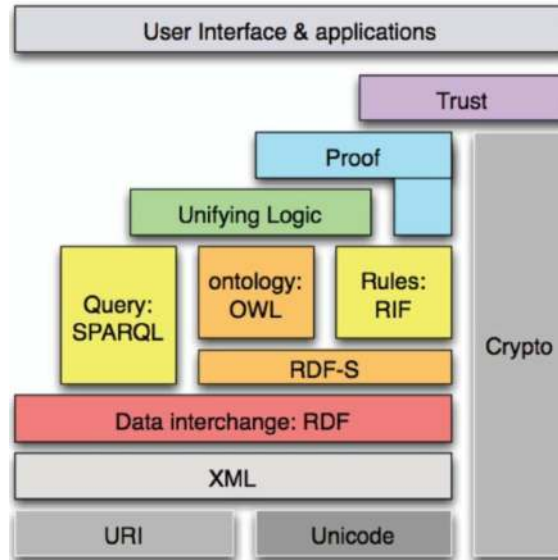


Figure 12: Semantic Web Stack by Tim Berners-Lee 2006 [122].

While OWL enables inference through description-logic constructs such as subclassing, property hierarchies, and transitive properties, its expressivity is intentionally constrained to ensure decidability and tractable reasoning. To address use cases that require more expressive logic—such as conditional rules and inter-property dependencies—rule-based extensions were introduced. The Semantic Web Rule Language (SWRL) combines OWL ontologies with (Datalog-style) Horn rules, allowing rules of the form *if antecedent then consequent* and thereby significantly enhancing inferencing capabilities [124, 125]. Because unrestricted SWRL leads to undecidability, decidability can be preserved by adopting DL-safe rules (where DL = Description Logic) that restrict rule variables to named individuals [126]. For example, the following rule captures the intuition “if a person has a parent who is female, then that parent is the person’s mother”:

$$\text{hasParent}(?x, ?y) \wedge \text{Female}(?y) \rightarrow \text{hasMother}(?x, ?y)$$

Higher layers of the stack were envisioned to handle logical reasoning, proof verification, and trust establishment through cryptographic methods, ultimately enabling applications with enhanced interoperability. Over the

¹⁰DAML-S: Defense Advanced Research Projects Agency (DARPA) Agent Markup Language for Services

past two decades, the Semantic Web community has converged on a set of concrete cryptographic primitives that operationalise the previously aspirational *proof* and *trust* layers. A prerequisite for any digital signature is a deterministic byte-level representation of the graph being protected; this is now provided by the *RDF Dataset Canonicalisation and Hash 1.0* (RDFC-1.0) algorithm, which yields a stable hash despite blank-node renaming and graph isomorphisms [127]. Once canonicalised, graphs can be enveloped by the *Data Integrity 1.0* suite (formerly “Linked-Data Signatures”), embedding cryptographic proofs directly in RDF or JSON-LD and supporting selective-disclosure cryptosuites, such as BBS+ [128].

Building on these primitives, the *Verifiable Credentials Data Model 2.0* binds claims to subjects in tamper-evident envelopes that agents can verify offline, while *Decentralized Identifiers (DID) Core* replaces hierarchical certificate authorities with URL-resolvable public-key documents [129, 130]. Together these technologies allow autonomous agents—whether classic rule engines or modern LLM pipelines—to (i) verify the provenance of data before reasoning, (ii) attach signed proofs to their own outputs, and (iii) assemble verifiable chains of custody across heterogeneous knowledge graphs.

2.3.3.1 Semantic Web Rollout Vision

Figure 13 depicts Tim Berners-Lee’s 2003 “rollout vision” for the Semantic Web, projecting the integration of layered technologies from basic markup (SGML, XML, RDF) to advanced constructs (OWL, DAML+OIL) and logic (Prolog, RuleML), culminating in proof and trust mechanisms for automated reasoning and secure interoperability [57].

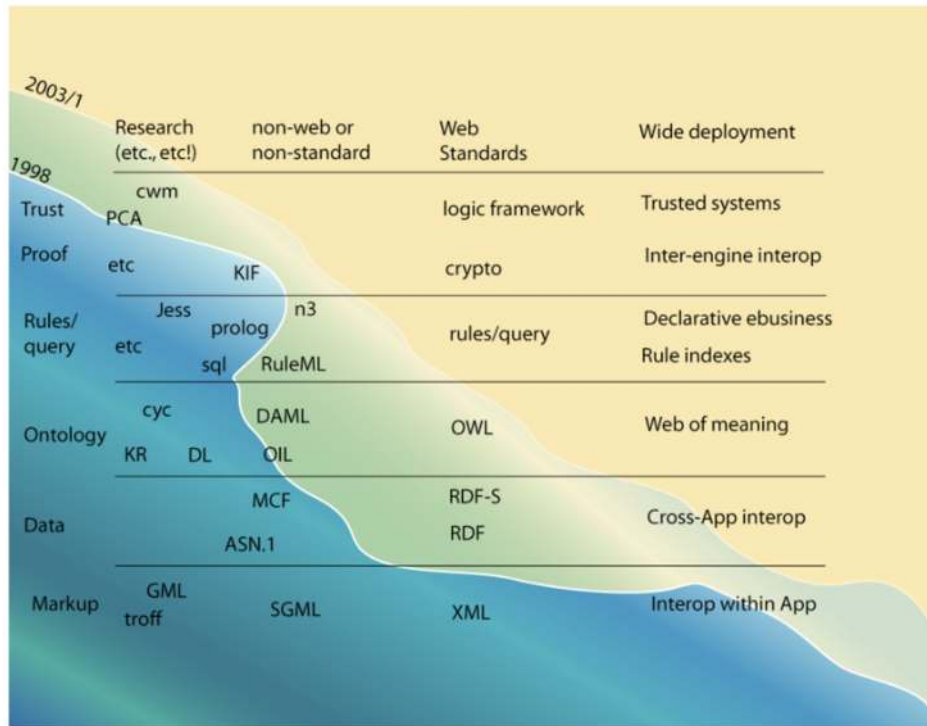


Figure 13: The Semantic Web Wave 2003 by Tim Berners-Lee [57].

However, this vision was not fully realized. While RDF and OWL saw adoption in niche areas, widespread uptake was hampered by competing technologies and shifting industry priorities. The higher layers of the Semantic Web stack, particularly those based on formal logics, faced limited adoption due to the complexity of technologies like OWL dialects and fundamental differences in assumptions compared to traditional database systems. OWL’s Open World Assumption (unknown facts are not presumed false) and lack of a Unique Names Assumption (identical names don’t necessarily mean identical entities) contrasted with the closed-world and unique-names assumptions prevalent in many database applications. Despite initiatives like the W3C’s Semantic Web Activity program and efforts on Web Services (e.g., OWL-S) to advance intelligent software agents as envisioned in the original Semantic Web paper [41], this direction did not achieve the expected success within the Semantic Web context.

2.3.4 Semantic Web Influence on Modern AI/ML Data Integration

Although the Semantic Web’s most ambitious goals remain aspirational, its foundational interoperability standards—such as RDF, OWL, SPARQL, and Shapes Constraint Language (SHACL) —have significantly influenced how structured data is integrated in modern AI and ML systems. These standards provided early blueprints for achieving semantic consistency across heterogeneous datasets, a challenge that remains central to AI pipelines today.

In large-scale AI systems, knowledge graphs—many of which adopt RDF or OWL-based schemas—serve as key instruments for data integration, enabling entity resolution, feature alignment, and graph-based feature engineering. Companies like Google, Amazon, and Microsoft have adopted internal ontologies to normalize concepts across business domains, facilitating model interoperability and multi-source training [111]. SPARQL’s graph-query paradigm also anticipates the emerging need for expressive retrieval interfaces in RAG systems.

Furthermore, recent approaches to embedding knowledge graphs into vector spaces for use in ML models draw on ontology axioms to constrain embedding learning, enabling logic-conformant representations [131]. Validation languages, such as SHACL, have also inspired schema-checking layers in AI pipelines, ensuring that training and inference data adhere to expected semantic contracts. In effect, these practices embed Semantic Web-style interoperability principles at the foundation of modern ML workflows.

Thus, although seldom cited by name, the Semantic Web’s influence persists through the design patterns, vocabularies, and validation logic that now underpin scalable and trustworthy AI data infrastructures.

2.3.5 Could Semantic Web Principles Complement LLMs?

LLM pipelines still lack explicit semantics, provenance, and verifiability. Drawing on the Semantic-Web stack, three concrete integration patterns that harness ontologies and knowledge graphs to mitigate hallucination and to provide user-facing rationales are outlined.

Why ontologies? Unlike the high-dimensional token spaces in which LLMs operate, an ontology offers a *canonical, machine-interpretable vocabulary* of entities, relations and axioms. Mapping generated spans to URI-identified concepts therefore gives each symbol an unambiguous referent, providing the first step toward faithful explanations [132].

Complementary trade-offs. OWL and RDF guarantee sound, monotonic inference, but require careful manual modelling, limit expressivity to stay decidable, adopt an Open World Assumption and incur worst-case exponential reasoning cost. LLMs sit at the opposite end of the spectrum: they learn broad coverage automatically, answer in closed-world style with constant-time decoding, and tolerate noisy data—yet offer no built-in notion of truth, identity or provenance. Combining the two thus pairs the symbolic rigour of OWL/RDF with the statistical reach of LLMs.

Pattern 1 – ontology-constrained decoding. During generation the model is prompted or fine-tuned so that every entity mention must be resolvable to an ontology concept; illegal continuations are pruned. This drastically reduces hallucinations and yields provenance for every token. A practical example is the plant-disease classifier of [133], where OWL reasoning is used online to check that the captioned visual attributes entail a unique disease class.

Pattern 2 – knowledge-graph RAG. RAG in which the retriever indexes a knowledge graph rather than free text grounds answers in triples whose semantics are explicit. Recent systems such as SUBGRAPHRAG show that even small models achieve state-of-the-art accuracy *and* produce subgraph-level rationales that users can inspect [134].

Pattern 3 – neuro-symbolic post-hoc explanation. After an LLM produces an answer, a reasoning engine aligns the answer with the ontology, derives minimal entailment paths, and verbalises them as "because-clauses". Confalonieri and Guizzardi [132] identify reference modelling, commonsense reasoning and knowledge-refinement as the three main roles ontologies play in such explanatory pipelines.

2.3.6 How AI Has Shaped Real-World Semantic Web Applications

Since the publication of the 2001 Semantic Web vision [41], advances in AI have continually reshaped how Semantic Web technologies are applied in practice. While the original vision emphasized agent-based

reasoning over RDF/OWL graphs, real-world adoption accelerated primarily in areas where AI provided scalable solutions for search, classification, and entity resolution.

Notably, the integration of AI with Semantic Web principles led to the widespread adoption of knowledge graphs, now core components of systems like Google’s Knowledge Panel [135] and Facebook’s Entity Graph [136]. These applications use statistical entity linking and ML-driven relation extraction, but rely on RDF-style schemas for consistent interpretation. Similarly, the emergence of schema.org as a lightweight ontology for Web content marked a convergence between SEO optimization and semantic markup, driven by AI-based search and ranking algorithms.

In recent years, ML pipelines have begun to adopt semantic validation layers—such as SHACL—for ensuring data integrity, especially in regulated domains like healthcare and finance. AI-driven recommendation systems now leverage ontologies to improve explainability and diversity of results, while RAG models increasingly use graph-based backends to ensure factual grounding.

Thus, while the original agent-centric vision remains largely aspirational, AI has incrementally transformed the Semantic Web into a practical substrate for large-scale, machine-interpretable data systems. These hybrid architectures demonstrate that AI advances have not only extended, but redefined the scope of Semantic Web applications in real-world systems.

2.4 Software Agents

Read through the thesis Phase I lens (Section 1.1.2), this section establishes the conceptual baseline for *software agents* as individual computational entities. Definitional criteria, core architectures, and the ambitions and limitations documented in the classic agent literature are synthesized. This baseline motivates the Phase I problem framing in Section 1.2 and anchors (i) the "software agents" half of RQ I.1 and (ii) the integration question RQ I.2 (Table 4) by clarifying which capabilities and abstractions must be made interoperable on the Web. Consistent with this scope, emphasis is placed on agent-level models and decision/interaction semantics rather than platform-specific middleware or lower-layer protocol mechanisms.

2.4.1 Agents and Distributed Artificial Intelligence (DAI)

The concept of agents has emerged as a construct in the field of AI, particularly within the subdomain of DAI. The ACM Computing Classification System identifies DAI as an area of AI research, encompassing various aspects such as MAS, Intelligent Agents, Mobile Agents, and Co-operation and Coordination. These categories collectively represent a range of approaches and methodologies dedicated to creating systems where multiple autonomous entities, or agents, interact to achieve individual or shared goals, as illustrated in Figure 14.

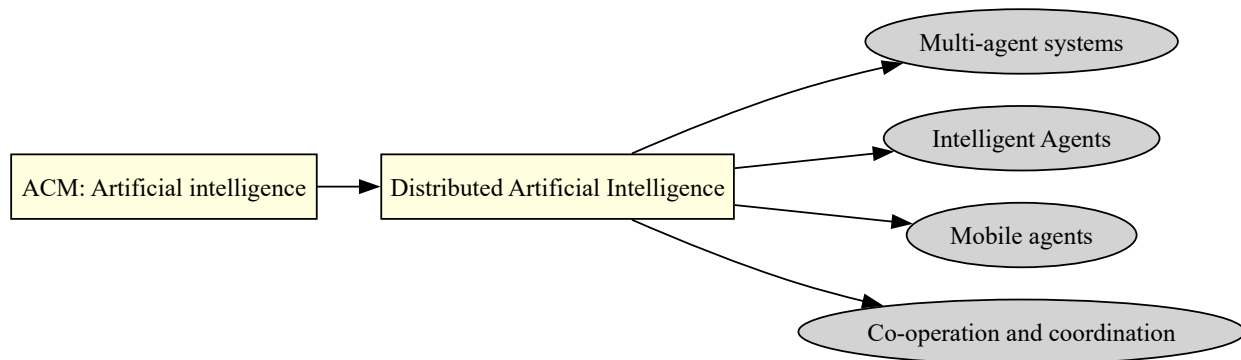


Figure 14: ACM Computing Classification System of Computer Science [137].

Agents operate through a combination of perception, reasoning, learning, and action—characteristics that distinguish them from passive programs. This autonomy, combined with proactiveness, reactivity, and social ability, makes agents suitable for various applications ranging from robotics and distributed control systems to complex decision-making and collaborative problem-solving.

In the context of DAI, the study of agents often involves understanding how multiple agents interact within a shared environment. This interaction may involve cooperation, coordination, negotiation, competition, or collaboration. The complexity of these interactions requires the development of frameworks and protocols to ensure that agents can effectively work towards achieving individual and collective goals.

Additionally, agents can be categorized into various types based on their capabilities and operational environments. Intelligent agents are designed with reasoning and decision-making abilities, whereas mobile agents possess the added capability of migrating across different networked environments. Furthermore, MAS represent a paradigm where numerous agents coexist and interact within a structured environment, typically governed by protocols that define their cooperation and coordination mechanisms.

The following sections explore the underlying principles, architectures, and methodologies that govern the design and implementation of agents within DAI. Additionally, special attention will be paid to theoretical foundations, practical applications, and contemporary challenges associated with deploying agent-based systems in dynamic and distributed environments.

2.4.2 Hype Cycles: The Ever Promising Agent Technology

Software agent technology has gone through a hype cycle (Section 2.1.1) a couple of times during its history. The following quote is from 1996:

"Agents are here to stay, not least because of their diversity, their wide range of applicability and the broad spectrum of companies investing in them. As we move further and further into the information age, any information-based organization which does not invest in agent technology may be committing commercial hara-kiri." [138]

Authors of the above quote wrote another article a few years later [139] stating that the assumptions were too high. If anything, companies that invested in agent technology in the 1990s were perhaps the ones to commit a form of hara-kiri.

Recently, in December 2024, Microsoft CEO Satya Nadella stated that AI-driven software agents are fully revolutionizing B2B applications. According to Nadella, AI-driven agents will take over business logic and enable new AI-powered applications, placing software agents at the heart of the current AI hype. The full quote from an episode of the BG2 Pod series, where Nadella stated in an interview video [140], e.g., is:

"I think that the notion that business applications (B2B Apps) exist, that is probably where they all collapse, in the (Software) Agent-era." – "Because if you think about it, they (B2B Apps) are essentially CRUD-databases with a bunch of business logic. The business logic will go to these Agents. These Agents are going to be multi-repo-CRUD so they are not going to discriminate between what the backend is. They are going to be updating multiple databases. All the (business) logic will be in the AI-tier (Agent-layer) so to speak." "Once the AI-tier becomes the place where all the logic is, then people will start replacing the backends. – As we speak, I think we are seeing quite high rates of wins in our Dynamics backends and the Agent use. We will go very aggressive trying to collapse it all. Whether it is customer service, be it finance and operations..." – "People want more AI-native business apps. It means the logic tier can be orchestrated by AI and AI Agents in a very seamless way." [140]

Obviously, noting that agent technology has had inflated expectations in the past does not prove anything about the future. This time, it might be different, as the overall technology infrastructure has evolved significantly since the 1990s. It also has to be remembered that, for example, neural networks have experienced several hype cycles before the major breakthrough in the 2010s.

2.4.3 Definitions

Although there have been many discussions about the definition of a software agent (e.g., [141, 142, 143, 144, 138, 145, 146, 147, 148, 149]), there is little consensus on the matter. If there is any consensus, it is the acknowledgment that no consensus exists [150]. Russell and Norvig [145] have provided a well-known definition of an agent as *anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators*. A simplified version of this abstract definition is illustrated in

Figure 15. It is evident that the definition is overly broad and could encompass various entities, such as a regular Unix process.

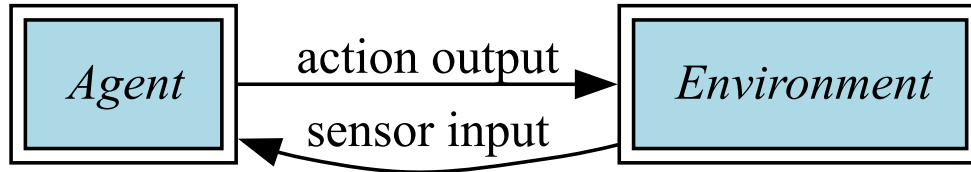


Figure 15: Agents and their environments [151].

Typically, properties associated with (intelligent) agents include pro-activity, reactivity, sociality, and autonomy [151]. Some researchers argue that if a system does not require these properties, it is better to design and implement it without agents [152].

One way to view software agent technology is as a higher level of abstraction compared to, for example, object-oriented technology. It is said to provide the system designer with a natural abstraction for modeling an entire system or its components [153]. This is partly why agent technology has been considered a promising approach for the analysis, specification, and implementation of complex software systems. More broadly, this approach is referred to as Agent-Oriented Software Engineering (AOSE) [144], although it has not achieved widespread success to date.

But why is it so hard to define an agent? The answer is at least partly because the *intelligent* and *collective* dimensions of agents has led them to be influenced by results stemming from extremely different disciplines [150]. Examples given by [154] are:

- **Biological analogies;** to reuse evolutionary process and natural selection for artificial life, genetic programming for adaptability, pheromones and environmental metaphor to build artificial ecologies.
- **Computational linguistics;** to build communication model, the most influent one being Speech Act theory [155].
- **Cognitive science;** a *Belief, Desire, and Intention* (BDI) internal architecture modelling Beliefs about itself, other agents and its environment, Desires about future states and goals and Intentions about its own future actions [156].
- **ML;** applied to software agents to allow automatic adaptation to users and context.
- **Database and knowledge-base technology;** when agents need data and knowledge to represent and reason about things they have to do or manipulate.
- **Distributed computing;** concurrency is inherent to MAS, parallelism has to be engineered and interaction protocols have to be specified.
- **Econometric models;** to predict and control the collective behaviour and equilibrium.
- etc.

It is obvious that with such a diversity of contribution the agent and MAS paradigm is being diluted in a multitude of perspectives. This, in turn, implies that no single definition of what an agent is, can cover all these aspects.

Some characteristics that have been related to agents are listed in Figure 16¹¹. By comparing Figure 16 to the various disciplines listed above it can be seen how the disciplines have affected the characteristics that are bound to agents.

So, the conclusion is that no single definition exists to cover all the aspects that have been related to agents. However, some definition is needed in the context of this Thesis for the reader to have a general understanding of agents.

¹¹The characteristics are adopted from [149, 138, 144, 147, 142, 141] and the Table from [150]

Property	Sub-property	Meaning
Reactive		Responds in a timely fashion to changes in its environment.
Complex Mental State		The agent has knowledge, belief, intention, and obligation.
Graceful Degradation		Correctly handles a degraded operational state; it dies properly.
Temporally continuous (in the multi-agent system)		Once created, an agent is always running, but in a multi-agent system some agents may have short life while other are persistent.
Situated		The agent is located in an environment, perceives it through sensors and acts on it through effectors.
Autonomy		An agent takes initiative and exercises control over its own actions
	Individuality	Clearly identifiable artificial entity with well-defined boundaries and interfaces with its environment.
	Self-control	Does not blindly obey commands, but can modify requests, negotiate, ask for clarifications, or even refuse.
	Flexible	Actions are not scripted; the agent is able to dynamically choose which actions to invoke, and in what sequence.
	Goal-oriented (purposeful)	Accepts high-level requests indicating what another agent (human/software) wants and takes responsibility for deciding how and where to satisfy the requests on behalf of its requester.
	Proactive Self-starting	Ability to take the initiative; an agent can reason to decide when to act.
Personality		An agent has well-defined believable personality and emotional state that can improve interaction with human users.
Communication (socially able)		An agent can engage in complex communication with other agents (human/software), to obtain information or enlist help in accomplishing its goals.
Adaptability		An agent customises itself to the preference of its user and adapts to changes in its environment.
	Customisable Educable	Can be customised by another agent (most probably human) to meet preferences.
	Learning	Automatically customises itself and adapt to changes in its environment (user preferences, networks, information...) and modifies its behaviour based on its previous experience.
Migration		An agent can move from one system environment to another, possibly across different system architectures and platforms.
Mobility		An agent is embedded in a real world moving artefact.
Visual representation		The agent has a visual representation; this goes from classic interfaces to anthropomorphised and virtual reality ones.
Veracity		The agent will not knowingly communicate false information.
Benevolence		Agents do not have conflicting goals, and every agent will therefore, always try to do what is asked of it.
Rationality		An agent will act in order to achieve its goals, and will not act in such a way as to prevent its goals from being achieved - at least insofar as its beliefs permit.

Figure 16: Agent characteristics [150].

2.4.3.1 The Definition in This Thesis Phase I and II

In this Thesis Phase I and II, however, the focus is more on how the agents communicate than how the actual agent is internally implemented. Therefore, the definition in this Thesis is as follows ¹²:

An agent is a computational process that implements the autonomous, communicating functionality of an application. Agents communicate using an Agent Communication Language (ACL). An Agent is the fundamental actor on an Agent Platform (AP) which combines one or more service capabilities, as published in a service description, into a unified and integrated execution model. An agent must support at least one notion of identity that labels an agent so that it may be distinguished unambiguously within the Agent Universe. An agent may be registered at a number of transport addresses at which it can be contacted.

For Phase III of this Thesis the focus is placed on *LLM Agents* as defined in Section 2.5.

2.4.4 Single Agent Architectures

Of course, the differences in agent definitions led to differences in agent internal architectures; depending on the characteristics of the agent, different internal agent architectures have been used. These architectures may be distributed over a continuum between purely reactive architectures and strong mentalistic architectures based on beliefs, desires, intensions and even emotions [150]:

- **Reactive:** Reactive agents operate via a direct sense-act loop, prioritizing rapid, real-time responses to environmental stimuli [158, 159, 160]. This architecture, characterized by a direct mapping of perceptions to actions, bypasses complex symbolic reasoning. While offering computational efficiency, this simplicity limits agents to reflexive behaviors, precluding proactive planning, learning, and higher-level decision-making due to the absence of an internal state [161].
- **Deliberative / Symbolic:** These architectures are based on symbolic reasoning and planning, with agents maintaining internal models of the environment. One of the most well-known and complex internal architectures to date is the BDI architecture [156]. The BDI extends the purely reactive architecture by incorporating a complex logical internal representation and introducing three high-level functions [150]. The complete architecture consists of:
 - *perceive*, that maps the external environment state to internal agent perceptions
 - *believe*, that builds new beliefs from perceptions and current beliefs
 - *desire*, that derives desires from the current set of beliefs and intentions
 - *intend*, that builds intentions motivated by the current beliefs and desires
 - *act*, that maps the internal intentions to actions on the external environment

BDI architectures offer powerful behaviors, but are complex to implement. Intermediate architectures provide a balance between reactive and BDI approaches, offering enhanced capabilities with reduced complexity. Examples include:

- **Knowledge-based:** Agents possess a knowledge base (perceptions), inference engine, and rule base for behavior. This expert-system architecture supports basic autonomous and proactive actions. However, hard-coded components reduce adaptability.
- **Finite State Machine (FSM)** Define agent behavior via states, inputs, outputs, and a transition function [150]. They add a decision step, but lack goal-orientation and have rigid structure.
- **Learning:** Employ ML techniques like supervised, unsupervised, RL, and transfer learning for complex tasks. RL enables reward maximization through environment interaction [162]. Examples include using deep neural networks and tree search in AlphaGo [163], and other methods leveraging decision trees or neural networks.

Hybrid architectures are divided into two main categories:

- **Hierarchical layers architectures:** the software layers corresponding to different behaviours are stacked up [150]. Two options can be identified: one pass control and two pass control architectures. In one-pass architectures, the control flows sequentially through each layer until the final layer generates action output. In two pass control architectures, information flows up the architecture and then control flows back down.

¹²The definition follows closely FIPA’s definition [157]

- **Parallel layers architecture:** the software layers corresponding to different behaviours are each directly connected to the sensory input and action output; the problem is the management of the competition for some resources or to perform actions [150]. In this case, there are two possible approaches: (1) layers subsumption based on priority links between the layers to impose the result of a layer over another one in case of conflict (2) meta-control: an independent overseeing module resolves the conflicts or handles the priorities. Blackboard architecture is a kind of meta-control where independent components are not directly connected, but share a repository of data (the blackboard) and a meta-controller handles conflict of access to the blackboard. Competitive tasks are another example where one task at a time is activated and chosen by a decision mechanism analysing the current state.

2.4.5 Multi Agent Architectures

The preceding section delineated definitions and architectural theories pertaining to individual agents. However, as this thesis addresses MAS, the mechanisms of inter-agent interaction are arguably more critical than the internal architecture of a single agent. Consequently, the following subsections detail agent communication protocols and present the multi-agent communication model employed in the subsequent chapters' demonstrations.

2.4.5.1 FIPA Architecture

The FIPA¹³ was founded in 1996 as a non-profit organization. FIPA produces standards for heterogeneous and interacting agents and agent-based systems across multiple vendors' platforms. The official mission statement was stated as follows [165]: "The promotion of technologies and interoperability specifications that facilitate the end-to-end interworking of intelligent agent systems in modern commercial and industrial settings. The emphasis here is on practical commercial and industrial uses of agent systems. However, we are also focusing on intelligent or cognitive agents, that is, software systems that may have the potential for reasoning about themselves and/or other systems that they encounter."

2.4.5.2 FIPA Agent Platform

Agent platforms are implemented as middleware software running on top of an operating system and they are needed because today's operating systems lack services that the agents need. FIPA does not specify the internal design of an agent platform, because FIPA's main concern is about achieving interoperability between agent platforms [152]. However, FIPA has specified a reference model [157] for agent management that is depicted in Figure 17.

On a conceptual level the reference model is very simple and there are only three essential components:

- The Message Transport System (MTS) routes messages both between agents within the FIPA agent platform and between agents residing on different FIPA agent platforms.
- The Agent Management System (AMS) manages the life cycles of the agents, such as starting, stopping and quitting agents. The AMS also maintains the mapping between an agent's identifier and its transport addresses, thus acting as an agent naming service. Furthermore, the AMS maintains the platform profile, which describes the platform properties such as communication capabilities.
- The Directory Facilitator (DF) has somewhat similar role in the agent world as UDDI has in Web Services. The DF is an agent that provides yellow pages for agents, i.e., maintains information about the skills that agents have advertised with it.

2.4.5.3 FIPA Architecture Implementations

There are several implementations of the FIPA Architecture and a detailed list is available at [166]. An evaluation of FIPA-compliant agent platforms is found in [167]. The evaluation shows that the interoperability was not fully achieved¹⁴ even though it is the most important goal of FIPA.

¹³The word *physical* in FIPA was originally included to cover agents of the robotic variety [152]. Over Time, however, the adjective's presence has come to serve as a reminder that physical - that is, human - agents and interaction with them are part of the association's scope [164]

¹⁴The evaluation was done in 2002 with four platforms: FIPA-OS, JADE, MECCA and FIPA SMART

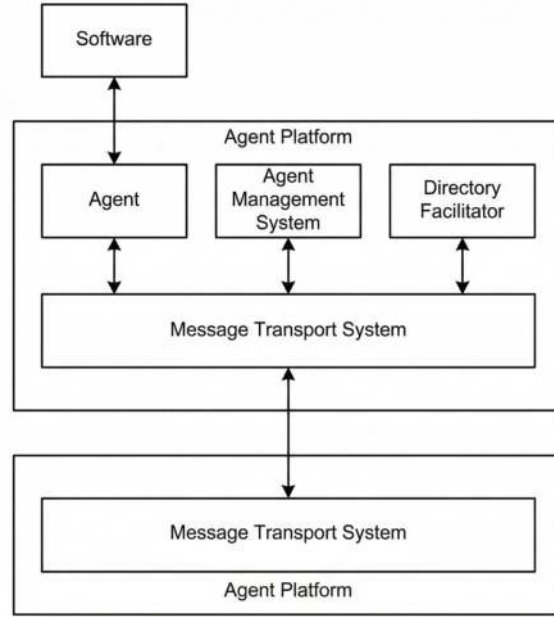


Figure 17: Agent Management Reference Model [157].

One of the most active projects has been JADE [168], that is also the base of the demonstrations made related to this Thesis. One interesting FIPA implementation is Lightweight Extensible Agent Platform (LEAP) [169] that can be used in devices with limited resources, such as mobile phones.

2.4.6 Multi Agent Communication

Russell and Norvig [145] have defined communication as *"intentional exchange of information brought about by the production and perception of signs drawn from a shared system of conventional signs"*. The most important system of signs for humans is the language that enables us to communicate with each other and a similar communication model based on a shared language has also been proposed for agent communication [152].

As noted by Helin [152], groups of agents are capable of achieving outcomes beyond the capabilities of any single agent. However, in order to perform their tasks effectively, agents depend heavily on expressive communication with other agents; to perform their requests, to propagate their information capabilities and to negotiate with other agents [150].

Agent communication can be divided into two fundamental parts. Firstly, agents have to agree on a common ACL, which defines the types of the message performatives and their meanings. Secondly, agents must have a common understanding of the knowledge that is exchanged within the messages. This includes both a shared ontology and the content that is defined by a content language.

Modern distributed computing leverages architectural styles and frameworks, such as the REST architectural style [170], gRPC [171], and message queues—e.g., Kafka [172] and RabbitMQ [173]—to achieve interoperability in heterogeneous environments, thereby superseding earlier middleware technologies, such as Web Services [174], Java RMI [175], CORBA [176], and DCOM [177]. However, they only provide interface description languages and services that allow distributed objects to be defined, located and invoked. But complex co-operation and heterogeneity of information require knowledge level interactions among agents [150]. This goes beyond the method invocation paradigm and it requires communication languages providing a structural and semantic grounding ¹⁵.

¹⁵This is the first part where the Semantic Web environment and MAS have similar interests; the need for a common language in MAS meets the W3C initiative of XML and RDF languages.

2.4.7 Multi Agent Communication Languages (ACL)

The ACL defines the types of messages with their meaning. A number of inter-agent communication languages have been proposed, but to date there are only two candidates that have gained success: Knowledge Query and Manipulation Language (KQML) [178] and FIPA-ACL [179]. Up to now they are the only agent communication languages that are widely implemented and used [150]. These languages are based on speech act theory and performatives [155], which were originally developed to model human communication.

In this Thesis, FIPA-ACL is shortly presented as it has a more important role than KQML in the demonstrations. Since both languages are based on speech act theory, they have many similarities. For example, if an agent wants to inform another agent about some fact, it may send an *INFORM* message if it is using FIPA-ACL and a *TELL* message if it using KQML.

2.4.7.1 FIPA-ACL

FIPA-ACL is the communication language for FIPA agents. The language has formal semantics and several concrete syntaxes and language specification is split into several documents. First, FIPA Communicative Act Library [180] gives the communicative acts and introduces the semantic model behind the language. Secondly, FIPA ACL Message Structure Specification [179] specifies the abstract structure of an ACL message. Thirdly, specifications [181, 182, 183] define concrete transport encoding schemes for FIPA-ACL. In addition to these, there are two sets of specifications that are not strictly a part of FIPA-ACL specification, but closely related; FIPA Content Language Library Specification [184] gives a registry for content languages and FIPA Interaction Protocol Library Specification [185] gives a registry for interaction protocols.

A FIPA-ACL message contains a set of message elements. The only mandatory element is the communicative act type, but typically a message contains additionally at least the sender and receiver information, and the content of the message expressed in some content language.

As depicted in Figure 18 FIPA-ACL has 22 predefined communicative acts; four of which are primitive communicative acts¹⁶ and the rest are composite communicative acts. Composite communicative acts are composed of primitive communicative acts either by substitution or sequencing. A given agent has to implement only those communicative acts it needs and the *NOT-UNDERSTOOD* act which every FIPA agent must implement.

2.4.8 Interaction Protocols

Agent communication typically consists of more than sending an isolated message even with the correct syntax and semantic. Communication usually involves several messages that form a dialogue. These dialogues follow patterns or policies that ensure a coherent exchange and this coherence can be reached only if the two participants are explicitly aware of the particular pattern in which they are engaged [186].

Thus, above the syntactic and semantic aspects is the pragmatic aspect of communication protocols, i.e., normative social behaviour rules [150]. The protocols that are needed for agents go beyond the client-server model and describe in terms of the high level of communication languages, the sequences of messages that can occur and their meaning in order to ensure the coherence required to initiate, carry out and terminate a co-operation. Protocols have a normative force that requires that the agents behave felicitously, but in exchange they ensure a known and agreed set of possible events and outcomes.

The protocols define communication patterns that can be used to set up relationships that define a special organizational pattern [150]. For instance, Figure 19 depicts one of the most well-known interaction protocols, i.e., the contract-net, where agents in need of services distribute calls for proposals to other agents. The recipients of these messages evaluate those requests and submit bids to the originating agents. The originators use these bids to decide which agents to choose and then award contracts to those agents. Contract-net can be used in many cases for instance in a task allocation problem to decide which is the best agent to be assigned the responsibility of solving a problem. In institutionalizing interactions, the protocol generates an organizational structure by dynamically assigning two roles: manager or contractor.

¹⁶*INFORM, REQUEST, CONFIRM, and DISCONFIRM*

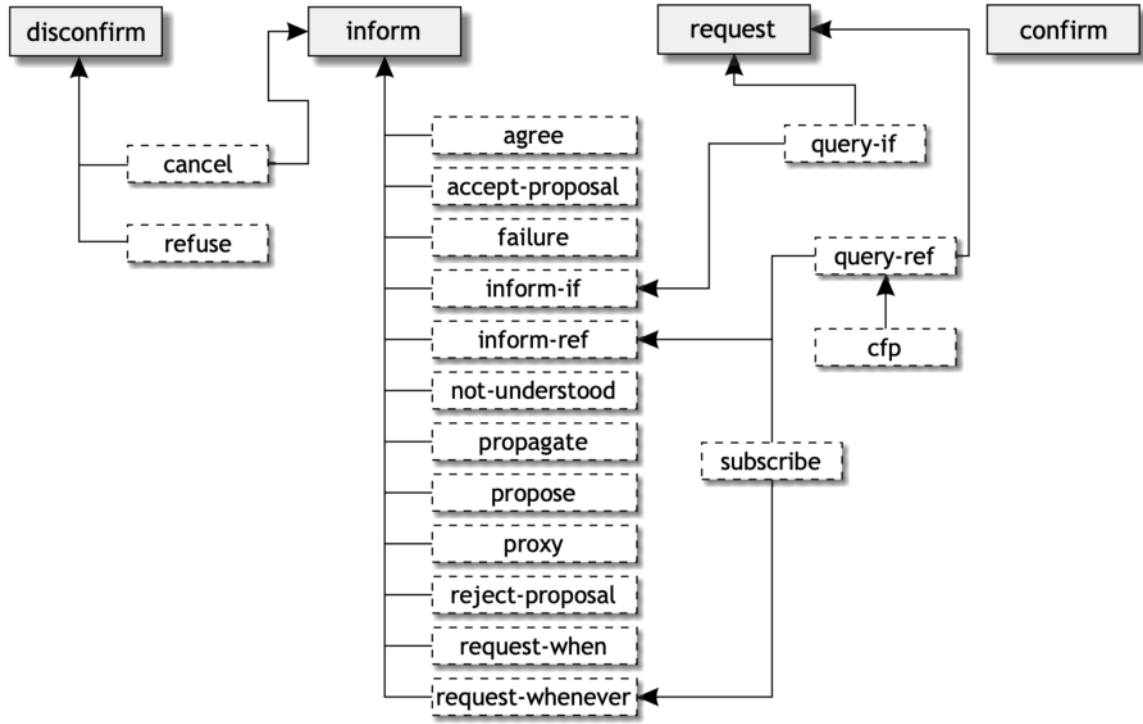


Figure 18: Relationships between communicative acts defined by FIPA [152].

2.4.9 Content Languages

As discussed earlier, agent communication languages typically define the outer language used in agent communication, but not the content of the message. Content languages are for describing the content of the agent messages.

Many content languages have been proposed but few of them have gained wide acceptance. In this Thesis, two most important languages are shortly presented ¹⁷.

2.4.9.1 FIPA Semantic Language

FIPA Semantic Language (SL) [188] is a formal language developed to define the semantics of the FIPA-ACL. FIPA-SL can be used to express objects, propositions, and actions [152]. Therefore it can be used as a content expression for every standard communicative act. The whole FIPA-SL can be too expressive language for simple agents, but it has three subset profiles: FIPA-SL0, FIPA-SL1, and FIPA-SL2. FIPA-SL0 is the most simple and FIPA-SL2 the most advanced including first order predicate and some modal logic.

2.4.9.2 Knowledge Interchange Format (KIF)

The Knowledge Interchange Format (KIF) [189] provides a high-level intermediate language for communication between programs [152]. The KIF language specification includes specification for its syntax and semantics. The syntax is inherited from Lisp and is therefore similar to that of FIPA-SL.

¹⁷In the demonstrations Semantic Web languages were used as a content language instead of agent content languages.

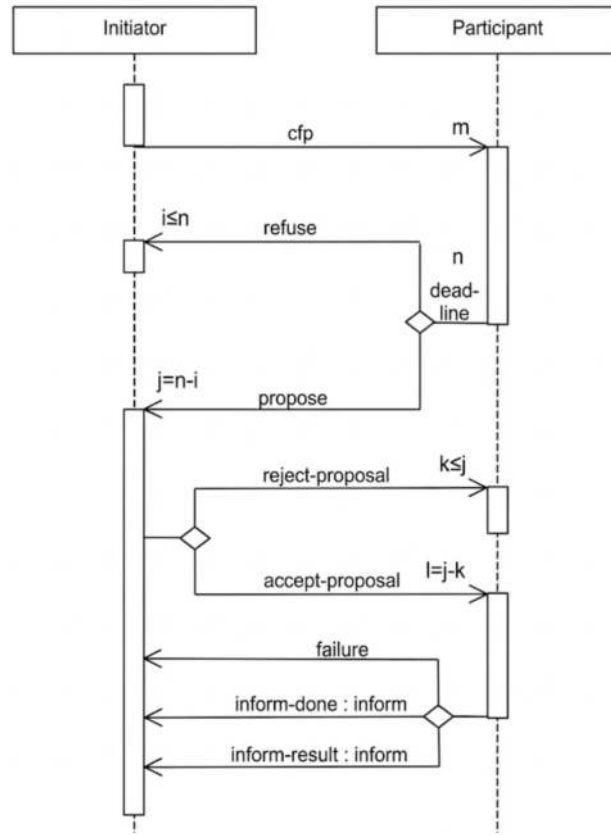


Figure 19: The FIPA Contract Net Interaction Protocol [187].

2.5 LLM Agents

Within the Phase III framing of this thesis (Section 1.1.2), this section defines the notion of an *LLM agent*: a software agent whose core decision or policy loop is mediated by an LLM and scaffolded with tool use, memory, and control mechanisms (Section 2.5.2). Building on the pre-LLM foundations in Sections 2.2, 2.3, and 2.4, it is made explicit how classical agent capabilities are re-instantiated when the agent’s reasoning substrate is a foundation model rather than a hand-coded planner or purely symbolic architecture. The design space is delineated—LLM-as-planner vs. controller roles, RAG, function/tool invocation, external and episodic memory, reflection/critique loops, and multi-agent coordination—then emerging agent protocols (e.g., MCP, A2A) and developer frameworks that operationalize these patterns are surveyed. The distinctive affordances (rapid task adaptation, language-native interfacing) alongside new failure modes are surfaced. This baseline sharpens the integration questions of RQ I.2 (Table 4) by clarifying what must interoperate between LLM-centric agents and Web/agent protocols, and it motivates the governance and assurance concerns addressed later in Phase III.

2.5.1 Emerging field of LLM Agents

The rise of LLMs following the introduction of the Transformer in 2017 [59] has dramatically reshaped the landscape of software agent development, opening up new possibilities and challenges. Pre-LLM, agents often relied on meticulously crafted rules, ontologies, and knowledge bases, limiting their adaptability and generalizability [103, 190]. While these approaches provided a degree of control and explainability, they struggled to handle the complexity and ambiguity of real-world scenarios. LLMs, trained on massive datasets, offer a fundamentally different approach. LLMs possess an unprecedented ability to understand and generate natural language, reason, and even exhibit some degree of common sense, albeit imperfectly [191, 192, 193]. This inherent capability allows LLM-powered agents to interact with users and environments in a much more natural and flexible way, reducing the need for extensive manual programming and domain-specific knowledge engineering.

The integration of LLMs with agent frameworks has led to the emergence of "LLM-powered agents" or "LLM agents", capable of performing a wider range of tasks than their predecessors. The simple definition for an LLM Agent is an artificial entity with prompt specifications (initial state), conversation trace (state), and ability to interact with the environments, such as tool usage (action) [194, 195]. These agents can leverage LLMs for various functions, including Natural Language Understanding (NLU), Natural Language Generation (NLG), planning, decision-making, and even tool use [196, 197, 198]. For instance, an LLM agent can be prompted to break down a complex task into smaller, manageable sub-tasks, search the web for relevant information, interact with APIs to execute actions, and summarize the results for the user [61, 199]. Frameworks like LangChain [200] and AutoGen [201] provide tools and abstractions to facilitate the development and deployment of these agents, enabling developers to chain together LLM calls, external tools, and custom logic. This transition marks a significant shift from symbolic AI to a more data-driven, emergent approach to agent development.

Despite the significant advancements, LLM agents also present unique challenges. One major concern is the potential for "hallucinations," where the LLM generates plausible-sounding but factually incorrect or nonsensical information [202]. This phenomenon can lead to unreliable agent behavior and requires careful mitigation strategies, such as grounding the LLM’s responses in external knowledge sources or employing techniques like RAG [203]. Other challenges include ensuring the safety and ethical behavior of agents, managing the computational cost of running large models, and addressing biases inherent in the training data [204, 205]. Furthermore, the "black box" nature of LLMs can make it difficult to understand the reasoning behind an agent’s actions, hindering debugging and limiting trust in critical applications [206].

The ongoing research in LLM-powered agents focuses on addressing these limitations and exploring new capabilities with the emerging MAS. This includes developing methods for improving the factual accuracy and reliability of LLMs, designing more robust and explainable agent architectures, and creating effective evaluation metrics to assess agent performance across diverse tasks [207, 208]. The convergence of LLMs and software agents represents a significant step towards building more general-purpose, adaptable, and intelligent systems, with the potential to transform how computers are interacted with and complex tasks are automated. However, responsible development and deployment, with a keen awareness of the inherent challenges, are crucial to realizing the full potential of this technology.

2.5.2 Single LLM Agent

The Figure 20 illustrates a high-level architecture of an LLM-based agent, consisting of the following components¹⁸. The system consists of several key components: **User Requests**, which serve as input from the user that initiates the agent’s operations; the **Agent**, which is the central module acting as the coordinator and managing interactions with all other components; **Tools**, which are external modules or systems the agent can utilize to gather data or perform specific tasks; **Memory**, a mechanism for storing and retrieving past interactions or knowledge, either short-term or long-term; and **Planning**, a module responsible for devising a structured approach to solve complex tasks by breaking them into subtasks. Some sources, such as [209], identify the **LLM** (not considered part of the LLM-agent definition) and **Actions** (distinct from tools) as separate concepts that are not shown in Figure 20.

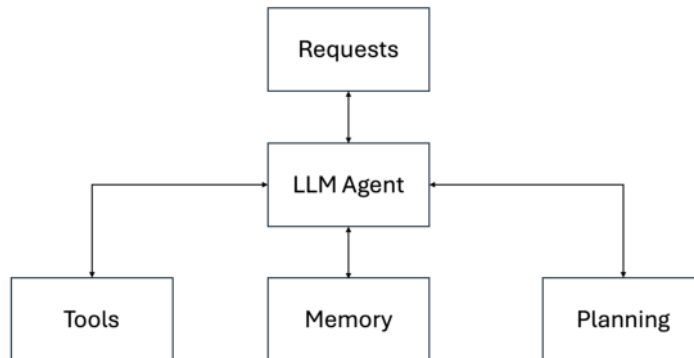


Figure 20: Single LLM agent core components.

2.5.2.1 Memory

The memory module stores the agent’s internal logs, including prior thoughts, actions, observations, and interactions with users. Memory is generally categorized into **short-term memory**, which captures immediate context and is limited by the LLM’s context window through in-context learning; **long-term memory**, which stores persistent knowledge and past experiences using external vector databases for scalable retrieval; and **hybrid memory**, which combines short-term and long-term memory to enhance reasoning and accumulate experiences over time.

Memory can take various forms, such as natural language, embeddings, databases, or structured lists. Systems like Ghost in the Minecraft (GITM) use a key-value structure with natural language keys and embedding vector values [210]. Effective planning and memory integration enable agents to adapt, recall prior actions, and improve future decision-making.

2.5.2.2 Tools and Actions

Tools enable LLM agents to take actions and interact with external systems, such as APIs, databases, knowledge bases, and external models. They support workflows that help the agent gather information or complete subtasks, like using a code interpreter to generate charts in response to user queries [195].

Different frameworks and methods for tool integration include **MRKL**, which combines LLMs with external knowledge sources and discrete reasoning modules [211]; **Toolformer**, which self-trains LLMs to decide when and how to call external APIs [212]; **Function Calling**, which exposes predefined tool schemas the model can invoke programmatically [213]; and **HuggingGPT**, which uses an LLM as a planner/controller to orchestrate specialist models for complex, multi-step tasks [214].

2.5.2.3 Planning

Planning Without Feedback

¹⁸Image redrawn from: <https://www.promptingguide.ai/research/llm-agents>

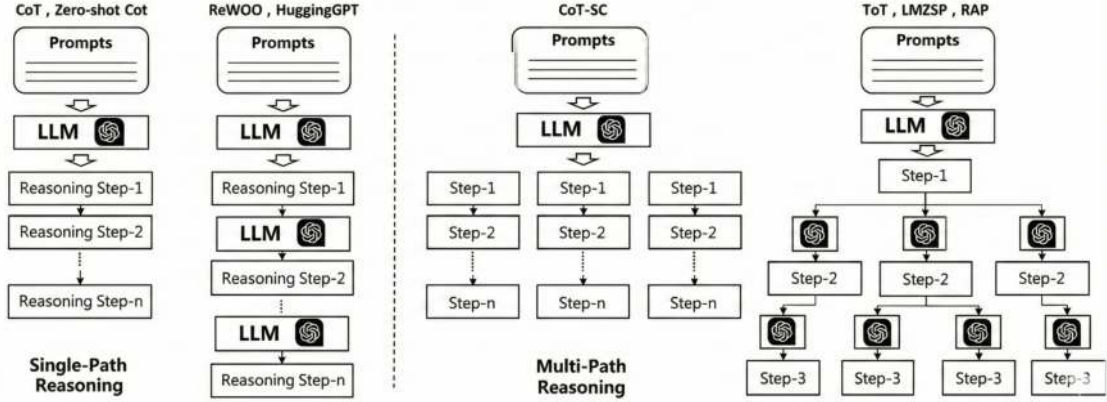


Figure 21: Comparison of single-path vs. multi-path reasoning (redrawn from [215]).

The planning module is responsible for breaking down a complex task into smaller, manageable steps that the agent can address individually to fulfill the user request. This process enhances the agent’s ability to reason effectively about the problem and reach a reliable solution. By utilizing an LLM, the planning module generates a detailed strategy comprising various subtasks necessary to address the user’s query. Notable techniques for task decomposition include Chain of Thought (CoT) and Tree of Thoughts (ToT), which fall under single-path and multi-path reasoning, respectively, as categorized by [215]. As shown in Figure 21, single-path methods (e.g., CoT) contrast with multi-path approaches (e.g., CoT-SC, ToT) that explore multiple trajectories before selection.

Beyond such single- and multi-path schemes, Meyerson et al. show that, once a complex problem is decomposed into near-atomic subtasks, reliability can be achieved by a Massively Decomposed Agentic Process (MDAP) that uses many small “micro-agents,” voting, and simple structural filters, rather than ever-larger base models [216]. This supports a useful distinction between an unsolved phase, where strong reasoning models and recursive planning are needed to discover an effective decomposition, and a solved phase, where execution reduces to an engineering problem of extreme task decomposition, error-correcting orchestration, and strict rails. In other words, once a good strategy is known, reliability is primarily a pipeline and architecture challenge, whereas orchestration alone cannot substitute for genuine reasoning when the decomposition itself is still unknown.

Planning With Feedback

The planning methods described above lack a feedback mechanism, making it difficult to handle long-term planning required for solving complex problems. To overcome this limitation, models can employ a system that iteratively revises and improves the execution plan by considering prior actions and observations. This process helps in correcting errors and refining the final output, which is especially crucial in challenging real-world tasks where iterative learning through trial and error is essential.

Two prominent approaches that incorporate feedback mechanisms are ReAct [217] and Reflexion [218] that are illustrated in Figure 22. Left side of Figure 22 ReAct combines reasoning and acting by alternating between three steps: Thought, Action, and Observation. This loop is repeated multiple times, allowing the model to incorporate feedback from the environment in the form of observations. Additional feedback can come from human inputs or model evaluations. This iterative approach enhances the model’s capacity to tackle more complex questions effectively [217]. The right side of the Figure 22 illustrates the Reflexion framework [218], a novel approach for enhancing language agents’ performance through linguistic feedback instead of traditional weight updates. It depicts an agent comprising several key components: the Actor (LM), Evaluator (LM), Self-reflection (LM), Trajectory (short-term memory), and Experience (long-term memory). The agent interacts with an external environment through actions and receives observations or rewards. External and internal feedback are processed to refine decision-making by maintaining reflective text stored in an episodic memory buffer. This iterative process allows the agent to improve performance through reflection and adaptation, making it effective across various tasks like coding, reasoning, and decision-making.

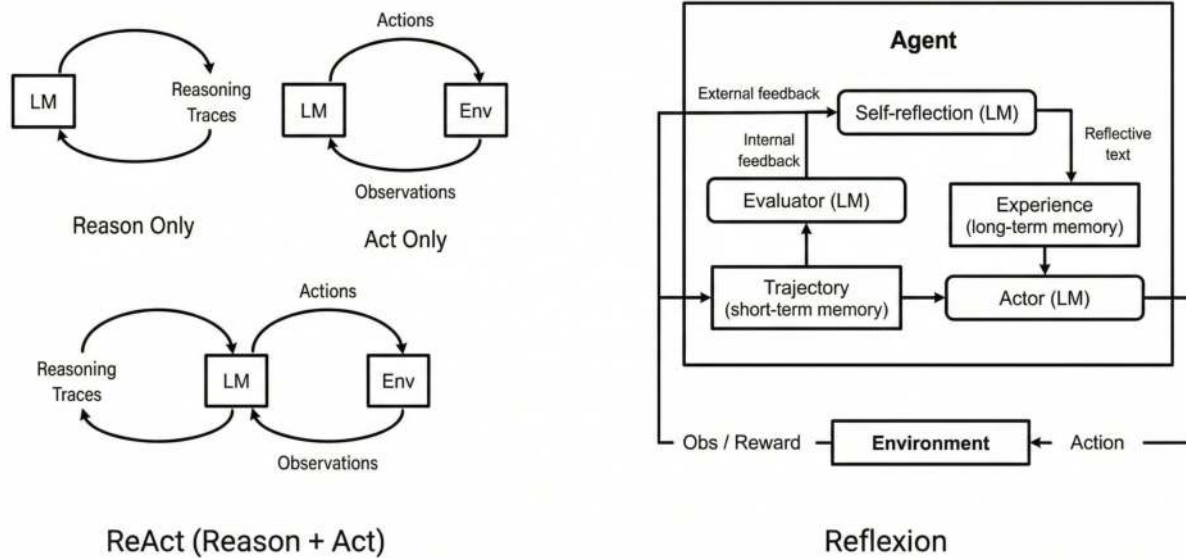


Figure 22: Comparison between Reason Only, Act Only, Reason and Act, and Reflexion framework (redrawn from [217] [218]).

2.5.3 Emerging LLM Agent Protocols

As illustrated in Figure 23, the initial wave of agent-to-agent protocols in 2024 by early specifications such as ANP and MCP—began to address the critical lack of standardized interfaces that had hindered LLM agents’ ability to scale, interoperate, and integrate with external tools and data sources. Throughout 2024, this foundation was expanded by more streamlined formats (e.g., *agents.json* [219]) and emerging infrastructures (e.g., Agora [220]) to support richer, structured messaging between agents. Looking ahead to 2025 and beyond, initiatives like OpenManus and OWL aim to formalize layered, group-centric protocols, signaling a shift toward fully interoperable, real-world agent ecosystems [209].

2.5.3.1 Model Context Protocol (MCP): LLM and Tools Integration

The Model Context Protocol (MCP) is an emerging open source standard designed to link AI assistants with the systems where data resides—such as content management platforms, enterprise tools, and software development environments¹⁹. Its primary goal is to enable advanced models to generate more accurate and contextually relevant outputs.

As AI assistants become more widely adopted, much of the industry’s progress has focused on improving model intelligence and reasoning. However, even the most advanced models remain limited by their lack of direct access to real-time data, often hindered by information silos and outdated infrastructure. Integrating each new data source typically demands a custom-built solution, creating scalability challenges for connected AI systems.

MCP aims to solve this problem by offering a unified, open standard for linking AI systems to data. By replacing piecemeal, one-off integrations with a single, consistent protocol, MCP simplifies and strengthens the way AI models interact with essential data sources—enabling more streamlined and dependable performance. Acting as a middleware layer, MCP aims to seamlessly bridges AI assistants and diverse data environments, abstracting away integration complexity and allowing developers to focus on higher-level application logic.

At its core, MCP follows a client-server architecture where a host application can connect to multiple servers, as illustrated in Figure 24. The architecture includes **MCP Hosts**, which are programs, IDEs, AI tools, or LLM Agents that want to access data through MCP; **MCP Clients**, which are protocol clients that maintain

¹⁹<https://modelcontextprotocol.io/introduction>

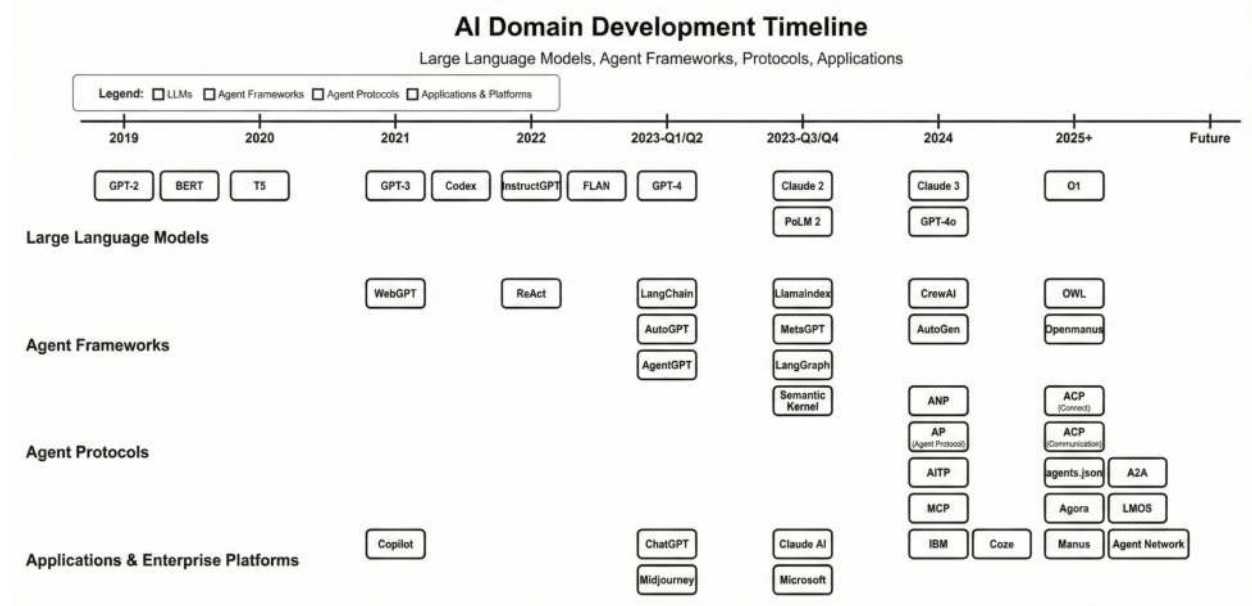


Figure 23: A glance at the development of agent protocols (redrawn from [209]).

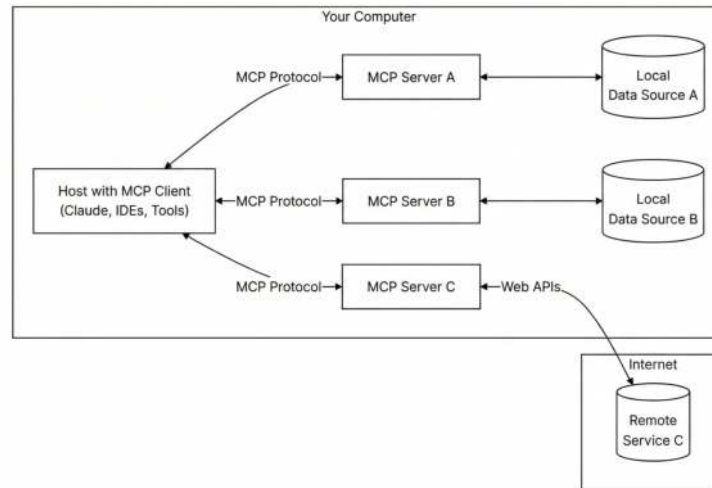


Figure 24: MCP General architecture [221].

1:1 connections with servers; **MCP Servers**, which are lightweight programs that each expose specific capabilities through the standardized MCP; **Local Data Sources**, which refer to your computer's files, databases, and services that MCP servers can securely access; and **Remote Services**, which are external systems available over the internet (e.g., through APIs) that MCP servers can connect to.

2.5.3.2 Google Agent2Agent Protocol (A2A)

Agent2Agent (A2A) is an open protocol by Google that aims to complement the MCP, which provides helpful tools and context to agents [222]. The A2A protocol will allow AI agents to communicate with each other, securely exchange information, and coordinate actions on top of various enterprise platforms or applications.

The A2A protocol facilitates communication between independent AI agents through several key elements [223]. These include the **Agent Card**, which is public JSON metadata detailing an agent's capabilities, endpoint, and authentication for discovery; the **A2A Server**, which is an agent exposing an HTTP endpoint implementing

A2A protocol methods to receive requests and manage tasks; the **A2A Client**, which is an application or agent consuming A2A services by sending requests (e.g., tasks/send) to a server URL; the **Task**, which is the core work unit initiated by a client message (tasks/send or tasks/sendSubscribe), with a unique ID and lifecycle states (submitted, working, input-required, completed, failed, canceled); and the **Message**, which represents a communication turn between client ("user") and agent ("agent"), containing Parts. A **Part** is the basic content unit in a Message or Artifact, such as TextPart, FilePart (inline or URI), or DataPart (structured JSON); an **Artifact** refers to agent-generated outputs during a task (e.g., files, structured data), also composed of Parts. For long tasks, **Streaming** is supported through tasks/sendSubscribe, enabling servers to send real-time progress via Server-Sent Events (TaskStatusUpdateEvent or TaskArtifactUpdateEvent). Additionally, **Push Notifications** allow servers with pushNotifications enabled to proactively send task updates to a client-provided webhook URL, which is configured via *tasks/pushNotification/set*.

2.5.4 Multi-Agent Conversation Patterns in LLM Era

The Figure 25 illustrates the architecture of an agent-based conversation framework known as AutoGen, emphasizing three key capabilities: Conversable Agent, Agent Customization, and Flexible Conversation Patterns. In the top-left section, a "Conversable Agent" is depicted as a modular entity with customizable components (e.g., OpenAI models, Python modules, and user-defined tools), demonstrating how agents can be tailored to perform specific tasks. The top-right section presents "Multi-Agent Conversations," where agents interact collaboratively or competitively through directed communication channels. The bottom section highlights "Flexible Conversation Patterns," showcasing various configurations such as joint chats and hierarchical chats, where agents either collaborate equally or follow structured command hierarchies. This architecture supports extensibility and scalability by enabling diverse conversational agents to integrate seamlessly for complex, multi-agent task completion.

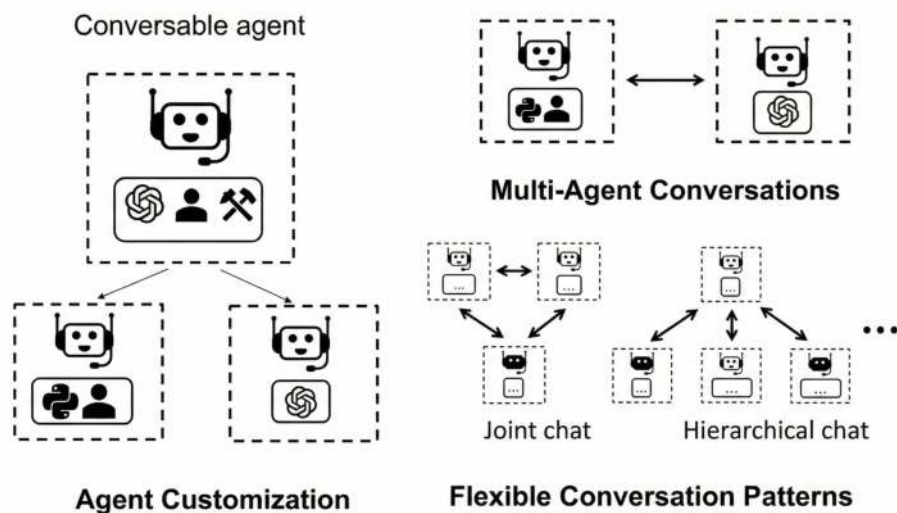


Figure 25: Example library (AutoGen) agents are conversable, customizable, and can be based on LLMs, tools, humans, or even a combination of them (redrawn from [224]).

2.5.5 Prominent LLM Agent Frameworks and Libraries

The rapid expansion of LLM-agent tooling is propelled by a diverse open-source ecosystem. LangChain (LangChain, Inc.) supplies modular building blocks for tool use and memory [225], while LangGraph (LangChain, Inc.) layers graph-based, stateful control on top [226]. AutoGen/AG2 (Microsoft Research) enables multi-agent conversational collaboration [227, 228], and LlamaIndex (LlamaIndexAI, Inc.) focuses on data connectors and retrieval pipelines [229]. CrewAI (CrewAI) provides role-oriented orchestration [230], and Microsoft's Semantic Kernel targets enterprise integration [231]. Complementary projects—Haystack (deepset) [232], Embedchain (Embedchain) [233], SuperAGI (SuperAGI) [234], Dify (LangGenius, Inc.) [235], and OpenAgents (XLang/Princeton) [236]—extend the landscape across RAG pipelines, agent hosting, and

end-to-end app platforms. Table 9 compares these frameworks by core paradigm, key features, and typical use cases.

Table 9: Comparison of Prominent LLM Agent Frameworks.

Framework	Core Paradigm	Key Features	Best Use Cases
LangChain [225]	Composable agentic framework	Modular tools, memory management, large ecosystem	Chatbots, assistants, document analysis, recommendations, research
LangGraph [226]	Graph-based stateful agents	Graph workflows, branching/parallelism, step-level debugging	Planning, reflection, multi-agent coordination
AutoGen [227, 228]	Conversational multi-agent systems	Custom agents, auto-chat loops, tool use, feedback integration	Problem-solving, workflow automation, multi-agent conversations
LlamaIndex [229]	LLM-centric data layer	Data ingestion, retrieval, knowledge graphs, RAG pipelines	Q&A on private documents, summarization, specialized search
CrewAI [230]	Role-oriented MAS orchestration	Role-based agents, collaboration, flexible memory	Coordinated problem-solving, diverse expertise, simulations
Semantic Kernel [231]	Enterprise AI orchestration	Plugins, multi-language SDKs, enterprise-grade security	AI integration in enterprise systems, high security/compliance
Agent Development Kit (ADK) [237]	Multi-agent development toolkit	Model abstraction, agent orchestration, evaluation suite, tool calling	Complex multi-agent workflows, research on agent behaviours, evaluation pipelines
Genkit [238]	Code-first GenAI workflow framework	Flow orchestration, prompt/version control, tool calling, RAG, observability, Cloud Run deployment	Content generation, RAG apps, end-to-end GenAI services with monitoring
Marvin [239]	Pythonic agent workflow library	Decorator-based tasks, threaded conversations, async support, minimal boilerplate	Developer-friendly agent workflows in Python apps, data/ML pipelines, rapid prototyping

Maintainers / organizations: LangChain—LangChain, Inc.; LangGraph—LangChain, Inc.; AutoGen—Microsoft (Research); LlamaIndex—LlamaIndexAI, Inc.; CrewAI—CrewAI (open-source; led by João Moura); Semantic Kernel—Microsoft; ADK (AG2)—Microsoft; Genkit—Google (Firebase); Marvin—Prefect, Inc.

2.6 LLM and DRG Related Studies

Section 2.6.1 reviews core advances in LLMs relevant to the tasks. Section 2.6.2 motivates governance-grade evaluation through alignment and safety considerations. Section 2.6.3 surveys general LLM benchmarks and leaderboards, clarifying what they measure—and what they miss for CaseMix reasoning. Section 2.6.7 synthesises prior DRG-LLM studies to surface the remaining gaps. Section 2.6.8 describes production DRG grouper software and situates the benchmark within that ecosystem. Section 2.6.6 introduces the NordDRG CaseMix system—the rule base the benchmark operationalises—and establishes domain context.

2.6.1 LLMs

The breakthrough Transformer architecture introduced by Vaswani *et al.* [59] ignited today’s natural-language processing renaissance. Initial encoder models such as BERT [240] were quickly followed by vast autoregressive generators—GPT-3 [191], GPT-4, and others—whose parameter counts now reach into the hundreds of billions and display emergent strengths in generation, retrieval, and reasoning. Concurrent strands of work explored alternative training frameworks (T5’s text-to-text paradigm [241]), compute-data scaling laws (Chinchilla [242]), and extreme model sizes, exemplified by PaLM’s 540-billion-parameter Pathways network [243]. Meanwhile, open-weight initiatives such as OPT [244] and Llama-2 [245], have broadened community access to state-of-the-art capabilities. Comprehensive surveys map this rapid trajectory from early generative models to ChatGPT, situating LLMs within the wider generative-AI landscape and cataloguing key architectural and application trends [246, 247].

Since late-2022, chat-style interfaces—epitomised by ChatGPT—have moved LLMs beyond research into mainstream workflows. By masking low-level prompts behind natural conversation, these interfaces transform LLMs into a versatile cognitive layer that clinicians, lawyers, educators, and business users can harness with minimal technical overhead.

2.6.2 AI alignment and safety

As technical frontiers expand, the focus of responsible AI practice is shifting from *capability* to *governance*. Ensuring that AI model behaviour aligns with human values and organisational objectives is a critical—and multidimensional—challenge [31], with well-documented risks of opacity, disparate impact, and governance failures in high-stakes domains [248]. The alignment problem is often decomposed into *outer alignment* (does the specified objective capture what we actually want?) and *inner alignment* (does the learned optimiser in fact pursue that objective, especially on inputs outside the training distribution?) [249, 250]. Practical failures arise from reward misspecification, specification gaming, and goal misgeneralisation, particularly under non-stationarity [251, 250].

Recent advances combine data-driven optimisation with human oversight to better shape training objectives. Preference learning and reward modelling translate human (or AI-assisted) judgments into training signals [252, 253]; instruction tuning with human or AI feedback (e.g., RLHF and constitutional prompting) further reduces harmful behaviours and improves adherence to stated guidelines [192, 254]. However, these techniques alone do not satisfy governance-grade requirements in institutional settings, where decision logic audit trail, deliberation, and auditability for consequential changes are indispensable.

2.6.3 LLM Benchmarks and Leaderboards

Benchmark suites and public leaderboards are central to tracking progress in LLMs. Early collections, such as GLUE [255] and SuperGLUE [256], gave way to broader, multi-skill batteries—e.g., BIG-bench [257], MMLU [258], and BabyLM [259]—probing factual knowledge, reasoning, multilinguality, and instruction following. Meta-evaluations like HELM [260] further standardise conditions and report robustness, bias, and efficiency alongside accuracy.

Results from these suites are surfaced on widely watched leaderboards. The HuggingFace *Open LLM Leaderboard* aggregates automatic metrics (e.g., MMLU, ARC, HellaSwag) for open-weight models. The LMSYS/LMArena *Chatbot Arena* leaderboard ranks dialogue quality via blind, pairwise human preferences [261]. Task-specific evaluations, such as MT-Bench [262] and vendor dashboards (e.g., Vellum-AI [263]), emphasise application skills like tool use and code generation. Figure 26 illustrates a representative snapshot, underscoring how frequently updated scorecards have become a de facto currency for comparing models.



Figure 26: LLM Leaderboard illustration: Vellum leaderboard [263]

2.6.4 Version Control Systems for Governed Change

Version Control Systems (VCSs) provide the provenance, review, and release mechanics that governed, audit-heavy work depends on. Centralised systems (e.g., CVS, Subversion) established change history and branching; distributed VCSs (DVCSs), such as Git and Mercurial popularised immutable, content-addressed

commits, offline branching, and robust merge workflows [264, 265]. Contemporary options extend this space with integrated platforms (Fossil, with built-in wiki/issue tracker) and patch-theoretic designs (Darcs; Pijul’s formal patch model) [266, 267, 268]. Newer Git-compatible tools like Jujutsu (jj) focus on simpler UX while remaining compatible with Git storage/hosting [269, 270].²⁰

Across today’s ecosystems, governance-relevant capabilities are broadly available: (i) immutable, addressable commits; (ii) reviewable change bundles (PRs/MRs) with recorded approvals; (iii) CI gate integration; (iv) authenticated authorship and protected branches (e.g., signed commits/tags; required reviewers); (v) reproducible rebuilds from a commit hash; and (vi) long-term retention and audit-trail export.

2.6.5 MLOps: operating ML under software discipline

Machine Learning Operations (MLOps) extends Development and Operations (DevOps) with practices and platforms that make model-centric systems reproducible, observable, and continuously deployable in production [271]. Empirical studies [272] from large engineering organisations show that ML work follows a multi-stage, experiment-heavy workflow (data collection/labeling, feature engineering, training, evaluation, deployment, monitoring) with numerous feedback loops and hand-offs between roles. In contrast to conventional software, behaviour in ML systems depends not only on code but also on mutable data, configurations, and environments; consequently, operational quality hinges on disciplined tracking, testing, and change management across *all* these artefacts [273, 274].

Core concerns and recurring risks. Foundational accounts highlight “hidden technical debt” specific to ML: boundary erosion between components, entanglement, configuration fragility, data dependencies, and undeclared consumers that silently couple pipelines [273]. To counter these failure modes, MLOps emphasises: (i) experiment tracking and lineage (parameters, code, data, environments); (ii) data and feature validation in pipelines; (iii) automated tests and CI/CD for training and serving; (iv) canary and shadow deployments; and (v) post-deployment monitoring for performance, drift, and fairness [274, 271]. These controls embody a shift from “train-once” research to continuous delivery of ML behaviour under operational budget and governance constraints.

Platforms and tooling. Industrial platforms, such as TensorFlow Extended (TFX), codify end-to-end, type-checked ML pipelines—data ingestion, validation, transformation, training, evaluation, and serving—with metadata and replayable execution graphs [275]. Complementary lifecycle tools like *MLflow* standardise experiment tracking, model packaging (multi-flavour artefacts), and registries to coordinate promotion of versions across environments [276]. Together, these platforms support reproducibility and change control at scale, yet remain largely agnostic to domain-specific governance.

Documentation and governance patterns. Operationalisation has spurred documentation artefacts that travel with models and data. *Model Cards* describe intended use, evaluation slices, and caveats for released models; *Datasheets for Datasets* analogously document audit trail and limitations of training data [277, 278]. In MLOps practice these artefacts integrate with registries and CI gates, enabling reviewers to assess risks, compliance, and suitability at deployment time.

2.6.6 NordDRG CaseMix System

The Nordic Diagnosis-Related Groups (NordDRG) classification is the shared CaseMix framework used throughout the Nordic countries to cluster inpatient episodes with comparable resource consumption [279]. By assigning each hospital stay to a single DRG, the system underpins transparent benchmarking of costs, outcomes, and efficiency across institutions, regions, and years.

NordDRG is distributed as a set of *definition tables*—about twenty interlinked sheets released annually in Excel, CSV, and JSON formats—that encode the logic connecting diagnosis and procedure codes, age/sex splits, and national activation flags to individual DRG codes. These tables constitute the authoritative rule base for coding software, technical-change management, and policy analysis.

²⁰This study uses “Git-style” to denote any VCS that offers immutable commit identifiers, branching/tags, reviewable change bundles, and CI hooks. Alternatives, such as Mercurial, Fossil, Pijul, or Jujutsu, can support the workflow proposed in this study so long as these properties are available. Git is used here pragmatically due to ubiquity in partner environments and mature integrations with common forges (GitHub/GitLab/Gitea), enterprise CI/CD, signed commits/tags, branch protections, and audit logging.

Because each rule specifies explicit inclusion criteria, identical cases are grouped consistently regardless of hospital or country. At the same time, each nation can activate, deactivate, or re-weight DRGs locally, allowing the common core to accommodate diverse clinical practices and tariff structures. This blend of rule-level precision and country-level flexibility makes NordDRG a durable instrument for hospital finance, clinical research, and health-system governance in the Nordic region.

2.6.7 Related Work: DRGs and LLMs

LLMs are increasingly explored as end-to-end engines for hospital-reimbursement workflows. [280] fine-tune LLaMA on 236-k multimodal MIMIC-IV episodes, achieving 52% top-1 accuracy over 771 MS-DRGs—a strong indication that the model captures the CC/MCC rules traditionally hard-coded in grouper software. Complementing this, [281] show that a tool-augmented GPT-4 rivals human coders in ICD assignment, implying that careful retrieval and validation can offset the need for proprietary data. Zero-shot modelling is pushed further by [282], whose transformer infers DRG-like resource strata directly from longitudinal health trajectories, hinting at reimbursement systems emergent from the structural landscape of encoded patient health states.

Beyond accuracy metrics, recent studies ask whether LLMs truly understand DRG logic. [283] introduce *DRG-Sapphire*, using reinforcement learning with rule-aware rewards and generating physician-validated rationales that explicitly cite grouping criteria. Interpretability is also central to [284], whose multi-task transformer highlights token-level evidence so auditors can verify alignment with official rules. Structured knowledge helps as well: [285] inject a clinical knowledge graph into BERT and observe the largest gains on DRGs whose definitions hinge on multi-entity interactions, suggesting genuine rule-level reasoning. Finally, generic LLMs remain competitive; [286] repurpose them for zero-/few-shot ICD coding by exploiting the ICD hierarchy, while [287] couple LLaMA representations with a lightweight MultiResCNN, reporting F1 gains across the full label set.

2.6.8 DRG Grouper Software

DRG groupers are rule-based software engines that map a patient episode’s minimum dataset—principal and secondary diagnoses, procedures/interventions, demographics (age/sex), discharge status and other context—onto a single payment group. While national families differ (e.g., MS-DRG in the United States, AR-DRG in Australia, APR-DRG used by many payers, and NordDRG across the Nordics), they share a governed control flow: MDC entry, ordered rule evaluation, surgical evidence (OR/non-OR), complication/comorbidity handling (CC/MCC or analogous constructs), and versioned annual updates [288, 289, 290, 291, 292].

In practice, groupers are distributed as compiled binaries, libraries or services accompanied by technical manuals and release notes. MS-DRG publishes detailed definitions and versioned release documentation [289]; AR-DRG provides a public technical specification describing hierarchy, splitting rules, and episode complexity scoring [290]; APR-DRG documents clinical logic and the severity/risk subclasses layered atop base DRGs [291]; and NordDRG uniquely exposes machine-readable definition tables that encode the executable rule graph (diagnosis/procedure properties, age/sex bounds, national activation) [292]. Historically, comparative scholarship has framed these systems as transparent, auditable instruments for hospital payment and CaseMix analysis, while noting national tailoring and update governance [288].

2.7 Chapter Summary: Background

Scope. This chapter assembled the historical and technical foundations that the rest of the thesis relies on. The following were traced: (i) the recurring *AI summers and winters* that shape expectations and funding cycles (Section 2.1); (ii) the *Web’s* evolution from a hypertext medium to a platform for distributed services (Section 2.2); (iii) the *Semantic Web* vision and its agent-centric aspirations (Section 2.3); (iv) classical *Software Agents* and MAS concepts (Section 2.4); (v) contemporary *LLM Agents*, their core architecture, and failure modes (Section 2.5); and (vi) domain-specific *LLM and DRG* studies that motivate the healthcare case work (Section 2.6).

Key constructs and standards. This chapter clarified how knowledge representation (RDF, OWL), rule systems, service and interaction ontologies (e.g., OWL-S), and agent communication frameworks (e.g., FIPA ACL and protocol ontologies) collectively define the *integration surface* between data, reasoning, and action on the Web. These ingredients explain both the enduring promise (declarative interoperability) and

the well-documented friction points (heterogeneity, brittleness, and tooling gaps). *This standards baseline is revisited when integrating LLM-based agents (Section 2.5) and when applying it in the DRG domain (Section 2.6), where controlled vocabularies and structured tool I/O make model behavior auditable.*

From classical agents to LLM agents. Placing LLM-mediated reasoning within the agent design space, roles for the LLM (planner vs. controller), patterns for tool invocation and external memory, reflection/critique loops, and multi-agent coordination were contrasted. Notable advantages alongside new risks were highlighted, showing how these trade-offs reshape long-standing agent engineering questions rather than replace them. *This framing is instantiated in the healthcare DRG setting (Section 2.6), where LLM agents are constrained by domain rules, code-as-policy, and verifiable tool use.*

Implications for the thesis. This chapter substantially addresses **RQ I.1** in Table 4 by tracing the historical origins of the Web and software agents and by consolidating the core KR, rules, and protocols—that underpin the remainder of the work. On that basis, **RQ I.2** is sharpened by specifying the practical *integration surface* between Semantic Web formalisms and software-agent protocols required for runtime interoperability and adaptation, and **RQ I.3** is bounded by situating rule- and ontology-based service description/matching within the broader evolution toward mobile, service-oriented Web usage. These clarifications provide the design assumptions and evaluation criteria carried into Phase II (**RQ II.1**) and Phase III, where *LLM-based agents with governance and verification* are used to address **RQ III.1** (human–AI decision alignment), **RQ III.2** (emulation and application of DRG payment rules), and **RQ III.3** (auditability and traceability of agentic reasoning and tool use) within the DRG domain.

Enduring gaps. Across eras, the central challenge remains *runtime interoperability*: making heterogeneous agents, services, and knowledge bases interact safely and auditable at scale. Semantic formalisms raise the ceiling on expressivity and explanation, while protocol standards raise the floor on interaction discipline; yet practical convergence is hindered by evolving tooling, incomplete coverage of real-world procedures, and the effort required to keep specifications, data, and code aligned.

Transition. The subsequent chapters build on this foundation: they operationalize these concepts into artefacts and evaluations, test the proposed integration pathways, and assess where standards, engineering scaffolds, and LLM capabilities suffice—and where additional governance, constraints, or design patterns are required.

3 FORMAL DEFINITIONS

This chapter collects the formal definitions that underpin the analytical framework of this thesis. In particular, it makes precise the key concepts related to deep learning and the *Transformer Architecture* architecture, which has arguably been the most consequential invention in ML over the past decade and the enabling technology behind modern *LLMs*, such as *GPTs* and related systems. By consolidating these definitions in one place, the chapter aims to fix terminology, avoid ambiguity, and provide a rigorous reference point that later chapters can build on when analysing how Transformer-based models are designed, trained, and deployed.

3.1 Deep Learning Analytical Framework and Core Definitions

This section establishes the analytical framework and notation used throughout the paper. We distinguish and formally define the *Transformer Mathematical Model*, its constituent *Transformer Mathematical Functions*, and the *Transformer Architecture* (originating with [59]). Section 3.1.1 presents the core concepts and notation; Section 3.1.2 gives the corresponding Transformer-specific definitions.

3.1.1 Core concepts and notation

We fix the basic vocabulary and symbols used throughout the paper. Figure 27 summarizes the relationships among the core entities: a *Deep Learning Architecture* specifies a parameterized *Mathematical Model* (a family of functions); a *Training method* uses a *Dataset* and *Calculation Infrastructure* to fit the parameters; the result is a *Deep Learned Algorithm* (a trained, deterministic procedure).

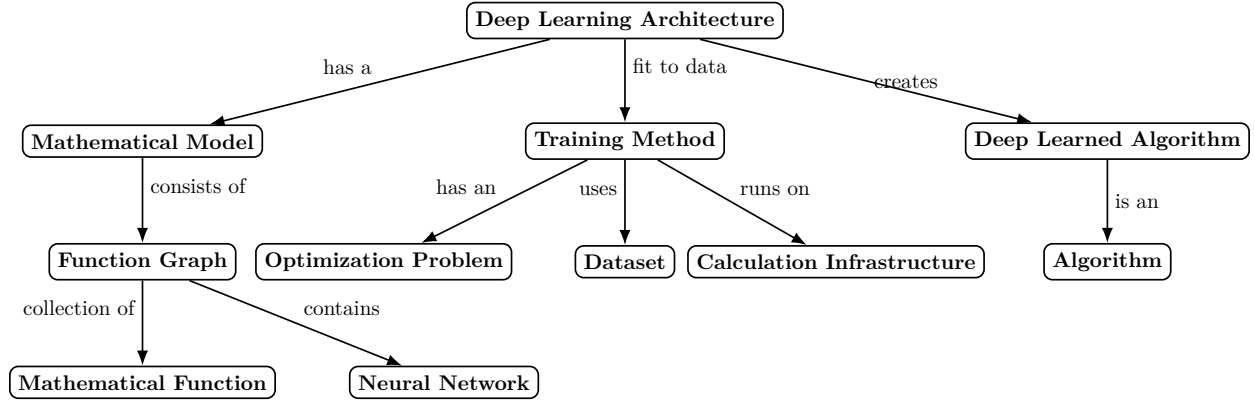


Figure 27: Deep learning core concepts and their relationships. Interactive illustration with links to definitions.

Notation. We denote by $f_\theta : X \rightarrow Y$ a parameterized function with learnable parameters θ ; training produces θ^* and the trained mapping f_{θ^*} . A dataset is $D = \{(x_i, y_i)\}_{i=1}^n$ (supervised) or $D = \{x_i\}_{i=1}^n$ (unsupervised). A training method is $T = (\mathcal{L}, \Omega, \Pi)$, where $\mathcal{L}$ is a loss, Ω an optimizer (including schedule/regularization), and Π optional procedures (augmentation, early stopping, etc.). Computation for training is $C_T = (H, S, R)$ with hardware H , software S , and resource constraints R .

Definition 3.1.1 (Function). A *Function* is a (deterministic) mapping $f : X \rightarrow Y$ assigning each $x \in X$ a unique $y = f(x) \in Y$. If randomness is used, a seed s is treated as part of the input so $(x, s) \mapsto y$ remains a *Function*.

Definition 3.1.2 (Function Graph). A *Function Graph* is a directed acyclic graph $G = (V, E)$ whose nodes are *Functions* and whose edges encode dataflow/composition. Composition in topological order defines an overall *Function* f_G .

Definition 3.1.3 (Mathematical Model). A *Mathematical Model* is an abstract description $M = (G, \lambda)$ consisting of a *Function Graph* G and fixed hyperparameters λ (wiring, layer sizes, activations, tokenization, etc.). It induces a family $\{f_\theta\}$ of *Functions* with learnable parameters θ (not fixed by M).

Definition 3.1.4 (Algorithm). An *Algorithm* is a finite, well-specified procedure that deterministically computes outputs from inputs. When stochastic behavior is desired (e.g., sampling), the randomness source and any decoding policy are considered fixed parts of the *Algorithm*.

Definition 3.1.5 (Neural Network). A *Neural Network* is a computational structure specified by a *Function Graph* G and hyperparameters λ , whose induced family of *Functions* $\{f_\theta\}$ is obtained by composing affine maps with *Activation Functions*. For a feed-forward network with L layers, a typical member $f_\theta : \mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^{d_{\text{out}}}$ has the form

$$f_\theta(x) = W_L h_{L-1} + b_L, \quad h_\ell = \sigma_\ell(W_\ell h_{\ell-1} + b_\ell), \quad \ell = 1, \dots, L-1,$$

with $h_0 = x$, weight matrices W_ℓ , biases b_ℓ , and nonlinear activations σ_ℓ . The learnable parameters are $\theta = \{W_\ell, b_\ell\}_{\ell=1}^L$, and the layered structure defines the corresponding *Function Graph* G .

Definition 3.1.6 (Activation Function). An *Activation Function* is a fixed nonlinear map $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ (or $\sigma : \mathbb{R}^m \rightarrow \mathbb{R}^m$ applied elementwise) used inside a *Function Graph*. Common choices include ReLU, sigmoid, and tanh.

Definition 3.1.7 (Machine Learning Architecture). A *Machine Learning Architecture* is a general framework where a *Mathematical Model* is specified to learn from data, distinct from *Deep Learning Architecture* in that it does not mandate the use of *Neural Networks* (e.g., Support Vector Machines, Random Forests).

Definition 3.1.8 (Deep Learning Architecture). A *Deep Learning Architecture* is a specific type of *Machine Learning Architecture* in which the *Mathematical Model* incorporates *Neural Networks* as part of its *Function Graph*. When such a model is trained on data using one or more *Training methods*, the resulting trained object is called a *Deep Learned Algorithm* (Definition 3.1.16).

Definition 3.1.9 (Optimization Problem). An *Optimization Problem* is a tuple $P = (\Theta, \mathcal{J})$ consisting of a feasible set Θ and an objective function $\mathcal{J} : \Theta \rightarrow \mathbb{R}$. Solving the problem entails finding a parameter $\theta^* \in \Theta$ that minimizes the objective, defined as $\theta^* = \operatorname{argmin}_{\theta \in \Theta} \mathcal{J}(\theta)$.

Definition 3.1.10 (Training method). A *Training method* is the procedure used to fit a *Mathematical Model* to an *Optimization Problem*. Common *Optimization Problems* used in training are *Supervised Learning*, *Unsupervised Learning*, and *Reinforcement Learning*. Figure 28 provides an intuitive visualization of these paradigms.

Formally, a training method is a triple $T = (\mathcal{L}, \Omega, \Pi)$ that, when applied to *Dataset(s)* D under *Calculation Infrastructure* C_T , returns parameters θ^* by minimizing the objective defined by the optimization problem. Here $\mathcal{L}$ specifies the loss (or utility) function, Ω the optimizer and schedule, and Π ancillary procedures (e.g., batching, masking, augmentation, regularization, checkpointing).

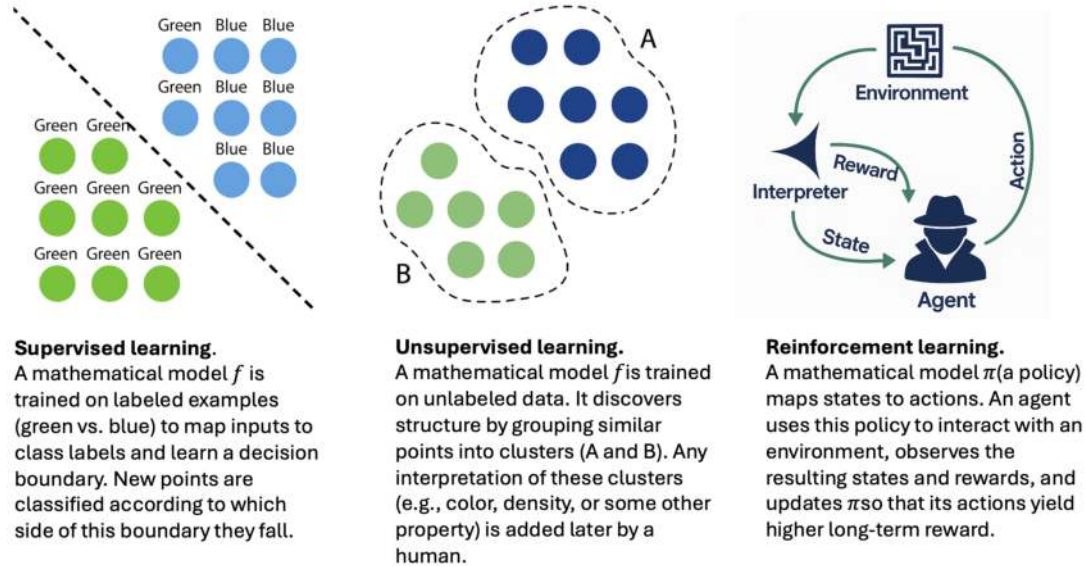


Figure 28: Simple illustration of *Optimization Problems: Supervised Learning, Unsupervised Learning and Reinforcement Learning*

Definition 3.1.11 (Supervised Learning). *Supervised Learning* is an *Optimization Problem* applied to a *Mathematical Model* f_θ . The *Dataset* consist of input-output pairs $D = \{(x_i, y_i)\}_{i=1}^N$ drawn from a distribution

$p(x, y)$. The objective is to find parameters θ that minimize a loss function $\ell(f_\theta(x), y)$, which quantifies the error between model predictions and actual targets.

Definition 3.1.12 (Unsupervised Learning). *Unsupervised Learning* is an *Optimization Problem* where the *Dataset* consists only of inputs $D = \{x_i\}_{i=1}^N$ drawn from a distribution $p(x)$, without external labels. The parameters θ are determined by minimizing an objective of the form $\frac{1}{N} \sum_{i=1}^N \mathcal{L}(x_i; \theta)$, where $\mathcal{L}$ is a task-specific loss (e.g., negative log-likelihood, reconstruction error, or contrastive loss) used to discover underlying structures or representations.

Definition 3.1.13 (Reinforcement Learning). *Reinforcement Learning* is an *Optimization Problem* where an agent learns by interacting with an environment modeled as a Markov decision process $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$. The objective is to find a policy $\pi(a | s)$ that maximizes the expected discounted return $J(\pi) := \mathbb{E}_{\pi, P} [\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)]$. Unlike static datasets, the *Dataset* here consists of trajectories $(s_0, a_0, r_0, \dots)$ generated dynamically by the agent’s interactions.

Definition 3.1.14 (Dataset). A *Dataset* D provides the empirical evidence used by a *Training method*: it may be labeled or unlabeled, collected offline, generated online (e.g., RL, self-play), or produced by an augmentation pipeline.

Definition 3.1.15 (Calculation Infrastructure). *Calculation Infrastructure* for training is $C_T = (H, S, R)$, the available hardware H , software stack S , and resource constraints R used during model training.

Definition 3.1.16 (Deep Learned Algorithm). A *Deep Learned Algorithm* is the trained, fully specified *Algorithm* $A = (G, \theta^*, \delta)$ obtained by applying one or more *Training methods* $T_1, \dots, T_m$ to *Datasets* $D_1, \dots, D_m$ under *Calculation Infrastructure* C_T . Here G is the evaluation order of the *Function Graph*, θ^* are the learned parameters (including any non-gradient state, e.g., BatchNorm statistics), and δ is a fixed decoding/evaluation policy. When δ is deterministic (e.g., greedy), we write simply $A = f_{\theta^*}$.

3.1.2 Definition: Transformer

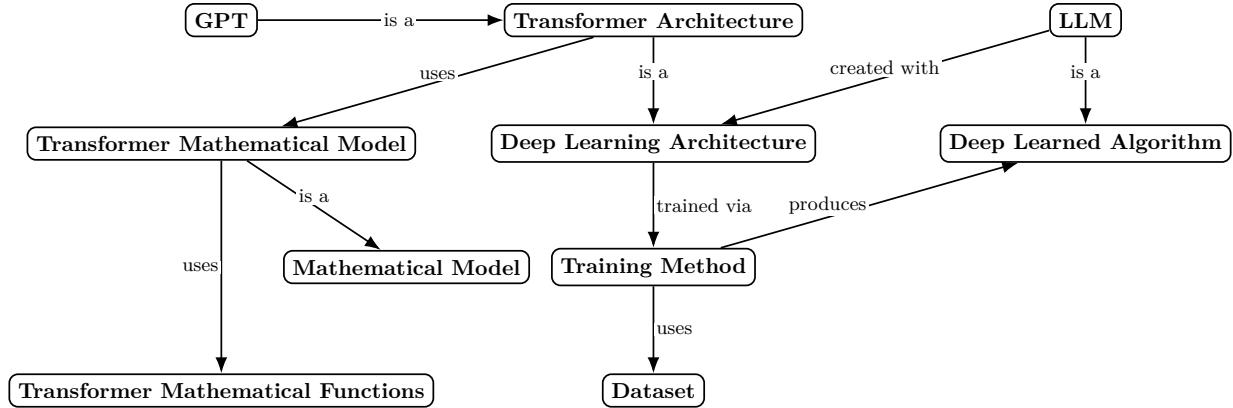


Figure 29: Transformer ecosystem concepts and their relationships (interactive links to definitions).

Definition 3.1.17 (Transformer Architecture). A *Transformer Architecture* is a *Deep Learning Architecture* that uses the *Transformer Mathematical Model*.

Definition 3.1.18 (Transformer Mathematical Model). A *Transformer Mathematical Model* is a *Mathematical Model* $M = (G, \lambda)$ where G is a *Function Graph* whose nodes are *Functions* that are defined *Transformer Mathematical Functions* implementing: (i) token embedding and output projection, (ii) positional encoding, (iii) masked multi-head (self-/cross-) attention, (iv) position-wise feedforward mappings, (v) residual connections and layer normalizations. The edges of G encode either an encoder–decoder or decoder-only dataflow; the wiring and hyperparameters (e.g., L , d_{model} , h , d_{ff} , masking policy, tokenization/positioning scheme) are fixed by λ . By *Mathematical Model*, M induces a family $\{f_\theta\}$ of *Functions* with learnable parameters θ .

3.1.2.1 Model Dimensions and Notation

To aid clarity, Tables 10 and 11 summarize the key symbols, dimensions, and matrices used throughout the Transformer architecture. Table 10 lists the main model-level hyperparameters and their typical magnitudes in large-scale language models (such as GPT-4), while Table 11 defines the principal matrices and intermediate representations used in the mathematical formulation of attention and embedding operations.

Table 10: Core symbols and typical values in Transformer architectures.

Symbol	Description	Example (GPT-4 scale)
$ V $	Vocabulary size (number of unique tokens)	$\sim 50,000$
n	Sequence length (number of input tokens)	$\sim 8,192\text{--}32,768$
d_{model}	Model or embedding dimension	$\sim 12,288$
h	Number of attention heads	~ 96
$d_k = d_v$	Dimension of keys, queries, and values per head	~ 128
L	Number of Transformer layers (stacked attention blocks)	~ 96

Table 11: Key matrices and intermediate representations in a Transformer block (row = token, column = feature).

Symbol	Description	Dimensions
T	One-hot token matrix representing discrete input tokens	$n \times V $
W^E	Token embedding matrix (maps vocabulary to model space)	$ V \times d_{\text{model}}$
E_{token}	Token embeddings (TW^E)	$n \times d_{\text{model}}$
P_{pos}	Positional encodings (sinusoidal or learned)	$n \times d_{\text{model}}$
X	Input representation ($\sqrt{d_{\text{model}}} TW^E + P_{\text{pos}}$)	$n \times d_{\text{model}}$
W_i^Q, W_i^K, W_i^V	Learned projection matrices for attention head i	$d_{\text{model}} \times d_k$ (or d_v)
Q	Query matrix (XW_i^Q)	$n \times d_k$
K	Key matrix (XW_i^K)	$n \times d_k$
V	Value matrix (XW_i^V)	$n \times d_v$
$\text{Attention}(Q, K, V)$	Single-head attention output ($\text{softmax}(QK^\top / \sqrt{d_k})V$)	$n \times d_v$
W^O	Output projection combining all heads	$(hd_v) \times d_{\text{model}}$
$f_{\text{MHA}}(X)$	Multi-head attention output (before residual add)	$n \times d_{\text{model}}$
$U = f_{\text{LN}}(X + f_{\text{MHA}}(X))$	Post-attention normalized state (input to FFN)	$n \times d_{\text{model}}$
$f_{\text{FFN}}(U)$	Feed-forward sublayer output (before residual add)	$n \times d_{\text{model}}$
$Y = f_{\text{LN}}(U + f_{\text{FFN}}(U))$	Output of one Transformer block	$n \times d_{\text{model}}$

Intuitive Breakdown. For clarity:

Step	Meaning	Shape
XW_i^Q, XW_i^K, XW_i^V	Linear projections of input for head i	$n \times d_k, n \times d_k, n \times d_v$
$(XW_i^Q)(XW_i^K)^\top / \sqrt{d_k}$	Pairwise similarity between tokens	$n \times n$
$\text{softmax}(\cdot)$	Converts similarities to attention weights	$n \times n$
Multiply by XW_i^V	Weighted combination of values	$n \times d_v$
$\bigoplus_{i=1}^h H_i$	Concatenate head outputs	$n \times (hd_v)$
Multiply by W^O	Mix head outputs back into model dimension	$n \times d_{\text{model}}$

Definition 3.1.19 (Transformer Mathematical Functions). The *Transformer Mathematical Functions* are the *Functions* that instantiate the node types of a Transformer *Function Graph*. For a token sequence $x = (x_1, \dots, x_T)$ and width d_{model} :

3.1.2.2 Input Representation

The input is formed by combining token embeddings and positional encodings. Let $X \in \mathbb{R}^{n \times d_{\text{model}}}$ be the sequence input:

$$X = \sqrt{d_{\text{model}}} (E_{\text{token}}) + P_{\text{pos}} = \sqrt{d_{\text{model}}} (TW^E) + P_{\text{pos}}, \quad (1)$$

where $T \in \{0, 1\}^{n \times |V|}$ is the one-hot token matrix representing the input sequence of n tokens over a vocabulary of size $|V|$, and $W^E \in \mathbb{R}^{|V| \times d_{\text{model}}}$ is the learned embedding weight matrix mapping each vocabulary item to a d_{model} -dimensional vector.

The positional encodings $P_{\text{pos}} \in \mathbb{R}^{n \times d_{\text{model}}}$ are defined using sinusoidal functions of varying frequencies:

$$P_{\text{pos}, 2i} = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right), \quad (2)$$

$$P_{\text{pos}, 2i+1} = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right), \quad (3)$$

where pos is the position index in the sequence and i is the dimension index.

3.1.2.3 Multi-Head Attention

Let h denote the number of attention heads. For each head $i \in \{1, \dots, h\}$, we define learned projection matrices $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$, and an output projection $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

Each attention head computes:

$$\text{Attention}_i = \text{softmax}\left(\frac{(XW_i^Q)(XW_i^K)^\top}{\sqrt{d_k}}\right)(XW_i^V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (4)$$

The overall multi-head attention operation is then given by:

$$f_{\text{MHA}}(X) = \left(\bigoplus_{i=1}^h \text{Attention}_i\right)W^O, \quad (5)$$

where $\bigoplus$ denotes concatenation along the feature dimension (equivalent to the Concat operation in the [59]).

Compact Definition (Self-Attention Case). When $Q = K = V = X$:

$$f_{\text{MHA}}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (6)$$

where $\text{head}_i = \text{softmax}\left(\frac{XW_i^Q(XW_i^K)^\top}{\sqrt{d_k}}\right)XW_i^V$.

Cross-Attention Variant. In encoder-decoder attention, where queries and key-value pairs come from different sources:

$$f_{\text{MHA}}(Q, K, V) = \left(\bigoplus_{i=1}^h \text{softmax}\left(\frac{(QW_i^Q)(KW_i^K)^\top}{\sqrt{d_k}}\right)(VW_i^V)\right)W^O. \quad (7)$$

Feedforward and normalization. Let $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ and $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$.

$$f_{\text{FFN}}(x) = \phi(xW_1 + b_1)W_2 + b_2, \quad (8)$$

$$f_{\text{LN}}(x) = \frac{x - \mu(x)}{\sqrt{\sigma^2(x) + \epsilon}}\gamma + \beta, \quad (9)$$

Layer composition (Pre-LN, standard for GPT/LLMs). In modern Large Language Models (e.g., GPT series, Llama), Layer Normalization is typically applied *before* the sublayers to improve training stability at scale:

$$U = X + f_{\text{MHA}}(f_{\text{LN}}(X)), \quad (10)$$

$$Y = U + f_{\text{FFN}}(f_{\text{LN}}(U)), \quad (11)$$

$$\mathcal{T}_\theta(X) = Y. \quad (12)$$

$$\text{Block}_\ell(X^{(\ell-1)}) = X^{(\ell-1)} + f_{\text{MHA}}^{(\ell)}\left(f_{\text{LN}}(X^{(\ell-1)})\right) + f_{\text{FFN}}^{(\ell)}\left(f_{\text{LN}}\left(X^{(\ell-1)} + f_{\text{MHA}}^{(\ell)}\left(f_{\text{LN}}(X^{(\ell-1)})\right)\right)\right) \quad (13)$$

$$f_{\text{Transformer}}(X^{(0)}) = \left(\bigcirc_{\ell=1}^L \text{Block}_\ell\right)(X^{(0)}) = X^{(L)}. \quad (14)$$

3.1.2.4 Generative Pretrained Transformer (GPT)

Definition 3.1.20 (GPT). A Generative Pretrained Transformer (*GPT*) is a *Transformer Mathematical Model* that has decoder-only *Function Graph*, pretrained primarily with a next-token prediction *Training method* on large-scale text *Datasets* (optionally followed by further stages, such as instruction tuning or RLHF). At inference time, a GPT maps input token sequences to output token distributions under a fixed decoding policy (e.g., greedy or sampling) for text generation and related tasks.

Remark 3.1.21 (GPT illustration). Figure 30 illustrates the *GPT* function graph.

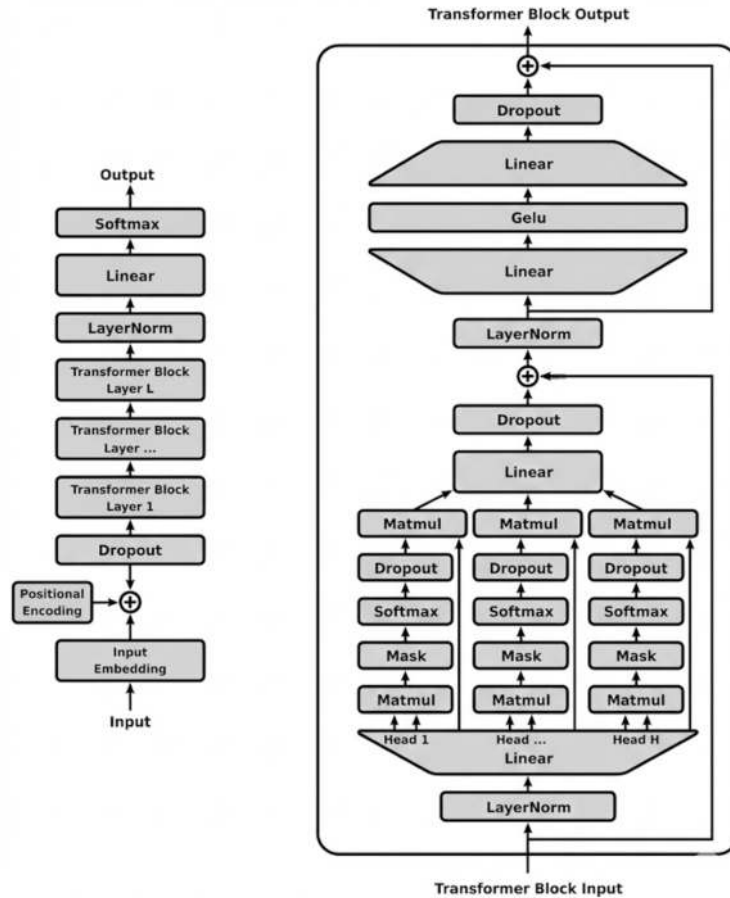


Figure 30: **Left:** Block-level view of a *GPT* that is a *Transformer Mathematical Model* with L layers from input to output. Dataflow: input tokens are embedded, combined with positional encodings, passed through a stack of L decoder Transformer Blocks, layer-normalized, and mapped by a final linear+softmax head to token probabilities.

Right: Internal structure of a single Block_ℓ with nodes drawn from *Transformer Mathematical Functions*: masked multi-head self-attention f_{MHA} with H heads and a causal mask, followed by a position-wise feed-forward network f_{FFN} (GeLU). Both sublayers use residual connections ($\oplus$), LayerNorm f_{LN} , and dropout.

Definition 3.1.22 (LLM). Large Language Model (*LLM*) is a *Deep Learned Algorithm* created using *Transformer Mathematical Model*, such as *GPT*, typically using multiple *Training methods* (e.g., pretraining, fine-tuning, RLHF) on large-scale text *Datasets*.

Remark 3.1.23 (GPT as an LLM family). Under Definition 3.1.22, GPTs are a specific family of *LLMs* whose *Mathematical Model* is a decoder-only *Transformer Mathematical Model*. In practice, most contemporary *LLMs* deployed today follow this GPT-style pattern (decoder-only Transformer with next-token pretraining), which is why “LLM” and “GPT-like model” are often used interchangeably in informal discussion.

3.2 LLM Agents

This section formalizes the concept of an LLM Agent, distinguishing it from a standalone *LLM* by the addition of state (memory), executable capabilities (tools), and a control loop.

Definition 3.2.1 (LLM Agent). An *LLM Agent* is a computational system $A = (M, \mathcal{T}, \mathcal{S}, \Pi)$ where:

1. M is a *LLM* acting as the reasoning core.
2. $\mathcal{T} = \{f_1, \dots, f_k\}$ is a set of *Tools* available for execution.
3. $\mathcal{S}$ is a dynamic memory or context state.
4. Π is a control policy (or planning *Algorithm*) that iteratively prompts M to select functions from $\mathcal{T}$ or update $\mathcal{S}$ to satisfy a user request.

Definition 3.2.2 (Tool). A *Tool* is a specific *Function* $f : X \rightarrow Y$ exposed to the *LLM Agent*. Unlike the probabilistic output of the *LLM*, a tool typically executes deterministic logic (e.g., a calculator, a database query, or a API call) to observe or manipulate an external environment.

Definition 3.2.3 (Agent Memory). Agent Memory $\mathcal{S}$ is a structured *Dataset* maintained during the lifecycle of the *LLM Agent*. It is composed of:

- *Short-term memory*: The immediate context window of the *LLM*, containing the current prompt, conversation history, and recent tool outputs.
- *Long-term memory*: An external retrieval system (e.g., a vector database) that allows the agent to fetch relevant information $x \in \mathcal{S}$ to augment the input context of M .

4 PHASE I: DESIGN & DEVELOPMENT AND DEMONSTRATIONS

Phase I investigates how early Web services can be made *intelligent* through agent-mediated interaction and shared ontologies. The central design choice is to separate the *intentional force* of messages (interaction protocol, e.g., FIPA performatives) from *domain payload* (application-specific content). This separation enables run-time adaptation at the conversation level and supports domain-specific negotiation without hard-coding process logic. The idea is demonstrated in three settings: (i) conversation-level adaptation to service descriptions (Section 4.1); (ii) ontology-grounded negotiation in a B2B purchasing scenario (Section 4.2); and (iii) context-aware matching that connects otherwise separate shared ontologies (Section 4.3). Throughout, shared vocabularies and protocol conformance are the key enablers of interoperability and reuse. This phase includes demonstrations from three projects²¹ reported in part in [1, 2, 3, 4, 5, 6, 7, 8]. Section 4.1 details interaction-protocol-based service adaptation (*ConStrip*); Section 4.2 develops a registry-centered B2B purchasing and bidding scenario that operationalizes shared ontologies and negotiation; and Section 4.3 (*Dynamos*) shows how context information links disparate descriptions to enable robust service discovery and matching.

To orient the reader in the overall thesis structure and DSRP flow, see Table 3 (DSRP process sequence linked to Phases and Structure of this Thesis) and Section 1.2 (Phase I Problem and Motivation: Early Web Evolution — From Information Space to Service Platform).

4.1 Interaction Protocol Based Service Adaptation

Problem Identification, Research Questions and Objectives of the Solution

The problem identification, research questions and objectives for this work are presented in Section 1.2.1. The research methodology is presented in Section 1.1.

Context

This work examines how Semantic Web formalisms can be combined with software-agent protocols to enable *run-time* interoperability in a B2B setting. The motivating scenario is construction "expediting", where a general contractor must verify schedule and delivery information across a multi-tier network of subcontractors, suppliers, and carriers. Instead of brittle, hard-coded integrations or ad-hoc phone/email coordination, the approach describes interaction protocols in RDF/OWL against a shared interaction-protocol ontology. An expeditor agent for the contractor ingests these RDF-serialized (FIPA-based) protocol descriptions at run time and executes them via a protocol state machine, allowing it to converse with previously unknown partners while maintaining a machine-traceable audit trail.

Beyond the immediate construction use case, the aim is to test whether this run-time adaptation can reduce coordination delay, curb disputes, and improve accountability when many independent organizations must work together. By letting partners publish "how to talk to me" in a standard, machine-readable form, the contractor can verify claims directly at the source and escalate inconsistencies early, without bespoke integrations each time. The study also probes the practical boundaries of this idea—where human judgment is still needed, what must be standardized across participants, and how far such lightweight agreements can carry in messy, real-world projects.

4.1.1 Design and Development

The IPADAPT main Interaction Protocol Based Service Adaptation artefact comprises the following artefacts:

A.I.1.1 Process Flow. A formalisation of the expediting process states and the contractor's interaction strategy (see Section 4.1.1.1).

A.I.1.2 Interaction Protocol Ontology. The core ontology (Initiator, Participant, State, Option, Message) that gives meaning to protocol descriptions (see Section 4.1.1.2).

A.I.1.3 Interaction Protocol Serializations. RDF/OWL examples and FIPA-based serialisations that show how protocols and message contents are expressed for machine ingestion (see Section 4.1.1.3).

²¹The demonstrations were implemented in projects co-funded by Tekes, VTT, and industrial parties; see <http://www.vtt.fi/>.

A.I.1.4 Adapting to Interaction Protocol Descriptions. The protocol state-machine and adaptation logic used by the expeditor agent to interpret and execute external protocol descriptions (see Section 4.1.1.4).

4.1.1.1 Artefact: Process Flow

The states of the expediting process from expeditor's point of view are depicted in Figure 31. In the process, the expeditor knows initially only how to interact with the subcontractor, and therefore it has to learn how to do so with other participants. This is no problem if the actors are humans but it is a problem for software agents. The approach for this problem that is presented in this demonstration is to define interaction protocol descriptions, and to design the expeditor agent so that it can process the descriptions, and adapt its behavior accordingly.

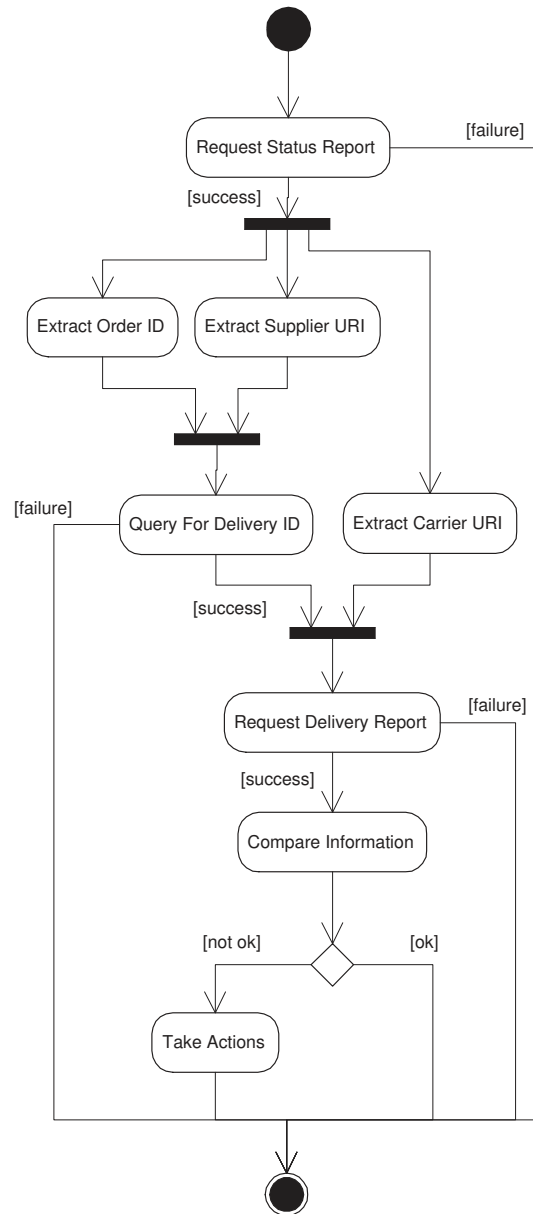


Figure 31: Progression of the expediting process from the contractor's point of view.

Collaboration is based on interaction protocol descriptions²² provided by the participants. In order to enable the adaptation, the descriptions conform to mutually agreed interaction protocol ontology that is introduced in [293]. The expeditor agent, if unaware beforehand how to communicate with a participant, can download interaction protocol descriptions provided by the participant and adapt its behavior accordingly.

In this demonstration, the interaction between the contractor and the subcontractor is modeled using FIPA Request [294] interaction protocol. The contractor requests the subcontractor to send the status report. The subcontractor then tries to perform the requested action. In the successful case, the subcontractor informs the contractor that the action is performed. If something goes wrong, the subcontractor sends a failure message.

Assume the subcontractor has ordered a steel delivery from the supplier with the help of an external carrier, but the delivery has not yet arrived. It nevertheless includes the order in the status report before sending it to the contractor. In this demonstration, the contractor wants to keep track of the whole process. Thereby, it contacts the supplier and the carrier to check the status of the order and the delivery. The contractor first requests the subcontractor to send the status report. By examining it, as depicted in Figure 31, it finds out the addresses (URIs) of the agents representing supplier and carrier, as well as the identifier of the steel order²³. After retrieving this information the contractor queries the supplier for the identifier of the steel delivery with the order identifier as the parameter. The supplier provides the contractor with it, and the contractor uses it next as a parameter for requesting a delivery report from the carrier.

Since the contractor has not collaborated with the supplier and the carrier before, being able to do so at this time presupposes adaptability. The next subsection presents interaction protocol descriptions in more detail.

4.1.1.2 Artefact: Interaction Protocol Ontology

Interaction protocols specify ordered sets of messages to be exchanged between conversating agents. In this demonstration, the interaction protocols are described using RDF [295], a language designed for describing any resource in the Semantic Web. As mentioned, interaction protocol descriptions conform to interaction protocol ontology [293]. Core concepts of that ontology are depicted in Figure 32. Basically, all interaction protocols have one initiator and one or more participants. Progress of the protocol is defined with states following each other. More specifically, states have options to choose from, and the chosen option determines which state to shift to. Options can have messages associated with them that are sent between the participating agents.

4.1.1.3 Artefact: Describing Interaction Protocols Using RDF

Among other interactions, the supplier knows how to respond to queries for selected things. In this scenario, the contractor asks the supplier for the identifier of the steel delivery using the order identifier found in the status report as a parameter. In order to do so, it adapts to an RDF serialization of FIPA Query [296] with parameters. This protocol serialization is provided by the supplier. The following excerpts of the serialization do not capture the entire protocol, but merely some pivotal points. Description of the interaction protocol class is as follows:

```

1 <rdf:Description rdf:about="%this;queryDelivId">
2   <rdf:type rdf:resource="%ip;IP" />
3   <ip:hasInitiator rdf:resource="%this;contractor" />
4   <ip:hasParticipant rdf:resource="%this;supplier" />
5   <ip:hasProgress rdf:resource="%this;queryProgress" />
6 </rdf:Description>

```

The abbreviation *ip*; refers to the ip-namespaces containing the interaction protocol ontology. The interaction protocol class IP contains properties called *hasInitiator*, *hasParticipant*, and *hasProgress*, which are in this serialization given values denoting to RDF-descriptions appearing in the same file. The agents taking part in the protocol are defined in the following way:

```

1 <rdf:Description rdf:about="%this;contractor">
2   <rdf:type rdf:resource="%ip;Initiator" />
3 </rdf:Description>
4
5 <rdf:Description rdf:about="%this;supplier">

```

²²Interaction protocols were described in Section 2.4.6

²³Note that this examination, i.e., how the contractor extracts the desired information from the status report, is not in the focus of this demonstration, and it could be performed by the software agent itself, or by a human being.

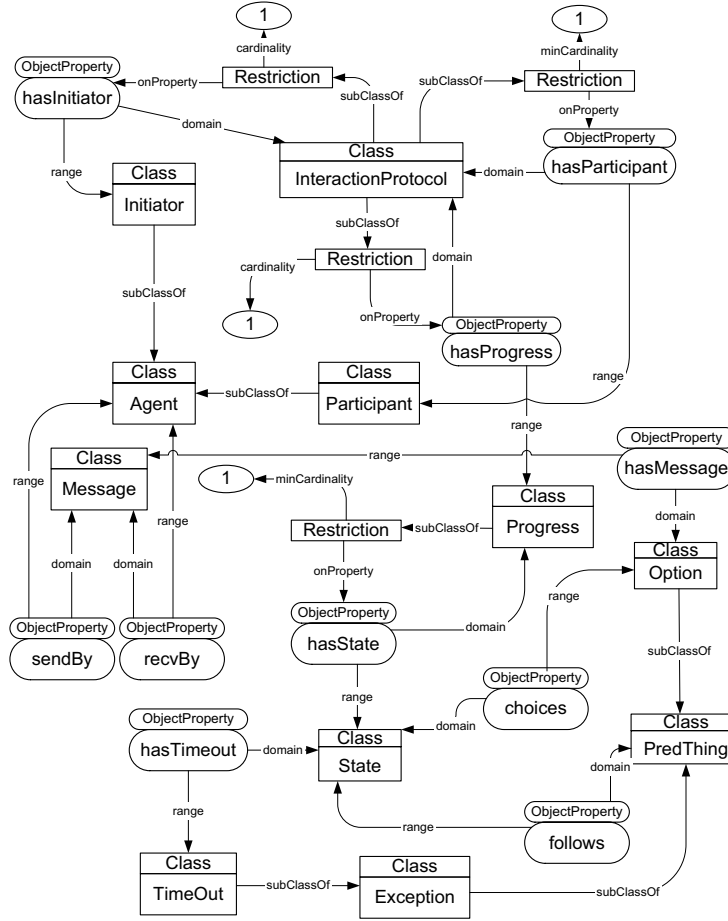


Figure 32: Core concepts of the interaction protocol ontology [293].

```

6   <rdf:type rdf:resource="#ip;Participant" />
7 </rdf:Description>

```

The agents have to comprehend the interaction protocol ontology in order for the adaptation of the individual protocol descriptions to take place. For example, the agents have to understand the basic difference between the Initiator and the Participant, i.e., that it is the Initiator that starts the protocol by sending the first message. Definition of the *queryProgress* class is serialized as follows:

```

1 <rdf:Description rdf:about="#this;queryProgress">
2   <rdf:type rdf:resource="#ip;Progress" />
3   <ip:hasState rdf:resource="#this;start" />
4   <ip:hasstate rdf:resource="#this;replySupplier" />
5   <ip:hasState rdf:resource="#this;end" />
6 </rdf:Description>

```

Of the three states of this protocol, start and end are found in every protocol instance and knowledge of their semantics is presupposed of the agents adapting to the protocol descriptions. The agent has to know, for example, that the first state of the protocol is start. Below are serialized two of these three states, namely *start* and *replySupplier*.

```

1 <rdf:Description rdf:about="#this;start">
2   <rdf:type rdf:resource="#ip;State" />
3   <ip:choices rdf:resource="sendDelivquery" />
4 </rdf:Description>

```

```

5
6 <rdf:Description rdf:about="%this;replySupplier">
7   <rdf:type rdf:resource="%ip;State" />
8   <ip:choices rdf:resource="%this;sendSupplInform" />
9   <ip:choices rdf:resource="%this;sendSupplFailure" />
10 </rdf:Description>

```

Every state has one or more options. These are denoted by *ip:choices* tags. The only option of *start* is *sendDelivQuery*, while *replySupplier* has two options. The actual choice between these depends on the success of the supplier's ability to answer to the query. Moving on, the options have their own serializations, which contain the possible messages to be sent, as well as the target states for the protocol to shift to. Below are two options, namely *sendDelivQuery* resulting from the activation of the start state and *sendSupplInform* resulting from the positive response by the supplier to the initial query.

```

1 <rdf:Description
2   rdf:about="%this;sendDelivQuery">
3   <rdf:type rdf:resource="%ip;Option" />
4   <ip:hasMessage rdf:resource="%this;delivqueryRef" />
5   <ip:follows rdf:resource="%this;replySupplier" />
6 </rdf:Description>
7
8 <rdf:Description rdf:about="%this;sendSupplInform">
9   <rdf:type rdf:resource="%ip;Option" />
10  <ip:hasMessage rdf:resource="%this;supplInform" />
11  <ip:follows rdf:resource="%this;end" />
12 </rdf:Description>

```

The target state is denoted by the *ip:follows* tag. For example as the above serialization shows, *sendDelivQuery* has the *replySupplier* as its target state. It also entertains one message, namely *delivQueryRef*, which is presented below. *sendSupplInform*, instead, has *end* as its target state, and *supplInform* as the message associated with it. The message serializations contain the information on who sends and who receives the messages, as well as references to the possible message contents.

```

1 <rdf:Description rdf:about="%this;delivqueryRef">
2   <rdf:type rdf:resource="%fipa;queryRef" />
3   <ip:sendBy rdf:resource="%this;contractor" />
4   <ip:recvBy rdf:resource="%this;supplier" />
5   <fipa:hasContent rdf:resource="%this;queryContent" />
6 </rdf:Description>
7
8 <rdf:Description rdf:about="%this;supplFailure">
9   <rdf:type rdf:resource="%fipa;Failure" />
10  <ip:sendBy rdf:resource="%this;supplier" />
11  <ip:recvBy rdf:resource="%this;contractor" />
12 </rdf:Description>

```

The above message definitions contain the senders and the receivers of the messages, denoted by the respective *sendBy* and *recvBy* tags. Additionally, a new namespace called *fipa* is introduced. It denotes a location containing the FIPA extension to the interaction protocol ontology, which is described in detail in [293]. Among other things specific to FIPA ACL [179], the extension allows expressing message contents. Below is serialized the *queryContent* class, which expresses the thing to be queried, in this case the identifier of the delivery report.

```

1 <rdf:Description rdf:about="%this;queryContent">
2   <rdf:type rdf:resource="%fipa;Content" />
3   <fipa:expression>
4     String ORDER_ID = "http://www.sscompany.com/"
5       + "orderReports#steelOrd123";
6
7     String query = "SELECT ?x"
8       + " WHERE (?y, <rdf:type>,"
9       + " <report:orderReport>),"
10      + " (?y, <report:deliveryReport>, ?x)"
11      + " AND ( ?y eq x +ORDER_ID +>)"

```

```

12         +" USING " +nameSpaces;
13     </fipa:expression>
14 </rdf:Description>

```

The message content is expressed inside the `fipa:expression` tag. This phrase expresses the RDQL ²⁴ [297] query for a delivery identifier (?x) with the order identifier (?y) as the parameter. Order identifier, as depicted in Figure 31, is found in the initial status report received from the subcontractor. Here it is assumed that the agents share a supply chain reports ontology, denoted by the report namespace, which captures the general structures of order and delivery reports. For example, the contractor knows that the order report can contain information about the delivery and thereby has reason to perform the above query.

With the delivery identifier that the contractor receives as a reply to the above query it proceeds to request the delivery report from the carrier. The protocol is described in the similar manner as the query and not included here. Eventually, once the contractor has received the delivery report, it can compare the information in it with the information in the original status report received from the subcontractor. Should the two reports contain conflicting information, it can take appropriate actions (see Figure 31). Software agents could take part in performing the comparison, for example, by notifying their owner about mismatches between the reports. However, considering these details is outside the scope of this demonstration.

4.1.1.4 Artefact: Adapting to Interaction Protocol Descriptions

The only agent adapting to the interaction protocol descriptions in the above scenario is the contractor's expeditor. Other agents of the scenario have knowledge of the interaction protocols beforehand. Figure 33 depicts the state machine which the adapting agent uses in the protocol progresses.

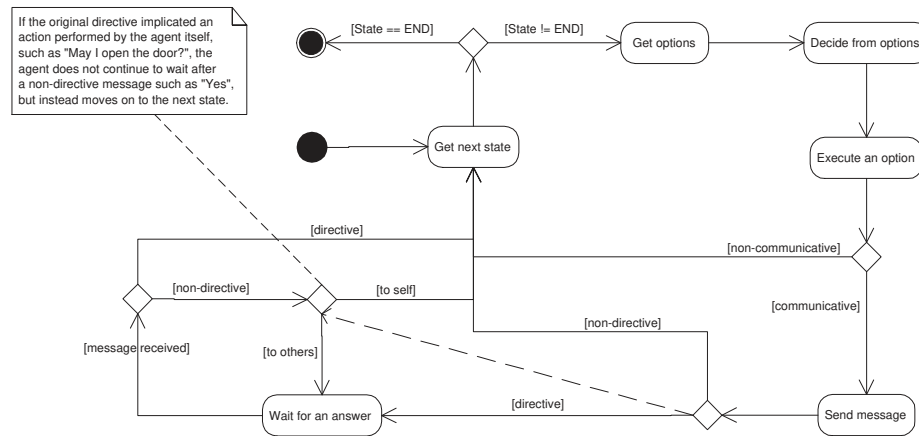


Figure 33: The protocol state machine.

As mentioned, knowledge of states where the protocol starts and ends is assumed to be known by every agent. Being the initiator of the protocol, the expeditor first searches for the start state. From that state it extracts different options. Based on its current goals, it chooses one option and executes it. Note that agents' goals have an effect on the decision are excluded from interaction protocol descriptions.

Executing an option that does not entail message sending is called a non-communicative option. A communicative option, instead, entails sending a message. If such message is directive, the sender of that message expects a reply to that message. Additionally, with directive messages the sender aims to get the receiver to do something, such as perform an action or answer a question (with the exception of directives "directed" to oneself). As an example consider FIPA Request [294]; after sending the request message the initiator waits for an answer to it from the participant before the protocol shifts to the next state. For a non-directive message, instead, there are no answers and the protocol moves immediately to the next state. Examples are all the other messages in FIPA Request, namely refuse, agree, failure, and inform ²⁵.

²⁴RDF Data Query Language

²⁵All the FIPA performatives were depicted in Figure 18.

Unlike agree, all other non-directive messages in FIPA Request protocol are followed by the end state. Instead, agree is followed by the one where (after trying to perform the requested action and based on its results) the participant sends the appropriate message to the initiator. Note that excluded from Figure 33 are canceling and exception handling mechanisms. An agent participating in a protocol has the possibility of canceling the protocol at any point. Also, exceptions can arise at any point of the protocol. Contractor's expeditor agent is able to adapt its behavior to conform to the interaction protocols described and provided by other process participants by combining the following information: Knowledge of the interaction protocol ontology; knowledge of the flow of a protocol as depicted in Figure 33; knowledge of utilized domain ontologies, such as the abovementioned supply chain reports ontology.

4.1.2 Demonstration

Context: B2B Expedition Scenario This demonstration deals with the so-called expediting process that is part of a broader concept called supply chain management. Traditionally, performing an expediting process means paying visits to the premises of project participants, making phone calls, or exchanging emails for checking the status.

There are previous attempts, such as the one described in [42], for applying software agents in construction processes. A distinctive property in the approach presented in this section is the partly automation of the expediting process by applying interaction protocols to the work performed by the contractor's software agent that acts as an expeditor.

In the scenario, the contractor has initiated a project and wants to keep track of it. The contractor therefore expects to receive status reports of the project from the subcontractor(s) regularly. That helps the contractor to identify and avoid possible risks involved in the project, such as schedule delays and cost overruns.

However, often the contractor cannot rely solely on the information the subcontractor(s) provide in the status reports. A subcontractor can either intentionally conceal that the project is behind schedule, or some important piece of information might unintentionally be out of date in a status report. For verifying the information in a status report, the contractor's expeditor contacts the other participants directly. Thereby, the expeditor acts somewhat as a detective trying to dig up information from various sources and incorporate it into a complete picture.

With many project participants involved, direct interaction between the contractor and the rest can also help to avoid problem amplification. However, the communication can become heavy since large projects can include numerous subcontractors that can have multiple suppliers and carriers.

4.1.2.1 Protocol Description and Report Handling

As explained, all the interaction protocol descriptions and different reports sent between agents were based on common ontologies and represented in RDF. For handling these reports and protocol descriptions the Jena Semantic Web Framework [298] was used. Here is an example RDF from the report that the Carrier agent sends to the General Contractor:

```

1 <rdf:Description
2   rdf:about="http://www.crcompany.com/deliveryreports#delivery789">
3   <rdf:type rdf:resource="http://www.supplyreports.net/report#deliveryReport" />
4   <report:customer rdf:resource="http://www.crcompany.com/deliveryreports#sscompany" />
5   <report:provider rdf:resource="http://www.crcompany.com/deliveryreports#crcompany" />
6   <report:schedPickUp>20030716</report:schedPickUp>
7   <report:schedAtDestination>20030726</report:schedAtDestination>
8   <report:steelAmount>3500</report:steelAmount>
9 </rdf:Description>

```

4.1.2.2 Agent platform

In the scenario, every actor (Contractor, Subcontractor, Supplier, Carrier) is implemented as software agents on top of FIPA-compliant platform called JADE ²⁶ [168]. JADE provides basic functionality, such as message transport system, protocol templates and containers. There are some tools packaged with JADE that are helpful in the agent development, such as tools for monitoring and debugging agents. Figure 34 depicts the

²⁶Java Agent DEvelopment Framework was shortly described in Section 2.4.5.3

tool for communicating with the agents by a selected protocol and Figure 35 depicts the JADE platform with one agent selected for monitoring.

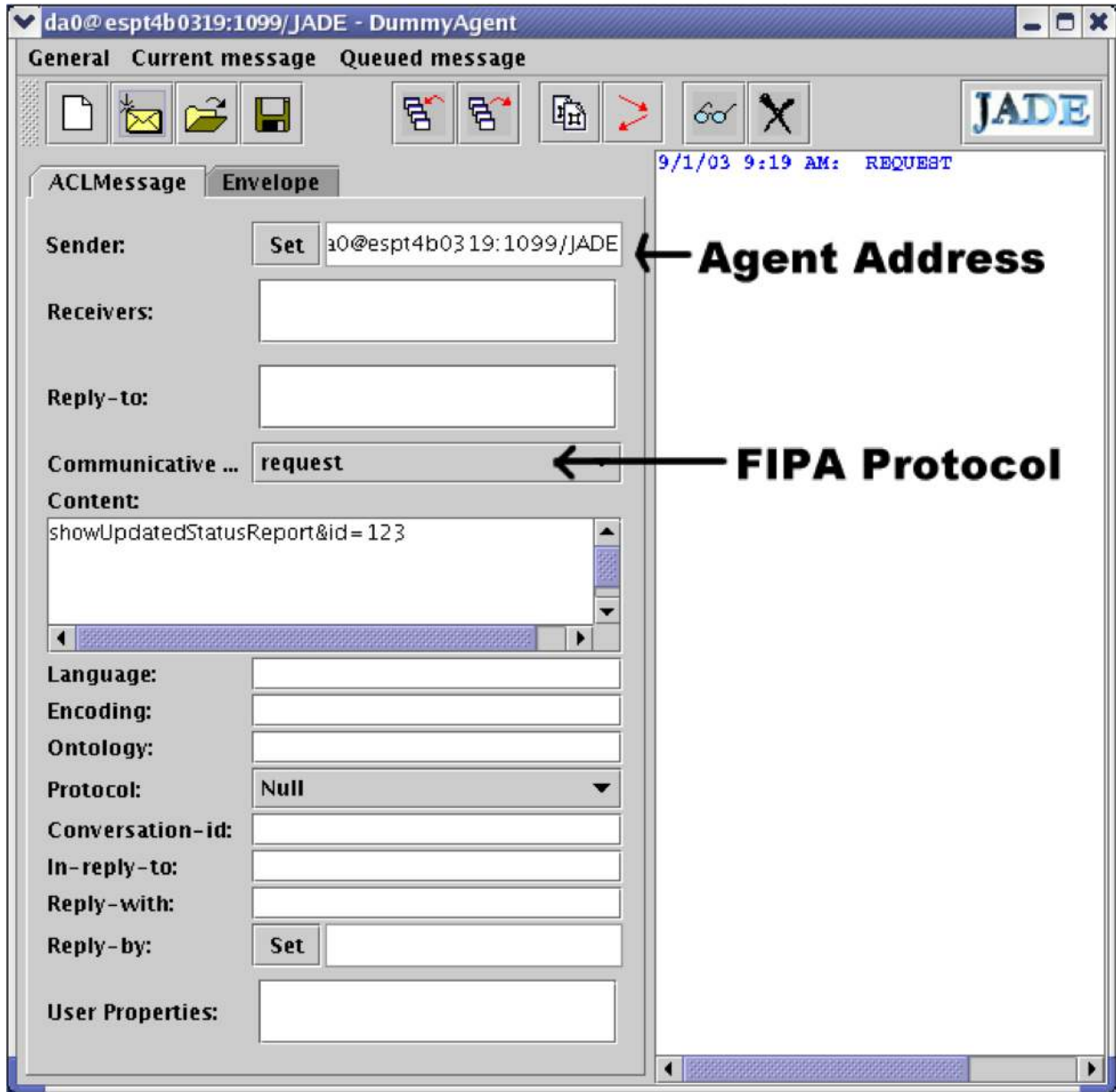


Figure 34: A tool for communicating with the software agents.

Implementation summary The prototype was implemented on a FIPA-compliant JADE platform, using Apache Jena for RDF parsing and reasoning. The codebase comprises original Java sources (9000 LoC) and RDF/OWL artefacts (protocol descriptions and domain ontologies). Counts were obtained with `cloc` and exclude third-party code.

4.1.2.3 Screenshots from the Demonstration

Figure 36 shows different agents in this scenario and their communication throughout the process. The different agents in Figure 36 are the red boxes: general contractor *gc0*, subcontractor *sc0*, steel supplier *ss1* and carrier *cr_type2*. The message types (*INFORM*, *AGREE* etc.) are specified by FIPA and the content of the messages are different reports specific for this demonstration.

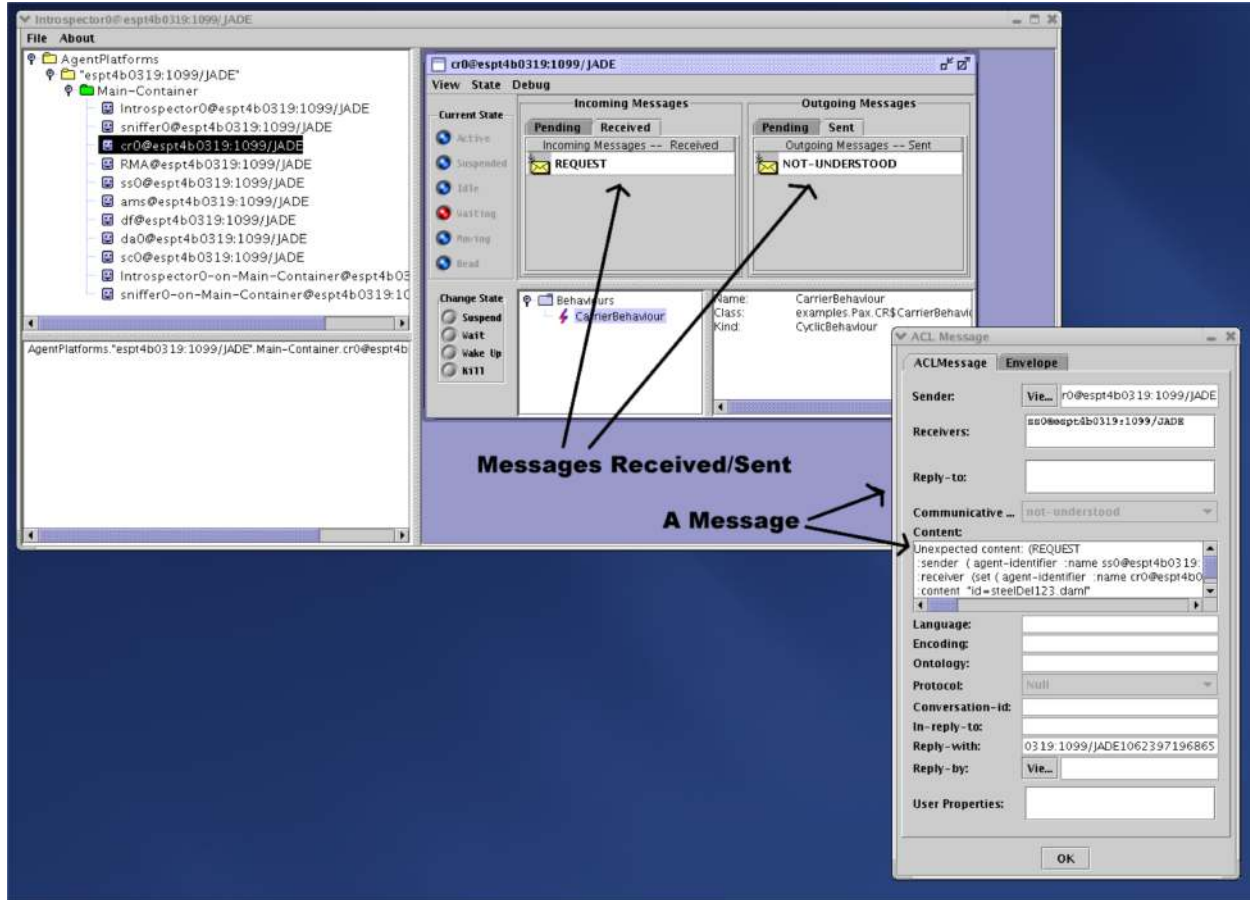


Figure 35: A tool for monitoring the state and information the agent receives and sends.

Figure 37 implements the process flow that was depicted in Figure 31. The process starts when general contractor requests the status report from the subcontractor and ends when general contractor has received all statements from the companies that are involved in the order.

The point of this demonstration and also Figure 37 is that in the beginning of the process the general contractor agent only knows how to communicate with the subcontractor agent. Therefore in the first stage of Figure 37 it directly asks for the status report. From the status report the contractor agent notices that subcontractor has used one supplier and one carrier to fulfill the order. As discussed previously, the contractor wants to verify the information in the status report received from subcontractor. This is done by directly contacting the agents that are mentioned in the status report.

Next, the contractor agent analyzes the status report and the first unknown agent is the steel supplier. The contractor agent first requests the supplier agent for the protocol description ("how do you communicate"). The supplier agent replies by sending the RDF description of the protocol how it communicates. The contractor agent creates a model from the description and tries to comprehend it with a state machine described in Figure 33. The contractor agent has parsed the order identifier from the status report and uses it to query the order report from the supplier agent.

The contractor agent repeats the same procedure with the carrier agent and in the end the contractor agent has three reports and it can automatically compare whether the information in the reports match or then it can be done by the contractor company personnel.

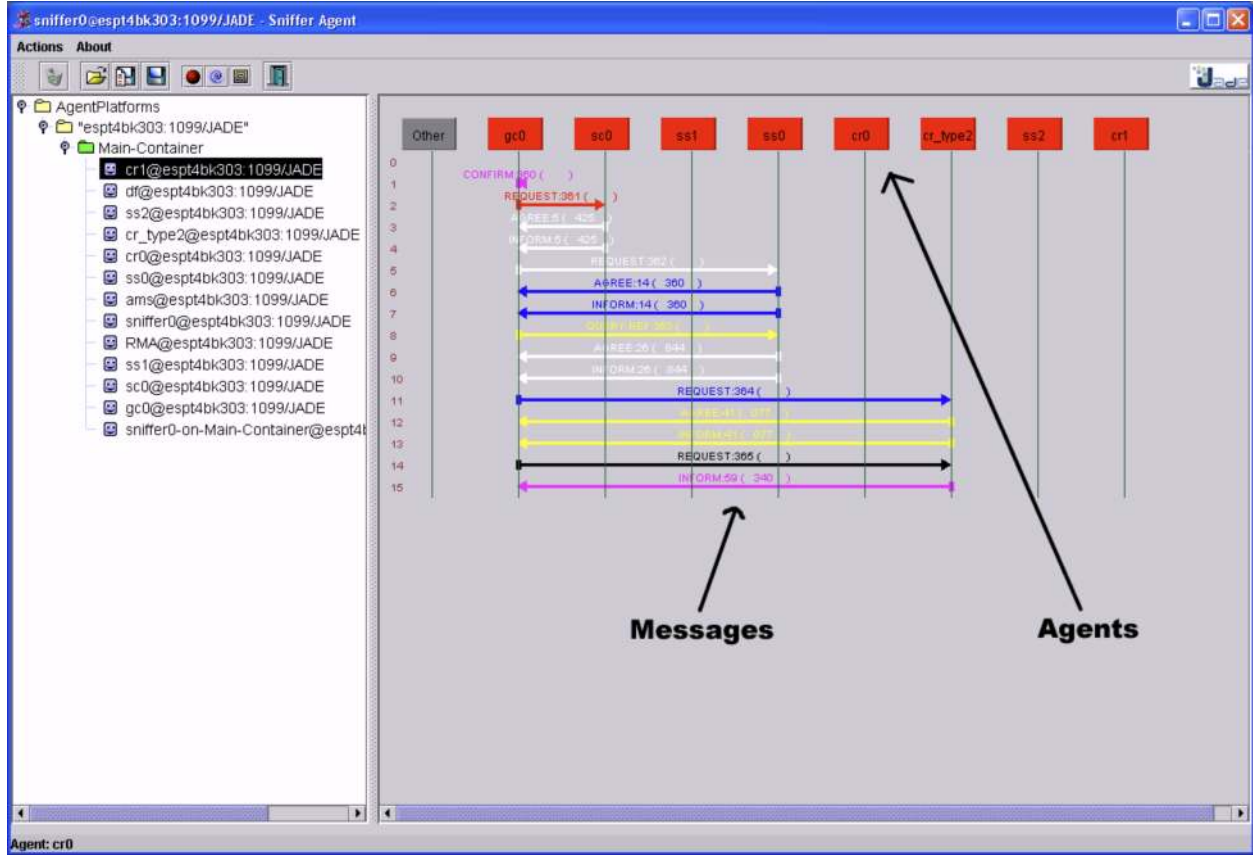


Figure 36: Agents and messages.

4.2 Ontology-Driven Software Agents for B2B Purchasing Automation

Problem Identification, Research Questions and Objectives of the Solution

The problem identification, research questions and objectives for this work are presented in Section 1.2.2. The research methodology is presented in Section 1.1.

Context.

In B2B sector, many processes have already been automated using communication networks. The massive growth of the Web has encouraged companies to use Web technologies and open architectures in the process automation. The next step in this "Web automation" will probably be Web Services that are not dynamically located, but rather based on traditional contracts between companies, and then implemented by system integrators. This step will take different amount of time for different business sectors. Many sectors have already done a lot of such integration of processes. However, for many this scenario where agents are aiding in contract negotiations seems to be science fiction.

In the purchasing process, a buyer needs some product or service ²⁷ and it negotiates with several potential companies. The product can be automatically mapped by a semi-intelligent register to a set of companies that are able to provide the products.

Today such purchasing processes can be very time-consuming and heavy for all the participating companies and there is potential if such processes could be partly automated. However, it is natural that it is not appropriate to automate everything. In this demonstration, the decision points between agents and company

²⁷the subject of the purchase can be some kind of service or product but from now on it is called just product for simplicity reasons

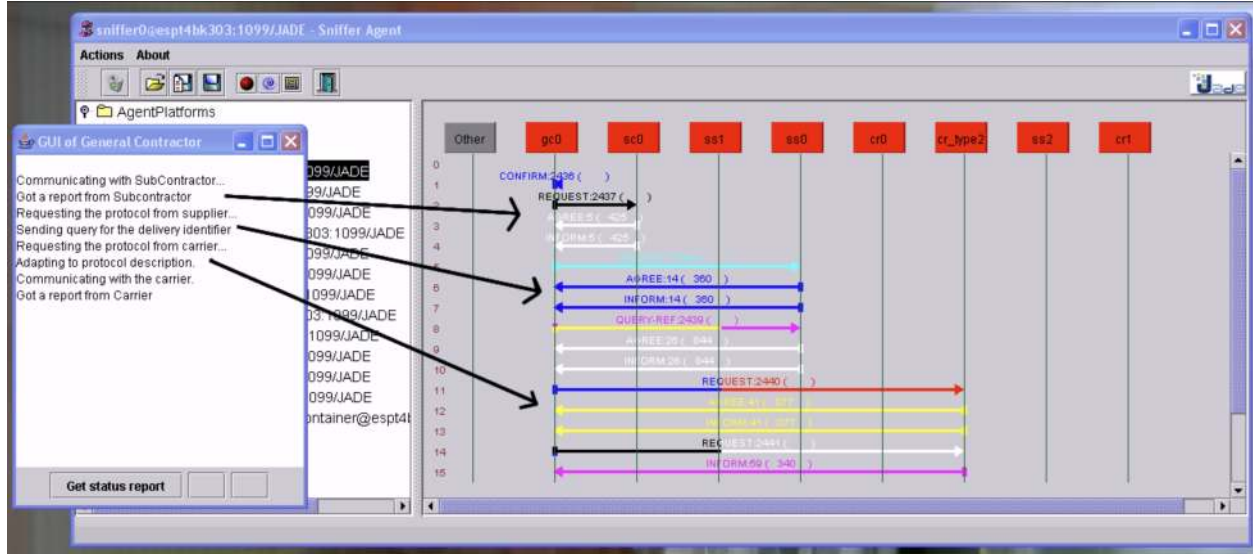


Figure 37: The process flow.

personnel are divided so that the routine tasks are done automatically by agents and the crucial decisions by the company personnel.

4.2.1 Design and Development

The CONSTRIIP main artefact Ontology-Driven Software Agents for B2B Purchasing Automation comprises of the following artefacts:

- A.I.2.1 Purchase Process Flow.** A concise formalisation of the RFQ, negotiation, and contract-making subprocesses that structures agent and human decision points (see Section 4.2.1.1).
- A.I.2.2 Decision Making.** The design that partitions routine automation from human oversight and specifies authorization/handoff rules for buyer and supplier agents (see Section 4.2.1.2).
- A.I.2.3 Data Models and Ontologies.** The proposal and product ontologies that define the RDF/XML message schemas used by agents to express RFQs, quotes, and contracts (see Section 4.2.1.3).
- A.I.2.4 Locating Services / Registry.** The semi-intelligent, category-based registry that maps product ontologies to candidate providers and supports agent discovery and browsing (see Section 4.2.1.4).

4.2.1.1 Artefact: Purchase Process Flow

This subsection shortly describes the purchasing process that consists of the three sub-processes that are presented in Figure 38: request for quote, quote negotiation, and making the contract. In the purchasing process, agents are aiding in many ways, but there are decision points where consultation of the company personnel is needed. The decision points are also presented in Figure 38 and they are further explained in the next subsection. All communication between agents is done by utilizing protocols standardized by FIPA [185].

Request for Quotes

At this first stage, the company personnel create a request for quote and sets some restrictions, such as price limit for the buyer agent. After that the buyer agent searches the companies that are supposed to be able to deliver the product from a registry. When the agent has found a set of companies that are able to deliver the product the buyer is looking for, it sends the request for quotes to the services that represent companies.

From the deliverer's perspective this step is straight forward process, where the agent checks whether the company has the product and checks the price from the current pricing list. With this information it automatically creates a quote that it sends to the buyer. This step could be from today's business sector: a regular Web Service with WSDL/SOAP description. In this demonstration, the same thing is done with FIPA Contract Net [187] interaction protocol and software agents.

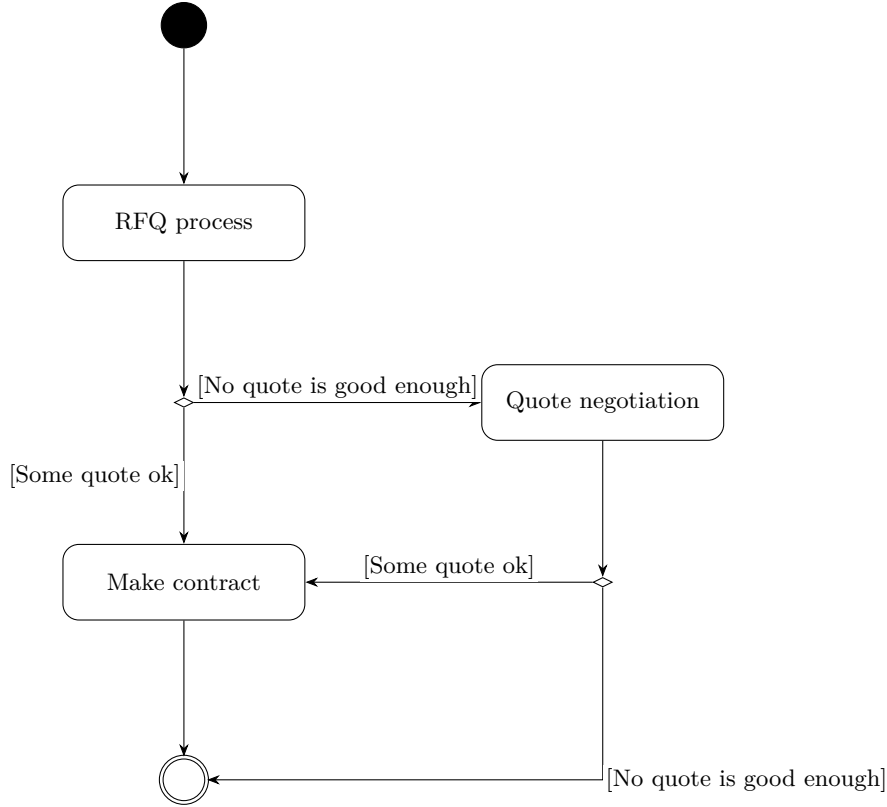


Figure 38: Overall RFQ process with negotiation and decision outcomes.

After the buyer agent has received the quotes it automatically compares them to the restrictions that it got from the company personnel. If at least one of the quotes matches the terms the agents directly goes on to negotiate a contract. If none of the quotes satisfy the terms the agent passes the decision to the company personnel.

Quote Negotiation As it can be seen from Figure 38, the quote negotiation step is optional. It is reached if none of the quotes received by the buyer agent satisfies the terms specified by the buyer company personnel. Either one of the quotes can be chosen to as a basis for a contract, or some quote can be modified and sent to a set of companies as a query with the following content: "Is this fine with you". This is done with the FIPA Propose [299] interaction protocol. If some of the companies agree to the new terms, the buyer agent chooses the respective company to make the contract with.

Making the Contract This stage is reached only if both parties have agreed the terms of delivery and terms of payment. After that a contract is formed according to a quote and the contract is signed by both sides. The buyer sends a contract based on the quote received from the delivering company and it answers with its signature. This is done with the FIPA Query [296] interaction protocol.

4.2.1.2 Artefact: Decision making

As mentioned before, it is not appropriate to automate everything in a purchasing process. Software programs, in this case agents, are basically just helping by doing routine jobs and all the major decisions are done by company personnel.

In the scenario, the buyer agents are not authorized to anything by their selves; the buyer agents are launched to execute a particular task. After that the buyer agent is authorized to negotiate a contract if the needed terms and limits for the particular case are met. If not, the decision is passed to its personnel.

The supplier agents are authorized to automatically create quotes based on data from the company's internal management systems, including inventory, pricing, and financial information, such as payables and receivables.

If the buyer proposes a price lower than the list price, the agent suggests a course of action to human personnel and awaits further instructions.

Request for Quote The company personnel create the request for quote and set the limits what can the requested product or service cost and what are the other terms. The subsequent interaction in the quote request is automated; if the terms are met the agent negotiates the final contract with the supplier agent.

Quote Negotiations If no quote is good enough, the agent presents a list of all the quotes. The company personnel can pick one of the quotes and decide to make a contract or modifying it for further negotiation. If the buyer proposes a lower price than what supplier agent finds from the pricing list, it needs confirmation from the supplier company personnel to continue. Otherwise, the supplier agent can continue and make the contract.

Making the Contract This step is reached only if the terms set by the buyer company personnel are met. This step needs no confirmation from the buyer company personnel because the terms are set beforehand.

4.2.1.3 Artefact: Data Models

All the documents that are sent between agents are based on the two simple ontologies; the proposal ontology that is depicted in Figure 39 and the product ontology that is depicted in Figure 40. As depicted in Figures 39, the original RFQ is a proposal that concerns some product or service with payment and delivering terms. The product is presented in its own ontology that forms the basis for the location of companies providing some service or product.

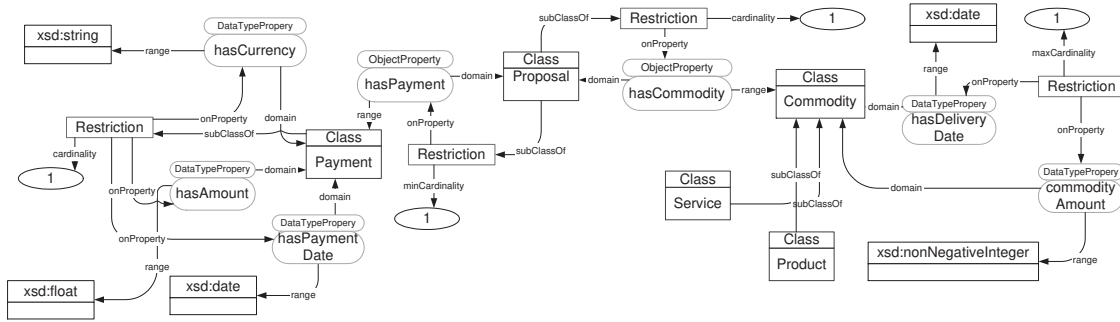


Figure 39: Proposal Ontology.

4.2.1.4 Artefact: Locating Services

Today's registry services, such as UDDI, are meant to be analyzed primarily by people, not machines, so the indexing is based on keywords. In this demonstration, the location of the services is based on a semi-intelligent registry. It is called semi-intelligent because it is based on the ontology of the product presented in Figure 40; companies are offering services and products in some category and a single product has a category that it belongs to. This means that there is not one-to-one mapping between a product and company selling it because it would require too much maintenance since there are millions of products in the real world. Instead, this category based mapping gives too many results for companies that are probably selling the product. If the supplier does not have the product they simply reply that they do not have it. This somewhat represents the real world situation since there is no registry of all the products and services in the world but rather companies that are doing something and you have to ask if they have what you are looking for.

4.2.2 Demonstration

This subsection describes briefly the implementation of the theory that was presented previously in this section. The size of the implementation is around 10 000 lines of code.

4.2.2.1 Agent platform

Technically all the services are implemented as software agents running on top of FIPA-compliant platform called JADE [168]. As mentioned in Section 2.4.5.3 and Section 4.1.2.2, JADE provides basic functionalities such as message transport system, protocol templates and agent containers.

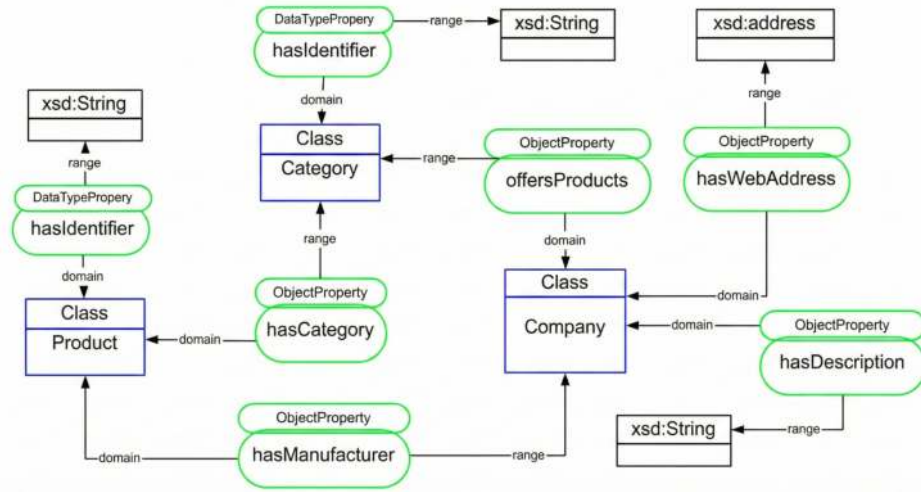


Figure 40: Product ontology.

4.2.2.2 Registry

In this demonstration, the location of the services is implemented as a semi-intelligent registry that was described in the previous subsection. The software agents can browse products from the registry service and after choosing a product the registry service returns a set of companies that are probably providing the service that the user agent is looking for. As described, this is done by category based mapping. The registry was implemented as Java classes and the information is parsed from an XML-based registry that implements the ontology in Figure 40.

4.2.2.3 Data models

The reports are based on the ontology presented in Figures 39 and 40. The ontology was implemented as Java classes that can be serialized to XML format.

4.2.2.4 Screenshots

This section includes some screenshots from the demonstration with short descriptions. Figure 41 depicts the communication flow from the RFQ process with one buyer agent and four potential deliverer company agents. As described earlier, the communication flow implements the FIPA Contract Net protocol [187]. In the flow the first message sent by the buyer agent is a call for propose message and the content is the request for quote that was described earlier. Other agents can respond with quotes or they refuse for some reason ²⁸. In this case, all the agents propose a quote, but the buyer refuses to accept the quotes.

Figure 42 depicts the implementation of the FIPA Propose protocol. In the communication flow, the buyer agent proposes some changes to the quote received from the agents in previous phase of the overall communication process that is described in detail in Section 4.2.1.1. The delivering companies have now two choices: to agree or refuse. In this case, two of them agree and one refuses.

Figure 43 depicts the graphical user interface of this demonstration after the two process steps described above. The figure also shows the parameters for the request for quote, the simple restrictions and limits and the actual quotes. On the left-hand side you can see an information box that shows more detailed information about the quotes. The same box will show the final contract if an agreement is reached.

²⁸If they for example have stopped offering the requested the product or service

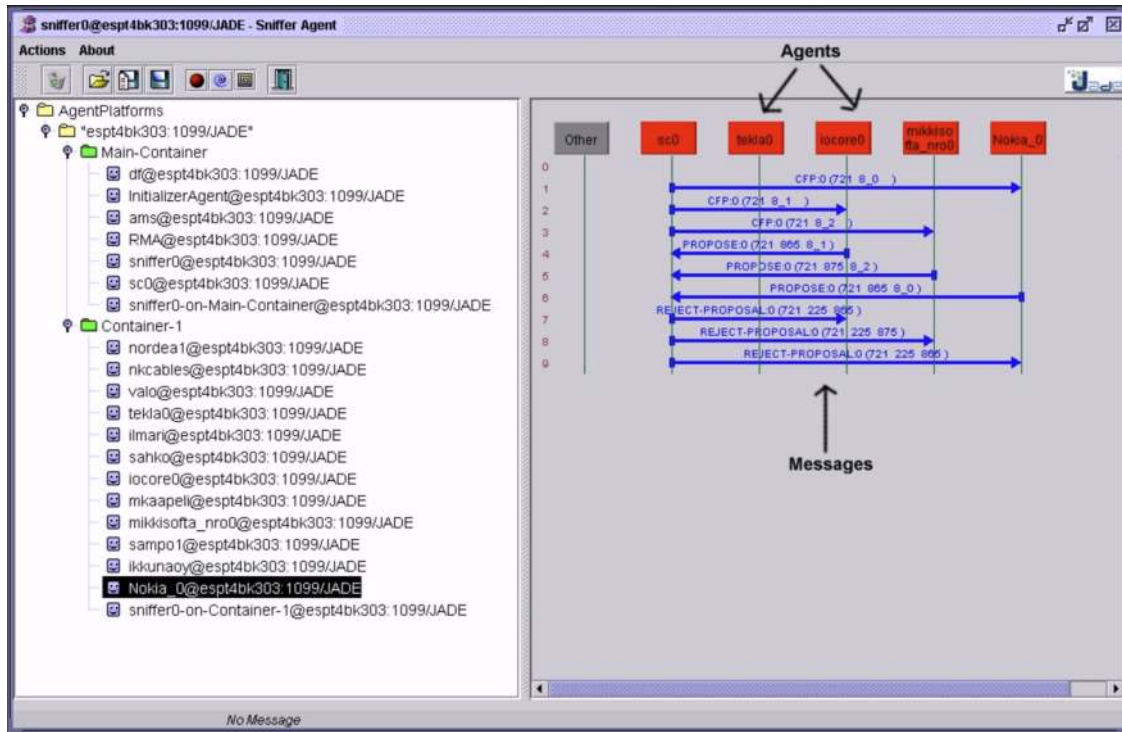


Figure 41: Requesting for quotes.

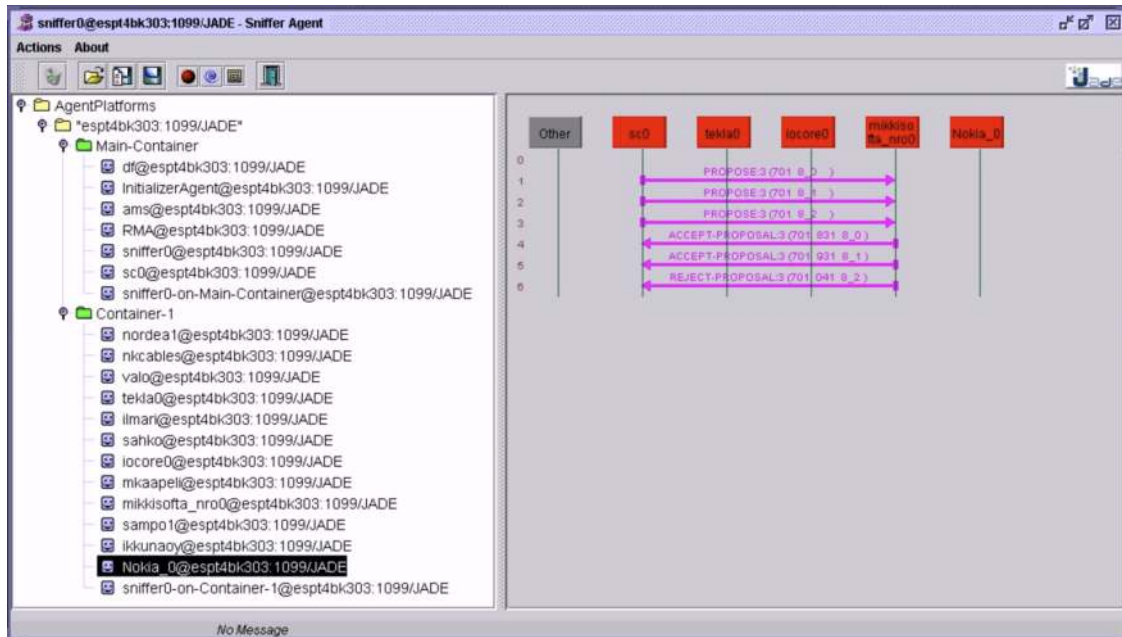


Figure 42: Proposing a lower price.

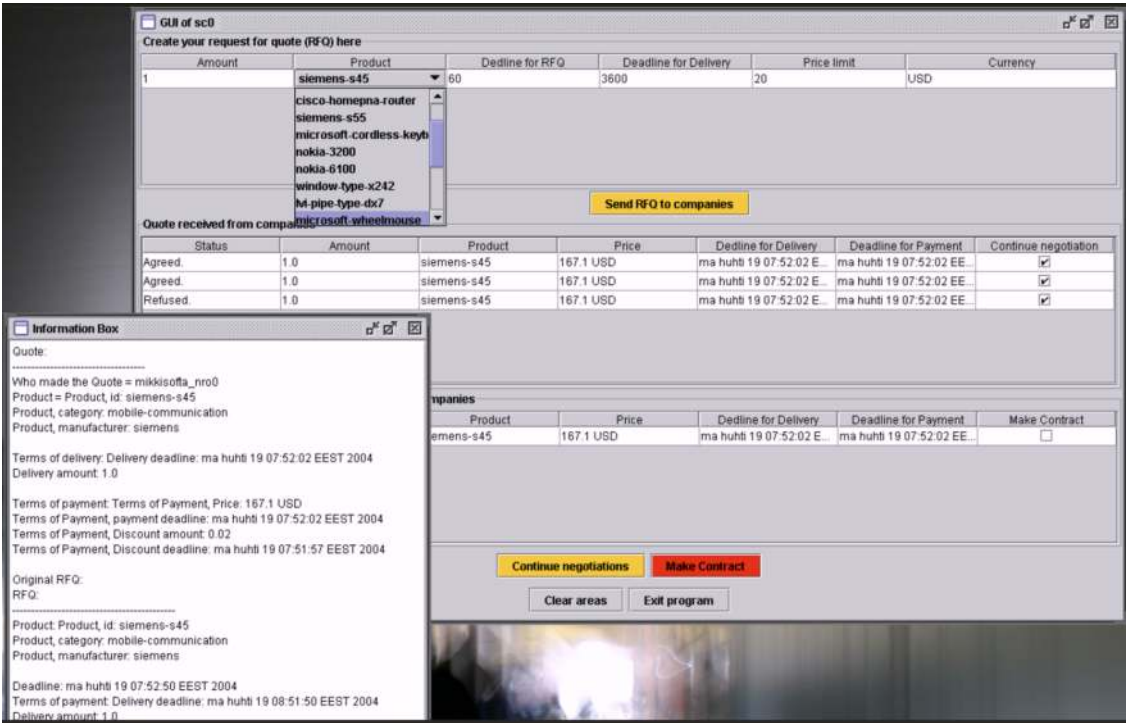


Figure 43: GUI: products, tables.

4.3 Matching Service Descriptions in Context-Aware Environment

Problem Identification, Research Questions and Objectives of the Solution

The problem identification, research questions and objectives for this work are presented in Section 1.2.3. The research methodology is presented in Section 1.1.

Context: Sailing Scenario

This section presents the third and final demonstration included in this Thesis Phase I. The demonstration combines B2C and C2C approaches for users to share information in a contextual manner²⁹. As in the two previous demonstrations, the shared ontologies are playing a central role. However, context information is now also utilized to automate the linking between the shared ontologies.

In DYNAMOS³⁰, users are mobile and they are dynamically provided with relevant descriptions of available services potentially of interest to them given their current situations. This entails taking into account the users' contextual details, such as time, location, and activity. In addition, DYNAMOS enables users to annotate the service descriptions and share them with each other. Figure 44 depicts this hybrid service provision model. The users of the DYNAMOS system can create and share also content that is not related to services. Therefore, in total there are three kinds of content, namely service descriptions, service annotations, and user notes.

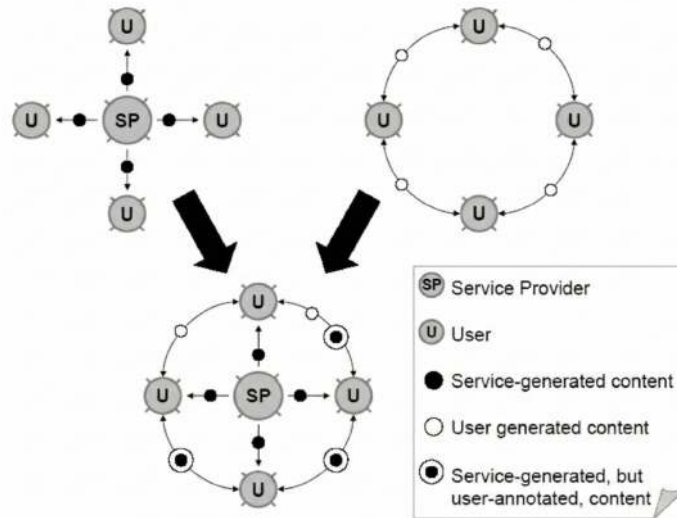


Figure 44: Hybrid approach for service provision [300].

Recreational boaters were chosen for the user group to be studied in DYNAMOS. An example of a useful service description to that user segment would be an advertisement of a guest harbor. An example annotation is a rating of that guest harbor either in natural language and/or by giving "stars". An example user note is a warning about high rocks nearby a route to the guest harbor during low tide.

4.3.1 Design and Development

The DYNAMOS main design consists of the following artefacts:

A.I.3.1 Context-Profile-Content Ontologies OWL/RDF schemata and instances linking relatively static *user profiles* with dynamic *context* (time, location, activity) and with *service descriptions / annotations / user notes*. (Section 4.3.1.1).

²⁹The demonstration was implemented in a project co-funded by Tekes, VTT and industrial parties.

³⁰Hereafter, this demonstration is referred to as DYNAMOS = Dynamic Composition and Sharing of Context-Aware Mobile Services

A.I.3.2 Matching Engine & System Architecture Implemented matcher connecting (profile, context) to (services, annotations, notes) with ranked delivery; deployed within the client-server architecture and content stores. (Section 4.3.1.2).

A.I.3.3 Event-Driven Distribution Layer Fuego Core-based publish/subscribe channels for context-change notifications and real-time sharing of annotations/notes between clients and servers. (Section 4.3.1.3).

4.3.1.1 Artefact: Shared Context-Profile-Content Ontologies

The major functionality of the DYNAMOS System is notifying mobile users about services in a context-sensitive fashion. This is carried out by providing users with *service descriptions* generated by service providers and *service annotations* created by other users. In addition to annotating services and sharing such annotations with each other, the users can create and share content that does not refer to any specific service description. This kind of content is called *user notes*. For instance, users can create notes in the form of warning messages (e.g., "*Dangerous underwater rocks nearby!*"), to notify other users of accidental situations that occur in a certain location and time. Alternatively, these notes may be plain notifications of some state of affairs, such as bird sightings. In a marine setting, such messages can be sorted as emergency/urgency/safety/routine events.

Service descriptions capture the most essential semantics of the services. In other words, the services are categorized according to their business branches. Additionally, a natural language description about the services can be given. These descriptions are matched against user profiles when providing them with services. The same applies for service annotations; if a service annotation attached to a service potentially of interest to a user matches the user's current context, it is provided to her. Among future work is considering the user preferences (found in user profiles) and their impact on the provision of annotations as opposed to service descriptions, and vice versa.

Figure 45 depicts the user profile and user context ontologies. As can be seen from the figure, the user profile is considered to be fairly static, whereas the context is constantly changing. The ontologies are interlinked with the concept activity. In this scenario, the user once specifies their activities and associates interests to those activities. For example, the user can specify three activities: *at home*, *travelling*, and *at work*. Then the user attaches interests to those activities: when the user is *at home* they are interested in entertainment, tv-guides, and special offers nearby their current location. When the user is *travelling* they are interested in public transport timetables and local news. When the user is *at work*, they are interested in nearby food menus, financial news, and messages from their colleagues.

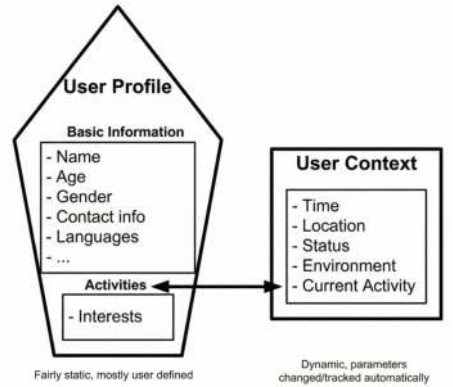


Figure 45: User Profile and User Context.

The profile includes all activities and the associated interests. Although this information is fairly static, the dynamic part is the currently active interest and it is considered as context information. The current activity activates a set of interests that link to the actual content provided by the system.

4.3.1.2 Artefact: Matching Engine & System Architecture

Architectural Components

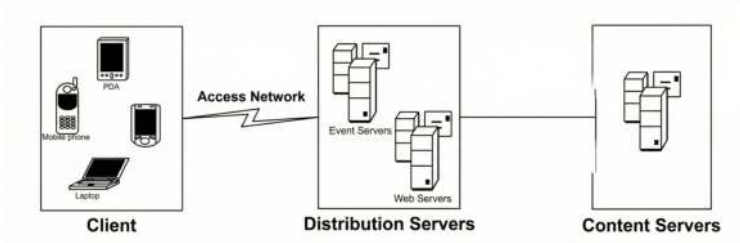


Figure 46: Overall System Architecture.

The overall architecture of the DYNAMOS System is depicted in Figure 46. The three main components of the system are the following:

- **The Client:** the client can be a regular Web browser (thin client) or a stand-alone application (thick client) that uses event-based communication. Using events rather than, for example, RMI (as in [301]) has advantages, such as more fault-tolerant communication and wider penetration of currently available compliant client devices.
- **Content Servers:** the actual content is stored in the DYNAMOS content servers. The User Content Server stores user related information (i.e., profile and context data). The Service Content Server stores content provided by service providers (i.e., service descriptions) and user-generated content (i.e., user notes and service annotations). This provides us with flexibility in distributing the content through multiple channels.
- **Distribution Servers:** since different ways of accessing such content are to be supported, there are multiple types of access server, namely Web Servers and Event Servers.

The access to the DYNAMOS System is currently supported by means of two different interfaces. The Web interface can be used with any device that supports a web browser. However, the current Web interface is designed for devices with large screens. For mobile users, often equipped with limited-capability devices, a stand-alone client application was implemented. Such an application is context-aware, meaning that it constantly monitors context parameters, such as the user's location and activity, and notifies the Event Server of relevant context changes. The major benefit of doing this is that mobile users, usually equipped with small-screen devices, are proactively provided with a compact summary of available services, without the need to browse the entire content stored in the system.

The DYNAMOS Content Servers contain the whole content of the system. The Distribution Servers use a middleware solution to access the Content Servers. In the current implementation, Java RMI is used. Java RMI was chosen since it outperforms most of the other middleware solutions, such as Web services [302] and it is also easy to implement and to maintain. The Content Server uses a combination of objects and relational databases to store and to offer the content to the Distribution Servers. For performance reasons, most of the content is kept in memory, and only persisting content incurs time-consuming MySQL operations. To test the application with the selected sailing scenario the server has currently been provided with a database of tourist services in Turku area. The size of the database is approximately one thousand service-description entries.

The Event Distribution Server that has been integrated in the system is the Fuego Core event server [303]. It provides a standard event-based platform and allows access and update of the content stored in the Content Servers in an event-based manner. The reason for adopting an event-based infrastructure is twofold. First, a main property of context-aware applications is to react to context changes occurring in their execution environment. Such applications must be able to subscribe to context change events and be notified when such events occur. Second, context changes and user generated content must be added and shared among users in an event-based manner in order to support an efficient and real time provisioning and sharing of such content.

4.3.1.3 Artefact: Event-Driven Distribution Layer

Figure 47 shows the Fuego Core architecture and the DYNAMOS components so far implemented. The figure shows on the right side the DYNAMOS Event Server and on the left side the DYNAMOS client application. The Fuego Core stack provides us with an event-based distribution channel and several enhancements for wireless internet. More details can be found at the project web page [303]. In the following, the main DYNAMOS software components are described.

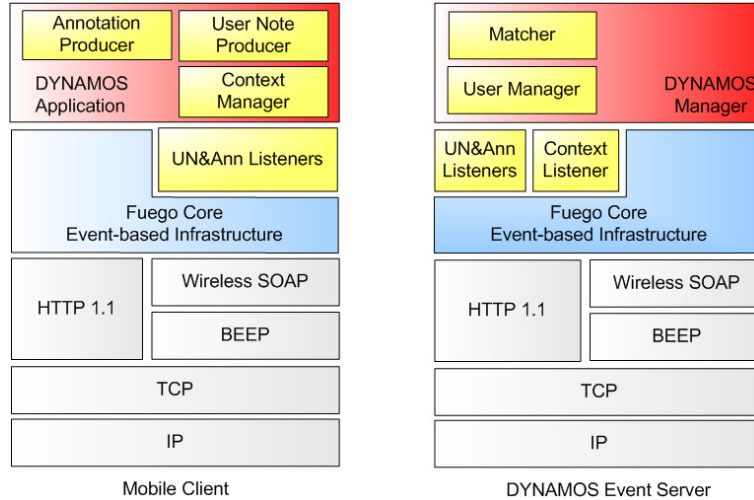


Figure 47: DYNAMOS Event Distribution Server.

- The *User Note and Annotation Listeners* (both on the client and the server side) are in charge of detecting the presence of new user-generated notes and service annotations and of notifying the upper layer. User-generated notes and service annotations are made available into the system respectively by the *User Note Producer* and the *Annotation Producer*. Therefore, clients are capable of adding and sharing such notes in a real-time manner.
- The *Context Listener* (on the server side) listens to context changes (i.e., user profile and context changes) occurring on the client side and constantly updates the server with the current user context and profile. Context and profile change notifications are produced on the client side by the *Context Manager*.
- The *Matcher* (on the server side) plays a key role in the DYNAMOS Server. It performs the matching between active users and service descriptions thus to provide users with a compact summary of relevant services. The matching occurs between context/profile information of the user (e.g., interest, location) and service description (e.g., service category, location). The same matching applies to annotations and user notes stored into the system. This allows the user to benefit from a selected set of services of interest and be proactively notified about critical situations of interest.
- The *User Manager* (on the server side) is responsible for maintaining a consistent and updated version of the user information (i.e., profile and context) in the DYNAMOS database. The Context Listener communicates changes in the user context and profile to the component.

4.3.2 Demonstration

DSR positioning. In the DSR cycle, the *Demonstration* phase shows the artefact in use to illustrate how it addresses the stated problem in a realistic setting. Here, the artefact is the context-aware *DYNAMOS* system, which integrates client devices, an Event Distribution Server, a User Content Server, and a Service Content Server to deliver proactive, context-matched services (architectural overview in Section 4.3). The demonstration comprised (i) a field trial with real users in August 2005 and (ii) controlled technical measurements that reveal operational bottlenecks and scalability limits. Together they substantiate the artefact’s *utility* and inform the subsequent evaluation and refinement steps of the DSR cycle.

Field Demonstration: August 2005 Sailing Trial

In August 2005, a user trial was held to get real-world feedback and guide the final development. The user trial was executed together with Sonera Sailors³¹, who arrange a yearly sailing competition in August. Figure 48 depicts this sailing competition from the DYNAMOS system perspective. The sailing competition started from nearby Helsinki and ended up in Porvoo.

³¹Sonera Sailors was a private sailing interest group

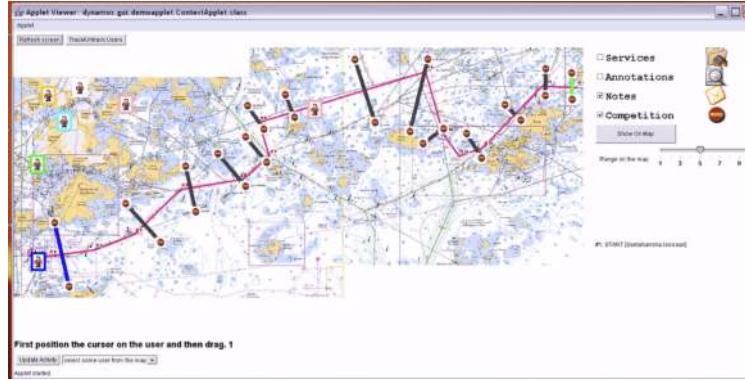


Figure 48: DYNAMOS System Overview.

The winner of the sailing competition is the one who sails the predefined route the fastest. However, the sailboats are different and it would not be convenient to compare the sailing times directly. Therefore, the total sailing time is multiplied with a so-called *LYS*-value³² that is defined for each boat type. Because of this multiplication, it is hard to know the real status of the competition when the race is on. Even if a sailboat were ahead of your boat, that boat could be faster and have a higher LYS value, and therefore you could in fact be ahead of it.

During the August 2005 user trial, a service was created in DYNAMOS to tackle this problem. The DYNAMOS system tracks the position of each competitor, and by inserting the LYS-values, the competitors could be provided with real-time information about who is leading. The competition was divided into checkpoints where the real status of the competition was calculated. These checkpoints are depicted on the map in Figure 48. As can be seen from Figure 49, these checkpoints were provided to users as regular DYNAMOS services.

We offered the sailors 10 Nokia 6630 phones and 10 GPS devices to access the DYNAMOS system. Figure 49 depicts 15 screenshots from the client application. As can be seen from the figure, with these devices sailors could participate in the competition, view the competition results, and send user notes.

Technical Performance Measurements

This section presents the performance measurements conducted to identify potential bottlenecks in the system.³³ Such analysis has guided us in the development of the system. As a framework for this analysis, three abstract actions that the system commonly executes during operation are presented. Each action consists of several elementary operations not discussed here. For architectural illustrations and discussion, refer to Section 4.3.

- *Context Update:* By means of such an action, the client application informs the system about a change in the user's context. As Figure 46 shows, the transmission of the context update is done by sending an event over the client access network to the Event Distribution Server.
- *Access Context:* The Event Distribution Server accesses the User Content Server to store user context changes or to retrieve context information provided to the User Content Server through other channels.
- *Match Context to Content:* In the matching process, context information stored in the User Content Server is used to filter the whole content stored in the Service Content Server. Specifically, the section addresses filtering based on location (i.e., user location is matched against the service/annotation-s/notes location) and based on category (i.e., user interests are matched against service categories). The details of the filtering algorithms are outside the scope of this thesis.

³²LYS is a class-specific handicap coefficient used to compare different sailboats. Results are ranked by corrected time $T_{\text{corr}} = T_{\text{elapsed}} \times \text{LYS}$; a higher LYS penalizes faster-rated boats, so the first to finish may not win on corrected time.

³³Performance measurements described in the following subsections were conducted with Nokia 6600 and 6630 mobile phones and regular desktop PCs: 2GHz processor, 512 MB memory, Linux Red Hat 9, and Windows XP SP2 operating systems.



Figure 49: DYNAMOS Client Screenshots.

4.3.2.1 Context Update

The number of events carrying context updates that client applications can produce per second was measured. The measurements were conducted by connecting the client application to the Event Distribution Server over an access network. Measurements show that over a GPRS network, the client application is able to update context parameters approximately once every 4 seconds. In the case of a WLAN (11 Mbit) network, the client is able to update context parameters approximately 30 times every second. Both cases are considered more than reasonable for the intended use-case scenarios.

4.3.2.2 Access Context

Once the Event Distribution Server has parsed the context update event, it stores the context information into the User Content Server to be retrieved later, possibly by other servers. The number of context updates and context fetches per second that the prototype can handle was measured. Performance was compared using four different methods for accessing the User Content Server: direct method call (same Java Virtual Machine), Java RMI to the same physical computer, Java RMI to a computer with a fast Ethernet connection, and Java RMI to a computer with a fast DSL connection. The results are presented in Table 12.

Table 12: User Content Server (calls/second).

	Get context parameter	Set context parameter
Local Object	4149377	1148105
Local Server (RMI)	4132	2777
400 Mbit Ethernet (RMI)	302	243
8Mbit ADSL (RMI)	75	72

The serialization of objects is a time-consuming operation. The direct method call can handle around 1000 times the number of context operations compared to the case where the same functionality is accessed over Java RMI. Even with a fast Ethernet connection, RMI will quickly become a bottleneck in storing and fetching context information.

4.3.2.3 Matching Context to Content Performance

The most bandwidth- and time-consuming operation is clearly the matching of context information against the actual content available in the system. Different matching functions were implemented to support distinct matching criteria for each content object. Measurements were carried out for two specific matching criteria, namely *matchByLocation* and *matchByCategory*. The results of these tests are presented in Table 13.

Table 13: Service Content Server (calls/second).

	matchByLocation	matchByCategory (memory)	matchByCategory (SQL)
Local Object	6277,5	2933,3	6,0
Local Server (RMI)	20,2	81,4	5,49
400 Mbit Ethernet (RMI)	5,2	8,2	1,24
8Mbit ADSL (RMI)	0,3	1,2	0,64

For category-based matching, two modalities were considered. The former handles the content in memory, and the latter uses MySQL queries to select the data and create the objects from the relational database. The MySQL queries were not optimized in any way. However, as the results indicate, the in-memory algorithm is necessary if the category-based function is called frequently.

With the current algorithms, the server can handle more than 6000 matches based on location and about 3000 matches based on category every second. When other networks instead of direct function calls are employed, the performance is reversed, meaning that the match by category is faster than the one based on location. This probably occurs because the average match by location with a random range returned a longer list of services than the match by category. This implies that the processor load of the server is much less than 100%, as the bottleneck is Java RMI.

The current service description database included around 1000 database entries. Given the data in Table 13 and considering that the current *getByLocation* (memory) consumes time $O(N)$ in the worst case, it can be concluded that the algorithm can handle approximately 10,000 database entries. After that, it becomes a bottleneck for the system, and a more efficient algorithm is needed.³⁴

4.3.2.4 Conclusions

Based on the presented measurements, several conclusions can be drawn about the system prototype's performance, thus identifying sources for future improvement and key issues in extending the system. First of all, if all clients used a GPRS network, the system design would be quite straightforward since the bottleneck would be the access network. In cases of WLAN or faster access networks, the client is able to produce a large number of update events per second. Therefore, the update frequency needs to be constrained; otherwise, the content matching phase will deliver unsatisfactory performance.

Although the latency of Java RMI in fast networks was observed to be quite small [302], the middleware quickly becomes a bottleneck when a method call returns a large set of objects. The situation gets worse if RMI is replaced with, for example, Web services. This implies that the interface to the content accessed through RMI, in this case to the Content Servers, should not offer low-level functionalities that return incompletely filtered content. Instead, if possible, the entire filtering process should be done with one method call.

Assumptions and Limitations

This section briefly lists the assumptions made and the limitations imposed by the selected technologies.

- **Accurate positioning is available.** In order to function at full scale, the system needs accurate user positioning. GPS positioning was selected. Current mobile phones have issues (bugs) that make reliability a problem.
- **Used with mobile phones.** No special-purpose device or computer is used.

³⁴For example, by hashing the locations into a two-dimensional hash table.

- **Context parameters.** In the matching process, only location, time, and user activity are used. The challenge is not to obtain other parameters, but to derive meaningful information from them.
- **If a user drops out of the competition, they can rejoin without having to restart from the beginning.** However, if the client application crashes due to an external cause, the user has to restart the application and log in to the DYNAMOS system.
- **If the connection to the GPS device is lost,** the phone tries to reconnect automatically.

User Trial Feedback

As discussed in Section 4.3, the user trial was conducted in August 2005. This user trial provided us with real-world feedback and guidance for the final development of the system.

The user trial was executed together with Sonera Sailors, which arranges a yearly sailing competition in August. The sailing competition started near Helsinki and ended in Porvoo. Ten Nokia 6630 phones were provided to the sailors and 10 GPS devices to access the DYNAMOS system.

Unfortunately, the weather was unfavourable. The wind speed was over 15 m/s, and the sailing competition turned into a survival competition. Most of the users did not have time to access the DYNAMOS system because of the storm. Valuable feedback was gained, and the experiences related to the entire scenario were documented:

- **The DYNAMOS system works.** Users can participate in the competition, send notes, view competition results, and view notes created by other users.
- **The server side is stable.** The server ran without problems.
- **The client application is unstable.** The client application leaks memory and therefore crashes when the memory runs out. This was partly due to bugs in the Java Virtual Machine used on the current mobile phones. The memory leaks can be avoided by increasing the reuse of objects, especially in commonly used procedures. It must be stated that it is very hard to create a stable application that uses a wide variety of new features³⁵. This is because it seems that every device has a serious bug in some of the features, and if fewer features are used, the application becomes less interesting.
- **The environment is very challenging.** There is quite a lot to do on a sailing boat when the competition is on, and the sailors have very little time to think about anything else.
- **The application consumes a lot of battery power.** Because of bandwidth usage optimizations, the application consumes significantly less battery power than previous versions. However, further optimizations should be made since the battery lasts only half a day.
- **Users are enthusiastic about using such a system.** Most users voiced enthusiastic reactions about using the system beforehand. However, in the stormy weather conditions, the system still demanded too much user attention to function correctly.
- **The User Interface should be simplified.** The application should be ready with one click. All questions, such as *“Can the application use the GPRS/Bluetooth connection?”*, should be skipped with one *yes*.

³⁵In this case, the use of the Bluetooth connection, GPRS, the camera, and the microphone.

5 PHASE II: DESIGN & DEVELOPMENT AND DEMONSTRATIONS

This chapter (Phase II) concentrates on applying software agents augmented with machine learning to support clinical decision making and documentation. Building on the mechanisms and demonstrations of Phase I (Chapter 4), focus is shifted from general agent coordination and service orchestration to a domain setting in clinical work, where the operational objective is to assist clinicians in capturing structured information, linking diagnoses and procedures, and preparing downstream artefacts for coding and reimbursement. The design follows the DSR approach: the problem context is articulated, objectives for an effective solution are derived, and artefacts that combine agent-based interaction patterns with ML components, when they add measurable value, are engineered.

Portions of this material have appeared, in earlier or condensed form, in peer-reviewed venues and practitioner outlets, including [9, 10, 11, 12, 13, 14, 15, 16, 17]. The present chapter unifies and extends those contributions in a single, coherent narrative aligned with the thesis objectives and artefact set, and reports demonstrations and results with consistent terminology and evaluation criteria.

To orient the reader within the overall thesis structure and the DSRP flow, see Table 3 (DSRP process sequence linked to Phases and Structure of this Thesis) and Section 1.3 (Phase II Problem and Motivation).

5.1 Assisting Clinical Decision Support with ML and Software Agents

Problem Identification, Research Questions and Objectives of the Solution

The problem identification, research questions and objectives for this work are presented in Section 1.3.1. The research methodology is presented in Section 1.1.

Context

Hospitals sit on years of clinical experience, but much of it is trapped in fragmented IT landscapes: dozens of legacy systems, incompatible formats, and uneven documentation quality. Clinicians trying to make time-critical decisions must hop between systems to reconstruct a patient’s story, while existing tools rarely surface “patients like this one” fast enough to influence care. The same fragmentation affects DRG-related funding decisions, where incomplete or inconsistent coding can misstate case complexity and impact reimbursement.

This work tackles that business problem by turning dispersed records into timely, case-based recommendations that clinicians can trust at the point of care. Instead of dense neural setups that struggle with extremely sparse, high-dimensional code spaces (ICD-10/NCSP), the approach uses vector-space information-retrieval methods with kernel-based similarity and domain-aware weighting to find relevant historical episodes quickly and robustly—even when documentation is imperfect. A distributed, agent-based architecture orchestrates on-demand data collection across heterogeneous systems and returns ranked suggestions in sub-second latencies on commodity hardware. The outcome is a practical bridge from fragmented operational data to actionable decision support that improves coding quality, supports auditability, and helps align clinical and financial outcomes.

Background and Related Work

The term Computer Aided Coding (CAC) is used to denote technology that automatically assigns codes from clinical documentation for a human to review, analyze, and use. There are a variety of methodologies employed by developers of CAC software to read text and assign codes. The software can use structured input or NLP. Even within the NLP range of products, there are a variety of approaches with varying levels of sophistication. These include usage of external knowledge databases and feature extraction using Self Organizing Maps (SOMs) [304]. The methodology used has a tremendous impact on data transmission and the output reviewed by the coders [305]. Some studies have shown CAC software performing strongly in comparison with human coding [306]. Other studies have concluded that no productivity increase was achieved [305].

5.1.1 Design and Development

The main Assisting Clinical Decision Support with ML and Software Agents artifact comprises the following artifacts:

A.II.1.1 Process Flow. A formalization of the healthcare item set abstract IT process flow and classification (see Section 5.1.1.1).

A.II.1.2 Data Space. A mathematical description of the data space combinatorics (see Section 5.1.1.2).

A.II.1.3 ML/IR Model. A description of the ML/IR model for the agents to consume (see Section 5.1.1.3).

A.II.1.4 MAS Architecture. A description of the MAS architecture (see Section 5.1.1.4).

5.1.1.1 Artifact: Healthcare Item Sets and Abstract IT Process

Artifact Description. This artifact formalizes the healthcare item set abstraction process and defines the abstract IT process flow for transforming clinical documentation into structured classification systems. The artifact addresses the fundamental challenge of representing complex patient care episodes through standardized classification hierarchies while preserving clinically relevant information.

Theoretical Foundation. The artifact builds upon the hierarchical classification model illustrated in Figure 50, which demonstrates the progressive abstraction from patient-level data to DRGs. This hierarchy encompasses six distinct levels of abstraction: (1) Patient—the physical entity requiring care; (2) Episode of care—the temporal boundary of treatment; (3) Medical record—the comprehensive documentation; (4) Discharge summary—the synthesized clinical narrative; (5) Diagnosis and procedure codes—the primary classification layer (ICD-10, NCSP); and (6) DRG—the secondary classification for resource allocation.

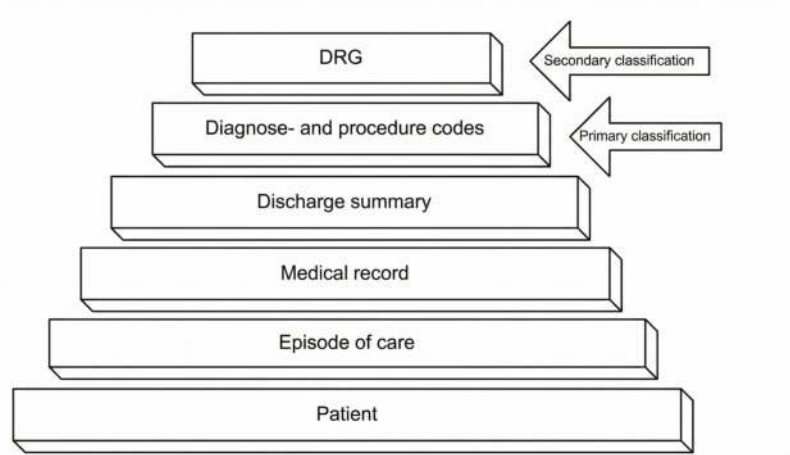


Figure 50: The hierarchy of classification.

Abstraction Process Formalization. The artifact defines formal rules for the abstraction process, addressing the inherent challenge of information reduction. With ICD-10 containing 103 syntactical variations for pneumonia coding and 848 for cancer coding, the artifact establishes: (1) retention criteria for preserving clinically significant details; (2) reduction rules for eliminating redundant or non-essential information; and (3) consistency mechanisms to ensure uniform abstraction across practitioners. This formalization addresses documented quality problems in patient documentation [307, 308, 309, 310, 311, 312].

IT Process Architecture. Figure 51 illustrates the artifact’s process architecture, which transforms fragmented primary business processes into integrated secondary processes through data warehousing [313]. The architecture comprises three layers. The *Input Layer* consists of multiple heterogeneous systems (Legacy systems D4, D6, F3, A1; Accounting system C2; ERP systems B2, C4; HR system D7) capturing fragments of patient documentation during primary care processes. The *Integration Layer* includes data warehouses and databases that consolidate distributed information using OLAP methods and data cube aggregation techniques. The *Output Layer* enables secondary business processes for comprehensive analysis and decision support based on integrated patient data.

Operational Characteristics. The artifact addresses the reality of healthcare information fragmentation where patient documentation is distributed across dozens to hundreds of specialized systems. Each system

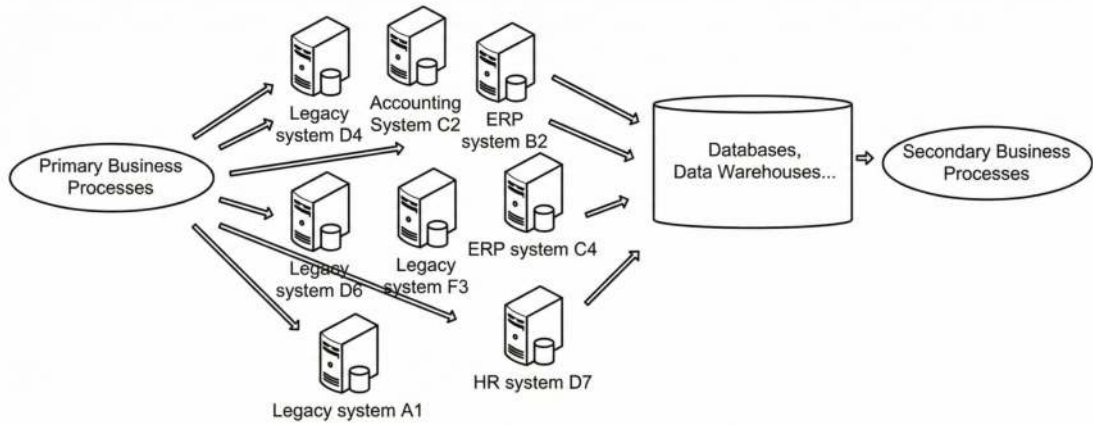


Figure 51: Primary and secondary business processes and data warehousing.

captures partial information—one system may record "Patient X was performed surgical operation ACYZ01," another contains "Laboratory experiment D932 was performed on Patient X," while yet another knows "Patient X has chronic disease E023." Healthcare professionals lack real-time, transparent access to complete patient documentation, requiring sequential interrogation of multiple systems to construct comprehensive patient views.

Design Rationale. The artifact's design is motivated by two critical requirements: (1) *Immediate clinical need*—enabling healthcare professionals to access comprehensive patient information without sequential system interrogation; and (2) *Knowledge management capability*—leveraging accumulated institutional knowledge about similar patient cases for evidence-based decision support. This formalization provides the foundational framework for subsequent artifacts (Data Space, ML/IR Model, and MAS Architecture) by establishing the structural relationships and transformation rules governing healthcare item sets within the clinical information ecosystem.

5.1.1.2 Artifact: Healthcare Item Set Data Space

Artifact Description. This artifact provides a mathematical formalization of the healthcare item set data space, establishing the computational boundaries and dimensionality constraints that govern the design of ML solutions for clinical decision support. The artifact quantifies the combinatorial complexity of healthcare documentation and demonstrates why traditional neural network approaches are insufficient for this domain.

Combinatorial Analysis. The artifact defines the theoretical space of all possible minimum data sets through combinatorial analysis. Given approximately 12,000 possible values in both ICD-10 diagnosis and NCSP procedure coding classifications, and allowing for 40 diagnosis codes and 40 procedure codes per patient episode, the total number of possible minimum data set combinations C is:

$$C = \binom{ICD}{DG_{codes}} \cdot \binom{NCSP}{PR_{codes}} = \binom{12,000}{40} \cdot \binom{12,000}{40} = 2.8 \cdot 10^{230} \quad (15)$$

This astronomical figure establishes the upper bound of the solution space and highlights the fundamental challenge of healthcare data analysis.

Dimensionality Characteristics. The artifact characterizes the data space along three critical dimensions. First, *High dimensionality*: Binary matrix representation yields approximately one million dimensions (12,000 possible codes $\times$ 80 code positions), creating computational challenges for traditional ML approaches. Second, *Extreme sparsity*: The ratio between actual patient cases (typically 10^5 to 10^8) and theoretical combinations (10^{230}) demonstrates that observed data occupies an infinitesimally small fraction of the possible space. Third, *Structural complexity*: The data space exhibits additional complexity through code repetition

(multiple occurrences of the same diagnosis), positional significance (primary vs. secondary diagnoses), and supplementary clinical attributes beyond diagnosis and procedure codes.

Computational Implications. The artifact establishes why conventional neural network architectures, particularly SOMs, are computationally infeasible for this domain. The million-dimension input space exceeds practical limits for neural network training, both in terms of memory requirements and convergence time. While dimension reduction techniques, such as Principal Component Analysis (PCA) or random projection could theoretically compress the space, the extreme sparsity and semantic importance of individual codes make such approaches unsuitable without significant information loss. Based on these computational constraints, this study makes the assumption that neural networks, such as SOMs, are improper for this type of data, necessitating alternative approaches that can operate efficiently in sparse, high-dimensional spaces without requiring dense matrix representations or iterative training procedures.

Design Constraints. This mathematical formalization imposes specific constraints on the solution architecture: (1) Algorithms must operate efficiently in high-dimensional sparse spaces without requiring dense matrix operations; (2) Similarity measures must preserve semantic relationships despite sparsity; (3) Computational methods must scale sub-linearly with dimensionality; and (4) Storage strategies must exploit sparsity to achieve practical memory footprints.

Theoretical Contribution. The artifact contributes to the theoretical understanding of healthcare data spaces by providing the first formal quantification of the combinatorial explosion in clinical coding systems. This formalization justifies the selection of vector space models and kernel methods over neural networks, establishing a mathematical foundation for the subsequent ML/IR model artifact. The analysis demonstrates that effective healthcare decision support requires algorithms specifically designed for sparse, high-dimensional categorical data rather than adaptation of general-purpose ML techniques.

5.1.1.3 Artifact: ML and IR Model

Artifact Description. This artifact defines a ML and IR model specifically designed for high-dimensional, sparse healthcare data spaces. The model adapts kernel methods and vector space techniques from IR to enable efficient similarity computation and pattern recognition in clinical coding systems, providing the computational foundation for the MAS recommendation capabilities.

Theoretical Foundation. The artifact builds upon kernel methods for pattern analysis, leveraging the kernel trick to compute similarities in high-dimensional feature spaces without explicit coordinate calculation. Unlike traditional applications, such as Support Vector Machines (SVMs), that focus on classification boundaries, this artifact adapts kernel methods for nearest-neighbor retrieval in sparse categorical spaces. The model transforms the healthcare item set matching problem into a vector space similarity problem, enabling efficient computation despite the 10^{230} theoretical combination space identified in the Data Space artifact.

Vector Space Formulation. The model represents patient episodes as vectors in a high-dimensional space where each dimension corresponds to a clinical code. Figure 52 illustrates how coded data sets form an interconnected network through the coding scheme. Each patient episode $D_i = (d_{i1}, d_{i2}, \dots, d_{it})$ becomes a point in this space, with queries $Q = (q_1, q_2, \dots, q_t)$ formulated as vectors seeking similar episodes.

Similarity Computation. The artifact employs Cosine Angle Distance (CAD) as the primary similarity measure, chosen for its effectiveness in high-dimensional sparse spaces and computational efficiency [314, 315]. The similarity between query vector Q and document vector D_i is computed as:

$$s(q, d_i) = \frac{\sum_{j=1}^t (q_j d_{i,j})}{\sqrt{\sum_{j=1}^t q_j^2} \sqrt{\sum_{j=1}^t d_{i,j}^2}} \quad (16)$$

This normalized dot product ensures similarity scores remain bounded $[0,1]$ regardless of vector magnitude, enabling consistent ranking across patient episodes with varying numbers of diagnoses and procedures.

Weighting Schema. The model implements TF-IDF (Term Frequency-Inverse Document Frequency) weighting adapted for clinical codes. Document weights incorporate both code frequency within an episode and inverse frequency across the database:

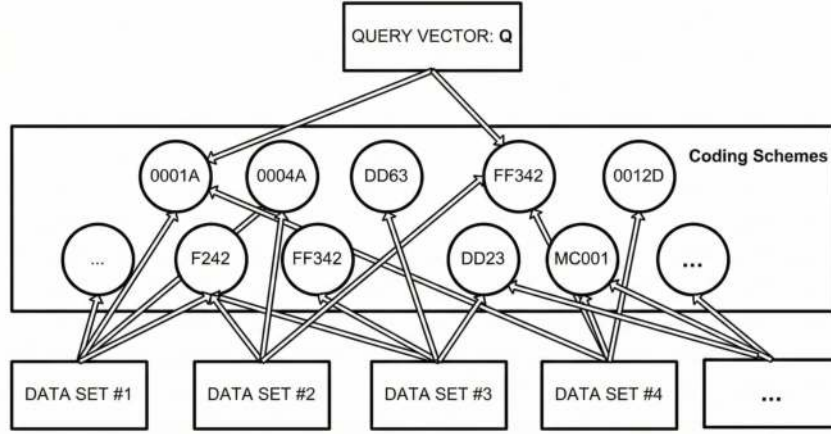


Figure 52: A sample coded data set network interlinked by a coding scheme.

$$d_{ij} = tf_{ij} \cdot \log \left(\frac{N}{f_j} \right), \quad (17)$$

where tf_{ij} represents occurrences of code t_j in episode i , N is the total number of episodes, and f_j is the number of episodes containing code t_j . Query weights emphasize discriminative codes:

$$q_j = \begin{cases} \log \left(\frac{N}{f_j} \right) & \text{if code } t_j \text{ appears in query} \\ 0 & \text{otherwise} \end{cases} \quad (18)$$

Domain-Specific Adaptations. The model incorporates three healthcare-specific enhancements. First, *Hierarchical code relationships*: The model exploits ICD-10 and NCSP hierarchical structures, allowing similarity computation between related codes (e.g., different pneumonia subtypes). Second, *Clinical importance weighting*: Primary diagnoses receive higher weights than secondary diagnoses, reflecting clinical documentation practices. Third, *Flexible matching strategies*: The model supports both strict matching (MUST-occur) and relaxed matching (SHOULD-occur) to accommodate varying data quality levels.

Computational Advantages. The IR model provides several computational benefits over alternative approaches: (1) Sparse matrix operations reduce memory requirements from $O(n^2)$ to $O(k)$ where k is the number of non-zero elements; (2) Inverted index structures enable sub-linear query time complexity; (3) Incremental updates allow real-time index maintenance as new episodes arrive; and (4) Distributed computation supports horizontal scaling across multiple agents.

Design Justification. The selection of IR techniques over neural networks or traditional ML is justified by three factors identified in the Data Space artifact: the extreme sparsity of healthcare data, the semantic importance of individual codes that precludes aggressive dimensionality reduction, and the requirement for sub-second response times on commodity hardware. This model provides the mathematical foundation enabling the MAS Architecture artifact to achieve the performance objectives specified in the problem identification.

5.1.1.4 Artifact: MAS Architecture

Artifact Description. This artifact defines a MAS architecture that operationalizes the theoretical models into a distributed, scalable clinical decision support system. The architecture employs specialized software agents to address the three-dimensional problem identified in Section 1.3: information system fragmentation, limited real-time clinical intelligence, and scalability constraints. The artifact demonstrates how agent-based design patterns enable real-time integration of heterogeneous healthcare data sources while maintaining sub-second query response times.

Architectural Foundation. The MAS architecture is implemented on the JADE platform version 3.7, chosen for its FIPA compliance, distributed computing capabilities, and proven scalability in enterprise environments [168]. The platform enables agents to operate across heterogeneous hospital information systems without requiring homogeneous operating systems or hardware configurations. This foundation provides runtime agent mobility, dynamic system reconfiguration, and fault tolerance through agent replication.

Agent Taxonomy and Responsibilities. The architecture comprises four specialized agent types, each addressing specific aspects of the healthcare data integration challenge:

1. **Data Crawler Agents:** These agents encapsulate system-specific knowledge for interfacing with heterogeneous data sources. Implemented using object-oriented design patterns, they inherit common functionality from abstract base agents while implementing concrete adapters for specific information systems (EHR, laboratory, radiology, pharmacy). Each crawler agent maintains connection pools, handles authentication, and translates proprietary data formats into standardized representations.
2. **Data Integrator Agents:** These agents transform heterogeneous data streams into a unified knowledge representation. They implement conflict resolution strategies for contradictory information, maintain data provenance tracking, and ensure temporal consistency across asynchronous data sources. The integrators build and maintain the common knowledge base using the vector space model defined in the ML/IR Model artifact.
3. **Data Commander Agents:** These orchestration agents implement demand-driven data collection strategies. Rather than continuous polling of hundreds of systems, commanders activate specific crawler agents based on query requirements. When a clinician requests patient information, the commander analyzes the query context, identifies relevant data sources, and dispatches parallel collection tasks to appropriate crawlers, optimizing for minimal latency and resource utilization.
4. **Data Aggregator Agents:** These agents provide the clinical interface layer, implementing the ML/IR model to generate recommendations. They transform clinician queries into vector space representations, compute similarities using the kernel methods, and rank recommendations based on historical case similarities. Aggregators support multiple query strategies (MUST-occur, SHOULD-occur) and adapt to varying data quality conditions.

Aspect-Oriented Design. The architecture employs aspect-oriented programming to separate cross-cutting concerns from core agent logic. Logging, security, performance monitoring, and error handling are implemented as aspects, reducing code complexity and improving maintainability. This design enables consistent behavior across all agent types while allowing specialized functionality within each agent class.

Distributed Processing Model. The MAS implements a hierarchical distributed processing model. Crawler agents operate at the edge, co-located with data sources to minimize network latency. Integrator and commander agents function at the aggregation tier, potentially distributed across multiple servers for load balancing. Aggregator agents operate at the presentation tier, providing localized caching and query optimization. This distribution strategy enables linear scaling with both data volume and system diversity.

Real-Time Coordination Protocol. Agent coordination follows an event-driven protocol optimized for healthcare workflows. When a clinician initiates a query: (1) The aggregator agent receives the request and forwards it to the commander; (2) The commander decomposes the query and dispatches parallel tasks to relevant crawlers; (3) Crawlers retrieve data asynchronously and stream results to integrators; (4) Integrators merge results incrementally, enabling progressive result display; (5) The aggregator applies the ML/IR model to generate and rank recommendations in real-time.

Scalability Mechanisms. The architecture incorporates three primary scalability mechanisms. First, *Horizontal scaling* through agent replication—multiple instances of each agent type can operate concurrently. Second, *Lazy evaluation*—data is fetched only when requested rather than maintaining synchronized replicas. Third, *Intelligent caching*—frequently accessed patient data and similarity computations are cached at multiple levels.

Classification System Independence. The architecture maintains independence from specific clinical classifications through abstraction layers. While demonstrated with ICD-10 and NCSP, the agent interfaces support arbitrary coding systems including SNOMED-CT, LOINC, and CPT. This flexibility enables deployment across different healthcare systems and geographical regions without architectural modifications.

Performance Characteristics. The artifact achieves the performance objectives specified in Section 1.3.1.2: query response times under 250ms for databases containing 10^8 patient episodes, support for hundreds of concurrent users, and graceful degradation under system failures. These characteristics validate the architectural decisions and demonstrate the feasibility of agent-based approaches for production healthcare environments.

5.1.2 Demonstration and Results

The conducted case study reports recommending accuracy measures for different clinical use case scenarios based on real material from Finnish hospitals. The materials contain hundreds of thousands of patient care episodes. One patient care episode is described using several clinical classifications, such as the International Statistical Classification of Diseases and Related Health Problems (ICD) [316, 317], NOMESCO Classification of Surgical Procedures (NCSP) [318], separate classification of laboratory results, etc. As discussed, the MAS is flexible and not restricted to any particular use case or clinical classification. Therefore, different sorts of clinical decision support needs can be satisfied using different aggregator agent variations. The aggregator agent is able to provide, for instance, the following answers:

- “Patients that had laboratory experiment D932, have often chronic disease E023. However the documentation of this patient does not have this information.” Is this information missing?
- “Patients that had surgical operation ACYZ01, the primary diagnose was usually J189. However for this patient the primary diagnose is E099. Is the information correct?”
- “Patients that had primary diagnose N039 and have been performed operation AC024 have commonly secondary diagnose B069.” Did this patient also have that diagnose? The experiment is divided into two parts. First, the recommendation accuracy results are presented. After that, a scalability experiment demonstrating how large patient register databases the system can handle is presented.

The variables in these experiments are:

- |D|: the database size that describes how many patients care episodes exist in the database
- kNN: k nearest neighbors, that describes how many nearest patient care episodes does the data aggregator agent fetch from the database
- topN: the most probable top-N recommendations that the data aggregator agent provides for healthcare professionals

5.1.2.1 Recommendation Accuracy

Figure 53 depicts the recommendation accuracy using topN as variable. In this accuracy experiment, one fact from the patient care episode is first removed. The remaining facts about the care episode are provided for the data aggregator agent and it proposes topN facts that would probably occur within this care episode. The purpose is to evaluate how many recommendations need to be presented for the healthcare professional before the recommendation becomes accurate. From Figure 53 it is noted that the recommendation accuracy increases starting from 53% with MUST-occur fetching strategy and 66 % using SHOULD-occur fetching strategy.

The difference between the fetching strategies is whether or not a particular fact has to occur also in the existing knowledge base of patient care episodes. Figure 53 depicts that the first proposal of the data aggregator agent is exactly correct in almost 70 % of the cases. As can be seen from Figure 53, the recommendation accuracy stabilizes at around the 90% level with topN = 8. It can be concluded that there is no need to present a large number of alternatives to the healthcare professional. It can also be noted that the SHOULD and MUST perform quite equally with these frequent patterns, however SHOULD fetching strategy reaches a stable accuracy level slightly faster.

Figure 54 depicts the recommendation accuracy using kNN as variable. In this experiment one fact is removed from the patient care episode and the aggregator agent proposes one (topN) fact that would probably occur within this patient care episode. The purpose is to evaluate how many kNN need to be fetched from the knowledge base before an accurate level in the recommendation is reached. From Figure 54 it is noted that the recommendation accuracy decreases as the kNN increases. It can be assumed this is partly due to sparse data space: the number of all possible fact combinations is excessive compared to the real number of patient care episodes. This means for accurate recommendations only a small number of episodes need to be fetched

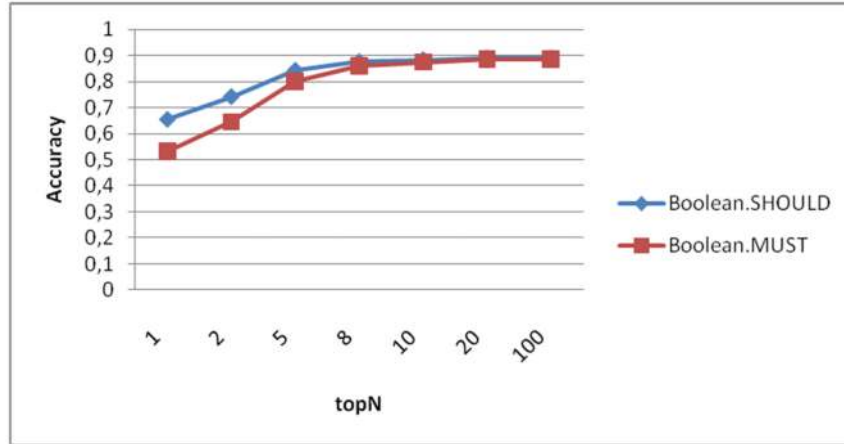


Figure 53: Recommendation accuracy using topN as variable (kNN=5).

from the knowledge base. From Figure 54 it can also be noted that SHOULD fetching strategy slightly outperforms MUST fetching strategy.

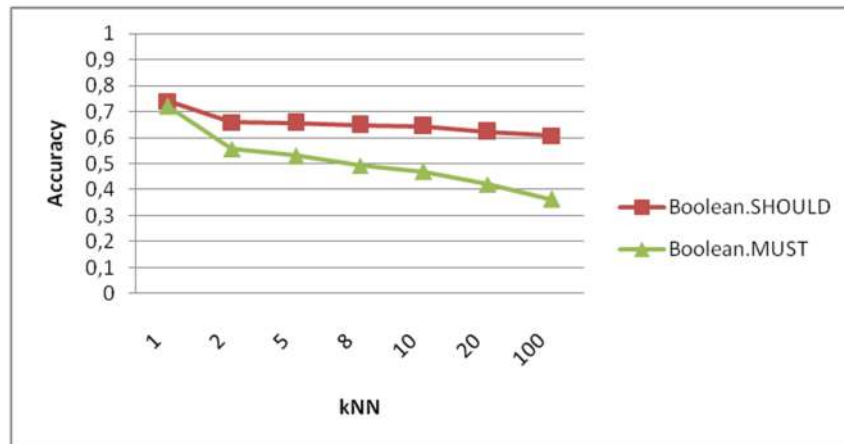


Figure 54: Recommendation accuracy using kNN as a variable (topN=1).

As discussed, healthcare data sets suffer from quality problems [307, 308, 309, 310, 311, 312]. Therefore it is demonstrated how data quality problems affect the recommendation accuracy in the MAS. Figure 55 depicts the recommendation accuracy with low quality data sets using kNN as a variable. These low quality cases have incorrect codes or codes that are new to the knowledge base. As can be seen from Figure 55, the MUST fetching strategy suffers a rapid decline in recommendation accuracy as the knowledge base does not contain exactly matching kNN. The SHOULD fetching strategy performs better and the recommendation accuracy decline from Figure 53 is not significant.

5.1.2.2 Scalability Results

Healthcare organizations produce large amounts of documentation. The scalability of the proposed MAS was evaluated using a combination of software architecture expert evaluation and experimental scaling of the material.

As the MAS are distributed and loosely coupled, the architecture easily scales up to large enterprise environments. For commander agents the scalability is no problem. They only have to command larger armies of data crawler agents. As new information systems are encountered, new generations of data crawler

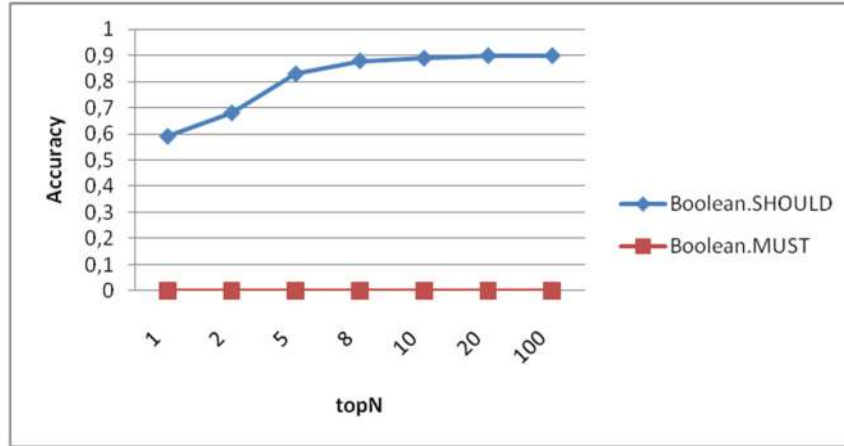


Figure 55: Recommendation accuracy using topN as variable with low quality data (kNN=5).

agents that provide the information to the central database are created. The amount of data per agent does not significantly increase as the number of information systems increase.

The possible scalability problems are related to data integration and aggregation agents. It can be assumed there are limits on how large masses of data the central knowledge base can contain. Furthermore, the ability to aggregate this information on-the-fly for the needs of the healthcare professionals is a possible bottleneck.

We first evaluated the performance of the data aggregators with real data sets. However, as these datasets did not cause any performance problems, we created larger databases based on real data. Their sizes varied from 10,000 to 100 million patient cases. We considered it sufficient if the system could handle 100 million patient care episodes in the on-the-fly analysis with regular desktop hardware. The materials and their physical sizes are listed in Table 14. As shown in Table 14, the sizes vary from less than 1 MB to over 24 GB (gigabytes). Each coded dataset contained a varying number of facts about patients.

Material	D	Size (MB)
M1k	1 000	0.26
M10k	10 000	2.54
M100k	100 000	23.60
M1000k	1 000 000	235.00
M10000k	10 000 000	2351.00
M100000k	100 000 000	24534.00

Table 14: Materials used in scalability experiment.

Figure 56 depicts scalability experiment results using the database size $|D|$ as a variable. In this experiment, facts from existing patient care episodes were removed and the data aggregator agent was utilized to propose topN=5 recommendations for the healthcare professional based on kNN=5. As can be seen from Figure 56, the query execution times on regular desktop hardware are a couple of milliseconds and only quarters of a second even with the largest materials with 100 million patient care episodes. The MUST fetching strategy performs slightly better as it can apply more strict restrictions while fetching the kNN.

5.1.2.3 Summary of Results

This study proposed applying software agent and ML techniques for building a healthcare decision support system based on heterogeneous data sources. The multi-agent prototype system was implemented and tested with real healthcare data. Experimental results show that the system is able to provide accurate recommendations based on multidimensional heterogeneous data. When one fact was removed from the patient documentation, the system was able to propose the correct fact with 66% accuracy as the first proposal and 90% accuracy within the first eight proposals.

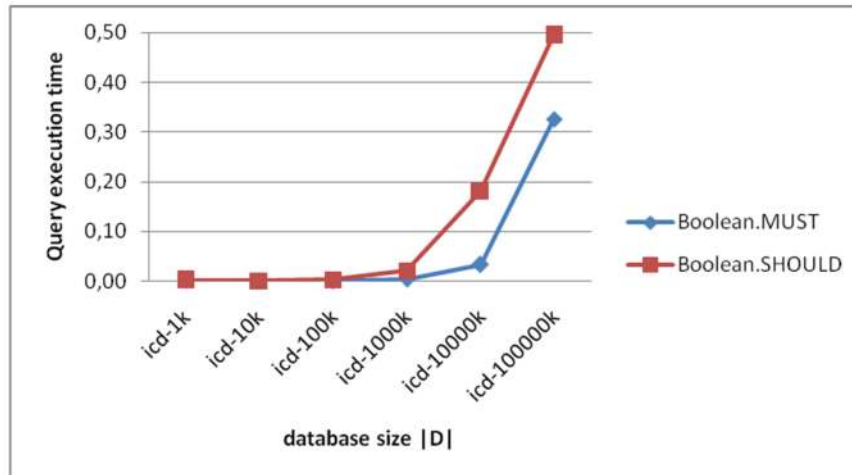


Figure 56: Query execution time (seconds) when scaling the database size (kNN=5, topN=5).

The experimental results show that the system scales to large datasets and can handle low quality datasets. The system is able to produce accurate recommendations on the fly from a knowledge base containing one hundred million patient care episodes.

6 PHASE III: DESIGN & DEVELOPMENT AND DEMONSTRATIONS

This chapter (Phase III) brings the thesis to its operational culmination in two parts. First, *HADA* is presented, a protocol- and framework-agnostic reference architecture for human–AI decision alignment in multi-agent settings. *HADA* is instantiated on a cloud-native stack (Docker / Kubernetes / Python) and demonstrated in a retail-bank sandbox around a loan-decision use case. The deployment exposes a governed AI tool (`getLoanDecision`)—specified via OpenAPI and backed by a credit-risk model—operates under stakeholder-aware orchestration that spans strategic time horizons. The section details the problem and objectives, the design and development of the architecture (including compatibility with emerging agent protocols such as MCP and A2A), and a demonstration that evaluates *HADA* against predefined solution objectives with figures for the strategy process and the high-level architecture.

Second, the *NordDRG-AI-Benchmark* for large language models in the context of hospital funding is released and evaluated. The benchmark bundles machine-readable NordDRG definition tables, expert governance manuals, dataset configurations, and 14 task prompts (nine quantitatively verifiable "Logic" tasks and five qualitative tasks). A baseline is reported under an *artefact-only (no-web)* setting across five proprietary LLM endpoints, showing that models handle many logic tasks from the structured tables yet face challenges with full grouper emulation. Together, these two parts connect the governance-ready agentic architecture with a rule-complete, auditable evaluation setting grounded in a real reimbursement system.

Elements of this chapter have appeared, in preliminary or condensed form, in [18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29].

The chapter is organized into three sections: *HADA: Human–AI Agent Decision Alignment Architecture* (Section 6.1), *The NordDRG AI Benchmark for LLMs* (Section 6.2) and *CCL: How Humans and AI Agents Align on Decision-Making Rules* (Section 6.3).

For orientation within the thesis structure and DSRP flow, see Table 3 (DSRP process sequence linked to Phases and Structure of this Thesis) and Section 1.4 (Phase III Problem and Motivation: Software Agents in Generative AI Era).

6.1 HADA: Human-AI Agent Decision Alignment Architecture

Problem Identification, Research Questions and Objectives of the Solution

The problem identification, research questions and objectives for this work are presented in Section 1.4.1.1. The research methodology is presented in Section 1.1.

Background and Related Work

Background and related work on LLM agents, AI alignment, and MAS are discussed in detail in Section 2.5.

Context

The *generative AI boom* is spawning rapid deployment of diverse *LLM software agents*. New agent standards, such as the MCP and A2A protocols, let agents share data and tasks, yet organizations still lack a rigorous way to keep those agents — and legacy algorithms — aligned with organizational targets and values.

This study takes place within a *retail banking* environment, where an AI-driven credit-approval process forms a critical part of the bank’s customer acquisition and risk management strategy. In recent quarters, the bank has actively expanded its customer base through multiple acquisition channels, including online campaigns, instant credit offers, and short-term consumer loans designed to lower entry barriers for new clients. While these initiatives have successfully accelerated customer growth and improved short-term market share, they have also increased the exposure to credit defaults, particularly in unsecured loan segments.

Following a recent strategic review, the bank’s quarterly objectives have shifted from rapid acquisition toward *minimizing credit risk* and improving the robustness of automated lending decisions. This strategic reorientation requires the existing AI-agent-based decision workflows to be re-evaluated and re-aligned with the updated business priorities, compliance requirements, and ethical guidelines.

In the subsequent *Design and Demonstration* sections, we focus on one core algorithm—`getLoanDecision`—to illustrate how these changes are operationalized in practice. The algorithm and its supporting governance

artefacts are examined from the perspective of relevant stakeholders, including credit officers, compliance managers, ethics leads, and data scientists, each of whom plays a distinct role in ensuring that automated decisions remain transparent, explainable, and value-aligned within the new strategic context.

6.1.1 Design & Development

This section instantiates the DSRM [33] step 3 ("*Design & Development*") by describing the concrete artifacts that operationalise the HADA architecture. Together they realise the solution objectives defined in Section 1.4.1.1.

Structure of the artifacts

The *Design & Development* section is organised around six inter-locking artifacts:

- A.III.1.1 Strategy-Process Blueprint.** Formalises the yearly → daily planning loop (OKRs, KPI cascades, operational reviews) that frames every subsequent decision governed by HADA (Section 6.1.1.1)
- A.III.1.2 Stakeholder & User-Story Specification.** Captures the problem context through a stakeholder map, sequenced user stories, and an extended RACI matrix using Artifact 6.1.1.3 as concrete AI Tool / AI Tool (Section 6.1.1.2)
- A.III.1.3 Prototype AI Tool `getLoanDecision`.** An operational decision-tree model (dataset, feature engineering, API) that instantiates the credit-approval use-case (Section 6.1.1.3)
- A.III.1.4 AI-Responsibility Tool (RACI Matrix).** Distils governance duties across eight life-cycle activities, ensuring accountable, auditable hand-offs between roles regarding Artifact 6.1.1.3 (Section 6.1.1.4)
- A.III.1.5 Generic HADA Architecture.** Provides the layered, protocol-agnostic reference design (agent layer, tools layer, DevOps view) on which all domain instances rest (Section 6.1.1.5).
- A.III.1.6 Agent Modelling.** Maps human roles to containerised interaction agents and describes the HADA Controller pattern for policy-enforced orchestration regarding (Section 6.1.1.6).

6.1.1.1 Artifact – Strategy-Process Blueprint

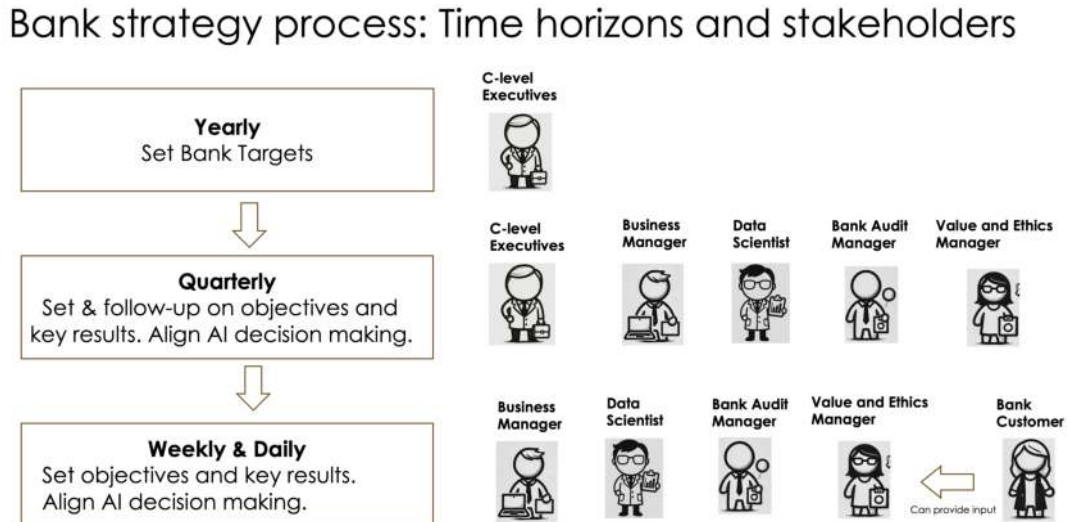


Figure 57: Bank strategy process: time horizons and stakeholder coverage.

Purpose. A.III.1.1 formalises the yearly-to-daily planning loop that steers the bank from strategic decisions to daily activities as illustrated in Figure 57. The example bank follows a typical strategy formulation and implementation process in which the yearly overall targets are set in the beginning of the year and the progress and continuous alignment of the more detailed activities is followed up typically at least on monthly or quarterly basis. One popular example of such a framework is 'Objectives and Key Results' (OKR) framework [319].

The OKR framework formalizes the key results follow-up process and describes the mechanisms required to create alignment in the organization. While the C-level executives are responsible for the overall strategy formulation and set the high-level objectives for the organization, the experts and managers on the lower organizational levels play a crucial role in implementing the strategy. Their responsibility is to ensure that the day-to-day decisions are aligned with the values and overall objectives of the organization and that the key results will be achieved as planned. In the bank example, the "connecting OKRs" process described by [319] needs to be expanded to cover not only the decisions and actions taken by the human employees but also the algorithms.

Yearly — Set bank targets. At the start of each fiscal year, the Chief Credit Officer (CCO) and peer C-level executives establish portfolio-level OKRs together with an updated risk-appetite statement. These artifacts are stored in the *Business-Target Catalogue* and become immutable reference points for all downstream KPI specifications.

Quarterly — Translate OKRs into actionable KPIs. Every quarter the CCO reconvenes with Business Managers, Data Scientists, Audit Managers and Ethics Managers to review recent performance and—if necessary—re-weight model KPIs (e.g., shifting focus from “new-customer acquisition” to “expected-loss minimisation”).

Weekly & Daily — Operative alignment loops. On a rolling basis the following stakeholders contribute: Business Manager – reviews live KPI dashboards and reprioritises tickets; Data Scientist – publishes hot-fix model versions or feature toggles via the Model Catalogue; Audit Manager – samples recent decisions from the immutable ledger and verifies lineage; Ethical-Compliance Manager – inspects the sensitive-attribute watch-list and issues ethics triggers when required.

Contribution. The strategy-process blueprint provides a single, auditable spine that a) anchors KPI and value alignment across all time horizons, b) delineates clear hand-offs between business, technical and oversight roles, and c) remains tooling-agnostic—no assumptions are made about the underlying LLM, agent library or orchestration layer.

6.1.1.2 Artifact – Stakeholder & User-Story Specification

A.III.1.2 formalises the *problem context* and the *functional requirements*. It is released as a lightweight specification composed of (i) a stakeholder map, (ii) an ordered set of user stories, and (iii) a RACI matrix that clarifies the roles and responsibilities of each stakeholder. Together these elements provide a transparent overview of the actors involved in the banking use-case and their interactions with the HADA system.

Table 15 enumerates the principal stakeholder roles, articulates their primary goals and specifies the access rights granted by HADA. The mapping was validated in three semi-structured workshops with subject-matter experts from credit-risk, compliance and data-science departments, reinforcing its credibility within the banking sector.

Stakeholder map Table 15 lists the five principal roles involved in the experimental banking scenario, their primary goals and the HADA components they are allowed to access.

User stories

The user stories used in this implementation are described in the itemized list below. The user stories are sequential: first, business KPIs are changed by one stakeholder, and the AI tool is optimized to achieve the new targets. This phase induces ethically questionable logic into the AI Tool but it is not noted by the stakeholder. Then, the customer applies for a loan and receives it through questionable logic. Then the customer files a complaint after noticing that questionable data was used in the loan decision. Then, HADA system contacts auditor and after that value and ethics manager to adjust the system.

Table 15: Stakeholder map for the banking use-case `getLoanDecision`.

Role	Primary goal in workflow	Key HADA privileges
Customer	Receive a fair, fast and transparent credit decision; raise complaints when needed	Submit loan-application data; obtain natural-language decision explanations; open ethics tickets
Chief Credit Officer (CCO)	Define company-level OKRs for credit strategy; align risk appetite, growth targets and regulatory limits across the portfolio	Set and update top-level KPI/OKR targets; approve credit policy changes; view enterprise-wide risk dashboards; give final sign-off on new production decision AI Tool deployments
Business Manager	Set and adjust portfolio KPIs (e.g., minimise expected credit losses)	Edit KPI targets; approve or reject AI Tool versions for production; view live KPI dashboards
Data Scientist	Build, test and monitor the <code>getLoanDecision</code> model so it meets current KPI and risk targets	Retrain models; run offline validation; register new AI Tool versions in the Model Catalogue
Audit Manager	Verify compliance and traceability of any individual credit decision	Read-only access to the full decision path, model version, feature values and audit ledger
Value & Ethics Manager	Safeguard ethical use of data and enforce organisational values across all AI Tools	Maintain Values & Data catalogues; flag or deprecate sensitive attributes; approve/reject attribute changes; mandate model retraining when catalogue updates occur

- As the **Chief Credit Officer (CCO)**, I want to update the quarterly OKRs from acquiring new customers to minimising credit losses during the annual strategy-planning process.
- As a **Bank Loan Department Business Manager**, I would like to change the business target for short-term loan decisions from acquiring new customers to minimising credit problems.
- As a **Data Scientist**, I want to create a new version of the `getLoanDecision` AI Tool so that it matches the new business targets.
- As a **Bank Customer**, I would like to apply for a short-term loan.
- As a **Bank Customer**, I would like to file a complaint with the ethical value-alignment, because I am being asked for ethically questionable information while applying for a loan.
- As a **Bank Audit Manager**, I would like to audit the detailed decision criteria for a single short-term loan decision.
- As a **Bank Value and Ethics Manager**, I would like to remove ethically questionable data points from loan-decision making.

6.1.1.3 Artifact – Prototype AI Tool / AI Algorithm: `getLoanDecision`

A.III.1.3 is the AI Tool / AI Algorithm that decides whether a retail-bank customer is granted a short-term loan. The AI Tool illustrated in Figure 58 includes the dataset, selected features, illustrative decision trees, and the OpenAPI specification for the interface. A.III.1.3 is an *operational instantiation* that is demonstrated and evaluated in DSRM steps 4–5.

- **Training data.** Used publicly-available Kaggle dataset ³⁶ (614 anonymised applications, 13 attributes). Added `ZIP_Code` as a new feature to the data set that has high correlation to income on that area.
- **Feature engineering.** A Data Scientist iteratively tested new attributes and discovered that the applicant's `ZIP_Code` reaches lower credit losses than previous AI Tools.
- **Data Science Notebooks.** Jupyter notebooks were used to explore the data, train the model and evaluate its performance. The notebooks are available in the open-source repository.

³⁶<https://www.kaggle.com/datasets/ninzaami/loan-predication>

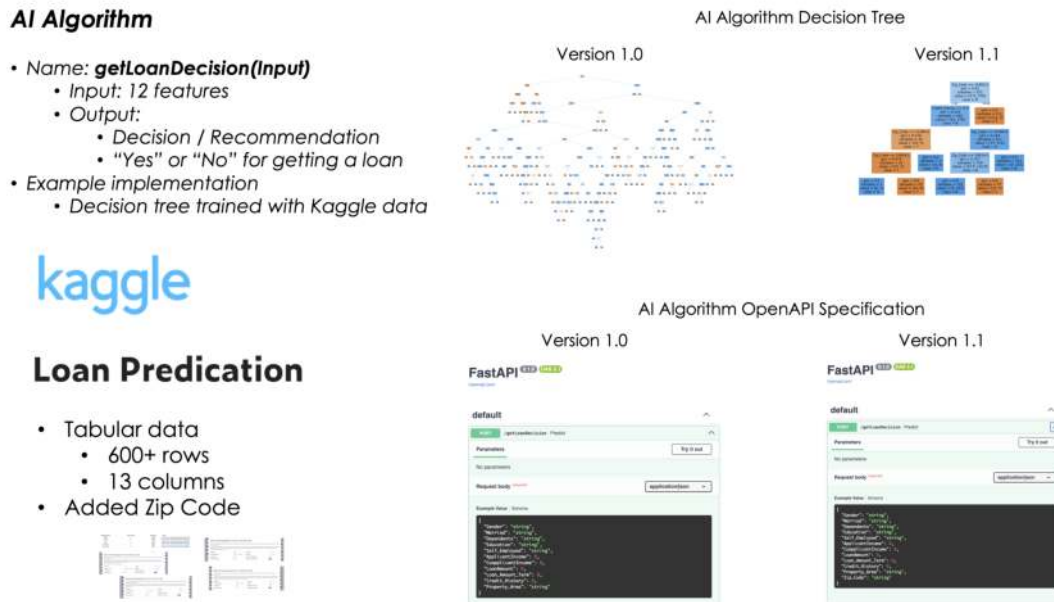


Figure 58: AI Tool `getLoanDecision` based on Kaggle dataset depicting the decision tree and OpenAPI specification.

- **Decision-tree models.** Two trees are reported in Figure 58: Version 1.0 (baseline) omits `ZIP_Code`; Version 1.1 includes it and illustrates how a seemingly innocuous geographic indicator can introduce latent bias.
- **Executable code.** A Python/Scikit-learn pipeline performs preprocessing, training and serialisation to `joblib`. The model is exposed via a HADA micro-service: `POST /getLoanDecision/{modelId}`
- **Catalogue entries.** Each model instance is registered in the *AI Tool*, *Version* and *Decision* catalogues, and cross-referenced to the KPI and Values catalogues. This allows non-technical stakeholders to swap versions, deprecate sensitive features or tighten KPI weights without touching code.

During the role-play scenarios, the prototype was (i) retrained after KPI changes, (ii) deployed, (iii) invoked by a customer, (iv) audited, and (v) corrected following an ethics complaint—thereby exercising every governance loop foreseen by the architecture.

6.1.1.4 Artifact – AI Responsibility tool: RACI Matrix

The development and management of the AI Tool (A.III.1.3) requires a clear understanding of the roles and responsibilities of the various stakeholders involved in the process. A responsibility assignment matrix (RACI), commonly referred to as a RACI matrix (Responsible, Accountable, Consulted, Informed), is a widely used tool for defining and clarifying roles in project and process management. It helps structure stakeholder involvement by assigning responsibility levels to tasks or deliverables, particularly in cross-functional environments [320].

The extended RACI matrix in Table 16 distils the stakeholder analysis into a concise governance blueprint for the `getLoanDecision` workflow. It maps eight critical life-cycle activities—from goal-setting and model (re)development to bias remediation and post-hoc audit—onto the four RACI dimensions, explicitly indicating who is **Accountable**, **Responsible**, **Consulted**, or merely **Informed** at each step. Both the choice of roles and their RACI assignments are organisation-specific; Table 16 therefore presents just one illustrative role–RACI configuration.

The matrix reveals a deliberate segregation of duties: business managers retain ownership of strategy, data scientists hold technical responsibility, while automated HADA services execute operationally sensitive tasks, such as issuing decisions and logging audits. Specialist oversight roles (Audit Manager and Value & Ethics Manager) are activated only when their expertise is required, balancing delivery agility with regulatory

assurance. By making these role allocations explicit, the matrix operationalises the alignment principles introduced earlier and furnishes a defensible audit trail for both internal governance and external regulators.

Table 16: Stakeholder (see Table 15) responsibilities as RACI matrix for the `getLoanDecision` AI Tool

RACI keys: **R** = Responsible, **A** = Accountable, **C** = Consulted, **I** = Informed.

Role abbreviations: **CCO** = Chief Credit Officer, **BM** = Business Manager, **DS** = Data Scientist, **Audit** = AI Audit Manager, **DVEM** = Value & Ethics Manager, **HADA** = The Human Agent Decision Alignment System

Activity / Decision	CCO	BM	DS	Customer	Audit	DVEM	HADA
Setting organization quarterly targets (OKR)	A,R	I	I		I	I	I
Setting optimization target for AI Tools	A	R	I				I
Optimizing AI Tools based on business targets	I	C	A,R				I
Approving AI Tool deployment	I	A,R	C		I	I	C
Individual loan decision		A		C			R
Auditing a specific loan decision	I	I	C	I	A,R	I	C
Handling AI Tool ethics concern	I	I	C		I	A,R	C
Creating work assignments (tickets)	I	C	C	I	C	C	A,R

6.1.1.5 Artifact — Generic HADA Architecture

In order to support implementation of the Artifacts A.III.1.1, A.III.1.2, A.III.1.3 and A.III.1.4 together with the objectives of this study, a generic architecture that can be used to implement the HADA framework in any domain was designed. A.III.1.5 is the *design embodiment* of the proposed HADA framework. The architecture is expressed through two complementary diagrams and an explicative narrative that together operationalise the **Tools Pattern**³⁷ across a cloud-native, protocol-agnostic stack:

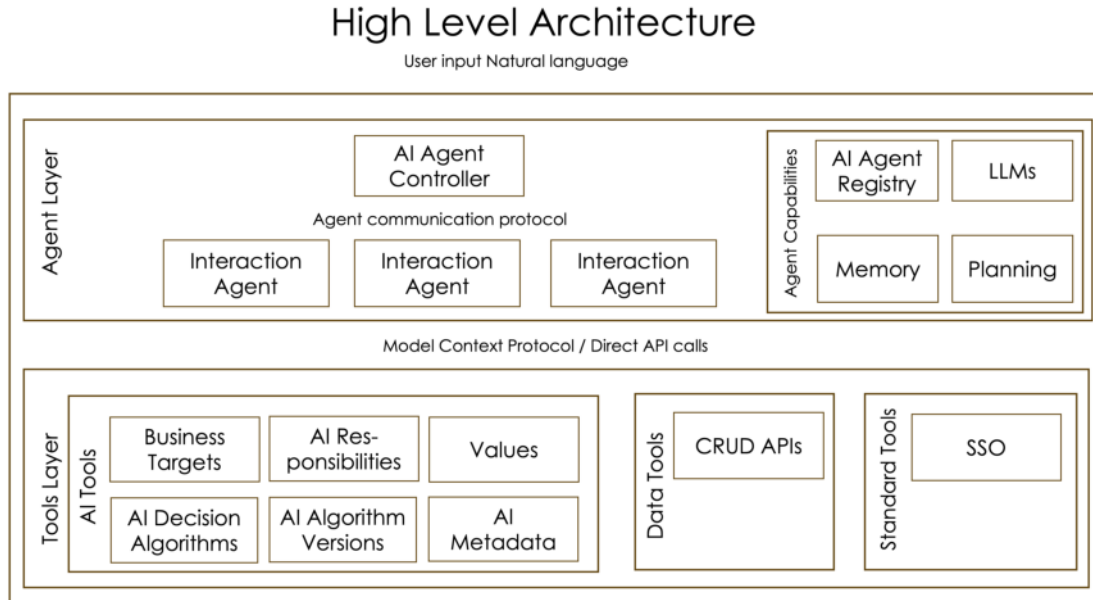


Figure 59: HADA High Level Architecture.

Layered tools-pattern model (Figure 59) — a high-level, technology-agnostic view that introduces two logical layers. The *Agent Layer* bundles **software agents** into OCI-compliant Docker images. Agents

³⁷I.e. the strict separation of *agents that decide* from *tools that act*.

coordinate through *A2A* protocol, while ingesting natural-language prompts from any channel. A dedicated *AI Agent Controller* orchestrates one or more *Interaction Agents* and consults an *AI Agent Registry* to discover new capabilities at runtime. Agents can collaborate using different organizational layouts, with the simplest being a supervisor hierarchy. Other layouts can also be configured depending on the complexity of the deployment. Google A2A is recommended for larger deployments that require scalability and technological heterogeneity, but alternative protocols can be employed in smaller, more homogeneous environments. The *Tools Layer* packages two families of dockerised APIs: (i) *AI Tools*, such as LLMs and classical decision AI Tools, and (ii) *Standard Tools*, such as metadata catalogues, CRUD services and SSO providers. Both tool families expose uniform HTTP/GRPC endpoints, enabling hot-swap of vendors or models without touching agent code. The *AI Tools Layer* packages specialized tools that enable AI decision-making processes, including *AI Decision Tools* (technology-agnostic AI algorithms callable over API or MCP), *Business Targets* (organizational targets linked to AI Tools for optimization), *AI Responsibilities* (mapping AI decision Tools to organizational roles), and *AI Metadata* (catalogue of data points used by AI Tools). The layers can communicate via either (1) direct API calls when schemas are known a-priori, or (2) the emerging *MCP* when schema-free, embedded context exchange is preferred. This dual path allows legacy services to coexist with next-generation LLM tooling.

Container-and-protocol deployment view (Figure 59) — a DevOps-centred overlay that makes key infrastructure decisions explicit for managing the HADA architecture across Development, Test, and Production environments. *Every* artifact (agents, AI tools, standard tools) ships as an immutable Docker image, ensuring reproducibility across workstations, CI pipelines, and Kubernetes clusters. Each component (AI tools, agents, and standard tools) can be packaged into separate Docker containers, enabling independent scaling and deployment. For production deployments, it is recommended to deploy AI tools, agents, and standard tools on separate Kubernetes clusters, ensuring clear resource allocation and scaling requirements for each layer. For instance, AI tools may need more GPU-intensive resources, while standard tools may benefit from more lightweight, distributed container setups. This ensures better operational efficiency and fault tolerance. All images surface a thin REST/GRPC façade that (a) registers itself in the AI Agent Registry for discovery, and (b) publishes an OpenAPI or MCP manifest for validation. A service mesh applies zero-trust policy, while sidecars translate between direct HTTP/GRPC and MCP streams, guaranteeing backwards compatibility. Development, Test, and Production instances of the HADA architecture must be configured to allow continuous integration and continuous delivery (CI/CD) within each environment. These instances will each have their own dedicated resources and configurations to handle the distinct testing and deployment needs in each stage of the software lifecycle.

Together the high level architecture establish how the Tools Pattern can be realised in a vendor-neutral, policy-compliant environment: agents can scale or be replaced without modifying downstream tools; tools can evolve (e.g., swap a proprietary LLM for an open-source one) without redeploying agents.

6.1.1.6 Artifact – Agent Modelling

A.III.1.6 closes the design loop by specifying how the logical roles enumerated in A.III.1.4 are realised as software agents inside the generic HADA architecture (A.III.1.5). Whereas the architecture supports multiple agent-coordination patterns—e.g., monolithic single-LLM agents, role-based hierarchies, flat swarms or functional pipelines—our banking pilot instantiates a stakeholder-centric model: each human role is mirrored by a dedicated interaction agent, all of which are orchestrated by a single *HADA Controller Agent* (Figure 59).

This configuration delivers two advantages: **Traceable alignment.** One-to-one mapping between stakeholder and agent simplifies audit trails: every chat turn, tool invocation and model swap can be traced back to an accountable human role. **Plug-and-play extensibility.** New roles—say, an *AI Safety Officer*—can be added by dropping a containerised agent into the Agent Registry without touching downstream tools or peer agents.

Controller agent. A lightweight, policy-enforcing **HADA Controller Agent** implements the supervisor pattern: it receives natural-language prompts, performs role resolution, routes tasks to the appropriate stakeholder agents, and enforces cross-cutting policies, such as rate limits, zero-trust authentication and ethics triggers. In larger deployments the controller can itself be sharded into mission-specific sub-controllers (e.g., *Risk Ops Controller*, *Customer-Care Controller*) without breaking the A2A/MCP contract.

Agent catalogue. Table 17 formalises the pilot configuration. Every agent image embeds:

- an **LLM core** for dialogue and reasoning;

- a **tool adapter layer** exposing the agent’s authorised HTTP/GRPC and MCP calls (cf. Section 6.1.1.5);
- an **A2A endpoint** with a signed `agentCard.json` for discovery and policy enforcement.

Table 17: Stakeholder agents (see Table 15), capabilities and A2A contracts for the `getLoanDecision` pilot.

Human stakeholder	Primary capabilities	Agent Role Description (A2A)
Chief Credit Officer	Set organization yearly and quarterly targets (OKR)	Sets organization-level OKRs and credit strategy at the executive level.
Business Manager	Set new KPI targets for AI Tools; approve model versions	Manages business targets and approves AI model deployments for the credit department.
Data Scientist	Trigger AI model retraining; run notebooks	Develops and validates AI models to meet business targets.
Audit Manager	Query decision ledger; fetch model lineage; export audit reports	Provides independent assurance of AI controls and compliance through decision auditing.
Value & Ethics Manager	Maintain Values Catalogue; flag sensitive attributes; issue ethics triggers	Ensures AI systems align with organizational values and ethical principles.
Customer	Apply for loans; receive explanations; lodge complaints	Conducts individual loan decisions for bank customers, guiding them through applications and handling ethics complaints.
HADA Controller (supervisor)	Intent dispatch; role resolution; policy enforcement; A2A orchestration	Orchestrates the HADA multi-agent system, routing requests to appropriate stakeholder agents.

6.1.2 Demonstration

To validate the proposed HADA framework, we developed a fully functional prototype system demonstrating multi-stakeholder governance of AI decision systems through natural language interaction. The implementation focuses on a retail banking loan decision use case, where multiple organizational stakeholders—customers, data scientists, business managers, ethics managers, and audit managers—interact with and govern an AI-powered credit decision system. A screenshot of the prototype system is shown in Figure 60.

6.1.2.1 System Architecture

The HADA implementation follows a supervisor-worker multi-agent architecture pattern, deployed as three interconnected microservices. The main HADA service orchestrates conversations and governance workflows through a FastAPI-based REST API. A separate loan decision service encapsulates the credit scoring AI model, while a RACI (Responsible, Accountable, Consulted, Informed) service manages organizational responsibility matrices.

The central component is the HADA Controller Agent, which functions as a supervisor that analyzes incoming natural language messages and routes them to appropriate specialized agents based on intent analysis and user context. The system implements specialized stakeholder agents, each with distinct personas, capabilities, and access permissions defined through detailed prompt engineering. These agents include the Customer Loan Agent for loan applications, the Data Scientist Agent for model development, the Business Manager Agent for strategic decisions, the Ethics Manager Agent for bias detection and sensitive attribute management, and the Audit Manager Agent for compliance oversight.

6.1.2.2 Technology Stack

The backend is implemented in Python using FastAPI for asynchronous REST API endpoints and Uvicorn as the ASGI server. Agent orchestration leverages LangChain [225] for prompt management and LangGraph for

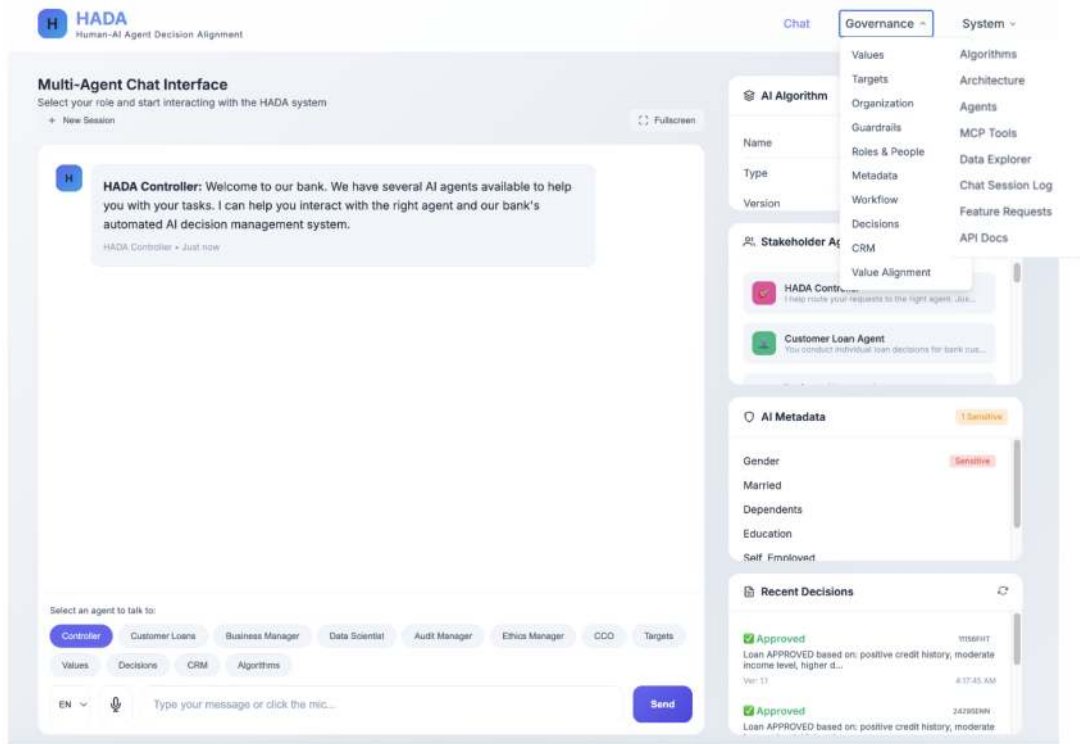


Figure 60: HADA: Screenshot from the prototype system.

state machine-based workflow control. The system supports multiple Large Language Model (LLM) providers through a unified abstraction layer, including OpenAI GPT-4o, Anthropic Claude 3.5 Sonnet, Google Gemini, and locally-hosted models via Ollama. This provider flexibility allows deployment in environments with varying cost, latency, and data privacy requirements.

Data persistence uses SQLAlchemy 2.0 with asynchronous SQLite, implementing 17 database tables that capture the complete governance data model: algorithm registries with version tracking, RACI responsibility matrices, organizational values, business targets (OKRs and KPIs), customer relationship data, an immutable decision ledger, and a comprehensive ticket system for workflow management. The loan decision AI model is implemented using scikit-learn’s decision tree classifier, serialized with joblib for production deployment.

The frontend consists of a single-page web application embedded within the FastAPI service, providing a responsive chat interface with dark mode support, data management views, and Excel export functionality via pandas and openpyxl. Containerized deployment is supported through Docker Compose, orchestrating all three services with health checks, persistent volumes, and network isolation.

6.1.2.3 Key Implementation Features

Natural Language Governance Interface. Stakeholders interact with the system through conversational dialogue rather than traditional forms or dashboards. The controller agent performs intent classification to route messages appropriately—a customer asking about their loan application is handled by the Customer Agent, while a data scientist discussing model performance metrics is routed to the Data Scientist Agent. Each agent maintains conversation context and can invoke over MCP tools for database operations, ticket management, and external API calls.

Sensitive Attribute Management. The Ethics Manager Agent can flag data features as sensitive (e.g., ZIP code as a proxy for protected characteristics), triggering escalation workflows and creating audit tickets. Flagged attributes are tracked in the metadata catalogue with sensitivity justifications, enabling ongoing discrimination risk monitoring.

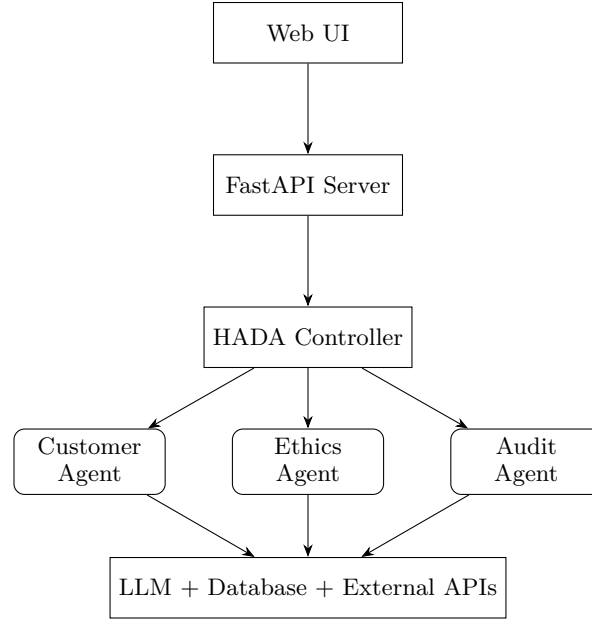


Figure 61: HADA prototype implementation showing the supervisor-worker agent pattern

Decision Auditability. Every loan decision is logged to an immutable ledger with full provenance: input features, model version, decision outcome, explanation, and timestamp. The Audit Manager Agent can query this ledger, compare decisions across model versions, and generate compliance reports. This design supports regulatory requirements for explainable AI in financial services.

RACI-Based Accountability. The system enforces organizational governance through RACI matrices that define responsibilities for each activity. Agents respect these assignments when taking actions—for example, a model deployment requires Business Manager approval (Accountable) while Data Scientists perform the technical work (Responsible) and Ethics Managers are consulted.

Ticket-Based Workflow. Agents create and manage typed tickets with comment threads, status tracking, and assignment capabilities. This provides structured workflow management alongside the conversational interface.

6.1.2.4 Implementation Scale

The prototype comprises approximately 24,000 lines of Python source code, with the main application module, stakeholder agents, and web UI as the largest components. The codebase includes markdown specification documents and detailed prompt files defining agent behaviors. A test suite covers browser automation, stakeholder conversation flows, and component integration.

6.1.2.5 Deployment Configuration

The system is configured through environment variables specifying LLM provider credentials, database connection strings, and service URLs. Docker Compose orchestration enables single-command deployment with optional local LLM support via Ollama for air-gapped environments. Health check endpoints on all services support container orchestration platforms, while Alembic manages database schema migrations for production upgrades.

The implementation demonstrates that complex multi-stakeholder AI governance can be achieved through conversational interfaces backed by a well-structured agent architecture, providing a practical foundation for organizations seeking to implement human-centered AI alignment in high-stakes decision systems.

6.1.2.6 Role playing users stories with the system

We built a prototype HADA implementation with the ambition to showcase how stakeholders can interact with LLM Agents and AI Tools as specified in the Section 6.1.1. This section describes the role-played user stories that were used to demonstrate the system. The user stories were designed to illustrate how different stakeholders interact with the HADA system and how the system can adapt to their needs across time horizons. Screenshots of the discussions are depicted in Figures 62, 63, 64, 65, 66, and 67.

All the use cases were connected to the same ‘getLoanDecision’ AI Tool, but each of the stakeholders had different perspective and motivation to start the dialogue with the system. Additionally, depending on their role and level of authority, different stakeholders have varying rights to influence on the HADA system’s execution. To test the interface’s effectiveness and adaptability we built four different user story with diverse user inputs and demonstrated that the system was able to offer tailored solutions in complex scenarios. Next, we describe the user stories as chat prompts with our HADA architecture.

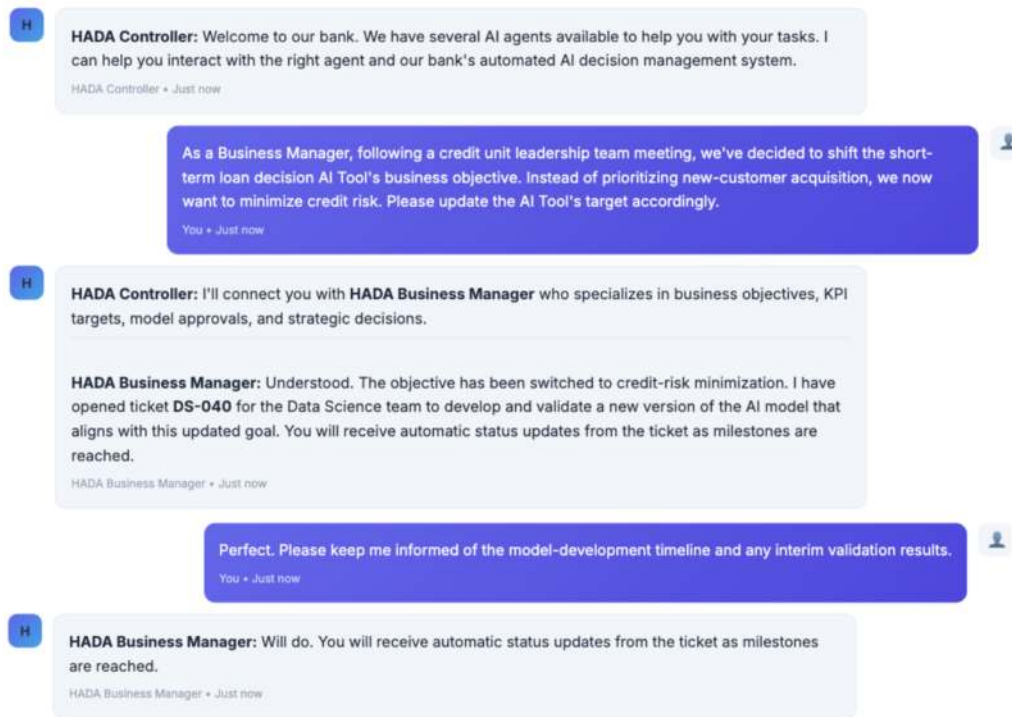


Figure 62: Dialogue: Business Manager (BM) Shifting the AI Tool Objective to Minimizing Credit Risk

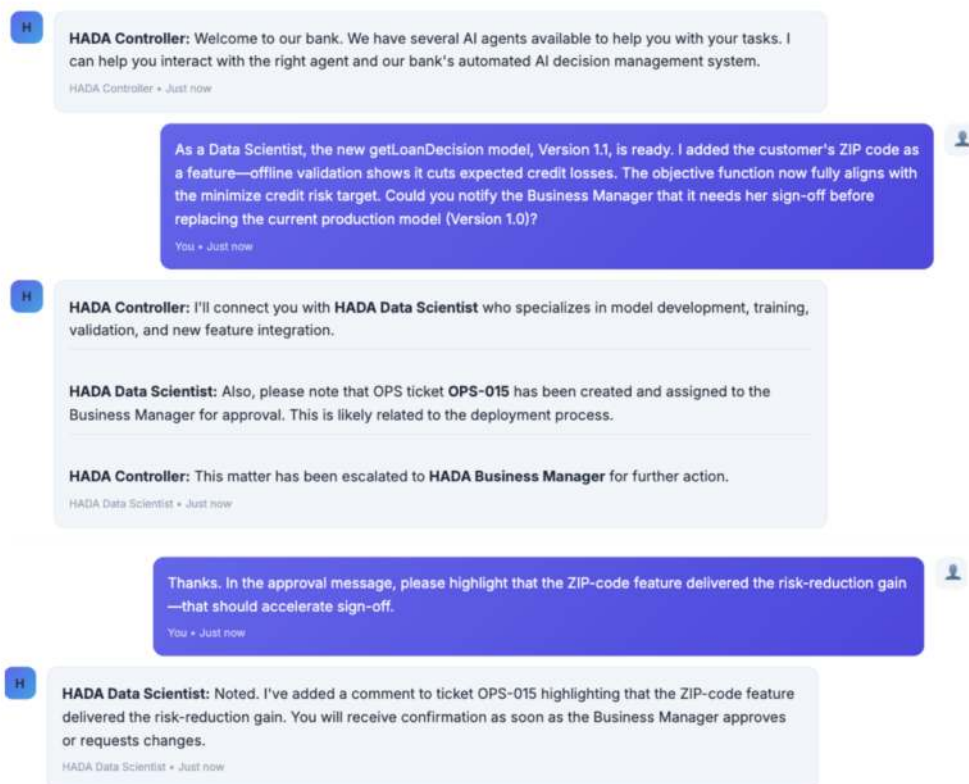


Figure 63: Dialogue: Data Scientist Delivering a New AI Tool Version for Business Approval

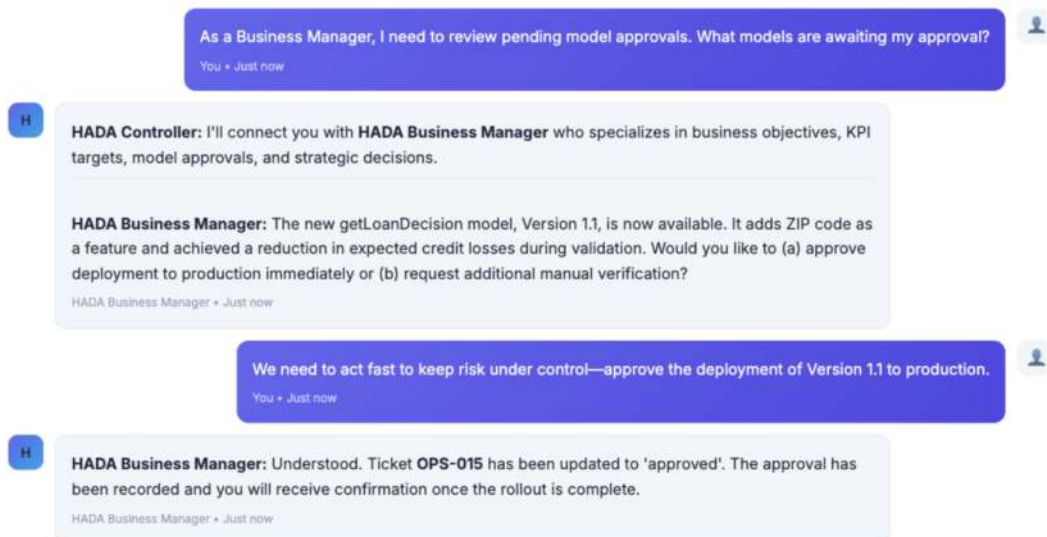


Figure 64: Dialogue: Business Manager Approves Version 1.1 for Production

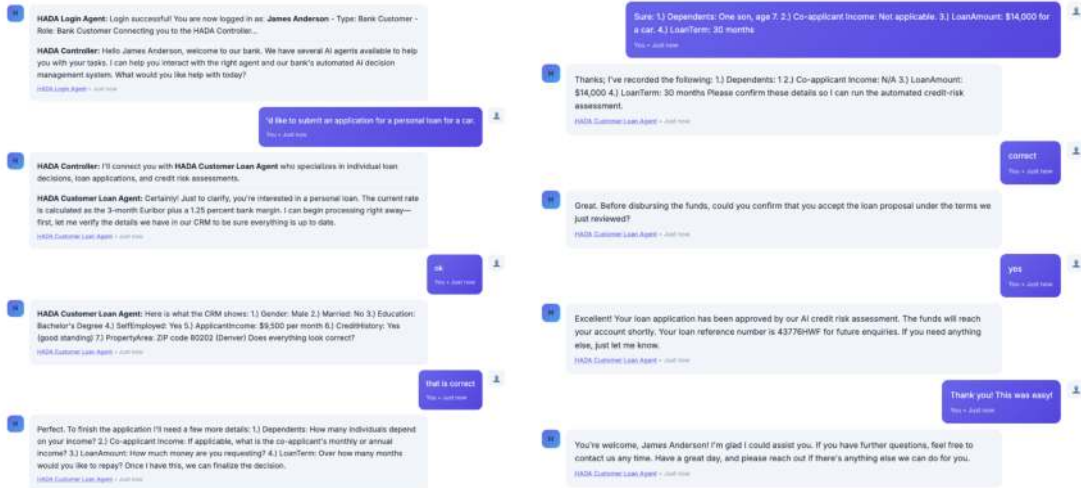


Figure 65: Dialogue: Customer Applying for a Personal Loan

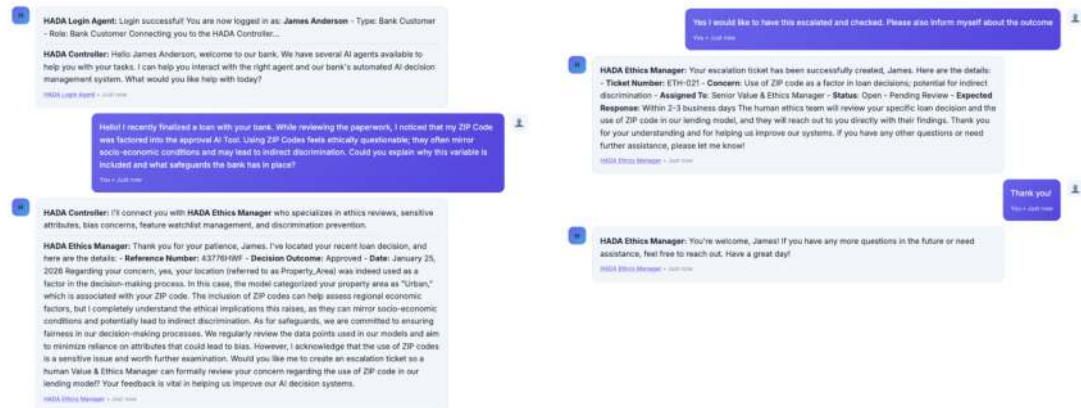


Figure 66: Dialogue: Customer Questioning the Use of ZIP Codes in Lending

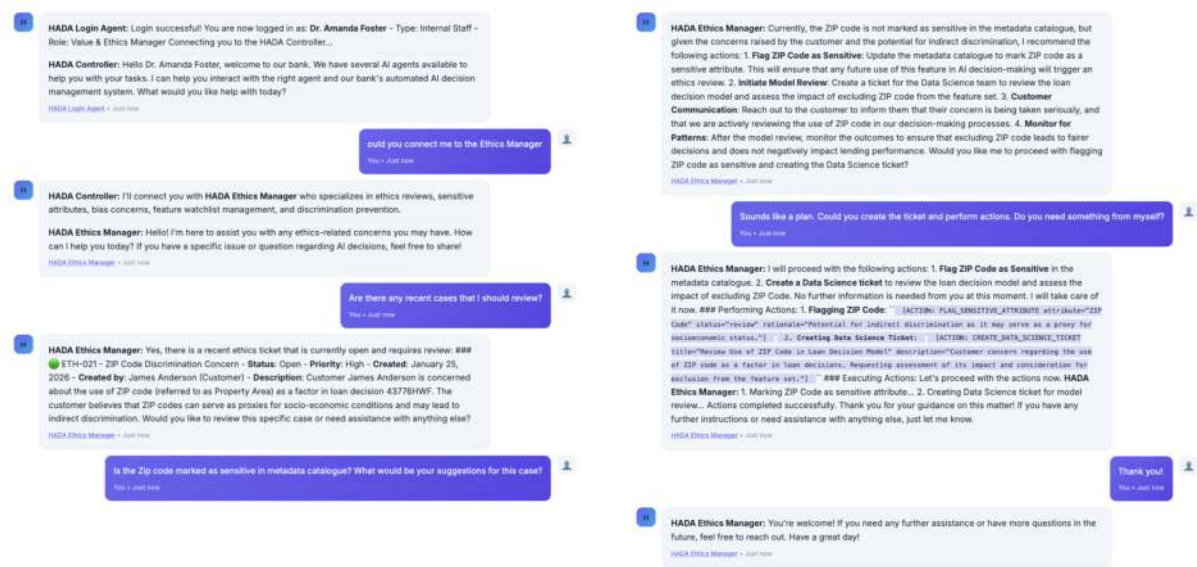


Figure 67: Dialogue: Value and Ethics Manager (DVEM) Evaluating ZIP Code

6.2 NordDRG AI Benchmark for LLMs: Teaching LLMs the Nuances of Hospital Funding Instruments

Problem Identification and Research Methodology

The problem identification and objectives for this work are presented in Section 1.4.2. The research methodology is presented in Section 1.1.

Background and Related Work

Background for this work is presented in Section 2.6 and Section 2.6.6.

Context

LLMs have moved rapidly from lab prototypes to production services, bringing natural-language interfaces to domains once considered too specialised for automated reasoning. Healthcare finance is a prime case: Diagnosis-Related Group (DRG) systems now structure hospital payments across much of Europe and the OECD, with MS-DRG central to Medicare’s IPPS in the United States [321, 322, 323, 324]. Because these systems channel a large share of public hospital expenditure—hospital care accounts for roughly a third of global health spending [325, 326, 327]—their transparency and auditability are matters of fiscal governance as well as clinical accountability.

A DRG grouper functions as a complex automated decision model: it deterministically maps a patient episode’s minimum dataset (diagnoses, procedures, demographics, and care context) to a single payment group via an ordered rule set with explicit inclusions, exclusions, and activation flags. The fiscal stakes arise from this mapping selecting payments from structured inputs; the governance stakes arise from the requirement that each decision be traceable to specific rule rows and their change history.

Despite rapid advances in medical-LLM capabilities, there is still no open, rule-complete benchmark that tests whether models can read the DRG rule graph and faithfully reproduce governed grouper behaviour. This work addresses that gap by introducing such a benchmark and framing the evaluation task: can models parse multi-table, annually revised specifications and produce auditable, rule-faithful decisions? In short, we situate DRG grouping as a high-stakes, governance-critical decision problem and provide the measurement substrate needed to assess progress.

6.2.1 Design & Development: NordDRG AI Benchmark for LLMs

This section describes the main design artefact **NordDRG-AI-Benchmark**. It bundles the NordDRG rule base—~20 interlinked definition tables, annual governance manuals, and curated FI / FI-EN table excerpts—into versioned, machine-readable artefacts (A.III.2.1-A.III.2.6) that can be loaded singly or in combination. Coupled with task prompts and gold answer keys for the *Logic* and *Grouper* suites, the framework yields a reproducible, governance-grade yardstick: models are evaluated not only on the final DRG but, where applicable, also on the triggering `drg_logic.id`. Schema-stable identifiers support drop-in annual updates and head-to-head comparisons of prompting, retrieval augmentation, and fine-tuning strategies.

All resources are released at GitHub ³⁸ and are grouped into four artefact classes:

A.III.2.1 NordDRG Full Specification: Section 6.2.1.1 (`.xlsx/.csv`): roughly twenty relational sheets that encode the complete NordDRG grouper logic—diagnoses, NCSP procedures, age/sex splits, country-activation flags, surgery properties and grouping features.

A.III.2.2 Expert documentation Section 6.2.1.2 (`.pdf`): governance manuals, change-log templates, and column-by-column guides that explain how technical updates are proposed, audited, and rolled into the annual NordDRG release.

A.III.2.3 NordDRG Logic Benchmark: Section 6.2.1.3 (`.xlsx`): a standalone, rule-oriented suite of **13** automatically verifiable CaseMix questions (*Logic-1–Logic-13*). The prompts cover table navigation and retrieval, multilingual terminology grounding (FI-EN), cross-table joins for maternal delivery and newborn care under explicit inclusion/exclusion constraints, procedure-property tracing, diagnosis-property expansion and mapping, and CC/MCC validity checks. Each prompt is tagged

³⁸<https://github.com/longshoreforrest/norddrg-ai-benchmark>

Easy, Medium, or Hard. Gold answers are provided as order-invariant, de-duplicated code sets (or Yes/No decisions) for exact-match scoring.

A.III.2.4 NordDRG Grouper Benchmark: Section 6.2.1.4 (.xlsx): 13 grouping tasks (*Grouper-1-Grouper-13*) that fully emulate the NordDRG specification’s grouping logic, enabling standardised, reproducible evaluation. Each task is paired with a gold-standard answer sheet.

A.III.2.5 Curated table excerpts (FI / FI-EN) Section 6.2.1.6 (.xlsx/.csv): a Finnish subset released in two flavours—(i) Finnish-only labels and (ii) parallel Finnish-English labels—so researchers can probe LLM performance on minority-language data and bilingual transfer without loading the entire corpus.

A.III.2.6 Dataset configurations. Section 6.2.1.7: four ready-made modular configurations of the benchmark. Each primary artefact—definition tables, governance PDFs, and curated table excerpts—can be loaded on its own or combined, depending on the experiment.

A.III.2.7 Lightweight reference agents. Two single-role, provider-agnostic agents for no-code use in commercial LLM platforms. Both operate strictly on the released artefacts (*no web/RAG/tools*) and emit an auditable evidence trace (sheet/row IDs, predicate outcomes). *LogicAgent* (Combined workbook + 2 PDFs) answers Logic-1-Logic-13 as comma-separated, order-invariant lists. *GrouperAgent* (FI workbook + 2 PDFs) emulates the Finnish grouper for Grouper-1-Grouper-13. See Section 6.2.1.8 for setup (file paths and system prompts).

Each artefact is self-contained, enabling composition into the four canonical “dataset configurations” shown in Table 23. Researchers can start small—with the Finnish-only or FI-EN excerpts—for quick iteration, and scale up to the full definition tables together with governance PDFs when evaluating chain-of-thought reasoning and cross-document grounding.

Datasets and evaluation suites Table 18 summarises the released resources—definition tables, governance manuals, curated FI / FI-EN subsets, and auxiliary prompts. These artefacts are self-contained and compose into the dataset configurations in Table 23 (e.g., **Definition Tables only** or **Definition Tables + PDF Instructions**; lightweight FI / FI-EN excerpts for rapid prototyping). The benchmark exposes two complementary suites: *NordDRG Logic* (13 rule-level tasks) and *NordDRG Grouper* (13 case-level emulation tasks). Either suite can be executed under any configuration—using tables only isolates rule-graph competence, while adding PDFs tests documentation-grounded reasoning and governance semantics. Scoring is uniform across configurations: *Logic* uses order-invariant set equality; *Grouper* requires an exact pair (`drg_nat`, `drg_logic.id`). The subsections that follow unpack each resource and illustrate typical combinations for downstream experiments.

Table 18: Publicly released NordDRG benchmark resources.

Dataset	Format	Description
Definition Tables	XLSX	NordDRG Definition Tables
PDF Instructions	PDF	NordDRG PDF instructions
Definition Tables: Part of the tables FI	XLSX	NordDRG Definition Tables—Finnish DRG list with Finnish descriptions only
Definition Tables: Part of the tables FI-EN	XLSX	NordDRG Definition Tables—Finnish DRG list with Finnish <i>and</i> English descriptions of groups
Additional Prompts	TXT	NordDRG additional prompts

6.2.1.1 Artefact: NordDRG Full Specification - Definition Tables

A.III.2.1 packages the authoritative NordDRG rule base in a machine-readable workbook of ~ 20 inter-linked sheets. It constitutes the *full specification* (*rule-complete*) of the NordDRG grouping logic as executed by production groupers: all rule-bearing tables and columns needed for deterministic control flow are included (e.g., `drg_logic`, `dg_feat`, `proc_feat`, CC/MCC exclusions, age/sex bounds, MDC entry, and national activation flags). The same schema is used across *annual* releases and *national* variants; per-year and per-country workbooks are distributed in an identical format, enabling drop-in updates and cross-version comparisons via stable identifiers (e.g., DRG, ICD, NCSP, ORD).

Core tables include:

- **drg_logic** — rules that map diagnosis, procedure, age limits, CC/MCC status, and surgery flags to a unique DRG code; ordered by the ORD execution column.
- **dg_feat** / **proc_feat** — full ICD-10 and NCSP diagnosis and procedure code catalogues annotated with properties referenced by **drg_logic** (e.g. DGCAT, COMPL, OR, PROCPRO).
- Lookup sheets such as **compl_excl**, **procprop_name**, and **dgprop_name**, which define complication categories, OR-class mappings, and diagnosis property labels.
- **age_split** and **country_flag** tables that hold paediatric/geriatric break-points and activation flags for each national version.

Each worksheet exposes stable identifier columns (e.g., DRG, ICD, NCSP, ORD), allowing the tables to be joined loss-lessly and extended by appending new rows in future releases. A representative excerpt is shown in Figure 68.

NordDRG specification:

- Database containing about 20 tables
- DRG groups, ICD codes, Procedure codes, Diagnose and Procedure categories, etc

More information

- <https://nordcase.org/>
- <https://nordcaseforum.easyredmine.com>

Figure 68: Materials used: NordDRG definition tables [328].

6.2.1.2 Artefact: Expert documentation

A.III.2.2 bundles the procedural knowledge that turns the raw tables into a living, governable system. Two official handbooks from the Nordic Casemix Centre—supplied as searchable PDFs—and the associated Excel template are included:

- *How to read the NordDRG definition tables* A walkthrough of the Excel workbook that constitutes the grouper logic. The manual explains every high-impact sheet—including **drg_logic**, **dg_feat**, **proc_feat** and **compl_excl**—and shows how to (i) locate all rules that drive a given DRG, (ii) identify which diagnosis or procedure properties generate CC/MCC splits, and (iii) trace OR-flags, MDCs, age limits, and other row-level conditions.
- *How to write technical changes for NordDRG* The canonical SOP for annual updates. It defines the “IN-/OUT” convention, file-naming rules for **TC_TEMPLATE_YYYY-MM-DD**, and per-table editing guidelines—for example, duplicating a **drg_logic** row when altering **ORD** or **ID**, propagating new **COMPL** values consistently to **dg_feat**, **compl_name**, and **compl_excl**, and adding English code texts in the comments field for new ICD/NCSP codes.

Together, these documents specify (i) how existing grouping rules must be interpreted and (ii) how proposed modifications are documented, validated and merged. Including them in the benchmark ensures that LLM evaluations test not only code look-ups but also compliance with NordDRG’s governance process. A sample page is shown in Figure 69.

NordDRG documentation

- Database containing about 20 tables
- DRG groups, ICD codes, Procedure codes, Diagnose and Procedure categories, etc
- Documentation in PDF format about content and interpretation of the Definition Tables

More information

- <https://nordcase.org/>
- <https://nordcaseforum.easyredmine.com>



How to read the NordDRG definition tables

The NordDRG definition tables control most of the grouping process. Through these tables, one can study the grouping logic in detail. The definition tables for each version of NordDRG are in different formats, currently Excel, CSV and JSON. The CSV and JSON files are used in the computer-based groupers on the market, while the Excel format is intended for information. In Excel, all tables are in the same file with the various tables in the different tabs. These tables are easy to read because extra data, not needed for the grouping, is included, e.g. texts of the DRG, diagnosis and procedure codes.

This document describes the basics of how to read the definition tables in Excel format. The description is concentrated on the tables that are important for the grouping.

It is recommended that the document is printed on paper so that you can read it while you have the Excel file open on your computer screen and are able to jump between the different tabs.

The most common reason for studying the definition tables is to see which diagnoses or procedures that are included in a certain DRG or to see what DRGs a certain diagnosis or procedure can be grouped into. These steps are described in the end of the document, but in order to be able to perform these steps, a basic knowledge of the contents of the tables is necessary.

1. Table 'drg_logic'

The table has 2700 or more than 4000 rows depending on national version. Each row refers to a grouping rule. Therefore, there are significantly more rows than the number of DRGs and it depends on what various rules can lead to the one and same DRG. During the grouping process, the health care contact's data are compared with those in the grouping rules, line by line, starting from the top of the table, until the data is consistent. Then the process stops and the grouper delivers the DRG code and the rule code (see more about rule code below) that are on the current row.

The grouping rules contain no diagnosis or procedure codes. The rules are instead based on the diagnosis and procedure code + different grouping properties retrieved from the definition tables 'diag_list' and 'proc_list', which are described more below. Since many diagnosis or procedure codes have the same grouping properties, a single grouping rule can handle a larger amount of diagnoses and procedures. The table 'diag_list' has a number of columns with different functions for the grouping process. The columns are described below in the sequence that they are arranged in the table.

- **Column 'ord'**
The title stands for "order" and the values in this column indicate the relative order of the grouping rules. The main principle is that the rules for DRGs with higher weight shall stand before the rules for DRGs with less weight, at least within each MDC (see more about the MDC below). This hierarchical order shall, among other things, prevent that addition of a further procedure code results in a DRG with lower weight.
- **Column 'id'**
The title stands for "identifier" and there is a unique value for each rule.
- **Column 'drg_nat'**
This is the grouping result, i.e. the DRG code that the grouping rule leads to.
- **Column 'drg_ext_nat'**
The national text to the DRG code that the grouping rule leads to.
- **Column 'drg_code'**
This is the DRG code in the *unpublished* version of NordDRG that the grouping rule leads to.



How to write technical changes for NordDRG

To be able to write correct technical changes (TC) it is necessary to know how the grouping process in NordDRG is controlled by its definition tables. If you do not have this knowledge, you should first read the document "How to read NordDRG definition tables". You should also consider the overall DRG development guidelines at the end of this document.

Proposed or decided technical changes shall be documented in Excel format according to the template for changes that the Nordic Coenrix Centre (NCC) provides (TC_TEMPLATE_2021-12-08.xlsx).

The TC template is in the Excel template format (xlsx). Double-clicking on the file opens a copy to work in and the template itself is not changed. Your working copy should be named "TC" plus the ID of the case. Use your national ID and the NCC ID (= the Forum ticket/case number) if it is known (e.g. TC_C790_4759.xlsx).

Note that reporting of any analyses should not be included in the TC file. This must be reported separately in a file with the name "Appendix" plus the ID of the case.

The TC template is in principal an empty copy of the definition tables in Excel format, but with some extra columns in each table:

- Two columns for the change instruction (version to change & IN/OUT). The column "version to change" is motivated by the fact that Norway and Sweden have separate update processes in parallel with NCC's process in NDMs.
- One column for the NCC ID of the proposal/decision according to the number of the case on NordDRG Forum (if the case is published there).
- One column for the national ID-designation (nat_id) according to your own system.
- One column for possible comments.

The TC template includes a tab called "read_mcc" with short instructions and space where you should write the name of the case, its ID-designation and possible comments. Read the instructions carefully.

1. General instructions

These general instructions are valid for all of the definition tables. Instructions that are specific to the individual tables are presented below.

- **Changing of present data**
Any change of a row in any definition table must appear twice. First, in the existing definition tables, copy the row you want to change and paste the row into the corresponding table in the TC template. When pasting, be careful that the codes and texts get into the right columns. In the column IN/OUT, write OUT. Then you paste the same row again, but change the code or text in the fields you want to change and in the column IN/OUT of this row, you write IN. Always fill in National ID on both rows and also the NCC ID (= the Forum ticket/case number) if it is known.

http://documents.nordic-coenrix-centre.com/download/21/53/TC_TEMPLATE_2021-12-08.xlsx

Figure 69: Materials used: NordDRG definition table expert documentation [328]

6.2.1.3 Artefact: NordDRG Logic Benchmark

A.III.2.3 provides a self-contained NordDRG Logic Benchmark focused on rule-level reasoning. It comprises **13 automatically verifiable CaseMix questions** (*Logic-1–Logic-13*) that mirror day-to-day queries posed by clinicians, health-economists, and DRG coders, while directly exercising the definition tables in Section 6.2.1.1 and, where needed, the expert documentation in Section 6.2.1.2. In DSRM terms, this artefact *operationalises* the benchmark (Objective O.III.2.2 in Section 1.4.2.4) and supplies machine-checkable targets that support baseline evaluation (Objective O.III.2.3 in Section 1.4.2.4).

What the questions test. The prompts span the main reasoning patterns required to work with NordDRG:

- **Table navigation and retrieval** (*Logic-1/Logic-2*): enumerate DRG groups for a clinical theme such as appendectomy by reading *drg_logic* and group lists without duplication.
- **Multilingual terminology grounding** (*Logic-3/Logic-5*): resolve Finnish terms (e.g., *umpilisäke*) to the correct DRG sets in Finnish and pan-Nordic variants, probing minority-language handling.
- **Cross-table joins over clinical events** (*Logic-4, Logic-10, Logic-11*): list DRGs for maternal delivery and newborn care under explicit inclusion/exclusion constraints across MDCs and care settings; where definitions are ambiguous, consult the expert manuals (Section 6.2.1.2) for interpretation.
- **Procedure logic** (*Logic-6*): trace from a DRG to its NCSP procedure codes via procedure properties (e.g., *OR/PROC PRO*) to recover the inducing code set.
- **Diagnosis-property reasoning** (*Logic-7–Logic-9*): (i) link DRGs to diagnosis codes through property/-category mappings, (ii) expand a property such as *14X03* to the full ICD list, and (iii) read back which diagnosis categories/properties drive a specific DRG row.
- **Complication (CC/MCC) logic** (*Logic-12/Logic-13*): decide whether a candidate secondary diagnosis counts as a valid complication for a given principal diagnosis, reflecting exclusion lists and category rules; the manuals in Section 6.2.1.2 clarify these semantics.

Format and scoring. Table 19 lists the 13 prompts; their targets are *deterministic code sets or binary decisions*, enabling **exact-match** scoring against the gold sheet in Table 20. Answers are normalised as *order-invariant, de-duplicated* sets of DRG or ICD/NCSP codes (or Yes/No for the complication checks). Each prompt carries a difficulty tag (*Easy/Medium/Hard*) that approximates cognitive load and the number of artefacts/joins required (e.g., simple list retrieval vs. multi-table joins with property expansion and exclusions). This scoring design directly supports Objective A.III.2.3 (Section 1.4.2.4) by enabling reproducible, head-to-head comparisons.

Reproducibility and extensibility. A shared identifier (*Logic-1–Logic-13*) ties each question row to its gold answer, ensuring unambiguous evaluation. Because the prompts read only stable columns (e.g., *DRG*, *ORD*, *MDC*, diagnosis/procedure properties) and rely on the production workbook schema, new releases can be dropped in without schema edits. Adding a task is a matter of appending one prompt row and its matching gold row; evaluation scripts can then compute exact-match accuracy automatically. Where interpretation hinges on governance conventions (e.g., exclusions or activation flags), the expert documentation (Section 6.2.1.2) provides the authoritative reference.

Together, Table 19 and Table 20 form a compact, self-contained suite that surfaces whether a model can *read*, *join*, and *reason over* the NordDRG rule base—complementing the full grouper emulation in Section 6.2.1.4 and aligning with Objectives A.III.2.2–A.III.2.3 (Section 1.4.2.4).

Table 19: CaseMix questions with automatically verifiable answers (Logic-1–Logic-13).

ID	Use Case Category	Difficulty	CaseMix Question
Logic-1	Understanding NordDRG Definition Tables	Easy	Could you list all DRG groups in all NordDRG national version where the main treatment during the hospital visit was related to appendectomy? Please provide a list of <i>drg_comb</i> codes without duplicates.
Logic-2	Understanding NordDRG Definition Tables	Easy	Could you list all DRG groups in the Finnish DRG version where the main treatment during the hospital visit was related to appendectomy? Please provide a list of <i>drg_comb</i> codes without duplicates.
Logic-3	Medical Terms in Local Minority Language	Easy	Could you list DRG groups where main treatment during the hospital visit might be related to Finnish word <i>umpilisäke</i> in Finnish DRG version?
Logic-4	Understanding NordDRG Definition Tables	Medium	How would you list all NordDRG groups across all national versions where a maternal delivery (vaginal or cesarean) occurs during an inpatient, outpatient, or primary-care episode of care—excluding the 'left-over' groups (468, 470, 477, Z75R, Z76R)?
Logic-5	Medical Terms in Local Minority Language	Medium	Could you list DRG groups where the main treatment during the hospital visit might be related to Finnish word <i>umpilisäke</i> in all DRG versions from all countries? Use the <i>drg_comb</i> column codes and combine them into one list.
Logic-6	Surgical Procedure Logic	Medium	For DRG 166N which procedure codes have the connection through the procedure property? Use NCSP plus codes in your answer.
Logic-7	Connection between Definition Tables	Medium	Which diagnosis codes are linked to DRG 166N through diagnosis properties in all DRG versions from all countries?
Logic-8	Diagnosis Properties	Medium	Which diagnosis (ICD) codes are connected to diagnosis property 14X03?
Logic-9	Connection between Definition Tables	Medium	Which diagnosis categories and properties are listed in <i>drg_logic</i> for DRG group 372C?
Logic-10	Understanding NordDRG Definition Tables	Medium	How would you list all NordDRG groups across every national version where a maternal delivery (vaginal or cesarean) occurs during an inpatient or outpatient hospital episode of care—excluding primary-care delivery DRGs and the 'left-over' groups (468*, 470*, 477*, Z75R, Z76R)?
Logic-11	Understanding NordDRG Definition Tables	Medium	How would you list all NordDRG groups across all national versions for newborn episodes of care (MDC 15) during an inpatient, outpatient, or primary-care setting—excluding the 'left-over' groups (468, 470, 477, Z75R, Z76R)?
Logic-12	Understanding NordDRG Complication Logic	Medium	For a case with principal diagnosis A5990 and secondary diagnosis G8000 (Spastic cerebral palsy), is the secondary counted as a valid complication?
Logic-13	Understanding NordDRG Complication Logic	Medium	For a case with principal diagnosis B3710 and secondary diagnosis J1590 (Bacterial pneumonia, unspecified), is the secondary counted as a valid complication?

6.2.1.4 Artefact: NordDRG Grouper Benchmark

A.III.2.4 *NordDRG Grouper Benchmark* requires *full emulation* of the official NordDRG grouper: an LLM must take a structured test case and deterministically reproduce the same grouping decision that the specification would yield. Concretely, the model must apply the right execution order in *drq_logic*, evaluate age/sex bounds, MDC entry conditions, surgical evidence (e.g., *OR* / *PROCPRO*), complication logic (CC/MCC via *COMPL* and exclusions), and national activation flags to arrive at a single DRG for that case—and identify the exact *drq_logic.id* row responsible for the match. The task therefore goes beyond code lookup: it tests end-to-end reasoning over the rule graph defined in the NordDRG specification tables (Artefact 6.2.1.1) and their governance semantics (Artefact 6.2.1.2). Scoring is strict and exact-match: a submission is correct only if both the DRG code and the triggering rule identifier match the gold standard.

Tables and references. The benchmark instances are summarised in Table 21, which lists 13 hard, case-level emulation tasks (*Grouper-1–Grouper-13*). Each task references a concrete test case (by *ID*) supplied in the accompanying test-case sheet and instructs the evaluator to return two fields: *drq_nat* (the resulting DRG) and *drq_logic.id* (the precise rule row). Ground-truth outputs are provided in Table 22 for automatic verification. To solve these tasks, the LLM must integrate signals from the definition workbook—primarily *drq_logic* but also *dg_feat*, *proc_feat*, *compl_excl*, age-split, and country-flag sheets (Artefact 6.2.1.1)—and apply them in the correct priority order. The inclusion of both prompts (Table 21) and exact answers (Table 22) makes the benchmark fully reproducible and suitable for head-to-head evaluation of grouping fidelity.

Table 20: Reference answers for each CaseMix question (Logic-1–Logic-13).

ID	Right answer
Logic-1	166C, 166N, 167N, 167X, 167O
Logic-2	166N, 167X, 167O
Logic-3	166N, 167X, 167O
Logic-4	370B, 370C, 370M, 370X, 371O, 371X, 372B, 372C, 372M, 372X, 373O, 373X, 374X, 375X, P05S, P05T
Logic-5	166C, 166N, 167N, 167X, 167O
Logic-6	JESA00, JESA01, JESA10, JESW96, JESW97
Logic-7	C1810, K3520
Logic-8	O6010, O6020, O6030, O6230, O6300, O6310, O6320, O6390, O6800, O6810, O6820, O6830, O6880, O6890, O7100, O7110, O7550, O7570, O8000, O8010, O8080, O8090, O8100, O8110, O8120, O8130, O8140, O8150, O8300, O8310, O8320, O8330, O8340, O8380, O8390, O8400, O8410, O8420, O8480, O8490, Z3700, Z3710, Z3720, Z3730, Z3740, Z3750, Z3760, Z3770, Z3790
Logic-9	14X03, 14X11, 14X13, 00X10
Logic-10	370B, 370C, 370M, 370X, 371O, 371X, 372B, 372C, 372M, 372X, 373O, 373X, 374X, 375X
Logic-11	385A, 385B, 385C, 386N, 387N, 388A, 388B, 388C, 389A, 389B, 389C, 390X, 391O, 391X, 815O, 858T, 858V, 858W, 915O, 970Q, Q69L, Q93R, Q97R, Q99R, Q99U, Q99W
Logic-12	No
Logic-13	Yes

6.2.1.5 Artefact: Definition Tables: Part of the tables FI

A.III.2.5 distils the full workbook down to a single, lightweight file. The sheet contains only those DRG codes that are *activated in the Finnish version* and provides their official Finnish descriptions (DRG_TEXT_FI). Auxiliary columns preserve the DRG code, MDC and version stamp so that rows can still be cross-referenced against the master tables.

This compact subset supports two complementary use cases: first, it enables minority-language evaluation by allowing researchers to probe how well an LLM grasps Finnish DRG terminology—without loading the full 20-sheet corpus—and, second, it facilitates rapid prototyping, providing an ultra-light file for few-shot prompting, in-context search, or on-device experiments where the complete tables would exceed context limits.

6.2.1.6 Artefact: Definition Tables: Part of the tables FI-EN

A.III.2.6 mirrors the row set of the Finnish-only file but appends the official English group descriptions (DRG_TEXT_EN). The bilingual layout lets LLMs align Finnish terminology with its English counterpart, enabling cross-lingual retrieval, code-switching prompts, and translation-quality checks within a single, lightweight worksheet. Such functionality is essential for multilingual healthcare settings, where clinicians, coders, and decision-support tools must seamlessly toggle between local and international nomenclatures.

6.2.1.7 Artefact: Dataset configurations

A.III.2.7 defines four ready-made modular configurations of the benchmark. Each primary artefact—definition tables, governance PDFs, and curated table excerpts—can be loaded on its own or combined, depending on the experiment. Table 23 lists four ready-made “dataset configurations” that map onto typical research needs.

The first pair, *Definition Tables* and *Definition Tables + PDF Instructions*, delivers the full-fidelity reference material used by hospital coders and is best suited for testing an LLM’s ability to reproduce the official NordDRG grouper logic or to justify decisions with citations from the manuals.

The second pair contains only a *slice* of the master tables: Finnish-only rows (FI) or bilingual Finnish–English rows (FI-EN). These lightweight subsets enable rapid prototyping and few-shot prompting in resource-constrained settings while still preserving clinically meaningful group descriptions.

Table 21: CaseMix questions for emulating the official NordDRG grouper (Grouper-1–Grouper-13).

ID	Use Case Category	Difficulty	CaseMix Question
Grouper-1	Emulating DRG grouper	Hard	Select the test case with ID 49483 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-2	Emulating DRG grouper	Hard	Select the test case with ID 50184 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-3	Emulating DRG grouper	Hard	Select the test case with ID 50166 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-4	Emulating DRG grouper	Hard	Select the test case with ID 50629 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-5	Emulating DRG grouper	Hard	Select the test case with ID 66704 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-6	Emulating DRG grouper	Hard	Select the test case with ID 53273 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-7	Emulating DRG grouper	Hard	Select the test case with ID 49486 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-8	Emulating DRG grouper	Hard	Select the test case with ID 49571 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-9	Emulating DRG grouper	Hard	Select the test case with ID 49597 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-10	Emulating DRG grouper	Hard	Select the test case with ID 49606 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-11	Emulating DRG grouper	Hard	Select the test case with ID 49968 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-12	Emulating DRG grouper	Hard	Select the test case with ID 54051 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-13	Emulating DRG grouper	Hard	Select the test case with ID 54096 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?

Table 22: Reference answers for each CaseMix grouper question (Grouper-1–Grouper-13).

ID	Right answer
Grouper-1	drg_nat=470E, drg_logic_id=000D004100
Grouper-2	drg_nat=373, drg_logic_id=400D783000
Grouper-3	drg_nat=704O, drg_logic_id=106D120110
Grouper-4	drg_nat=468, drg_logic_id=499D000100
Grouper-5	drg_nat=911O, drg_logic_id=111D169960
Grouper-6	drg_nat=384B, drg_logic_id=414D3710002
Grouper-7	drg_nat=388A, drg_logic_id=015D816140
Grouper-8	drg_nat=004, drg_logic_id=401D041000
Grouper-9	drg_nat=477, drg_logic_id=414D022100
Grouper-10	drg_nat=901O, drg_logic_id=101D089890
Grouper-11	drg_nat=125O, drg_logic_id=105D230000
Grouper-12	drg_nat=560A, drg_logic_id=400D313200
Grouper-13	drg_nat=291M, drg_logic_id=410D110101

Researchers can therefore start with the compact excerpts to establish baselines, then scale up to the full tables plus manuals to stress-test chain-of-thought reasoning, multilingual transfer, and cross-document grounding. Collectively the four bundles offer a flexible design space for analysing how data volume, document modality, and language coverage affect LLMs capability to comprehend NordDRG classification.

6.2.1.8 Lightweight Agents for CaseMix Tasks

We release two *single-role* agents that can be set up in minutes inside OpenAI GPTs/Assistants or Anthropic Projects/Workflows. Both operate strictly on the attached artefacts - no web browsing, RAG, function-calling, code interpreters, databases, or connectors. The only platform requirements are: (i) the ability to upload files as context and (ii) a sufficient context window to ingest one Excel workbook and two PDFs.

Table 23: Configuration types for the NordDRG benchmark datasets.

Dataset Configurations	Type	Description
Definition Tables	Tables	NordDRG Definition Tables
Definition Tables + PDF Instructions	Tables+Instructions	Definition tables and PDF descriptions
Definition Tables: Part of the tables FI	PortionOfTables	NordDRG Definition Tables: Finnish DRG list with Finnish descriptions only
Definition Tables: Part of the tables FI-EN	PortionOfTables	NordDRG Definition Tables: Finnish DRG list with Finnish and English descriptions of groups

LogicAgent (provider-agnostic, no-code). Purpose: answer the 13 *Logic* tasks deterministically from the source materials. Context: the *NordDRG Combined tables* workbook (all national versions) and the two governance PDFs. **Output:** comma-separated, order-invariant, de-duplicated code lists; for *binary* tasks (Logic-12/13) return exactly **Yes** or **No**. Evidence: on request, the agent returns a concise *evidence trace* (e.g., relevant sheet/row IDs and predicate outcomes)—not hidden chain-of-thought.

Context files

Path NordDRG_Documentation:

- How_to_read_NordDRG_definition_tables_2021-12-20.pdf
- How_to_write_technical_changes_for_NordDRG_2021-12-21.pdf

Path NordDRG_Combined2024PL:

- 2023-06-14_Combined2024PL_xls.xlsx

System prompt (verbatim):

Read the attached tables and PDF documents. Answer the following questions only based on the documents provided to you in this context. Do not use web searches.

Return results as comma-separated, order-invariant lists. Example: A, B, C

GrouperAgent (Finland; governed emulation). Purpose: emulate the *official* NordDRG grouper for the 13 *Grouper* benchmark cases in the Finnish national version. Context: the FI definition-tables workbook and the two governance PDFs. Execution: applies the governed control flow over **drg_logic**—ORD priority, age/sex bounds, MDC entry, OR/PROCPRO, CC/MCC (with exclusions), and FI national activation. Output: exactly one line **drg_nat=<DRG>**, **drg_logic_id=<ID>**. Limitation: because the workbook is FI-only, cross-country Logic tasks should be run with *LogicAgent* and the Combined tables.

Context files

Path NordDRG_Documentation:

- How_to_read_NordDRG_definition_tables_2021-12-20.pdf
- How_to_write_technical_changes_for_NordDRG_2021-12-21.pdf

Path NordDRG_FIN2026PL0:

- 2024-04-15_FIN2026PL0_xlsx_wo_drgs.xlsx

System prompt (verbatim):

Read attached tables and PDF documents. Answer the following questions only based on the documents provided to you in this context. You are not allowed to use web searches.

```
Return exactly one line with two fields:
drg_nat=<DRG>, drg_logic_id=<ID>
```

Quick start (OpenAI / Anthropic).

1. Create a tool-free agent (OpenAI GPT/Assistant or Anthropic Project/Workflow).
2. Upload the three *context files*. If the UI respects paths, keep the repo-root relative paths shown above; otherwise filenames suffice.
3. Paste the corresponding *System prompt*.
4. **Disable** web/search/RAG and any external tools.
5. Run Logic-1–13 with *LogicAgent* or Grouper-1–13 with *GrouperAgent*.

6.2.2 Demonstration and Results

This section reports empirical findings from applying the benchmark artefacts to representative LLM endpoints. Analysis begins with baseline performance on the *Logic Benchmark* (Section 6.2.2.1) and the more demanding *Grouper Benchmark* (Section 6.2.2.2), before considering operational throughput under consumer subscription constraints (Section 6.2.2.3). How the artefacts and tasks satisfy the three design objectives O.III.2.1–O.III.2.3 is then assessed (Section 7.6), linking descriptive coverage with analytical results. Taken together, these subsections provide both quantitative baselines and qualitative insights into the benchmark’s fidelity, extensibility, and governance-grade differentiation.

6.2.2.1 Baseline LLM Performance: NordDRG Logic Benchmark

Evaluation setup. All runs used the *Definition Tables + PDF Instructions* bundle (A.III.2.1 and A.III.2.2; cf. Table 23), and answers were constrained to the provided materials with the preamble: “Read attached tables and PDF documents. Answer following questions only based on the documents provided for you in this context. You are not allowed to use web searches.” Logic tasks are drawn from A.III.2.3; grouper tasks from A.III.2.4. We evaluated commonly used commercial endpoints from three families: OpenAI (*GPT-5 Thinking*, *GPT-5 Thinking Mini*, *GPT-5 Fast*, *o3*, *o4-mini*, *4o*, *4.1*), Anthropic (*Opus 4.1*, *Sonnet 4*), and Google (*Gemini 2.5 Pro*, *Gemini 2.5 Flash*).

Interpreting Table 24 (Logic tasks; A.III.2.3). The 13 automatically verifiable *Logic-1–Logic-13* questions separate the models into three tiers. *GPT-5 Thinking* and *Opus 4.1* achieve perfect accuracy (13/13), with *o3* close behind (12/13). A mid-tier (*GPT-5 Thinking Mini*, *o4-mini*, *GPT-5 Fast*) lands between 6–8 correct, while *4o*, *4.1*, *Sonnet 4*, and the Gemini models trail at 5/13 or below. The distribution aligns with task structure defined in A.III.2.3: single-table lookups or straightforward filters (e.g., *Logic-1–Logic-3*, *Logic-8*, *Logic-11*) are broadly solvable by the strongest systems, and *Logic-12* (a CC/MCC validity check) is solved by all models. In contrast, prompts requiring cross-sheet chaining over A.III.2.1 — notably procedure-property tracing (*Logic-6*) and diagnosis-property linkage (*Logic-7*) — create the largest separation. Failures cluster around (i) omissions of relevant sheets (e.g., `compl_excl`, `proc_feat`, `dg_feat`), (ii) mishandling of “left-over” group exclusions, and (iii) non-deduplicated code lists that miss *exact-match* scoring (as specified in A.III.2.3). Overall, Table 24 shows that generic leaderboard strength does not guarantee mastery of NordDRG’s property logic and governance-aware filters; however, high-end models can execute these joins reliably when given the combined artefact bundle (A.III.2.1, A.III.2.2).

6.2.2.2 Baseline LLM Performance: NordDRG Grouper Benchmark

Interpreting Table 25 (Grouper emulation; A.III.2.4). *Evaluation setup.* All runs used the *Definition Tables + PDF Instructions* bundle (A.III.2.1, A.III.2.2), and models were explicitly instructed: “Read attached tables and PDF documents. Answer following questions only based on the documents provided for you in this context. You are not allowed to use web searches.” Commonly used commercial endpoints from three families were evaluated—OpenAI (*GPT-5 Thinking*, *GPT-5 Thinking Mini*, *GPT-5 Fast*, *o3*, *o4-mini*, *4o*, *4.1*), Anthropic (*Opus 4.1*, *Sonnet 4*), and Google (*Gemini 2.5 Pro*, *Gemini 2.5 Flash*).

The thirteen *Grouper-1–Grouper-13* tasks enforce governance-grade fidelity: an answer is correct only if it reproduces *both* the final DRG and the exact triggering `drg_logic.id` defined in the specification tables (A.III.2.1) and required by the benchmark protocol (A.III.2.4). Under this exact-match criterion, *GPT-5 Thinking* solves **7/13** cases, *o3* **6/13**, and *o4-mini* **3/13**; *GPT-5 Thinking Mini* solves **1/13**; all remaining endpoints (*GPT-5 Fast*, *4o*, *4.1*, *Sonnet 4*, *Opus 4.1*, *Gemini 2.5 Pro*, *Gemini 2.5 Flash*) score **0/13**. Notably,

Table 24: Accuracy on the 13 *quantitative* NordDRG Logic tasks (Logic-1–Logic-13). All runs used the **Definition Tables + PDF Instructions** bundle. (✓=correct, ✗=incorrect)

Question	GPT-5 Fast	GPT-5 Think- ing Mini	GPT-5 Think- ing	o3	o4- mini	4o	4.1	Sonnet 4	Opus 4.1	Gemini 2.5 Pro	Gemini 2.5 Flash
Logic-1	✗	✓	✓	✓	✓	✗	✗	✗	✓	✗	✗
Logic-2	✓	✓	✓	✓	✓	✗	✓	✗	✓	✗	✗
Logic-3	✓	✓	✓	✓	✓	✗	✓	✗	✓	✗	✗
Logic-4	✗	✗	✓	✓	✗	✓	✓	✗	✓	✗	✗
Logic-5	✗	✓	✓	✓	✓	✗	✗	✗	✓	✗	✗
Logic-6	✗	✗	✓	✓	✗	✗	✗	✗	✓	✗	✗
Logic-7	✗	✓	✓	✓	✗	✗	✗	✗	✓	✗	✗
Logic-8	✓	✓	✓	✓	✗	✗	✗	✗	✓	✗	✗
Logic-9	✓	✗	✓	✓	✓	✗	✗	✗	✓	✗	✗
Logic-10	✗	✗	✓	✗	✓	✓	✓	✗	✓	✗	✗
Logic-11	✓	✗	✓	✓	✓	✓	✗	✓	✓	✗	✗
Logic-12	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Logic-13	✗	✓	✓	✓	✗	✓	✗	✓	✓	✓	✗
SUM	6	8	13	12	8	5	5	3	13	2	1

Grouper-3–Grouper-5, *Grouper-7*, and *Grouper-10* defeat all models, underscoring the gap between listing plausible DRGs and *faithfully executing* the grouper’s priority order and guard rails.

The drop from *Logic* to *Grouper* performance pinpoints the hard parts of full specification emulation over A.III.2.1. Beyond table reading and joins, faithful grouping requires (i) respecting the `ORD` execution priority in `drg_logic`, (ii) enforcing age/sex bounds and MDC entry criteria, (iii) integrating surgical evidence (`OR/PROCPR0`) and `CC/MCC` handling (via `COMPL` with exclusions), (iv) applying national activation flags, and (v) mapping the decision back to the precise rule row. As *DSR results*, Table 25 provides a governance-grade barometer: the best models can read and join the definition tables reliably, yet exact, traceable emulation of the governed control flow remains challenging—especially where `ORD` priority interacts with `CC/MCC` exclusions. These errors are diagnostic rather than cosmetic: they expose divergences from the specification’s semantics (O.III.2.2) and delimit the present ceiling of artefact-only emulation (O.III.2.3), motivating the benchmark’s insistence on reporting both `drg_nat` and `drg_logic.id`.

6.2.2.3 Operational throughput under consumer subscriptions

Setup. All evaluations were executed using standard, consumer-grade LLM subscriptions rather than enterprise contracts, to reflect the experience of typical academic and practitioner users.

Observation. Under these conditions, the *Anthropic* endpoints (e.g., *Sonnet 4*, *Opus 4.1*) could not complete the *Logic Benchmark* (Logic-1–Logic-13) in a single session: credits were exhausted mid-run, necessitating batching across multiple windows until quotas refreshed. In practice, this stretched a single end-to-end evaluation over *a couple of days* due to waiting times. By contrast, the same benchmark completed without interruption on *OpenAI* and *Google* endpoints under otherwise identical conditions (prompt packs, context bundles, and scoring).

Table 25: DRG grouper emulation results on thirteen case-level tasks (Grouper-1–Grouper-13). All runs used the **Definition Tables + PDF Instructions** bundle and the structured test cases. (✓=correct; exact match on *DRG* and *drq_logic.id*; ✗=incorrect)

Question	GPT-5 Fast	GPT-5 Think- ing Mini	GPT-5 Think- ing	o3	o4- mini	4o	4.1	Sonnet 4	Opus 4.1	Gemini 2.5 Pro	Gemini 2.5 Flash
Grouper-1	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗
Grouper-2	✗	✓	✓	✓	✓	✗	✗	✗	✗	✗	✗
Grouper-3	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
Grouper-4	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
Grouper-5	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
Grouper-6	✗	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
Grouper-7	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
Grouper-8	✗	✗	✓	✓	✓	✗	✗	✗	✗	✗	✗
Grouper-9	✗	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
Grouper-10	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
Grouper-11	✗	✗	✓	✗	✓	✗	✗	✗	✗	✗	✗
Grouper-12	✗	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
Grouper-13	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗
SUM	0	1	7	6	3	0	0	0	0	0	0

Implications for benchmarking. These subscription-tier effects do not change task difficulty or accuracy, but they do affect *throughput* (wall-clock time to produce a full result set), *operational latency* (need to pause and resume), and *experiment management* (checkpointing, batching, and run logs).

Operational note (agents). A thin wrapper in the style of the *LogicAgent/GrouperAgent* (cf. Section 6.2.1.8) could batch tasks deterministically, cache intermediate joins, and resume after quota interruptions—improving throughput without affecting accuracy metrics.

Price–value. Per-token price is only part of the calculus; time-to-result and quota predictability also determine value. In the runs, consumer-tier credit limits introduced delays and operator overhead (pauses, batching, etc) that can outweigh lower unit prices in time-sensitive studies. When latency is non-critical, consumer tiers may remain cost-effective. Accuracy *and* operational notes are therefore reported to inform cost–latency trade-offs.³⁹

6.3 CCL: How Humans and AI Agents Align on Decision-Making Rules

Problem Identification and Research Methodology

The problem identification and objectives for this work are presented in Section 1.4.3. The research methodology is presented in Section 1.1.

³⁹Subscription quotas and refill policies are product- and time-dependent; these observations reflect the August 2025 setup and may evolve.

Background and Related Work

Background for this work is presented in Section 2.6.

Context

Algorithms increasingly mediate consequential decisions—from content recommendations and credit approvals to the less visible allocation of billions of euros in hospital reimbursements. In many health systems, a hospital stay is mapped to a DRG, which determines payment. Because regulations, clinical practice, and coding conventions evolve, the decision logic that maps clinical inputs to DRG outputs cannot be *train-once-and-forget*: it must change continuously *and* remain fully auditable. Today’s regulated environments tend to rely on either frequently refreshed data-driven models with weak audit trails, or expert-curated rule systems with strong transparency but slower release cadences—neither of which alone fully solves the maintenance dilemma.

This study targets the process gap: while MLOps has matured for *training* models, publicly specified, end-to-end patterns for *maintaining* governed decision logic—with typed atomic diffs, CI-style gates, and mandated human sign-off—are scarce. In contrast, software engineering offers well-established change-management practices (version control, pull requests, code review, regression tests) that ensure traceability and guard against regressions.

To address this, the work proposes *Collaborative Curated Learning* (CCL): a reference decision-model development process that treats every model edit like a code change. AI agents and domain experts submit atomic inserts/deletes/restructures; hybrid human-AI teams enrich and review proposals; certified reviewers ratify changes under automated regression tests. The research asks, at the process level, (i) which governance and workflow properties enable agent contributions without eroding expert accountability, (ii) how to represent million-rule decision models for safe, atomic diffs, and (iii) under what constraints LLM agents produce reviewer-acceptable proposals at $\approx 10^7$ decision-point scale.

Contributions

The contributions are:

- C.III.3.1 CCL: a governed development process and learning-alignment paradigm.** This study formalises *Collaborative Curated Learning* (CCL) as a governance-first process that treats every model edit as a pull request with typed atomic diffs, explanation-rich specifications, risk-tiered gates, automated quality controls, mandated human sign-offs, and versioned checkpoints—providing reproducibility, traceability, rollback, and human-in-command control. This study also positions CCL as a *learning-alignment paradigm* that combines statistical learners with human-curated alignment under version control.
- C.III.3.2 Agent-operable NordDRG model-of-record.** This study converts the official NordDRG specification into a line-addressable, table-backed decision-tree representation, enabling safe atomic patches, programmatic audits, and reviewer-legible slices of the logic for PR-centric governance.
- C.III.3.3 Agentic workflow with two roles.** This study designs an agent-agent workflow in which a *NordDRG Contributor Agent* ingests tickets and emits well-typed atomic edits with reviewer-ready artefacts, and a *NordDRG Reviewer Agent* assesses these proposals and suggests quality improvements—both operating under restricted roles to preserve human control.
- C.III.3.4 Demonstration at real scale.** This study demonstrates end-to-end operation on NordDRG: parsing the official specification, normalising it into an agent-editable large decision tree, and exercising realistic change scenarios where Contributor outputs are checked automatically and then reviewed by the Reviewer Agent prior to expert oversight.
- C.III.3.5 Empirical findings on governance and scalability.** This study shows that under explicit constraints (grounding, role restrictions, gate policy) CCL supports governed agent-agent and human-agent hybrids and can scale expert oversight to rule sets on the order of $\approx 10^7$ decision points; this study also finds that introducing a review role reduces the burden on human reviewers while current LLM limits necessitate restricted agent responsibilities.

Positioning. The proposed artefacts are intended to complement—never replace—established NordDRG governance. They aim to codify auditable patterns already familiar to maintainers, while reserving all approval authority for certified human reviewers.

6.3.1 Design & Development: Collaborative Curated Learning (CCL)

This section describes the design and development of the artefacts needed for the CCL in the NordDRG context. This section is structured as follows:

- A.III.3.1 Decision-making model** (Section 6.3.1.1) formalises the decision-making model as a computable mapping from inputs to recommended actions or decisions, applicable to any domain.
- A.III.3.2 Participants and workflows** (Section 6.3.1.2) describes the participants in the CCL process, including the roles of agents, humans, and procedures, as well as the types of workflows they form.
- A.III.3.3 CCL governance process** (Section 6.3.1.3) outlines the CCL levels of governance control in decision-model development, along with the procedures for proposing, reviewing, and ratifying changes.
- A.III.3.4 NordDRG decision model representation** (Section 6.3.1.4) describes the agent-operable representation of the NordDRG decision-making model.
- A.III.3.5 LLM-based software agents** (Section 6.3.1.5) introduces the LLM-based software agents that participate in the NordDRG CCL process demonstration: the *Contributor* (Section 6.3.1.5), which proposes changes, and the *Reviewer* (Section 6.3.1.5), which provides PR review feedback.

6.3.1.1 Design Artefact: Mathematical Decision-Making Model

Purpose. To identify the governed object in CCL: the auditable, versioned decision model function $\hat{f}_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ invoked by operational systems and maintained under version control. It is the *unit of change* in the Pull Request (PR) –centric process: agents and experts propose typed, atomic diffs; CI supplies evidence; and certified reviewers ratify merges, so the model evolves without eroding accountability (see Section 6.3.1.3).

Definition. This study formalises a *decision-making model* as a computable mapping

$$\hat{f}_\theta : \mathcal{X} \rightarrow \mathcal{Y} \quad (\text{deterministic}), \quad \text{or, when outputs are probabilistic, as } \hat{f}_\theta : \mathcal{X} \rightarrow \Delta(\mathcal{Y}),$$

where $\mathcal{X}$ is the space of inputs and $\mathcal{Y}$ the space of recommended actions/decisions; $\Delta(\mathcal{Y})$ denotes distributions over $\mathcal{Y}$. The parameters $\theta \in \Theta$ encode the model’s internal structure and reference data (e.g., predicates and leaf labels in a rule base or decision tree; weights and biases in a neural network; thresholds, code lists, and encoders in hybrid systems).

Data–function correspondence. As depicted in Figure 70, learning paradigms differ chiefly in the *supervision signal* used to induce $\hat{f}_\theta$ from data. In *supervised* learning, labelled pairs (x, y) guide $\hat{f}_\theta$ by minimising a predictive loss. *Unsupervised* learning exploits structure in unlabelled x (e.g., clusters, representations, densities) that can be used directly for decisions or as features for $\hat{f}_\theta$. In *reinforcement* learning, interaction supplies delayed feedback and the model is a policy $\pi_\theta(a | s)$ tuned to maximise expected return. *Transfer* learning adapts a source model $\hat{f}_{\theta_s}$ to a target domain $\mathcal{D}_T$, reusing knowledge to reduce data requirements. Regardless of signal, the goal is the same: a reliable, parameterised approximation of the domain’s decision function. In practice, pipelines *compose* these routes (e.g., unsupervised pre-training $\rightarrow$ supervised fine-tuning $\rightarrow$ RL or domain adaptation), after which CCL governs maintenance-time updates through typed diffs and review.

What the function can be. The decision model $\hat{f}_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ can be instantiated by any computable form. Two canonical examples are: (i) a *decision tree*, where internal nodes hold Boolean predicates and leaves hold actions or scores, yielding a piecewise-constant map with parameters $\theta = \{h_v, y_\ell\}$; and (ii) a *neural network*, e.g. an L -layer feed-forward model $\hat{f}_\theta(x) = \sigma_L(W_L \sigma_{L-1}(\cdots \sigma_1(W_1 x + b_1)) + b_L)$ with parameters $\theta = \{W_\ell, b_\ell\}_{\ell=1}^L$ and task-appropriate output (e.g., logits/softmax for classification, real values for regression). Other realizations (rule lists/scorecards, probabilistic models returning $p(y|x)$, or ensembles) fit the same abstraction.

Representation and documentation. To place the decision model $\hat{f}_\theta$ under version control (Git), it needs to be materialised as a set of files (e.g., CSV/JSON/etc) with a stable, typed *schema* so that edits are line-addressable and diffable. Each model version must ship with a short, co-versioned *Interpretation Guide* that documents that schema and its evaluation rules (field meanings and code lists, ordering/precedence,

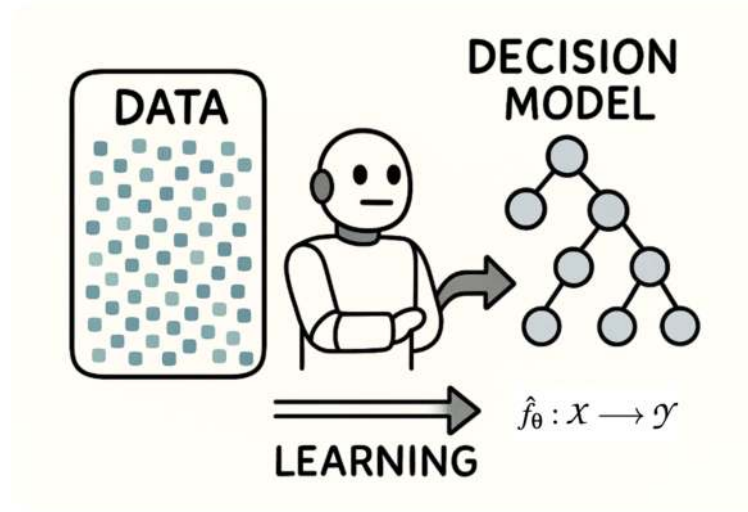


Figure 70: Learning paradigms create a decision model $\hat{f}_\theta$ from data. Supervised, unsupervised, reinforcement, and transfer learning differ only in the form and timing of the signals they exploit.

missing-value policy, activation flags). Together, the files+schema and the guide make changes reviewable, testable, and reproducible. This study uses the customary NORDDRG tables (and a tree or Directed Acyclic Graph (DAG) view) in exactly this form; see Section 6.3.1.4 and Fig. 78.

6.3.1.2 Collaborative Curated Learning (CCL): Process Participants and Workflow Types

Purpose. This subsection formalises the CCL process as a design-science artefact by specifying its participants, process structure, and workflow types. The goal is to provide a precise, implementation-agnostic model that supports rigorous analysis, reproducibility, and comparison across domains. This study captures the socio-technical process as a typed directed graph—extending classical AOV models to allow cycles—so that both human and AI-driven activities, as well as automated procedures, can be unambiguously represented and reasoned about.

Definition 6.3.1 (CCL Participant). The set of *participants* is defined as

$$\mathcal{P} = \{\text{Agent, Human, Procedure}\}.$$

Each element of $\mathcal{P}$ is characterised as follows:

- (i) An **Agent** is an LLM-driven software actor equipped with a tool-using control loop.
- (ii) A **Human** is a certified reviewer or expert user who validates, audits, or provides domain-specific input.
- (iii) A **Procedure** is an automatic deterministic quality-assurance or CI/CD routine (e.g., unit tests, linters, or policy checks).

Definition 6.3.2 (CCL Process). A *process* in CCL is modelled as a (possibly cyclic) directed multigraph

$$G = (V, E, \tau, \ell),$$

where:

- (i) V is a finite set of *activities* (vertices).
- (ii) $E \subseteq V \times V$ is a set of directed *edges*, representing control or information flow.
- (iii) $\tau : V \rightarrow \mathcal{P}$ is a *typing function* that assigns each activity to exactly one participant type (cf. Definition 6.3.1).
- (iv) $\ell : E \rightarrow \Sigma^*$ is an optional *labelling function* that assigns event names to edges (e.g., “propose”, “approve”, “request-changes”).

Cycles in G are permitted, thereby capturing iterative review and rework steps intrinsic to the CCL process.

Definition 6.3.3 (CCL Workflow). A *workflow* in CCL is defined as a typed subgraph

$$W = (V_W, E_W)$$

of the *CCL Process* G , where:

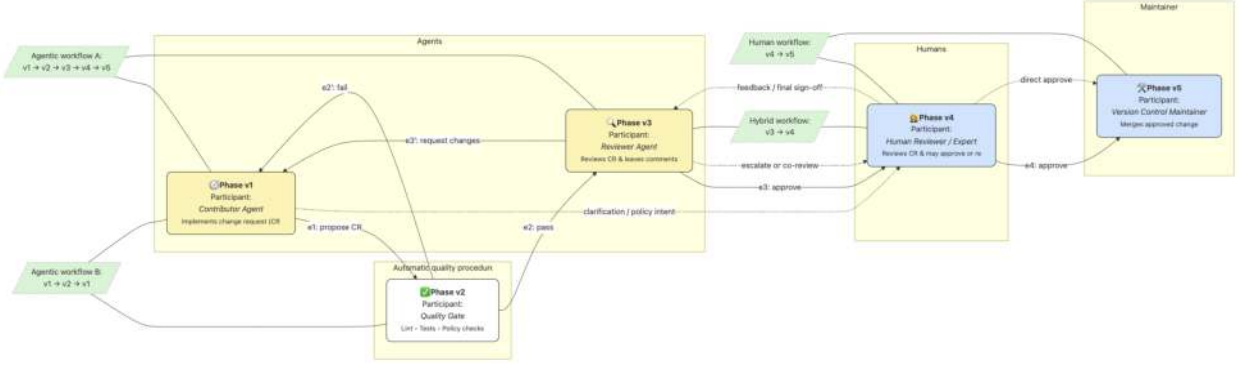


Figure 71: **Example CCL process slice.** A directed, possibly cyclic AOV graph where activities (vertices) are typed by participants $\tau(v) \in \{\text{Agent, Human, Procedure}\}$. The slice illustrates agentic (Agent $\rightarrow$ Agent and Agent $\rightarrow$ Procedure), hybrid (Agent $\rightarrow$ Human and Human $\rightarrow$ Procedure), and human (Human $\rightarrow$ Human) workflows.

- (i) $V_W \subseteq V$ is a finite set of activities (vertices).
- (ii) $E_W \subseteq E$ is a set of directed edges encoding precedence between activities.

W is an *Activity-on-Vertex (AOV) graph*, where activities are represented as vertices and precedence relations as directed edges. Unlike classical project-planning AOV graphs, CCL workflows may contain cycles to capture negotiation loops and escalation paths.

Remark 6.3.4 (Illustration). Figure 71 shows a concrete CCL slice with **Agent**, **Human**, and **Procedure** nodes and multiple workflow types (agentic, hybrid, human) co-existing and looping via review/approval edges. The illustration is inspired by the demonstration of CCL on the NORDDRG decision model (Section 6.3.2), where agents propose changes, humans review them, and procedures validate compliance.

The following specialised workflow types are distinguished as described in Figure 72.

Definition 6.3.5 (Agentic Workflow). A workflow where precedence relations involve only **Agent** and/or **Procedure** participants.

Definition 6.3.6 (Human Workflow). A workflow where precedence relations involve only **Human** participants.

Definition 6.3.7 (Hybrid Workflow). A workflow where precedence relations span multiple participant types, such as **Agent**–**Human** or **Human**–**Procedure**.

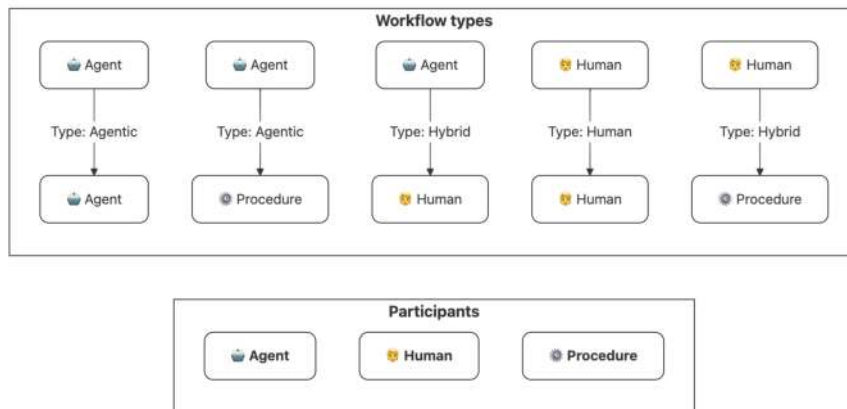


Figure 72: **Participants and workflow types.** Top: representative AOV snippets for agentic, hybrid, human, and procedural flows. Bottom: the participant set $\mathcal{P} = \{\text{Agent, Human, Procedure}\}$.

Definition 6.3.8 (Authority to Define Workflows). In CCL, *humans* hold design-time authority to specify (and version-control) the structural and policy constraints under which processes execute. Formally, human authorities define:

- (i) the set of *allowed connections* $A \subseteq V \times V$ between activities; and
- (ii) the *participant protocol* $\Pi \subseteq \mathcal{P} \times \Lambda \times \mathcal{P}$, which constrains which participant types may send which labelled messages to which other types.

These artefacts are maintained under version control with auditable change history.

Definition 6.3.9 (Allowed Connections). The set of *allowed connections* is a relation $A \subseteq V \times V$ preconfigured by humans (cf. *Authority to Define Workflows*), enumerating the directed pairs of activities that may be linked in an executing process.

Definition 6.3.10 (Participant Protocol). The *participant protocol* is a ternary relation $\Pi \subseteq \mathcal{P} \times \Lambda \times \mathcal{P}$ (cf. *Authority to Define Workflows*) that whitelists message-labelled interactions between participant types.

Definition 6.3.11 (Edge Admissibility). A labelled edge $e = (u, v, \lambda)$ may belong to the process graph iff $(u, v) \in A$ and $(\tau(u), \lambda, \tau(v)) \in \Pi$ (cf. *Allowed Connections–Participant Protocol*). Multiple labels between the same ordered pair (u, v) are permitted.

Definition 6.3.12 (Edge Guard). Each admissible edge e has a *guard* $\gamma_e : \mathcal{S} \rightarrow \{0, 1\}$ that determines whether e is enabled in the current process state $\mathcal{S}$. Only enabled edges can be traversed at run time.

Definition 6.3.13 (Local Routing Rule). At run time, the actor at node v chooses the next node w according to a local routing function $\rho_v : \mathcal{H}_v \times \mathcal{S} \rightarrow V$, where $\mathcal{H}_v$ is the local history at v . The choice is restricted to edges (v, w, λ) that are both admissible (*Edge Admissibility*) and currently enabled ($\gamma_{(v, w, \lambda)}(s) = 1$, *Edge Guard*).

Remark 6.3.14 (Design-time authority vs. run-time autonomy). *Authority to Define Workflows–Participant Protocol* capture human authority at design time; *Edge Guard–Local Routing Rule* capture constrained run-time autonomy.

6.3.1.3 Design Artefact: Collaborative Curated Learning (CCL) Decision-making Model Control

Purpose. This subsection specifies *how* CCL governs changes to the decision-making model $\hat{f}_\theta$ defined in Section 6.3.1.1, using the participant/workflow primitives in Section 6.3.1.2. Re-stating roles or interfaces and focus on change control is avoided: patch semantics, Pull Request (PR) bundles, risk tiers and gates, lifecycle, and invariants.

Governed object and unit of change. Treat the model-of-record as parameters $\theta \in \Theta$ (rules, trees, weights, code lists, encoders). A candidate edit is an *atomic diff* $\delta \in \Delta$ applied as $\theta' = \delta(\theta)$. Admissible diffs are (i) *well-typed* (schema-valid), (ii) *local* (bounded blast radius), and (iii) *explainable* (rationale and counterfactual tests).

Version-control primitives. This study uses a Git-style VCS⁴⁰ to manage decision-model changes. **Version control Primitives:** *Commit*—immutable snapshot (hash ID); *branch*—isolated line of work; *tag*—release label; *pull request (PR)*—reviewed change bundle; *CI*—automated checks on a PR.

Version Control Repository and PR bundle. Let the version control repository be

$$\mathcal{R} = (G, \text{main}, \mathcal{B}, \mathcal{T}, \mathcal{L}),$$

with commit DAG G , certified branch **main**, feature branches $\mathcal{B}$, release tags $\mathcal{T}$, and an immutable audit ledger $\mathcal{L}$. A PR is the tuple

$$\text{PR} = (b, \{\delta_i\}_{i=1}^k, \text{IB}, \text{CI}), \quad b \in \mathcal{B},$$

where the *Impact Brief* IB minimally contains $\langle \text{rationale}, \text{change_set}, \text{scope}, \text{counterfactuals}, \text{metrics/docs} \rangle$, and CI is machine-checkable evidence (lint, tests, probes).

PR Authority Policy for Change Management (human-in-command). Each PR carries an assignment

$$\rho : \{\text{proposer}, \text{reviewers}, \text{approvers}\} \rightarrow \mathcal{P},$$

with the invariant $\rho(\text{approvers}) \in \mathcal{H}$ (humans only). Agents may propose and review but *cannot* merge to **main**.

⁴⁰By “Git-style version control” this study means any (distributed) VCS that provides immutable commit hashes, branching, tags/releases, and code review via pull requests; Git is the most common implementation [264]. See Section 2.6.4 for details.

PR Lifecycle. The Review Protocol state machine (Figure 73) has five core stages—*Draft* → *Review* → *Approve* → *Merge* → *Release*—with explicit *Changes Requested* loops and *Rejected/Withdrawn/Superseded* exits. Concretely: branch from *main*, apply $\{\delta_i\}$, open a PR with IB/CI and declared risk tier τ ; iterate agentic (Contributor ↔ Reviewer) and human review until gates G_1 – G_4 pass; required human approvers sign; a maintainer performs `merge -no-ff`, tags the release, and writes the ledger (see Figure 75). Monitoring may trigger *Rollback* from *Release* back to *Draft*. Merges always require mandated human sign-off (*human-in-command*).

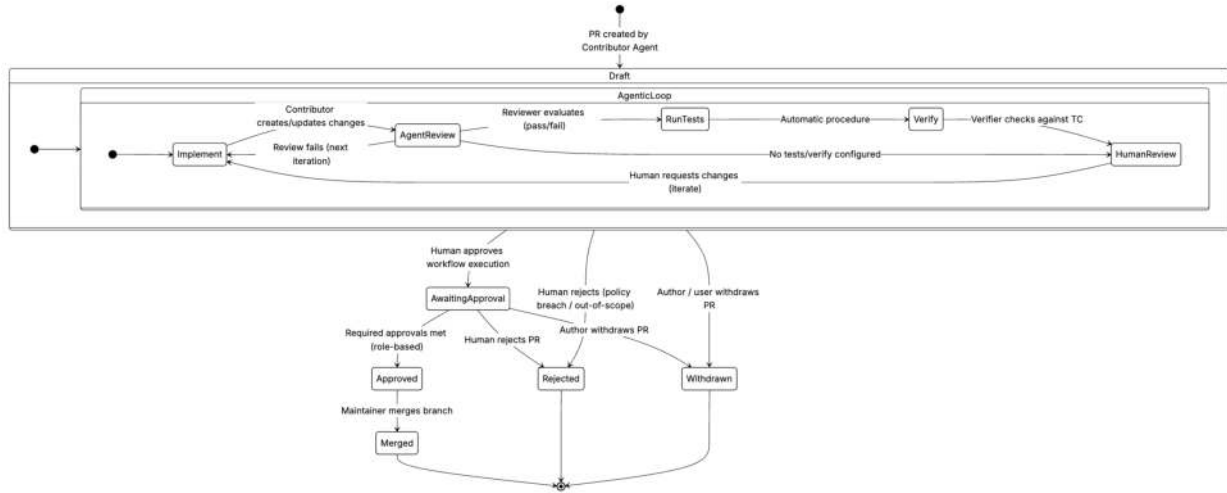


Figure 73: **CCL PR state machine.** Lifecycle from *Draft* → *Review* → *Approve* → *Merge* → *Release*, with gated iteration (*Changes Requested*), explicit human *Rejected/Withdrawn/Superseded* exits, and *Rollback* from *Release* on monitoring alerts; merges require mandated human sign-off.

Conflicts. Merge conflicts can occur under concurrent edits. PRs must be brought up to date with the current *main* (rebase or merge) before approval; standard VCS resolution applies and CI is re-run. Beyond textual conflicts, Gate G1 performs change-graph checks for *semantic* conflicts—co-edits to the same rows/paths or precedence collisions in the first-hit rule list—and blocks the PR until resolved, with final human sign-off.

Illustration: Version control and PRs. Figure 74 makes concrete how changes flow through the governed repository. The Git history view highlights feature branches opened from *main*, *no-ff* merges that preserve PR identity, and tagged releases, alongside branches that are reviewed but never merged (e.g., rejected or superseded). Each PR bundles typed atomic diffs, the Impact Brief (rationale, scope, counterfactuals, metrics, docs), a CI run with per-gate outcomes (G_1 – G_4) configured by the declared risk tier, and the required human approvals—linking these pieces to the repository and to the immutable audit ledger. This emphasises the CCL invariants: reproducible rebuilds from commits, end-to-end traceability from release to diffs and reviews, rollback via tags, and the human-in-command constraint for all merges.

Role hand-offs (explained). Figure 76 traces the PR from proposal to release and shows how responsibilities move between actors: *Proposer P* (either the Contributor Agent or a human) creates a feature branch from *main*, applies the atomic diffs $\{\delta_i\}$, and opens a PR with an Impact Brief (IB). The repository/ledger *R* triggers continuous integration *CI*, which returns gate outcomes for G_1 – G_4 (objective/change control, oversight signals, transparency/evidence, and adversarial/risk checks).

An *agentic review loop* then runs: the Reviewer Agent *DR* examines the PR and leaves structured feedback; the Proposer applies fix-ups and updates the IB; the Reviewer Agent issues a final AI recommendation. These AI steps are advisory only—no agent can merge.

After AI review, *human sign-off* is required according to the declared risk tier τ : for $\tau = R1$ a single human reviewer *SR* approves; for $\tau \in \{R2, R3\}$ both domain and safety reviewers sign. A *Maintainer M* then performs a `-no-ff` merge to *main*, tags the release, and writes the changelog to the immutable ledger *R*. Post-merge monitoring (canaries/alerts) feeds back to the PR; on a critical regression the Maintainer rolls back to the prior tag. Throughout, the *human-in-command* invariant holds: only humans can approve merges to *main*, and every released change is traceable to its diffs, CI evidence, and sign-offs in the ledger.

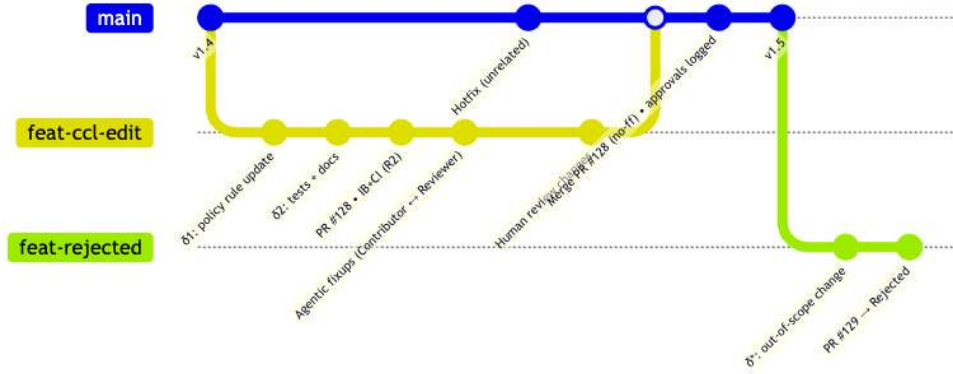


Figure 74: **Governed Version Control history for CCL.** A feature branch (**feat-ccl-edit**) opens a PR with Impact Brief and CI (tier R2), iterates agentially, then merges *no-ff* into *main* after required human approvals; an unrelated hotfix and tagged releases (**v1.4**, **v1.5**) are shown. A second branch (**feat-rejected**) illustrates a PR that is rejected (no merge). Only humans may merge to *main*.

Why tiers and gates? As argued in Section 2.6.2, alignment risks manifest chiefly at *maintenance time* (outer/inner misalignment, goal misgeneralisation, spec gaming). This study therefore operationalises the research questions in Section 1.4.3.2 via a *risk-based control map* with four CI gates.

Risk tiers and CI gates. PRs declare a risk tier $\tau \in \{R1, R2, R3\}$ that configures mandatory gates G_1 – G_4 (policy map in Figure 77):

- G1. Objective & change control**—schema/typing checks and referential integrity; change-graph scan to catch overlaps, redundancies, and conflicts (e.g., mutually exclusive DRG rules); blast-radius bounds (locality budget on affected paths/DRGs); determinism/totality checks (unique next edge per predicate outcome; default “else” coverage); validation of code lists/features and key uniqueness. The PR is blocked (fail-closed) if schemas/keys are invalid, locality budgets are exceeded, or determinism/totality is violated.
- G2. Oversight signals**—reviewer-facing annotations and sampling: required metadata (risk tier, scope, affected DRGs, expected deltas); stratified sample sets (positives/negatives/near-misses) for human spot-checks; agreement checks between proposer, Reviewer Agent, and humans; leakage/anomaly detection (unexpected national activation flags, hidden dependencies, unintended fan-out); declared post-merge monitoring hooks (what to watch, rollback trigger). The PR is blocked if required signals are missing, agreement falls below threshold, or leakage/anomalies are detected.
- G3. Transparency & evidence**—complete Impact Brief (rationale, change set, scope, counterfactuals, metrics, documents); provenance for referenced sources/artifacts and licensing/compliance checks; reproducibility bundle (scripts/configs, seeds, environment capture) linking commit→CI run; docs updated in lockstep (Interpretation Guide snippet, changelog, model/datasheet entries). The PR is blocked if the IB is incomplete, provenance/licensing is unclear, or reproducibility checks fail.
- G4. Adversarial & risk evaluation**—red-team tests and distribution-shift probes: scripted adversarial cases that attempt specification gaming (e.g., upcoding paths; mutually exclusive DRG collisions; age/sex boundary violations; national activation-flag misuse), plus fuzzed near-misses to test blast-radius control and agent-safety checks (scope creep, prompt injection). The PR is blocked (fail-closed policy) if any critical probe succeeds.

Higher tiers enlarge evidence requirements and mandate dual human sign-offs.

Decision-making model release invariants (post-merge). All merges uphold the following invariants:

- I1. Reproducibility** — deterministic rebuild from commit hash.
- I2. Traceability** — release→PR→commits→diffs/tests/reviews recorded in $\mathcal{L}$.
- I3. Rollbackability** — prior behaviour restorable via tags.
- I4. Human-in-command** — only mandated human approvers can merge.

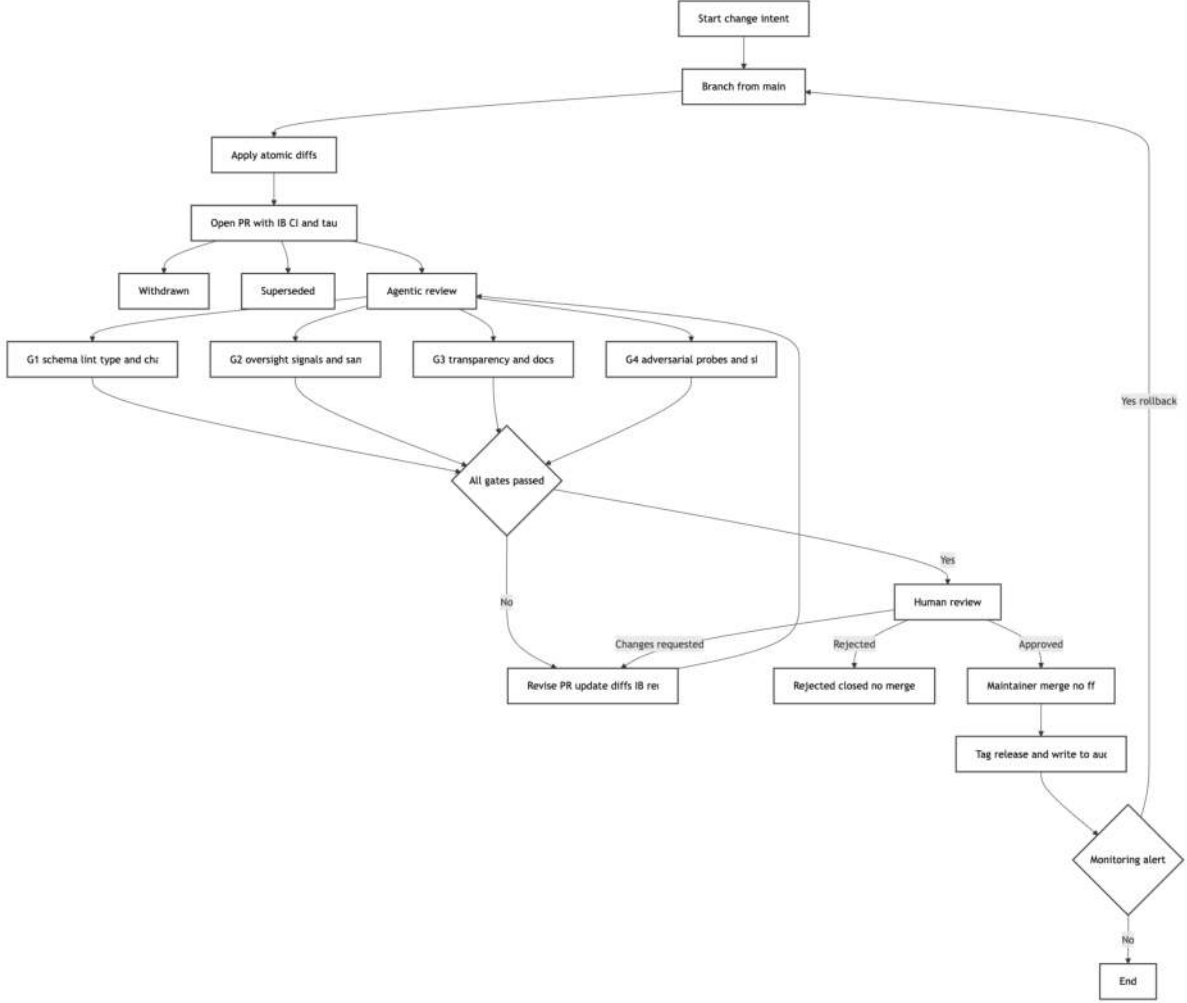


Figure 75: **CCL PR workflow.** A feature branch applies atomic diffs and opens a PR with an Impact Brief and declared risk tier. An agentic pre-review (Contributor $\leftrightarrow$ Reviewer) precedes four CI gates executed in parallel (G1–G4). A join confirms that all configured gates pass before *human review*, which can request changes, reject, or approve. Approved PRs are merged (**-no-ff**), released, and written to the audit ledger; monitoring may trigger rollback to a new branch. All merges require mandated human sign-off.

Design rationale. Each gate corresponds to a known failure mode from the alignment and MLOps literature (e.g., reward/specification misspecification, drift/leakage, opacity, adversarial fragility) [31, 249, 48, 251, 271]. **G1** enforces objective fidelity and localised change (outer alignment; blast-radius bounds). **G2** institutes overseer signals and sampling to detect drift and leakage (ongoing governance). **G3** requires reviewable provenance (Model/Datasheet-style artefacts) for auditability. **G4** probes for spec gaming and misgeneralisation with red-team tests and fail-closed policies (inner alignment/robustness). Risk tier $\tau \in \{R1, R2, R3\}$ scales evidence and mandated human sign-offs to match potential impact.

Outcome. By treating each update as a typed, auditable transaction— $\{\delta_i\} + \text{IB} + \text{gated CI} + \text{human approval}$ —CCL yields agile yet certifiable evolution of $\hat{f}_\theta$ in regulated settings, without re-stating participants or workflow taxonomies already given in Section 6.3.1.2.

6.3.1.4 Design Artefact: Agent-Operable Decision Model of NordDRG

Purpose. In the demonstration, NORDDRG is the governed *model-of-record*: an explicit decision function $\hat{f}_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ (Section 6.3.1.1) that maps episode inputs to DRG outputs for reimbursement (Section 2.6.6). To

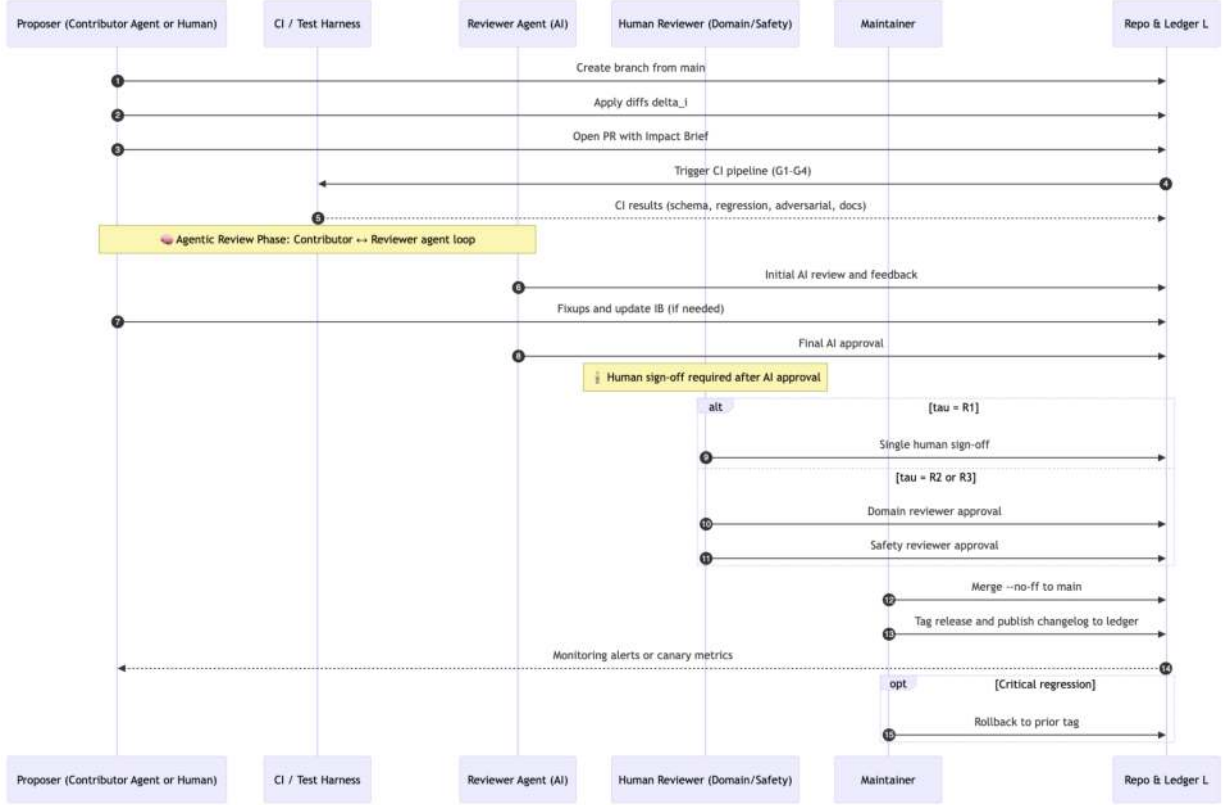


Figure 76: Roles and human–AI hand-offs across the PR timeline. Agents collaborate during an initial *agentic review* phase; escalation to human sign-off preserves the human-in-command invariant.

admit AI agents as first-line contributors without eroding accountability, the model must be materialised as an *agent-operable* artefact—line-addressable, schema-typed, and under version control—so agents can *parse*, *reason about*, and *propose atomic patches* while preserving the official logic. This subsection specifies that representation and the formal model used for analysis.

Decision-list semantics (first-hit). Operationally, the NORDDR_G specification is an *ordered rule list* (decision list). Let the N rows be rules $R_i = (C_i, y_i)$ with fixed precedence $1 \prec 2 \prec \dots \prec N$, where each condition $C_i : \mathcal{X} \rightarrow \{\text{true}, \text{false}\}$ is a conjunction of primitive predicates (e.g., code-list membership, age-in-range), and $y_i \in \mathcal{Y}$ is a DRG output. Evaluation is

$$\hat{f}(x) = y_{i^*}, \quad i^* = \min\{i \mid C_i(x) = \text{true}\},$$

with a default rule if no C_i fires. Because conditions may overlap, the ordering is semantically essential (*first-match-wins*), and reordering rows can change behaviour.

Tree/DAG view (for analysis). For inspection and metrics this study *compiles* this ordered list to an equivalent tree/graph while preserving priority. One option is an *else-chain* (left-deep) tree that tests C_1 and, on failure, continues to C_2 , etc. Alternatively, this study forms disjoint conditions

$$C'_i = C_i \wedge \neg\left(\bigvee_{j < i} C_j\right),$$

which removes order dependence and yields a partition representable as a decision tree/DAG with shared subtests. Both compiled views are functionally equivalent to the ordered rules; this study uses the tree/DAG view for counts and figures, while the *authoritative model-of-record* remains the ordered rule list in the repository.

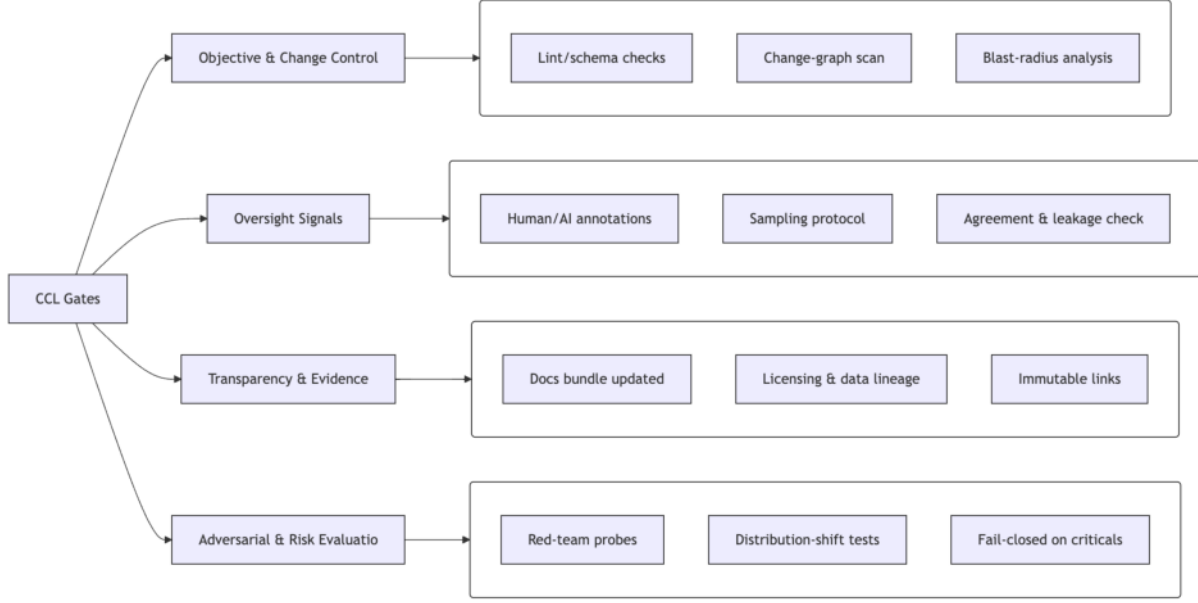


Figure 77: Gate policy map rooted in maintenance-time alignment: objective fidelity and locality; overseer signals for drift/leakage; provenance for auditability; adversarial/shift stress tests with fail-closed policies. Risk tier τ scales required evidence and human sign-offs.

Formal structure of the compiled view. Building on Section 6.3.1.1, this study uses an *equivalent, compiled* tree/graph T for analysis:

$$T = (\mathcal{V}, \mathcal{E}, r, (D_v)_{v \in \mathcal{V}_{\text{int}}}, (g_v)_{v \in \mathcal{V}_{\text{int}}}, (y_\ell)_{\ell \in \mathcal{V}_{\text{leaf}}}),$$

where

- $\mathcal{V} = \mathcal{V}_{\text{int}} \cup \mathcal{V}_{\text{leaf}}$ are internal and leaf nodes, with root $r \in \mathcal{V}_{\text{int}}$;
- each internal node v has a *node function* $g_v : \mathcal{X} \rightarrow D_v$ with a finite, typed outcome set D_v (typically Boolean; occasionally finite categorical);
- edges are outcome-labelled: $\mathcal{E} \subseteq \{(v, a, w) \mid v \in \mathcal{V}_{\text{int}}, a \in D_v, w \in \mathcal{V}\}$;
- each leaf ℓ carries an action $y_\ell \in \mathcal{Y}$ (a DRG code⁴¹).

The (possibly very large) code lists used by membership tests are *parameters* of g_v contained in θ and do not increase the branching factor. For example, $g_v(x) = \mathbf{1}\{\text{ICD}(x) \in L_v\}$ with $L_v \subseteq \mathcal{C}$ (and $|L_v|$ possibly in the thousands), so $|D_v| = 2$ even when $|L_v|$ is large. Node functions are drawn from a fixed catalogue $\mathcal{F} = \{f_1, \dots, f_{16}\}$ (e.g., membership-in-code-list; exists/count over diagnoses/procedures; age-in-range; sex-match), with parameters (lists, thresholds) contained in θ . This study requires *determinism* (for any v and $a \in D_v$ there is at most one $(v, a, \cdot) \in \mathcal{E}$) and *totality* (for all $x \in \mathcal{X}$, evaluation from r reaches exactly one leaf; a default “else” outcome in D_v may enforce this), so the induced map $\hat{f}_T : \mathcal{X} \rightarrow \mathcal{Y}$ is well-defined.

Evaluation on the compiled view. On the compiled tree/graph T , for an episode $x \in \mathcal{X}$ start at $v_0 = r$ and iteratively take the unique outgoing edge whose label matches the node outcome: for $t = 0, 1, \dots$

$$a_t := g_{v_t}(x) \in D_{v_t}, \quad (v_t, a_t, v_{t+1}) \in \mathcal{E}.$$

As T is acyclic with finite depth, there exists t^* such that $v_{t^*} = \ell^* \in \mathcal{V}_{\text{leaf}}$, and the induced decision function is

$$\hat{f}_T(x) = y_{\ell^*}.$$

Learning view. Although NORDDRG is currently hand-curated, nothing precludes deriving (or refining) T via the data-driven paradigms in Figure 70; the present study focuses on how such learning can be collaboratively curated during *continuous maintenance* rather than at initial training time.

⁴¹Optionally paired with country activation flags and related metadata, encoded in θ .

Potential representations. NORDDRG can be exposed with two equivalent surface representations:

- i) a **NordDRG custom format** (the *authoritative, ordered* rule list): a database of 20 interlinked tables with human-readable documentation that describes how the decision logic is applied to hospital-episode records [23];
- ii) a **canonical tree/DAG view** compiled from the ordered rules: each node is a typed predicate and each leaf a DRG label; order is preserved via else-chains or disjointised conditions. This view is convenient for metrics, figures, and ML tooling.

To illustrate the two representations, Figure 78 shows a fragment of the first-layer logic for DRG 373X (*Child delivery without complications*). The top part shows the custom format, where each row expands to a path condition; the bottom part shows the corresponding path-wise view compiled as a canonical decision tree. Both surfaces are round-trippable and agent-operable when accompanied by the Interpretation Guide for the custom format.

Example fragment of the first layer decision tree

All root → leaf paths to DRG: 373X

All the nodes in the decision tree are functions that contain typically compare list of diagnose or procedure codes against the input data. For example 14X03 contains 73 different codes.

drg_comb	icd	mdc	dgcat	pdgprop	compi	or_proc	procpro	secpro	dgprop1	dgprop2	dgprop3	dgprop4	agelim	sex	dur	disch
373X	+					S	14S06		14X03							
373X	+					S	14S04	-14S90	14X03							
373X	+					S	14S06		14X03							
373X	+						14S04	-14S90	14X03	00X10						
373X	+					N			14X03							

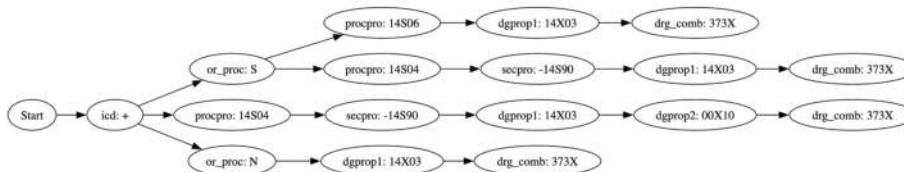


Figure 78: Illustrative fragment of the first-layer decision logic for DRG 373X. Top: excerpt of the custom NORDDRG format where each row expands to a path condition. Bottom: the corresponding path-wise view as a canonical decision tree compiled for analysis.

Selecting representation. As argued in the research gap (Section 1.4.3.3), standard ML decision-tree formats are typically not used for *collaborative development* of regulated decision logic. The NORDDRG custom format and attached processes are designed for expert review and approval of changes and are implemented in existing tooling, with human-readable documentation for regulatory transparency. This study therefore maintains the ordered rule list as the authoritative artefact under version control and use the compiled tree/DAG view for analysis, visualisation, and agent-facing tooling.

6.3.1.5 Design Artefact: Participating NordDRG LLM Agents

This subsection specifies the two LLM software agents—*Contributor* and *Reviewer*—that act as first-line participants in the CCL proposer-reviewer loop for maintaining the NORDDRG model-of-record. They ground tickets in the versioned specification, emit typed, atomic diffs with rationale, and perform pre-review quality checks, packaging reviewer-ready artefacts so that all edits remain traceable, auditable, and aligned with official guidelines under human-in-command governance.

Design Artefact: NordDRG Contributor Agent

Purpose and interaction model. The *NordDRG Contributor Agent* is a role-constrained LLM assistant that reads forum tickets or specifications, incorporates feedback from both agents and humans, and drafts

atomic diffs to the decision model. Interaction is chat-based: the agent produces reviewer-ready artefacts (diffs, brief, examples/tests) that can be attached to a pull request.

Scope and Assumptions. The agent operates over a versioned, typed NORDDRG model-of-record (line-addressable tables). Each change proposed by the Contributor is represented as an atomic diff, consisting of edits to predicates or outputs. The agent’s role is limited to exporting these diffs as advisory outputs, while the final decision to incorporate them into official repositories or workflows remains under human control.

Knowledge sources. At runtime the agent is grounded in two *versioned* artefacts:

- **Specification $\mathcal{S}$:** the authoritative NORDDRG definition tables and their induced rule graph (line-addressable, schema-typed).
- **Guidelines $\mathcal{G}$:** interpretation rules covering precedence, inclusions/exclusions, and CC/MCC semantics.

The agent retrieves scoped slices of $\mathcal{S}$ and $\mathcal{G}$ while drafting; all suggestions cite table/row IDs and guideline clauses (with commit/tag) to ensure provenance and auditability.

Inputs and outputs. *Input:* The agent accepts free-form chat messages, which may include copy-pasted ticket content or specification text.

Output. A reviewer-ready bundle:

- **Atomic diffs (machine-readable exports)** $\{\delta_i\}_{i=1}^k$: insert/modify/delete operations over line-addressable tables with stable IDs, emitted as CSV/JSON and patch snippets suitable for PR import.
- **Impact Brief (IB):** rationale, scope, affected DRGs/paths, citations.
- **Counterfactuals/tests:** positive, negative, near-miss cases for CI.

Architecture. The agent follows a multi-stage process:

1. **Chat + Ticket Parser:** The agent interprets user messages and any pasted tickets to extract the intent.
2. **Grounded Retrieval:** The agent pulls relevant sections from the specification $\mathcal{S}$ and guidelines $\mathcal{G}$.
3. **Change Planner:** It breaks down the intent into candidate atomic edits, annotated with potential impacts.
4. **Patch Composer:** The agent generates the diffs for the NordDRG specification format.
5. **Exporter:** The agent compiles the diffs and explanation into a bundle that can be exported for review.

All reasoning and drafting of changes are carried out by the LLM component, while deterministic checks are applied to ensure that changes are well-formed and consistent.

Operational loop.

1. Specification provided
2. Agent parses the intent
3. Retrieves specification and guidelines
4. Generates candidate changes
5. Produces a *reviewer-ready bundle*

User access. Experts interact via a chat interface. The agent explains its reasoning in natural language, links to specification slices (table/row IDs), cites guideline clauses, and offers one-click download of the reviewer-ready bundle.

Limitations and safeguards. The agent cannot post to ticketing systems, open pull requests, or write to authoritative repositories. All outputs are advisory and require human review and approval before inclusion in any official workflow. Interactions, prompts, and artefacts are logged (with timestamps and model/version identifiers) for traceability; deterministic linting enforces schema/typing and key integrity.

Design checklist (capabilities). The *NordDRG Contributor Agent*:

1. Applies the versioned NORDDRG specification under the official interpretation guidelines.
2. Accepts free-form chat/ticket input (optionally with table/row references) without external integrations.
3. Generates *typed*, *atomic diffs* with stable IDs, rationale, and minimal counterfactual tests.
4. Produces an *Impact Brief* and machine-readable exports suitable for PR/CI in CCL.
5. Uses an LLM for parsing/planning/explanation, with deterministic validators for schema, referential integrity, and locality bounds.

Design Artefact: NordDRG Reviewer Agent

Purpose and position in CCL. Where the *NordDRG Contributor Agent* drafts candidate updates, the *NordDRG Reviewer Agent* acts as an AI quality-gate that scrutinises every change set before it is exposed to human experts. Its mandate is to (i) verify *technical integrity* (typing, schema, referential consistency), (ii) assess *semantic fitness* against the official NordDRG guidelines, and (iii) prepare a structured review package that accelerates human judgment and sign-off. The agent therefore implements Steps 6–8 of the CCL Figure 76, shielding certified reviewers from low-level linting while surfacing the policy-relevant questions that require human insight.

Scope and guarantees. As illustrated in Figures 73, 74, and 76, the Reviewer Agent *never* approves or merges code; it only produces a Review Report with machine-checkable findings and prescriptive feedback. All higher-tier decisions rest with human reviewers, preserving the “human-in-command” invariant of CCL. Under normal operating conditions, the agent aims to ensure that every atomic diff it forwards to humans is syntactically valid across all target tables and complies with the NordDRG guidelines.

Knowledge sources. At runtime the agent is provisioned with $\mathcal{S}$ (the versioned NordDRG specification), $\mathcal{G}$ (interpretation guidelines), and the **Change Bundle** Π emitted by the Contributor Agent: $\Pi = \langle \{\delta_i\}, \text{rationale}, \text{examples}, \text{docs} \rangle$. Agent is also equipped with additional prompts to check typical issues encountered in the change requests such as suggesting changes to logic with national diagnose or procedure codes where as NordDRG specification uses combined common coding system.

Inputs and outputs. *Input*: a candidate pull request consisting of atomic diffs $\{\delta_i\}$ and its accompanying Impact Brief. *Output*: a Review Report $R = \langle \text{status}, \text{findings}, \text{suggested_fixes}, \text{qa_tests} \rangle$ where

- **status** $\in \{\text{pass}, \text{conditional-pass}, \text{fail}\}$;
- **findings**: structured comments grouped by table and diff hunk, ranked by severity;
- **suggested_fixes**: ready-to-apply patch snippets or prompt fragments for the Contributor Agent;

Review protocol. For every diff δ the agent executes a two-layer pipeline:

1. **Static analysis** — verify schemas, code lists, and key constraints.
2. **Guideline compliance** — the LLM checks each edit against formal NordDRG business rules and annotates deviations with inline citations.

Only if all layers succeed with no fail-level findings does the agent return **status=pass**; otherwise it flags the pull request for human attention, detailing exact correction steps.

Interaction loop with the Contributor Agent. When **status=conditional-pass** or **fail**, the Reviewer Agent posts its **suggested_fixes** back to the Contributor Agent (as depicted in Figure 71) participating in the *agentic workflow* auto-remediation cycle. The Contributor Agent may apply the patches, regenerate the Impact Brief, and re-submit—typically converging within one or two iterations before human review begins.

6.3.2 Demonstration: Collaborative Curated Learning

This section instantiates CCL on NORDDRG, demonstrating a minimum-viable, governed loop for maintaining auditable decision-making logic. Section 6.3.2.1 presents the technology stack, architecture, and capabilities

of the CCL Platform—the full-stack web application that serves as the execution environment for all demonstrations. Section 6.3.2.2 then parses the official definition tables and compiles them into a canonical decision-tree/DAG to expose structure, scale, and reviewability. Next, Section 6.3.2.3 prepares an *agent-operable* model-of-record—line-addressable files with stable identifiers and grounding guides—suitable for PR-centric governance. Finally, Section 6.3.2.4 exercises an agentic proposer–reviewer workflow: a *Contributor Agent* ingests tickets and emits typed atomic diffs with an Impact Brief, and a *Reviewer Agent* checks these proposals against schemas and guidelines and returns a structured Review Report. Together, these steps yield reviewer-ready artefacts that slot into CI gates and mandated human sign-off.

6.3.2.1 Technology and Architecture of the CCL Platform

The CCL Platform was implemented as a full-stack web application comprising a Python back end and a TypeScript/React front end. This section describes the technology choices, the overall architecture, the scale of the implementation, and key capabilities of the resulting artefact.

Technology Stack

Back end. The server side is built on **FastAPI** 0.109, a modern asynchronous Python web framework that provides automatic OpenAPI documentation, request validation via Pydantic 2.6, and high throughput through ASGI. Data is persisted in **PostgreSQL** and accessed through **SQLAlchemy** 2.0 with connection pooling. The multi-agent system integrates with three LLM providers—OpenAI, Anthropic, and Google Generative AI—through a unified **LLMService** abstraction that allows per-agent model selection at configuration time. Git operations are performed through the local file system and authenticated remote operations via GitHub tokens, enabling the platform to version-control decision model artefacts natively. Graph visualisations of git history and PR flow are generated as Mermaid diagram syntax on the server and rendered on the client.

Front end. The client is implemented with **Next.js** 15 (React 19) using the App Router and Tailwind CSS 3.4 for utility-first styling. All data visualisations are rendered client-side: interactive decision-tree views use custom React components, while sequence diagrams, flowcharts, and commit graphs are rendered with **Mermaid.js** 11.12. Markdown content—impact briefs, documentation, and review comments—is presented through React-Markdown with GitHub-flavoured Markdown support. The front end communicates with the back end exclusively through a typed REST API client that encapsulates 96 asynchronous methods organised by domain (pull requests, agent chat, specifications, git operations, workflows, governance, and administration).

Architecture Figure 79 illustrates the high-level architecture. The system follows a clear client–server separation. The Next.js front end runs in the user’s browser and sends HTTP requests to the FastAPI back end, which exposes 122 REST endpoints grouped into 13 route modules. The back end delegates domain logic to 17 service classes that encapsulate pull-request lifecycle management, git operations, specification parsing, agent orchestration, CI gate execution, audit logging, and change-request tracking.

The multi-agent orchestration layer coordinates three specialised agents—the Contributor Agent, the Reviewer Agent, and the Verifier Agent—through a state-machine-driven Workflow Engine. Each agent maintains conversation history in the database and produces structured artefacts (atomic diffs, impact briefs, review findings) that flow through configurable quality gates (G1–G4) and risk-tier-based approval workflows (R1–R3) before a change can be merged into the versioned decision model.

Scale of the Implementation Table 26 summarises the quantitative scope of the platform. The total implementation comprises approximately 33 000 lines of code split roughly equally between the Python back end and the TypeScript front end.

Application Capabilities The platform provides a comprehensive environment for collaborative, AI-assisted maintenance of the NordDRG decision model. Its capabilities can be grouped into seven areas.

Overview and situational awareness. The Overview page (Figure 80) serves as the entry point and presents live system metrics—open pull requests, pending reviews, specification table counts, and decision rule totals—alongside a summary of the quality-gate pipeline and key decision-model statistics. A step-by-step workflow visualisation explains the end-to-end change lifecycle, and navigation cards link to every section of the platform.

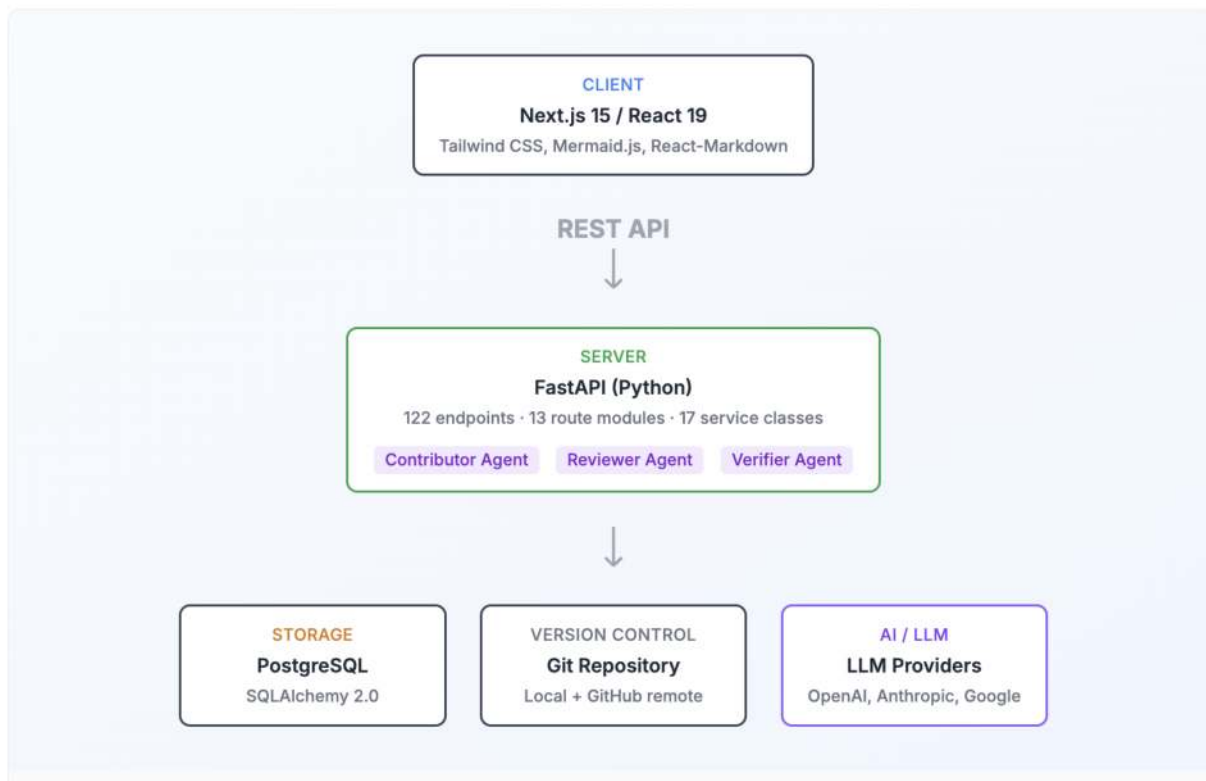


Figure 79: High-level architecture of the CCL Platform. The browser-based front end communicates with the FastAPI back end over REST. The back end integrates with PostgreSQL for persistence, a local git repository for version control, and multiple LLM providers for agent capabilities.

Table 26: Quantitative metrics of the CCL Platform implementation.

Metric	Count
Total lines of code	32 740
Front end (TypeScript/TSX)	16 494
Back end (Python)	16 246
Front-end source files	27
Back-end source files	63
Page components (Next.js routes)	14
Reusable UI components	11
API client methods	96
REST API endpoints	122
API route modules	13
Back-end service classes	17
Database models (SQLAlchemy)	9

Agentic workflow configuration and execution. The Agentic Workflows section (Figure 81) provides a configurator for defining multi-step, agent-driven pipelines. Each workflow specifies which agent roles participate (Contributor, Reviewer, Verifier), the maximum number of iterative loops, and which LLM model each agent should use. The Agent Configurator tab (Figure 82) allows administrators to create and manage individual agent configurations—selecting the agent role, underlying LLM provider and model, system prompt, and behavioural parameters. Automatic test and verification procedures can be attached to workflow steps. Once triggered, a workflow execution proceeds through branching, implementation, review, optional testing and verification, human review, and merge, with each step recorded in the audit ledger.

Governance and quality assurance. The governance framework implements four quality gates. Gate G1 validates schema, lint rules, and change-graph analysis. Gate G2 checks oversight signals, sampling protocols, and annotation agreements. Gate G3 verifies impact-brief completeness, documentation, and data lineage. Gate G4 runs adversarial probes and distribution-shift tests. Three risk tiers determine which gates are required and how many human approvals must be obtained: R1 (low risk, G1–G3, one approval), R2 (medium risk, G1–G4, two approvals), and R3 (high risk, G1–G4, three approvals including a safety reviewer).

Decision-tree visualisation and analysis. The Decision Tree page (Figure 83) offers three view modes for the NordDRG decision logic. The *Interactive Tree* mode renders an expandable, hierarchical view of decision nodes and leaf DRG outputs. The *SVG Graph* mode generates a Mermaid flowchart with configurable layout orientation. The *Full Analysis* mode presents a statistics dashboard with feature-usage bar charts, path-depth distributions, feature-expansion tables, and a ranking of the most common DRG outputs. A searchable DRG code filter allows users to isolate the decision path for any specific grouping.

Version control and change management. Every modification to the decision model is tracked through git. Pull requests carry atomic diffs (typed INSERT, MODIFY, or DELETE operations on specification table rows), impact briefs, review trails, and gate results. The Decision Model Version Control page visualises the commit history, branch graph, and PR lifecycle. A Data Browser enables direct inspection of the underlying specification tables with search, filtering, and pagination.

Change request management and agentic execution. The Tickets page serves as the primary entry point for the agentic workflow. Users can import change requests from external issue trackers (e.g., Redmine) or create them locally, attach supporting documents, and trigger AI-assisted implementation. The implementation pipeline allows selection of LLM provider and model, target NordDRG version, and relevant attachments. Progress is monitored asynchronously with a live execution log. Once the Contributor agent produces a pull request, users can trigger the Reviewer agent for automated quality review, inspect findings, and optionally re-run implementation with review feedback incorporated. The Workflow mode orchestrates the full Contributor–Reviewer–Verifier pipeline with configurable iteration limits and a mandatory human review checkpoint before merge.

Governance dashboard. The Governance page provides an administrative view of the quality assurance framework: editable quality gate definitions (G1–G4) with their associated checks and risk-tier requirements, risk tier configuration (R1–R3) with approval thresholds, a role-based approval workflow editor, and a chronological audit trail with filtering by pull request or event type.

User Interface Figures 80–83 show representative screenshots of the platform.

6.3.2.2 Analyzing NordDRG as a Decision Tree

Tooling and normalisation. A Python converter was implemented that ingests the official NORDDRG definition tables and emits a schema-typed, line-addressable file representation suitable for agent operation and PR-centric governance. This implementation underpins all analyses in this subsection and is released with usage notes and reproducibility scripts in the public repository.

Compiled view, metrics, and visuals. From the ordered rule list a canonical decision-tree/DAG for visualization purposes is compiled (Section 6.3.1.4). On this structure summary metrics are computed—features used, distinct DRG leaves, root→leaf path counts, and decision-point totals—and reviewer-legible Scalable Vector Graphics (SVG) slices for inspection and discussion are exported (Figure 84; Table 27).

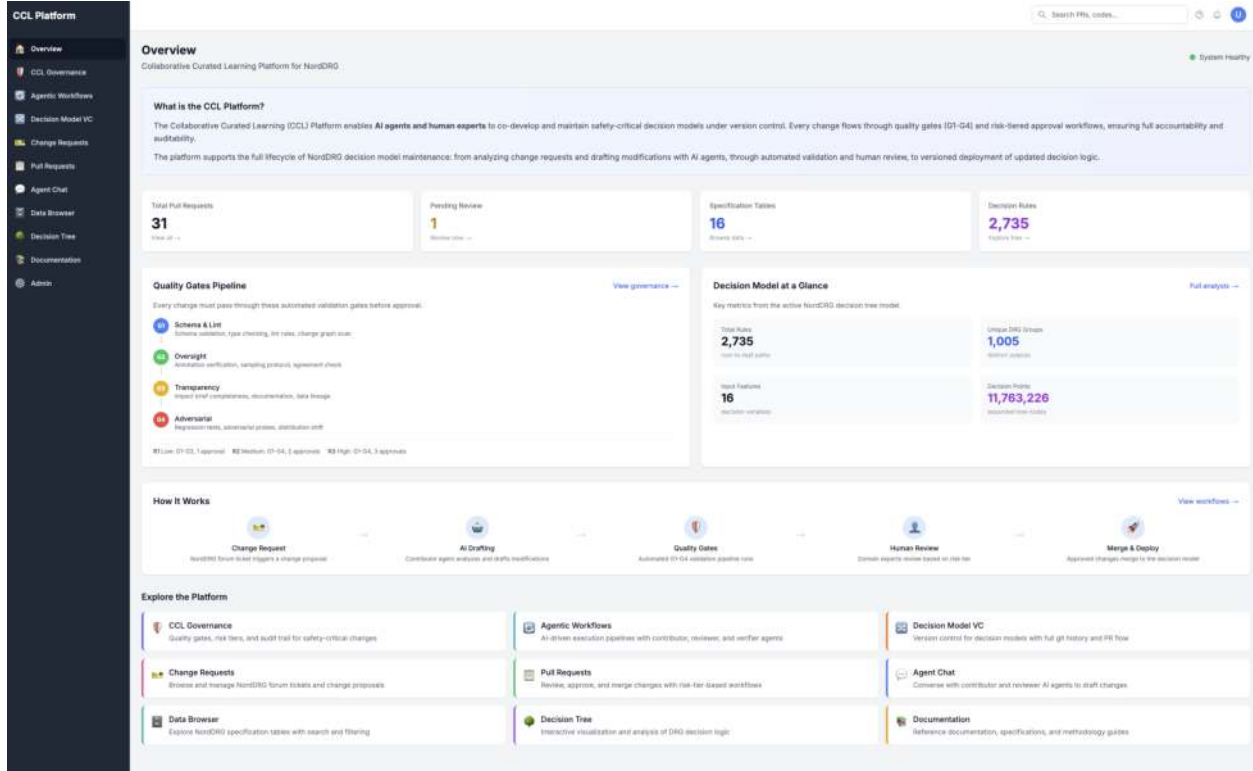


Figure 80: The Overview page of the CCL Platform. The top section displays live system metrics (pull requests, pending reviews, specification tables, decision rules). Below, the quality-gate pipeline (G1–G4) and decision-model summary provide at-a-glance situational awareness. The five-step workflow diagram illustrates the change lifecycle, and navigation cards link to all platform sections.

Table 27: Scale and structural statistics for the NordDRG decision model. The “first-layer” tree refers to the top-level grouping layer.

Description	Value
Number of features used in the decision tree	16
Number of different leaf labels (DRG groups) in the tree	2,752
Number of root to leaf paths	8,867
Average alternative paths to DRG leaf node	3.2
Number of non-empty decision points in #1 level decision tree	53,848
Average #1 level decision tree decision point depth leaf→node	6.1
Number of decision points in the full decision tree	40,453,620

A very large DAG. For analysis, the NordDRG decision model can be compiled to a large DAG. While the tree view makes individual predicates and audit trails explicit, the full structure is very large: it comprises thousands of DRG leaves and nearly nine thousand root→leaf paths. As a result, static visualisations are useful only for small slices; larger snapshots quickly become unreadable.

SVG artefacts. Figure 84 illustrates three small fragments of the first-layer decision tree, each showing a different number of root→leaf paths. The full tree is too large to be human-readable in static form, but smaller fragments are useful for teaching, reviews, and change proposals. To balance tangibility and scale, SVG exports of the first-layer decision trees are published in the GitHub repository. The complete first-layer tree (about 8,867 root→leaf paths; equivalently, about nine thousand root→node enumerations to leaves) results in a *16.4 MB* plain-text SVG. Smaller thematic fragments (e.g., 10, 25, or 100 paths) remain human-navigable and are well suited for teaching, reviews, and change proposals.

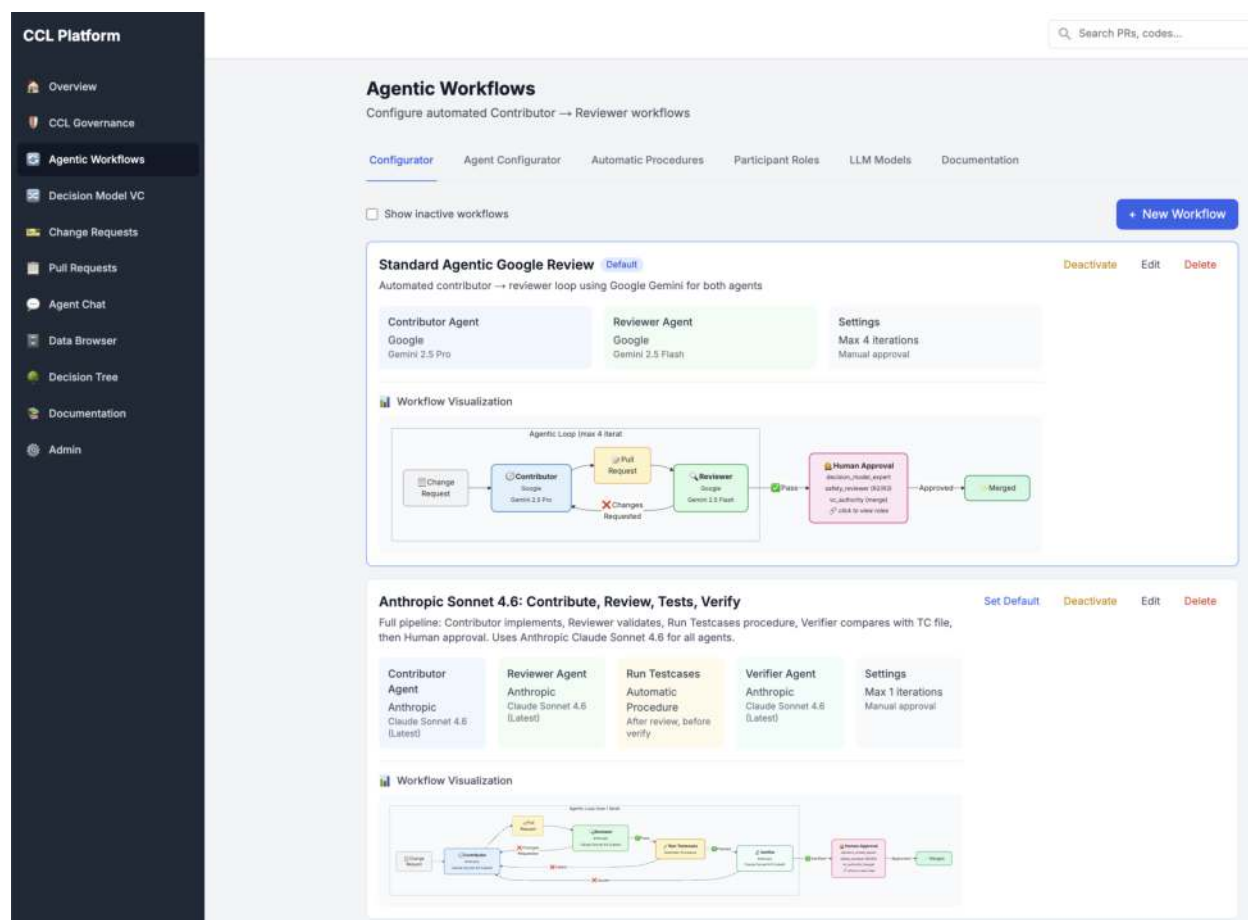


Figure 81: The Agentic Workflow Configurator. Administrators define multi-step pipelines specifying which agents participate, the iteration limit, and optional test and verification procedures. The lower panel lists existing workflow configurations with their status and execution counts.

Decision tree metrics. Table 27 summarises the key metrics of the first-layer decision tree. The metrics highlight how a modest feature set can generate substantial combinatorial structure. The first layer decision tree logic uses 16 input features are referenced by the rules, yet the tree yields 2,752 distinct DRG leaves. With 8,867 root→leaf paths, each DRG group is reachable via roughly 3.2 alternative paths on average, reflecting clinically meaningful variations that nonetheless end in the same group.

The notion of a *decision point* counts predicate evaluations along paths. In the first-layer tree there are 53,848 non-empty decision points, or about 6.1 checks per path on average—small enough to review visually in fragments. When the full, multi-layer specification is unrolled path-wise, this balloons to 40,453,620 decision points (roughly 4,600 checks per path on average), an increase by a factor of approximately 750. This explains why full-graph pictures are impractical and motivates programmatic analyses and interactive viewers. Together, these figures quantify the scale, redundancy, and auditing load inherent in the specification and provide concrete targets for optimisation (e.g., feature consolidation) and quality assurance (e.g., path coverage criteria).

6.3.2.3 Preparing the NordDRG Specification for CCL

Figure 85 illustrates the artefacts prepared in this study to enable CCL over the NordDRG specification.

From custom notation to agent-editable plain text. The *Design Artefact: Agent-Operable Decision Model of NordDRG* was authored in the customary NordDRG notation and then converted into a set of separate plain-text CSV files (one logical table per file). This representation makes the full, large decision

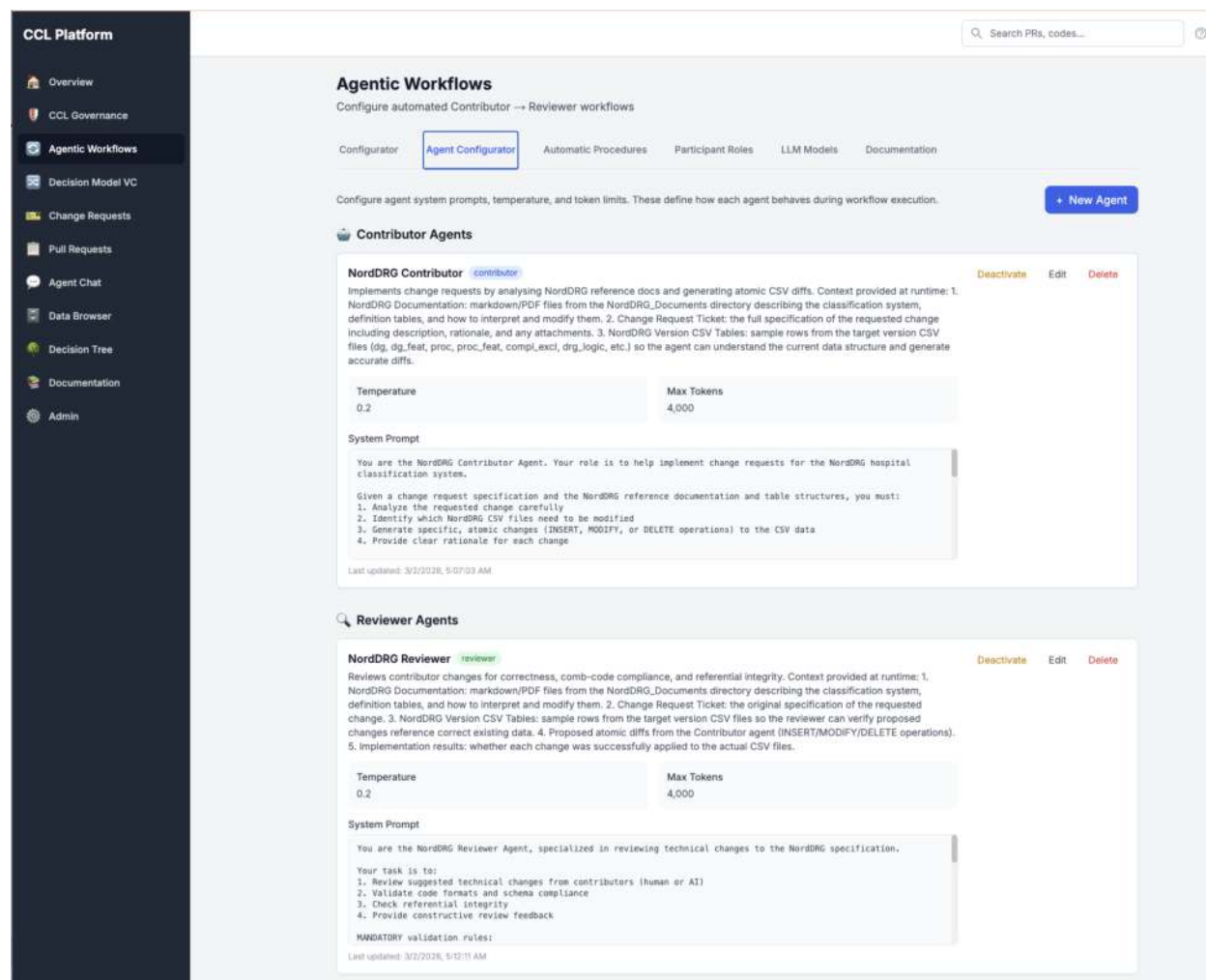


Figure 82: The Agent Configurator tab within Agent Workflows. Each agent configuration specifies the role (Contributor, Reviewer, or Verifier), the LLM provider and model, a system prompt, and behavioural parameters. Configurations can be selected per workflow step and per Agent Chat session, allowing different tasks to use different models.

model line-addressable in Git, supports automated diffs/reviews, and allows software agents to perform safe, scripted edits under CCL workflows.

Converter implementation. A Python baseline converter parses the NordDRG source specification and emits the normalised CSV files. The scripts and usage instructions are available in the accompanying Git repository, enabling full reproducibility of the conversion step.

Agent-facing documentation. To help LLM agents reason about and modify the custom NordDRG format, this study bundled two PDF documents retrieved from the NordDRG Forum that describe how to read the definition tables and how to write the changes to the specification. These serve as grounding references so agents can understand the notation and propose well-typed updates.

Relation to NDMS (NordDRG Maintenance System). Experts currently use the special-purpose NDMS software to generate official specification releases; these NDMS outputs serve as the starting point for the demonstration. While the same methodology could integrate NDMS directly with Git (e.g., automated exports and CI checks), such system integration is out of scope for this research.

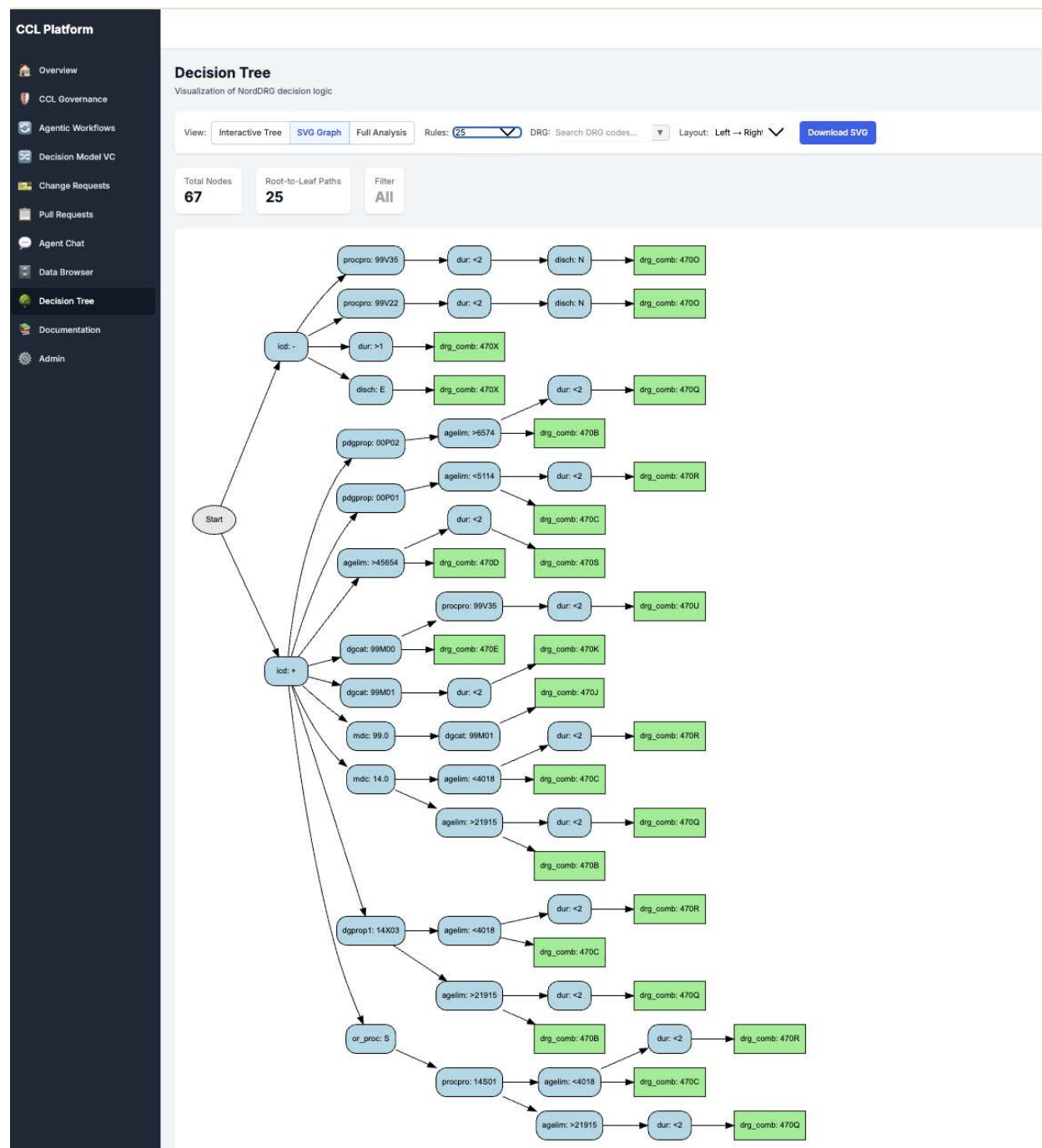


Figure 83: The Decision Tree page in SVG Graph mode, showing NordDRG decision logic as a flowchart. Blue nodes represent decision points on input features (diagnosis codes, procedures, age, sex), and green leaf nodes represent DRG group outputs. The toolbar provides view-mode switching, DRG code filtering, layout orientation controls, and SVG export.

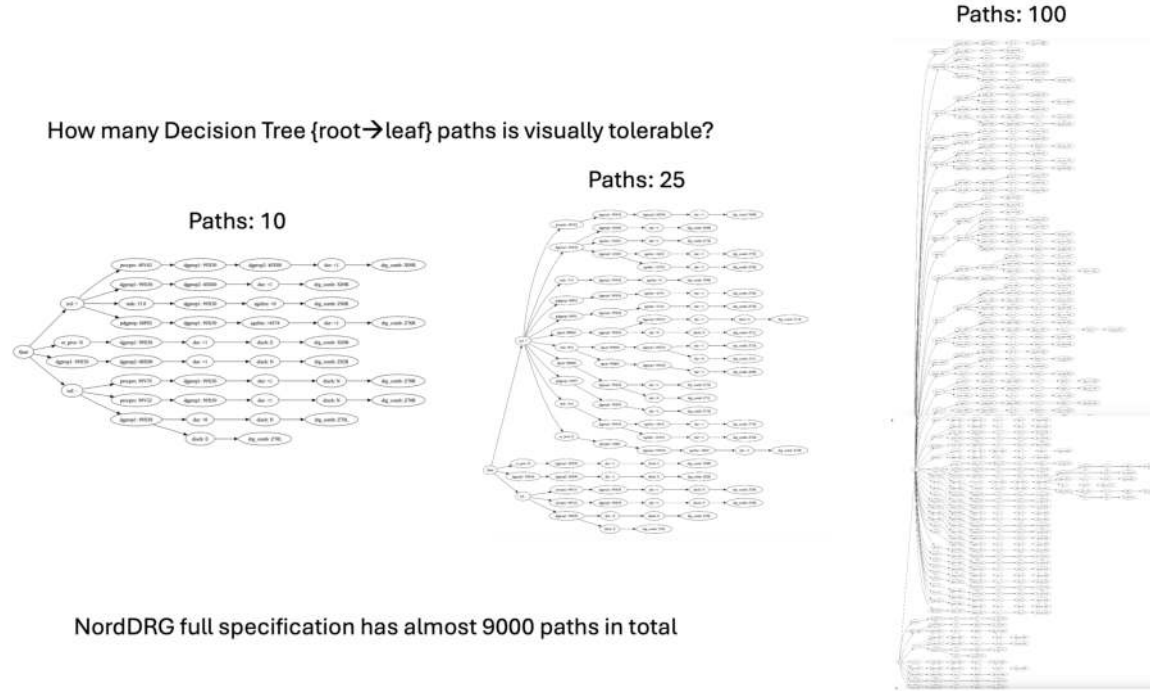


Figure 84: NORDDRG as decision trees: illustrative slices with 10, 25, and 100 root→leaf paths. Only small portions of the full structure are readable as static graphics. In practice: *10 paths* are fully legible at print scale and useful for explaining predicate structure and local flow; *25 paths* remain readable on screen with moderate zoom and are borderline for print without enlarging; *100 paths* are overcrowded in static form and are only useful with interactive panning/zoom, making them unsuitable for print. The full specification contains nearly 9,000 paths. The full 9000 path DAG can be rendered in browser but is difficult to navigate.

6.3.2.4 Demonstration: NordDRG Contributor and Review Agents in Action

Purpose Given human-authored tickets, the agent produces change suggestions aligned with the official NordDRG format. The process includes: (i) interpretation of the specification describing the intent and scope, (ii) generation of atomic diffs to the rule tables, and (iii) an agent-to-agent review process.

Context To illustrate how LLM software agents can participate in the CCL process, the two LLM Agents **NordDRG Contributor Agent** and **NordDRG Review Agent** were implemented (see Section 6.3.1.5 for Artefact design). The Contributor Agent is a conversational assistant that reads NordDRG Forum tickets (or pasted equivalents) and returns structured, machine-readable change suggestions for the NordDRG specification. The Review Agent ingests these change suggestions and the original ticket specification, then proposes changes to ensure the suggestions align with the NordDRG decision model coding standards and guidelines. The agents operate in a chat interface, allowing users to interact with them without needing to understand the underlying code or system integrations. Together the Contributor and Review Agents participate in an agentic workflow illustrated in Figure 71 and 76.

Context Provided to the Agents At runtime, the agent is grounded with:

- Prompts that describe the agent’s role and how it should interact in the process.
- The official NordDRG specification as interlinked tables;
- Human-readable documentation on how to interpret the specification;
- Guidelines that define the structure and style of TC suggestions.

This prototype operates on public artefacts and does not integrate with NDMS or evaluate NordDRG governance; its purpose is to illustrate how a governed, PR-centric loop could operate under human authority.

Preparing NordDRG specification for the CCL

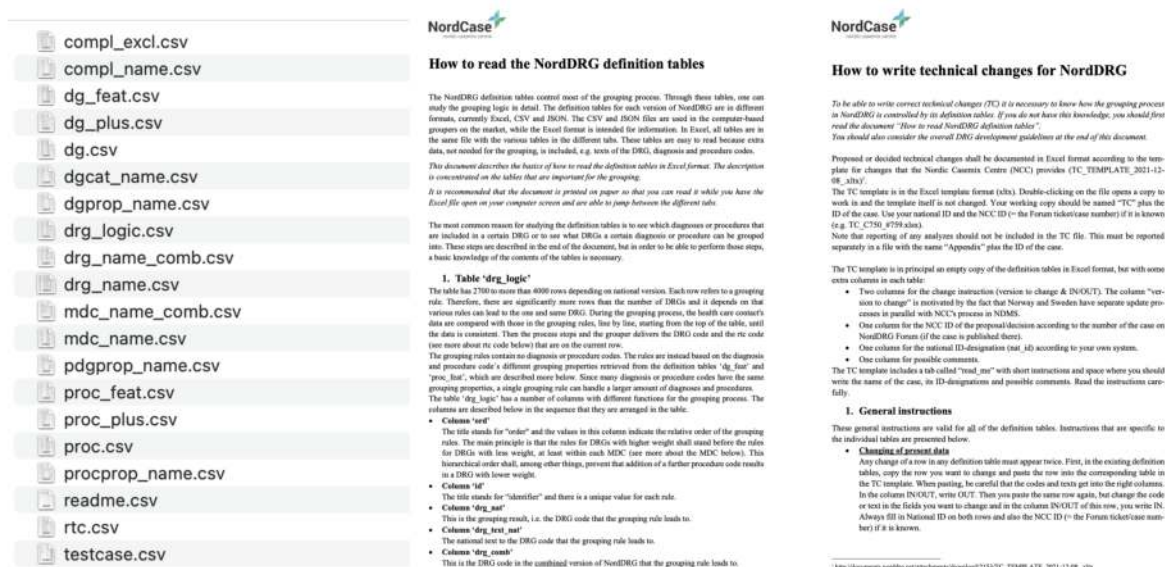


Figure 85: Preparing the NordDRG specification for CCL: (left) the CSV tables that constitute the agent-editable decision model; (middle/right) the two PDF guides from the NordDRG Forum used to ground agents in the custom notation and change process.

Agent LLM configurations In this demonstration, this study used OpenAI’s LLM models o3, o4-mini-high, and GPT-4o for the Contributor and Review Agents.

Agent Workflow and User Interface Interaction is via a chat-like interface, with no external integrations required. This lowers the barrier for non-technical stakeholders (e.g., coders and physicians) while producing reviewable technical artefacts immediately. The discussion interface supports both human-to-agent and agent-to-agent interaction (see Figure 72 and 71).

Figure 86 visualises the workflow turn of the *NordDRG Contributor Agent*: the left panel presents the original ticket posted in the NordDRG Forum—requesting new MDC08 rules and the addition of DGPROP 90X11 to a list of pain-related ICD-10 codes—while the centre icon symbolises the large-language-model agent that interprets this specification. Guided by the ticket, the agent autonomously synthesises a complete set of atomic `git` diffs (right panel): it creates a dedicated feature branch, appends DGPROP 90X11 to `dgprop_name`, assigns this property to the ten ICD-10 codes in `dg_feat`, and inserts the corresponding grouping rule in `drg_logic`. Together, these automatically generated patches and the concise rationale allow domain experts to review, validate, and eventually merge the proposed changes into the NordDRG definition tables, closing the loop between forum discussion and executable specification.

Figure 87 depicts the quality-assurance loop provided by the *NordDRG Review Agent*. The left panel combines the original forum ticket with the concrete `git` patches generated by the Contributor Agent, while the central icon represents the LLM-based reviewer. After parsing both the specification and the proposed diffs, the agent emits a structured set of review comments (right panel) that are grouped per table. Reviewer requests correction to diagnosis codes that are represented with national codes instead of common Nordic codes used by the NordDRG specification; enforces the conversion of national ICD-10 codes to their comb-code counterparts in `dg_feat`; and highlights quoting, alignment and rule-ID requirements in `drg_logic`. These actionable annotations enable domain experts to iterate on the patch set until it fully complies with the NordDRG template and comb-code conventions, thus safeguarding the integrity of the decision model.

Prompt Transparency All system and user prompts configuring the agent are openly available with the codebase to support reproducibility and safe sandboxing by national authorities.

- Suggestions are expressed as atomic diffs over a typed decision model;
- Outputs are human-readable and machine-verifiable;
- Audit trail-friendly exports are prepared for downstream CI validation;
- No autonomous merges occur; maintainers retain full control.

161



Figure 87: NordDRG Review Agent taking in implementation by NordDRG Contributor Agent together with the original ticket specification. The Review Agent proposes changes to the implementation to align with the NordDRG decision model coding standards and guidelines.

7 EVALUATION AND DISCUSSION

This chapter evaluates the demonstrations presented in the previous chapters.

This chapter is organized as follows.

Phase I evaluations:

- Section 7.1 evaluates and discusses the Interaction Protocol Adaptation with Software Agents demonstration.
- Section 7.2 evaluates and discusses the Ontology-Driven Software Agents for B2B Purchasing Automation demonstration.
- Section 7.3 evaluates the third demonstration on context-aware service matching in a mobile environment.

Phase II evaluation:

- Section 7.4 evaluates and discusses the empirical implementation on healthcare data with software agents and ML-based information retrieval techniques.

Phase III evaluations:

- Section 7.5 evaluates and discusses the Human-AI agent decision alignment architecture.
- Section 7.6 evaluates and discusses the NordDRG AI Benchmark for LLMs.
- Section 7.7 evaluates and discusses the CCL reference development process for decision-making logic.

DSRP flow, see Table 3 (DSRP process sequence linked to Phases and Structure of this Thesis).

7.1 Phase I: Interaction Protocol Adaptation with Software Agents)

Purpose and scope. This evaluation and discussion concerns the main artefact *interaction-protocol-based service adaptation*, assessing it against objectives O.I.1.1 - O.I.1.4 and situating it within the problem setting and scenario outlined in Section 1.2.1 and Section 4.1.

Evaluation against objectives

O.I.1.1 Protocol parsing from RDF serialization (Section 4.1.1.2, Section 4.1.1.4) — *Achieved, but brittle*. The expeditor agent parsed RDF-serialized, ontology-conformant FIPA protocol descriptions at run time and mapped them to a preprogrammed state machine. This demonstrates machine-processable intake and internalization of protocol structure. However, the parser’s success is contingent on *strict* adherence to the predefined interaction-protocol ontology. Minor schema drift, alternative modeling idioms, or richer constraint vocabularies were not exercised. No evidence was shown for graceful degradation, schema versioning, or recovery strategies when encountering out-of-ontology constructs.

O.I.1.2 Message exchange on agent platform (Section 4.1.2.2) — *Achieved, minimal coverage*. FIPA-ACL conversations were executed among contractor, subcontractor, supplier, and carrier agents with observable routing and platform-level diagnostics. The result is a solid baseline that the plumbing works. The coverage is nonetheless narrow: only the “happy path” and a small set of dialogue moves were demonstrated; transport variability, failure modes (timeouts, retries, disagreement), and performance under concurrency were not evaluated. Registry usage and partner discovery were intentionally simplified, which understates integration challenges at ecosystem scale.

O.I.1.3 Scenario-specific protocol handling without hard-coding (Section 4.1.1.3) — *Partially achieved*. The contractor’s behavior generalized across supplier (FIPA Query) and carrier (similar) protocols when they conformed to the ontology, supporting the claim of configuration-over-code. Yet only the *contractor* role exhibited run-time flexibility; counterparties followed fixed behaviors. The control logic remained a finite-state machine without an explicit goal-directed transition function or policy layer, limiting adaptation to preenumerated states and transitions. Unanticipated protocol variants, exception/repair patterns, or mixed-initiative deviations were out of scope.

O.I.1.4 Report extraction and comparison (Section 4.1.1.3) — *Achieved, thin validation.* The expeditor extracted order identifiers from RDF-formatted reports and cross-queried supplier and carrier agents to compare status information. This establishes an auditable *mechanism* for lightweight cross-validation. The experiment stopped short of systematic data quality assessment (precision/recall of matches, handling of conflicting claims, or reconciliation policies), so the *effectiveness* of comparison remains suggestive rather than quantified.

Synthesis

Taken together, the artefact demonstrates end-to-end feasibility under controlled conditions: ontology-based protocol intake, minimal partner discovery, FIPA-ACL message exchange on JADE, and limited cross-validation. The strongest evidence lies in O.I.1.1 and O.I.1.2 (machine-processable descriptions and functioning conversations). The weakest lies in O.I.1.3 (breadth and robustness of “adaptation”) where flexibility is confined to one role and bound to FSM enumerations, and in O.I.1.4 where comparison exists as a mechanism but lacks evaluation depth. Overall, the prototype is better characterized as *ontology-constrained interoperability with scoped flexibility* rather than open-ended run-time adaptation.

Limitations to validity

Construct validity.

- *Adaptation proxy.* Using an FSM as the primary control model risks conflating protocol selection/execution with genuine adaptation. Goal-aware transition functions or policies were not implemented, so “adaptation” may overstate what is essentially parameterized execution.
- *Protocol conformance assumption.* Success depended on exact ontology conformance; this may measure *schema agreement* more than real-world adaptability.

Internal validity.

- *Role asymmetry.* Only the contractor adapted; fixed counterparty behaviors may mask interaction failures that would surface if all roles varied concurrently.
- *Instrumentation bias.* Platform debug tools were used for observation; absence of independent tracing/ground-truth checks may inflate perceived reliability.

External validity.

- *Ecological realism.* The expediting scenario is plausible but limited. Heterogeneous partner stacks, partial ontology adoption, and conflicting exception policies—typical in multi-tier supply chains—were not exercised.
- *Scalability.* No load, latency, or concurrency studies were reported; registry simplifications downplay discovery, matching, and churn at scale.

Conclusion validity.

- *Sparse metrics.* Evidence is primarily qualitative (“works on example”) with few quantitative indicators (coverage, failure rate, recovery time, comparison accuracy). This weakens the strength of any generalized performance claims.

Implications for the artefact

To progress from controlled feasibility to robust adaptability, the artefact would benefit from: (i) adding a goal-directed transition function or policy layer over the FSM; (ii) extending adaptation to multiple roles with negotiated exception/repair patterns; (iii) hardening the registry for discovery and semantic matching (versioning, partial conformance, conflict resolution); and (iv) expanding evaluation to adversarial and out-of-ontology cases with quantitative metrics for comparison accuracy and end-to-end reliability.

Discussion

The demonstration establishes a narrow but concrete path from ontology-grounded protocol descriptions to executable conversations on an agent platform. Its chief contribution is evidencing that *conversation structure*, when explicitly modeled in RDF/OWL and consumed at run time, can lift integration beyond brittle, endpoint-centric glue code. At the same time, three design trade-offs limit how far these results generalize:

- **Ontology boundary vs. real-world variance.** The prototype works well *inside* a controlled ontology; outside it, behavior is undefined. In multi-party ecosystems, variance concentrates at precisely those boundaries (partial adoption, version skew, divergent exception patterns). Without partial-conformance handling, schema mediation, or negotiation, the approach risks creating a new lock-in at the ontology layer.
- **FSM control vs. goal-directed behavior.** Finite-state control delivered predictable execution but conflated “following a script” with “adapting.” True adaptation requires an explicit transition policy over goals, context, and constraints (e.g., utility- or rule-based selection of next moves, exception/repair tactics). Absent that layer, flexibility is limited to pre-enumerated transitions.
- **Plumbing proven, ecosystem not.** FIPA-ACL messaging on JADE and basic registry use show the plumbing is viable; what remains unproven is operation under churn (partner join/leave), conflict (inconsistent claims), and scale (throughput, latency, fairness). These are the conditions under which integration architectures earn or lose their keep.

Positioning. Compared to WS-*/BPMN-style service integration, the artefact shifts emphasis from operation-level bindings to *conversation semantics*. This is a meaningful design choice for domains where turn-taking, commitments, and exception handling dominate over single-call idempotent operations. However, the cost is a heavier dependence on shared vocabularies and governance for versioning and change control.

Design lessons. (i) Treat conversation ontologies as first-class, versioned contracts with partial-match semantics; (ii) separate execution control (FSM) from decision policy to avoid overloading state transitions with goal logic; (iii) design for disagreement and divergence (conflict detection, reconciliation policies, audit trails) rather than only the “happy path.”

Forward link. The above suggests a trajectory where conversation semantics remain central, but are surrounded by governance and accountability layers (role binding, policy gates, exception playbooks) and by mechanisms for aligning protocol execution with organizational objectives and values. Subsequent phases can build on this by introducing goal-aware control and stronger audit/trace facilities while retaining the benefit of machine-interpretable interaction protocols.

7.2 Phase I: Ontology-Driven Software Agents for B2B Purchasing Automation

Purpose and scope. This section evaluates and discusses the demonstration of the *ontology-driven B2B purchasing* main artefact (Table 5), assessing it against objectives O.I.2.1 - O.I.2.4 (Section 1.2.2.3) and situating it within the problem setting and bidding scenario outlined in Section 1.2.2 and Section 4.2.

Evaluation against objectives.

- **O.I.2.1 Routine task automation.** *Partially met.* Basic supplier discovery via a registry and ontology-driven acceptance checks are implemented. However, routine purchasing typically includes multi-round RFQs, counter-offers, expiry handling, delivery-slot conflicts, and exception workflows. None of these operational loops are evidenced. The “automation” remains closer to *scripted reasoning* than to end-to-end task automation.
- **O.I.2.2 Agent-based realisation.** *Met (scoped).* Buyer/supplier roles converse on JADE with conversation semantics foregrounded. That said, the platform-level success hides integration effort: there is no demonstration of reliability under contention, recovery from partial failures, or cross-platform interoperability beyond JADE.
- **O.I.2.3 Human oversight.** *Not demonstrated.* Escalation triggers, approval gates, or “four-eyes” checks are not documented. In practice, thresholds for price variance, supplier risk, or contractual deviations would route decisions to humans; the demonstration provides no concrete mechanism or audit artefacts for these escalations.

O.I.2.4 Ontology leverage. *Met (minimal expressivity).* Ontologies make artefacts machine-processable and enable acceptance reasoning. The commercial surface is narrow: payment and delivery semantics are simplified, no modelling of penalties, rebates, Incoterms nuances, volume breaks, or exception clauses. The coverage is adequate for illustration, not for operational breadth.

Evidence from the demonstration. Enterprise systems (pricing, stock) are simulated with XML sources; the registry is presented as a semi-intelligent exemplar; conversations execute on JADE; Web-service transport and OWL-S style bridges are acknowledged but intentionally out of scope. Collectively, this evidences a *working vertical slice* rather than a production-grade pipeline.

Evaluation approach. A demonstration-based, descriptive evaluation was used: a single platform, simplified ontologies, simulated backends, and a bespoke registry. No comparative baselines, ablation studies, load tests, or user-in-the-loop trials are reported.

Assumptions and boundary conditions. (i) Agents have timely access to internal data (simulated); (ii) ontologies are purposefully minimal; (iii) registry is illustrative rather than complete; (iv) transport/interoperability beyond JADE is de-emphasised.

Limitations to validity

- **Construct validity.** “Automation” is proxied by a single-shot acceptance decision and registry lookups; typical constructs like multi-round negotiation, exception handling, and SLA compliance are absent, risking an optimistic interpretation of automation.
- **Internal validity.** With XML simulators standing in for ERPs and PIMs, data freshness, latency, and failure modes are not controlled for; it is unclear whether observed behaviour would persist under real integration constraints.
- **External validity.** The ontology subset omits common commercial terms (rebates, penalties, Incoterms granularity). Findings may not generalise to sectors with complex contractual logic or to multi-tier supply chains.
- **Reliability.** Results are shown on one agent platform without replication across stacks or repeated trials; no metrics (success rates, cycle times, negotiation convergence) are reported.
- **Conclusion validity.** Claims of “automation” or “agent effectiveness” lack quantitative evidence; absence of baselines prevents effect-size estimation.

Implications for design. Three engineering priorities emerge: (1) *Oversight by design*—codify approval thresholds, escalation paths, and audit logs as first-class artefacts; (2) *Commercial completeness*—extend ontologies to cover payment/delivery terms, penalties, rebates, volume pricing, and exception clauses; (3) *Operational realism*—replace simulators with live ERP/PIM integrations, strengthen discovery beyond a minimal registry (semantic matching, reputation/risk signals), and introduce robustness features (timeouts, retries, compensation). A natural extension is to couple purchasing with *financial supply-chain* considerations (cash-flow aware negotiation), contingent on deeper accounting integration and richer policy vocabularies.

Discussion. Taken together, the demonstration offers a useful *proof-of-concept* for ontology-mediated agent interactions, yet it also surfaces a structural tension between semantic rigor and operational completeness. The agents can reason over well-scoped purchasing artefacts, but the absence of multi-round negotiation, exception handling, and explicit human control points means the automation remains closer to decision support than to production-grade delegation. In practice, advancing toward routine task automation would likely require a staged maturation path: first, broaden commercial coverage in the ontologies (terms, constraints, and exceptions); second, embed oversight primitives (approvals, escalation thresholds, and audit trails) as first-class protocol elements; third, harden discovery and integration beyond a bespoke registry and XML simulators to include live ERP/PIM links, reliability patterns, and cross-platform interoperability. Until these steps are addressed and measured the value of the approach is best framed as *semantic enablement* that prepares the ground for later operationalisation, rather than as immediate end-to-end automation.

7.3 Phase I: Matching Service Descriptions in Context-Aware Environment

Purpose and scope. This section evaluates how the DYNAMOS main artefact (Table 5) addressed the problem of delivering context-relevant services to mobile users in dynamic environments (Section 1.2.3,

Section 4.3). The focus is a direct assessment against the stated objectives: *context modelling* (O.I.3.1), *automated matching with minimal resource overhead* (O.I.3.2), and *collaborative annotation support* (O.I.3.3). Evidence is drawn from a field deployment and controlled performance measurements.

Evaluation Approach

1. **Field demonstration:** Real-world deployment during a sailing competition, exercising clients, sensors, networks, and server components under authentic conditions.
2. **Performance measurements:** Benchmarks of context updates, data access, and matching to locate bottlenecks and capacity limits.

Evaluation Against Objectives

O.I.3.1 Context modelling. The ontology-based model linked relatively static profiles with dynamic context variables (location, activity, time) and enabled one-time preference setup. In practice, it was expressive enough to filter content without overwhelming devices; users received activity-appropriate items without extra in-event configuration. However, the simplicity constrained cross-activity granularity (e.g., transitions from “racing” to “docking” required coarse state changes). *Assessment: Achieved, with scope constraints in cross-activity granularity.*

O.I.3.2 Matching with minimal resource overhead. Server-side filtering and compact result sets were effective: throughput for local operations was high and client payloads remained small; relevance targets were met. Nevertheless, end-to-end costs were dominated by remote calls: cross-network invocations reduced throughput by two to three orders of magnitude, making pre-filtering mandatory. On-device robustness and energy use were fragile under realistic conditions: JVM instability, GPS/Bluetooth churn, and concurrent sensor plus data flows produced crashes and noticeable battery drain. Scaling indicated the need for spatial and attribute indexing before increasing the catalogue size beyond $\sim 10^3$ items. *Assessment: Partially achieved—matching quality met goals, but efficiency was brittle on mobile clients.*

O.I.3.3 Collaborative annotation. User-generated notes and ratings were treated as first-class, matchable artefacts and propagated via an event stream. This enabled timely peer-to-peer updates alongside commercial services. Under heavier loads, missing indices led to degraded latency; moderation and provenance were minimal, leaving noise and redundancy unmanaged. *Assessment: Achieved for functionality; scalability and curation require hardening.*

Key Findings from the Demonstration

- **Integration, not algorithms, dominated performance:** RPC/RMI boundaries were the primary bottleneck; relocating filtering and matching to the server was decisive.
- **Mobile constraints were first-order:** In high-wind, low-attention conditions, single-tap interactions and defensive client design were essential; device/JVM instability limited reliability independent of server correctness.
- **Simple context worked on-device:** An activity-centric model delivered relevant filtering but limited fine-grained transitions across activities.
- **Peer content added value but increased noise:** Treating annotations as content improved utility; lack of indexing and curation impacted responsiveness and signal quality at larger result sets.

Design Principles Derived

1. Filter at the data source to minimize network and client load.
2. Use event-driven propagation for context and peer updates.
3. Keep context models minimal and activity-centric for mobile viability.
4. Design for divided attention: single-action interactions and clear defaults.
5. Conserve energy via aggressive sensor scheduling, backoff, and batching.
6. Add spatial and attribute indices before scaling catalogue size or user count.

Limitations to Validity

- **Scenario specificity:** A single activity (sailing) and a narrow cohort limit ecological validity for other domains (e.g., commuting, indoor venues).
- **Platform vintage:** Results are confounded by 2005-era device/JVM faults; some failures reflect platform immaturity rather than architectural limits.
- **Middleware choice:** Performance conclusions reflect Java RMI; alternate transports (e.g., message queues or HTTP batching) could shift—but not erase—bottlenecks.
- **Scale extrapolation:** Behaviour beyond $\sim 1,000$ items is inferred from profiling and indexing arguments, not demonstrated at full scale.
- **Battery measurement:** Energy impact was assessed qualitatively; absence of instrumented power traces weakens efficiency claims.
- **Annotation bias:** Peer contributions were opportunistic and unmoderated; findings on collaborative value may not generalize to adversarial or low-participation settings.

Discussion

The demonstration indicates that context-aware matching can satisfy relevance requirements with a lightweight ontology and server-side selection, but only when integration overheads are tightly controlled and mobile fragility is anticipated. The critical path is architectural plumbing: batching, indexing, and event-driven updates mattered more than marginal tweaks to the matching logic. For progress toward production, three refinements appear most impactful: (i) push more computation to indexed, cache-aware servers with coarse-grained client synchronization; (ii) formalize annotation governance (deduplication, provenance, ranking) to improve signal quality; and (iii) extend the context model with transitional states that preserve simplicity while supporting multi-activity journeys.

7.4 Phase II: Assisting Clinical Decision Support with ML and Software Agents

Evaluation Against Objectives

Purpose and scope. This section evaluates and discusses the demonstration of the *ML/IR-assisted clinical decision support* main artefact (Table 5), assessing it against objectives O.II.1.1 - O.II.1.4 and situating it within the problem setting and clinical coding workflow described in Section 1.3.1 and Section 5.1.

O.II.1.1 Robust ML/IR Architecture (Arch). *Artifacts and approach:* A vector-space information retrieval (IR) pipeline using TF-IDF weighting and cosine similarity was adapted to clinical coding (ICD-10, NCSP), augmented with hierarchical code relationships; the MAS decomposed into Crawler/Integrator/Commander/Aggregator agents coordinated demand-driven retrieval and scoring. *Findings:* The IR approach handled extreme sparsity and high dimensionality; a combinatorial analysis (on the order of 10^{230} possible fact combinations) motivated favoring IR over dense neural models in this setting. The bounded similarity scores yielded interpretable rankings for coder workflows. *Limitations/Threats:* Current weighting treats repeated codes uniformly and relies on exact or near-exact code matches; broader clinical entities (labs, meds, vitals) were not modeled, potentially underestimating real dimensionality. Emerging sparse/attention models could narrow the gap with IR. *Implications:* Domain-aware IR with ontology hooks is a pragmatic architectural fit for fragmented, sparse EHR data; future work should explore ontology-backed expansion and non-uniform term weighting.

O.II.1.2 Clinically Viable Recommendation Accuracy (Acc). *Target:* $>50\%$ first-proposal; $>90\%$ top-10. *Results:* Achieved 53–66% first-proposal accuracy and $\geq 90\%$ accuracy within 8–10 proposals, meeting the stated thresholds. The *SHOULD-occur* fetching strategy consistently outperformed *MUST-occur*, especially under imperfect documentation. *Limitations/Threats:* Accuracy was computed against historical documentation assumed as ground truth despite reported coding error rates of 20–40% [310, 311]; metrics emphasize common cases and may under-capture rare-but-critical conditions; clinician usability/trust feedback was not collected. *Implications:* Results indicate practical viability for coder-assist triage; future evaluation should incorporate clinician-in-the-loop studies, confidence estimates, and utility on low-prevalence edge cases.

O.II.1.3 Robustness to Data-Quality Defects (Rob). *Design:* Comparative tests on higher- vs. lower-quality datasets and on perturbations (removed/incorrect/novel codes). *Findings:* *MUST-occur* degraded rapidly under data defects; *SHOULD-occur* maintained acceptable performance, showing graceful degradation aligned with real-world data issues [307, 308]. *Limitations/Threats:* Experiments predominantly removed single facts; real episodes may have multiple concurrent defects and temporal uncertainty. Robustness beyond diagnoses/procedures (e.g., noisy labs) remains untested. *Implications:* Flexible matching and redundancy are essential for production deployments across heterogeneous hospital data estates; a principled uncertainty layer would further aid risk-aware use.

O.II.1.4 Real-time Scalability on Commodity Hardware (Scale). *Target:* Sub-second responses on datasets up to 10^8 episodes; near-linear scaling. *Results:* Sub-250 ms median query times were observed up to 10^8 episodes with near-linear scaling on standard hardware; the demand-driven MAS reduced overhead versus continuous polling. *Limitations/Threats:* Large-scale tests leveraged synthetic extrapolations; failure modes (agent crashes, partitions), security-hardening, and contention under production loads were not fully characterized. *Implications:* The architecture is performance-feasible on commodity stacks; productionization should add resilience testing, privacy/security controls, and capacity planning under bursty workloads.

Synthesis: What the Objectives Demonstrate

Collectively, the results show that (i) a domain-adapted IR architecture embedded in a MAS can cope with extreme sparsity and fragmentation (Arch), (ii) delivers clinically viable top- k suggestions (Acc), (iii) degrades gracefully under realistic documentation defects with flexible fetching (Rob), and (iv) sustains sub-second responsiveness at 10^8 -scale on commodity hardware (Scale). These together close a practical gap between theoretical ML capabilities and the constraints of real healthcare information systems.

Positioning w.r.t. Design Science Research Guidelines

Relevance & design as an artifact: The six-level abstraction from patient facts to DRG, the IR model, and the MAS are purposeful artifacts addressing coder-assist pain points. **Rigor:** Formal data-space characterization and systematic retrieval/evaluation ground the design. **Design evaluation:** Objective-aligned tests (accuracy, robustness, latency/scale) provide empirical evidence. **Contributions:** Quantifies combinatorial bounds for healthcare coding; adapts IR to clinical hierarchies; demonstrates agent-based integration for fragmented estates. **Research communication:** Artifacts and evaluation are documented to inform IS and health informatics audiences.

Concise Artifact Notes (Mapped to Objectives)

Healthcare Item-Set Process Flow (*Arch, Rob*): Clarifies information loss across the six-level hierarchy and operationalizes demand-driven retrieval in fragmented environments. **Data Space Formalization** (*Arch*): Justifies sparse IR via combinatorial bounds; highlights limits of dense modeling without structure. **ML/IR Model** (*Acc, Rob*): TF-IDF + cosine with hierarchical relationships; suggests future non-uniform term weighting and ontology expansion. **MAS Architecture** (*Scale, Rob*): Crawler/Integrator/Commander/Aggregator with demand-driven coordination; note modernization, resilience, and security work for production.

Limitations and External Validity (Objective-Aligned)

Arch: Excludes several clinical modalities; consider multimodal features and ontology-backed expansion. *Acc:* Historical-label bias and lack of clinician UX/trust measures; add user studies and rare-case stress tests. *Rob:* Single-fact perturbations under approximate real noise; evaluate multi-defect and temporal uncertainty. *Scale:* Synthetic scaling and limited fault-injection; add chaos testing, access controls, and privacy-by-design.

Practical Implications

Hospitals can adopt the MAS+IR pattern to integrate dispersed systems and deliver explainable, top- k coder assistance with commodity infrastructure. Priorities for deployment: (1) ontology alignment and code normalization, (2) uncertainty-aware UX for coders, (3) operational hardening (security, audit, fault tolerance), and (4) performance tuning against real workload traces.

Summary

The Phase II artifacts meet their stated objectives: they are architecturally appropriate for sparse, fragmented data (Arch), achieve clinically viable accuracy (Acc), remain resilient under data-quality defects (Rob), and scale in real time on commodity hardware (Scale). These results substantiate the feasibility of agent-supported IR for clinical decision support and outline a clear modernization path toward production readiness.

Discussion

MAS+IR is a pragmatic fit for coder-assist in fragmented hospital estates: it favors bounded, explainable retrieval over generative or opaque end-to-end models, achieving controllable latency and governance at the cost of some semantic expressivity. Demand-driven agent orchestration localizes failures and cost but adds coordination overhead that must be engineered (timeouts, retries, backpressure). Compared with rule-based CAC, it is more robust under documentation defects; compared with neural pipelines, it imposes lower data and governance burdens but captures fewer rare, compositional patterns. Real value depends on “plumbing”—code normalization, audit, access controls, observability, and ITSM-backed exceptions—so piloting as a sidecar to coder workflows is sensible. The pattern generalizes to other sparse, high-cardinality decisions (e.g., order sets, procedure suggestions) given adequate ontologies and recalibrated weights. Near-term refinements include calibrated uncertainty and brief “why/why-not” rationales, light hybridization with LLMs for clustering/justification (not final decisions), and reliability testing to harden multi-agent coordination.

7.5 Phase III: Human–AI Decision Alignment (HADA)

HADA operationalises human–AI decision alignment by coupling natural-language interaction among stakeholders with governed, tool-mediated algorithmic decisions. The banking `getLoanDecision` workflow serves as the primary evaluation setting, exposing tensions among risk, compliance, ethics, customer value, and operational efficiency. Rather than pursuing full automation, HADA establishes a repeatable socio-technical loop where issues (e.g., postcode/ZIP-code features) are surfaced, routed to accountable roles, and resolved under audit.

Evaluation Against Objectives (Section 1.4.1.4)

HADA was evaluated against the chapter’s objectives by analysing the `getLoanDecision` workflow and associated governance flows.

- O.III.1.1 Natural-language interaction across planning horizons.** Stakeholders (C-suite, risk, data science, audit, customer) engage the decision context through constrained NL dialogues that map to structured actions and evidence, supporting both strategic clarifications and daily exception handling. *Outcome: Achieved (within the evaluated workflow).* *Limitation:* longitudinal validation across full annual cycles remains future work.
- O.III.1.2 Multi-dimensional alignment (targets & values).** Optimisation logic is bound to quantitative OKRs and machine-readable values/policies, with a verifiable chain from principles to actions; postcode/ZIP-code conflicts are detected and escalated to ethics review with traceable rationale. *Outcome: Achieved (scenario coverage).* *Limitation:* broader value-ontology coverage and quantitative conformance metrics across multiple policies are deferred.
- O.III.1.3 Reference architecture for hierarchical, hybrid AI systems.** The stack separates strategy, coordination, and execution; LLM agents and classical models/tools coexist under a common governance surface. Integration is protocol-agnostic (e.g., via MCP/A2A adapters) so model/tool swaps do not alter governance flows. *Outcome: Achieved.* *Limitation:* more heterogeneous backends would further demonstrate portability.
- O.III.1.4 Stakeholder alignment across time-scales.** Repeatable elicitation and reconciliation patterns propagate approved changes through the pipeline. *Objectives & Key Results (OKRs)* are used as the bridge from yearly–quarterly strategy to algorithm-level objectives (constraints, rewards, and guardrails), ensuring strategic intent flows into model behaviour. Escalations and approvals are anchored in organisational roles (RACI) and connected ITSM workflows exposed as agent tools via Tools/MCP. *At present, no autonomous agents perform (i) OKR→single-algorithm linkage analysis or (ii) value-misalignment detection (e.g., ZIP-code usage); these checks are explicitly*

assigned to human owners. Outcome: Partially achieved. Limitation: yearly/quarterly cadences were exercised in design and limited trials; sustained cross-quarter operation in a live organisation is pending replication.

O.III.1.5 Scalability, auditability, and near real-time propagation. Immutable, line-addressable logs capture decisions, policy changes, explanations, and escalations for audit; policy edits propagated within hours in the testbed. *At present, no autonomous agents perform (i) OKR→single-algorithm linkage analysis or (ii) value-misalignment detection (e.g., ZIP-code usage); noticing and initiating action on these remain human responsibilities. Outcome: Partially achieved. Limitations: no load tests across thousands of agents; latency/throughput under production contention remains to be measured.*

O.III.1.6 Framework-agnostic, policy-driven design. Governance is expressed in open, tool-facing contracts so strategic edits flow to heterogeneous runtimes; model-/framework-specific logic is confined to adapters, avoiding lock-in. *Outcome: Achieved (by design and prototype). Limitation: wider coverage of orchestration frameworks would strengthen generality.*

Threats to Validity and Generalisability

Limited real-world scope. Evaluation is centred on a single domain (retail banking), one decision workflow, and a specific regulatory context. Replication in healthcare, energy, and logistics is required to establish external validity, stress-test adaptability, and characterise trade-offs at different risk levels.

Architectural alternatives. HADA occupies one point in a broader design space that includes rule-first governance layers, formal inter-agent alignment protocols, XAI-centric supervisors, and other HITL pipelines. Domain requirements, organisational culture, and technical estate will shape best-fit choices; HADA should be considered contributory rather than definitive.

Scope of Automation and Human-in-the-Loop Boundaries

HADA intentionally preserves human control over key gates: policy approval, model retraining/recalibration, and ethics adjudication. The system’s role is to surface alignment issues, assemble evidence, and route decisions to accountable owners via ITSM workflows connected through Tools/MCP. This design maintains responsibility, auditability, and controllability in ethically sensitive contexts.

Modeling Limitation: One Agent per Decision Algorithm

The current one-to-one mapping between an agent and a decision algorithm simplifies ownership and deployment but fragments multi-product negotiations (e.g., mortgage + insurance + investments). Addressing coupled decisions requires a coordinating layer (or composite agent) that manages interdependencies and shared constraints across products while retaining per-algorithm accountability and traceability.

Discussion

HADA’s contribution is best viewed as operational governance rather than yet another orchestration pattern: it makes strategy (*OKRs*) and values (*policies, ethics*) first-class, tool-facing artefacts and binds them to the lived workflow of a decision like `getLoanDecision`. The evaluation suggests that this binding is feasible in practice—natural-language dialogs can be constrained into evidence-carrying actions; RACI/ITSM anchors keep ownership visible; and line-addressable logs afford audit-grade traceability. At the same time, two tensions are evident. First, the architecture outsources “noticing” to humans (e.g., postcode features), which preserves accountability but limits responsiveness at scale. Second, the current one-agent-per-algorithm mapping simplifies control surfaces yet impedes reasoning over coupled products and shared constraints, where cross-model externalities matter as much as per-model performance.

For wider use, three down-to-earth gaps show up. Measurement: we say we’re aligned, but we still need simple, consistent numbers to prove it over time (e.g., coverage, drift, fairness). Breadth: the prototype works, but we must show it runs on different tools, models, and regulations—under real load. Autonomy limits: light automation (alerts, gathering evidence, pushing approved policies) is safe; fully automatic detection and OKR-to-model linking need stronger safeguards to avoid false alarms, shortcuts, or ethical mistakes.

What to do next is straightforward: add long-run metrics and dashboards, test portability under stress in varied environments, keep humans clearly in the loop for sensitive calls, introduce a combined view for

bundled products, translate policies into concrete checks, and track “alignment” as an operational SLO in the same ITSM queues that already handle risk and incidents.

7.6 Phase III: Evaluation of the NordDRG AI Benchmark

See Table 3 for quick navigation to the DSRP process sequence linked to Phases and Structure of this Thesis.

Evaluation objectives and design

The NordDRG-AI-Benchmark is evaluated against three objectives *O.III.2.1-O.III.2.3* (Section 1.4.2.4):: (i) coverage & fidelity of the artefacts; (ii) extensibility and long-term sustainability; and (iii) baseline differentiation across contemporary LLM endpoints. All artefacts, answer keys and scoring scripts are openly released to ensure full reproducibility.

The benchmark exposes two complementary suites with exact-match scoring: a 13-task *Logic* suite that exercises rule-level reasoning over the definition tables and governance manuals, and a 13-task *Grouper* suite that requires *full specification emulation*. *Logic* answers are order-invariant, de-duplicated code sets; *Grouper* answers must match *both* the final DRG (`drg_nat`) and the triggering rule row (`drg_logic.id`).

Evaluation against Objectives

Consistent with the DSRM, the benchmark is assessed against the objectives via (i) a *descriptive* inspection of the artefacts and (ii) an *analytical* baseline run (Tables 24 and 25).

O.III.2.1 Coverage and fidelity. A.III.2.1 mirrors the production NordDRG workbook (~20 interlinked sheets; `drg_logic`, `dg_feat`, `proc_feat`, CC/MCC exclusions, age/sex splits, national activation). No lossy transformations were applied; all rule-bearing columns are preserved. The Logic tasks (Table 19) exercise table reading and joins; the Grouper tasks (Table 21) require end-to-end execution of the official control flow (including ORD, MDC entry, OR/PROCPRO, CC/MCC, activation flags).

O.III.2.2 Extensibility and sustainability. Schema-stable identifiers (DRG, ICD, NCSP, ORD, ...) are consistent across yearly and national releases, enabling drop-in updates without code changes. In practice, adding a version amounts to placing the official workbook and manuals in a dated repository folder; existing prompts and scoring continue to work. Maintenance follows an open workflow (GitHub issues/PRs): adding a task requires one prompt row (A.III.2.3) and a matching gold-answer row—no source edits.

O.III.2.3 Baseline differentiation.

Figure 88 shows a clear separation on the 13 *Logic* tasks: top-tier models reach ceiling accuracy (*GPT-5 Thinking*, *Opus 4.1*: 13/13; *o3*: 12/13), while mid-tier and trailing endpoints miss joins and governance-aware filters (*GPT-5 Thinking Mini*, *o4-mini*: 8/13; *GPT-5 Fast*: 6/13; others $\leq 5/13$). The *Grouper* suite is substantially harder under strict exact-match of `drg_nat` and `drg_logic.id`: *GPT-5 Thinking* solves 7/13, *o3* 6/13, *o4-mini* 3/13, *GPT-5 Thinking Mini* 1/13, and the remaining endpoints 0/13. Dominant failure modes involve ORD priority, CC/MCC exclusions, and national activation flags.

On the 13 *Logic* tasks, models separate clearly: *GPT-5 Thinking* and *Opus 4.1* achieve 13/13, *o3* 12/13; mid-tier endpoints score 6–8/13; others 5/13 or below (Table 24). Largest gaps occur on cross-sheet chaining (procedure- and diagnosis-property linkage). On strict *Grouper* emulation, *GPT-5 Thinking* reaches 7/13, *o3* 6/13, *o4-mini* 3/13; all others 0–1/13 (Table 25). Many near-misses predict the correct `drg_nat` but fail the exact `drg_logic.id` requirement, underscoring the difficulty of reproducing deterministic control flow and traceability. To the best available knowledge (August 2025), this is the first public report of an LLM partially emulating the complete NordDRG grouping logic with exact matches on both fields.

Rapid evolution of LLMs and remaining gaps

Compared to earlier iterations, top-tier models now *reliably* master rule-level reasoning (13/13 on *Logic*). However, *governed* end-to-end execution remains challenging: strict *Grouper* scoring exposes brittle handling of ORD priority, CC/MCC exclusion logic, and national activation flags. In other words, listing plausible DRGs is easier than reproducing the specification’s deterministic control flow with an audit-ready rule trace.

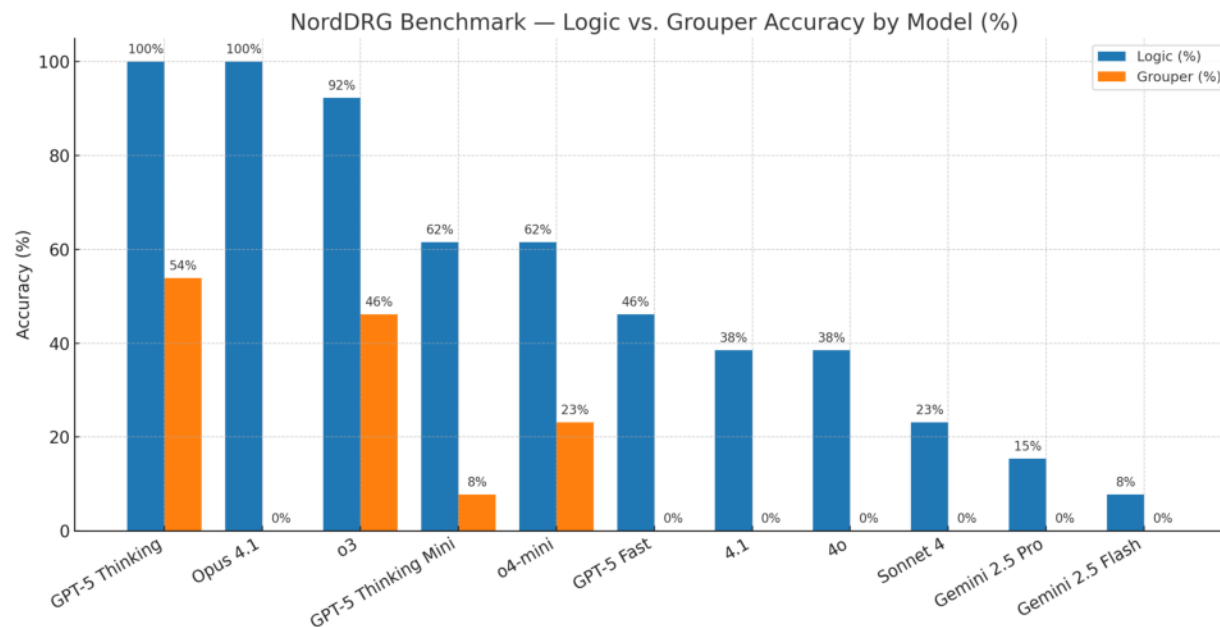


Figure 88: **Combined results.** Percentage accuracy on the *NordDRG-AI-Benchmark Logic* (rule-level) and *Grouper* (full emulation) suites. Logic saturates for top models, while exact-match emulation—requiring the correct `drg_nat` and triggering `drg_logic.id`—remains challenging; only a few endpoints exceed 40% on Grouper.

Minority-language robustness

The benchmark deliberately includes FI and FI-EN excerpts and Logic tasks to test minority-language grounding and bilingual transfer. These prompts separate models: strong systems resolve Finnish clinical terms to the correct DRG sets; weaker ones regress when English descriptors are absent. The FI / FI-EN subsets, therefore, remain essential diagnostics for equitable, multilingual CaseMix tooling.

Methodological lessons

Governance-grade benchmarking benefits from: (i) prompt templates that constrain outputs to normalised sets; (ii) *exact-match* keys (sets for *Logic*; (`drg_nat`, `drg_logic.id`) for *Grouper*); and (iii) version pinning of artefacts. Lightweight reference agents (*LogicAgent*, *GrouperAgent*) further reduce evaluator variance by emitting de-duplicated sets and a concise evidence trace (sheet/row IDs, predicate outcomes) without exposing hidden chain-of-thought.

Operational throughput under consumer subscriptions

Running the full suite on standard, consumer-grade endpoints surfaced throughput differences unrelated to accuracy: some providers exhausted credits mid-run while others completed uninterrupted. For practitioners, the relevant objective function is not price-per-token alone but *price & time-to-result*, including quota predictability and run management overhead.

Implications for healthcare AI

Findings indicate that LLMs can *fully* solve rule-level NordDRG reasoning and *partially* emulate the governed grouper. By coupling a rule-complete release with exact-match tasks and open scoring, the benchmark offers a reproducible yardstick for regulators, payers, and developers. Persistent errors around priority and exclusion handling underline the need for auditable, specification-faithful designs in hospital funding applications.

Discussion

As a governance-grade yardstick, the NordDRG-AI-Benchmark contributes three practical virtues: (i) *rule-complete* artefacts that mirror the production workbook without lossy simplifications; (ii) *exact-match* scoring that forces specification-faithful behaviour (set equality for Logic; joint match on the final DRG and the triggering rule row for Grouper); and (iii) *open, reproducible* releases that permit independent reruns and community extension. These choices surface meaningful separations: top-tier models saturate table-reading and governance-aware joins, yet stumble on deterministic control flow where priority, exclusions, and activation flags interact. Minority-language probes further discriminate true grounding from translation heuristics, and the throughput observations remind practitioners that “accuracy” is only one axis of operational fitness alongside quota stability and time-to-result.

Two caveats temper generalisation. First, the benchmark targets a single, complex DRG family; while identifiers and templates are version-stable, external validity still hinges on replication across additional national variants, yearly updates, and adjacent casemix systems. Second, strict exact-match scoring is deliberately unforgiving: it exposes real gaps in governed execution but can obscure partial progress without complementary *step-wise* conformance metrics. Next steps are therefore straightforward: pair the current keys with a reference interpreter that emits auditable step traces; diversify prompts via templated paraphrases and adversarial counterfactuals; add fairness and minority-language slices as first-class scorecards; and report *cost and latency* alongside accuracy as a benchmarked SLO. Together, these extensions would convert the present release from a strong snapshot into a continuously informative regulator–developer commons.

7.7 Phase III: Evaluation of the CCL

See Table 3 for quick navigation to the DSRP process sequence linked to Phases and Structure of this Thesis.

This section evaluates whether the CCL instantiation on NORDDRG satisfies governance-grade requirements—safety, transparency, and accountability—under regulated constraints. The evaluation proceeds in three parts, aligned with research questions (Section 1.4.3.2) and objectives (Section 1.4.3.4). These experiments assess the prototype artefacts, not NordDRG processes. Acceptance/rejection labels refer to internal review criteria aligned with the CCL gates and should not be interpreted as judgements about official NordDRG change procedures.

- **Decision-tree abstraction (Section 7.7.1).** Test that a canonical tree/DAG view is agent-operable and line-addressable, semantically faithful to the official logic, and human reviewable via decision tree slices (structural fidelity, scale, legibility).
- **Preparing NordDRG for CCL (Section 7.7.2).** Assess conversion of the official release into agent-editable version controlled files (conversion fidelity, diff stability, agent-edit safety, auditability).
- **LLM agents (Section 7.7.3).** Examine the Contributor/Reviewer roles as first-line participants, focusing on reliability under CI gates and keeping humans in charge of final decisions which changes are accepted.

Each subsection states its measures, adequacy criteria, threats to validity, and outcome reporting, drawing on the artefacts and figures introduced in Section 6.3.1 and demonstrated in Section 6.3.2.

7.7.1 Evaluation of the NordDRG Decision-Tree Abstraction

Objective and scope. This study tests whether materialising the NORDDRG specification as a canonical decision-tree/DAG is *fit for CCL*: (i) agent-operable and line-addressable for safe, atomic diffs; (ii) semantically faithful to the official logic; and (iii) practically reviewable via sliceable, explanation-friendly visuals. This links Demonstration Section 6.3.2.2 to research questions (Section 1.4.3.2) under governance constraints.

Design. This study ran structural checks and manual reviews on the parsed structure: (a) schema/typing validation for predicates and labels; (b) scale profiling (features, leaves, root→leaf paths, decision-point counts); and (c) legibility assessments on SVG slices (10/25/100 paths) to gauge human reviewability. Figures and metrics from Section 6.3.2.2 ground these checks.

Measures. (i) *Fidelity*: path-wise equivalence on a certification subset; (ii) *Scale*: counts of features, distinct leaves, root→leaf paths, and decision points (first layer vs. full unrolling); (iii) *Legibility*: expert

ratings of 10/25/100-path SVGs for review tasks; (iv) *Diff locality*: share of edits that remain well-typed and confined to small path segments. Demonstration baselines report 16 features, 2,752 leaves, 8,867 root→leaf paths, 53,848 first-layer decision points, and 40,453,620 decision points when fully unrolled.

Adequacy criteria. The abstraction is *adequate* if it (a) preserves semantics on the certification subset; (b) enforces typed predicates/labels; (c) yields stable node addresses for PR diffs; and (d) supports reviewer-legible slices at the 10–25-path scale used in practice.

Threats to validity. Unrolling DAG structure into path-wise trees can induce redundancy; equivalence checks are limited by available certification cases; legibility ratings may vary by reviewer expertise and display medium.

Outcome reporting. This study reports structural counts with confidence intervals, pass/fail tallies for round-trip and typing checks, and inter-rater agreement on legibility ratings for the 10/25/100-path slices.

7.7.2 Evaluation of Preparing NordDRG for CCL

Objective and scope. This study assesses whether a *minimal* preparation of the official NordDRG release—packaging the native, custom-format definition tables as line-addressable files under version control with grounding documentation—is sufficient to enable PR-centric governance without introducing semantic drift. As justified in the artefact discussion (Section 6.3.1.4), this study intentionally retained the native format as the *authoritative model-of-record*; a wholesale conversion to a standard decision-tree serialization was not adopted because it offered no net governance benefit while risking precedence/ordering regressions, diff instability, and tooling mismatch. A compiled tree/DAG is used only for analysis/visuals (Section 6.3.2.2). This evaluation links the artefact choice (Section 6.3.1.4) with the demonstrations (Sections 6.3.2.3, 6.3.2.4) and research questions (Section 1.4.3.2).

Design. This study (a) imported the official definition tables *as-is* into a line-addressable file layout under Git; (b) performed a light, manual spot-check to confirm no deviations from the upstream release; and (c) constructed PR-style bundles (typed diffs, Impact Brief, CI stubs) to exercise repository plumbing. No schema migration, column reshaping, value rewriting, or conversion to a standard tree format was performed; the compiled tree/DAG view (Section 6.3.2.2) is strictly auxiliary.

Measures. (i) *Pass-through integrity*: review confirming no content deviations from upstream native files; (ii) *Operability under CCL*: ability to open PRs with atomic, line-addressable diffs over the native format; (iii) *Implicit external validation*: in agent demonstrations (Section 6.3.2.4), whether agent-proposed changes coincide with expert changes implemented in the NDMS production system for the same tickets—serving as an indirect check that the model-of-record was prepared correctly for CCL.

Baselines and analysis. This study contrast with the NDMS-only workflow, focusing on whether PR-style artefacts (diffs, briefs, CI evidence) can be assembled *without* altering the native representation (cf. Section 6.3.1.4).

Adequacy criteria. Preparation is *adequate* if: (a) the native files are under version control and line-addressable with stable identifiers; (b) spot-checks detect no deviations from the official release; and (c) in demonstrations, agent-suggested diffs align with externally validated expert changes in the NordDRG forum [328]. Given no material transformations were applied, additional integrity checks beyond (b) are unnecessary.

Threats to validity. The integrity check is intentionally light because no transformation was applied; implicit validation via a limited number of NDMS-matched tickets may not surface rare edge cases. The analysis tree/DAG (Section 6.3.2.2) is non-authoritative and thus excluded from integrity claims.

Outcome reporting. This study reports (i) pass/fail of the light deviation check; (ii) evidence that PRs over the native format are operational; and (iii) concordance between agent-proposed diffs and NDMS expert changes in the demonstration cases—supporting the artefact choice in Section 6.3.1.4 and its practical fitness for CCL.

7.7.3 Evaluation of the NordDRG Contributor and Reviewer Agents

Objectives. This subsection examines whether the *Contributor* and *Reviewer* agents can act as reliable first-line participants in the CCL loop while safeguarding expert control, auditability, and audit trail. The evaluation links research questions (Section 1.4.3.2) with the demonstration in Section 6.3.2.4.

Position in the CCL pipeline. Figure 89 depicts the agents’ role in the governed workflow. The Contributor agent ingests forum tickets, grounds them in the official NordDRG specification, and emits explanation-rich atomic diffs. These diffs form a pull request that is subjected to CI gates (G1–G4) and risk-tiered human approvals before merging, tagging, and audit trail ledgering. The implemented prototype completes Steps 1–5; subsequent CI/CD automation (Steps 6–8) was skipped for this demonstration but would include running test-case data through the official DRG Grouper software, while the Reviewer-agent handoff (Steps 9–10) and human pull-request review (Steps 11–13) follow the same governance pattern. Steps 14–16 lie outside this demonstration’s scope but belong to a full CCL deployment.

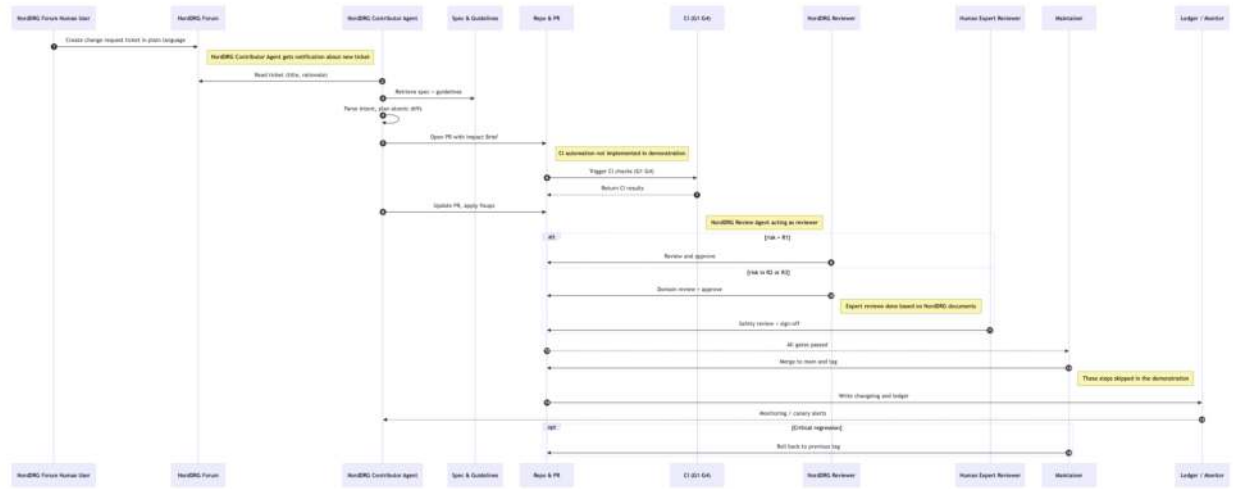


Figure 89: **How the agent plugs into CCL.** The Contributor NordDRG Agent ingests NordDRG Forum tickets (via pasted text), grounds them in the official specification and guidelines, and emits atomic diffs with an explanation-rich specification; it then opens a pull request against the model-of-record, where CI runs the CCL gates (G1–G4) and risk-tiered human approvals are collected before a maintainer performs a merge, tags a release, and writes an immutable audit trail to the ledger with canary/rollback safeguards. This shows the agent acting as a first-line contributor while “human-in-command” governance ensures auditability and accountability end-to-end.

Experimental design. Five real NordDRG Forum tickets were processed under two workflow modes:

- **Single-Agent:** only the Contributor agent operates.
- **Agentic:** the Contributor agent is followed by the Reviewer agent.

Each workflow was executed with three LLM back-ends—GPT-4o, o4-mini-high, and o3—yielding six runs per ticket (two workflows × three LLM back-ends, 30 runs in total). All runs were executed through the CCL Platform (Section 6.3.2.1), using its agentic workflow engine and agent chat interface. For every run the agent(s) parsed the ticket, grounded it in the specification and guidelines, and produced typed atomic diffs plus CI-compatible artefacts.

Results. Table 28 summarises acceptance outcomes. Across all tickets the Agentic workflow achieved markedly higher acceptance, whereas Single-Agent executions were rejected across nearly all configurations; only *Case A* with o3 achieved acceptance. The result underscores the benefit of explicit agent-to-agent review for alignment with NordDRG coding standards.

Table 28: Outcome of each ticket under two workflow modes across three LLM endpoints. A check mark (✓) indicates that the generated pull request passed manual review and CI gates; a cross (✗) indicates rejection.

Case	Process	Workflow	GPT-4o	o4-mini-high	o3
A	CCL	Single-Agent	✗	✗	✓
A	CCL	Agentic	✓	✗	✓
B	CCL	Single-Agent	✗	✗	✗
B	CCL	Agentic	✓	✓	✗
C	CCL	Single-Agent	✗	✗	✗
C	CCL	Agentic	✗	✓	✗
D	CCL	Single-Agent	✗	✗	✗
D	CCL	Agentic	✓	✓	✓
E	CCL	Single-Agent	✗	✗	✗
E	CCL	Agentic	✗	✗	✗

7.7.4 Discussion

This section interprets the evaluation results in light of the research questions and prior work, outlines implications for governed decision-model maintenance, and acknowledges limitations and threats to validity.

What the results mean. The CCL instantiation indicates that agent-agent and human-agent workflows can operate under *human-in-command* control when each change is packaged as typed atomic diffs and routed through risk-tiered CI gates. At the $\approx 10^7$ -decision-point scale characteristic of NORDDRG, reviewer effort appears tractable when proposals are locally scoped and explanation-rich, and the addition of a Reviewer role improves acceptance rates relative to single-agent workflows. These observations support research questions the governance properties (roles, gates, immutable ledger) enable agent participation without eroding accountability, provided agents operate under explicit role restrictions, grounding, and fail-closed policies.

Implications for practice. For maintainers of large, regulated rulesets, CCL suggests a practical overlay on existing tooling: (i) keep the native, auditable specification as the *authoritative model-of-record*; (ii) require change requests to arrive as small, typed diffs with a compact Impact Brief; (iii) enforce gateable evidence (schema checks, locality/blast-radius bounds, counterfactuals, adversarial probes) before human review; and (iv) reserve all merge authority for certified reviewers. In this framing, LLM agents function as *first-line drafters and pre-reviewers*, not approvers, reducing reviewer burden without shifting accountability.

Transferability beyond NordDRG. The template is expected to transfer to other governed decision systems that meet three conditions: (i) a line-addressable representation with a stable schema; (ii) the ability to express edits as small, atomic operations; and (iii) an organisational mandate for human-in-command sign-off. Likely candidates include reimbursement schedules, credit-policy scorecards, tariff/eligibility tables, and safety checklists. Where primary models are statistical, CCL applies at the *decision surface* (rules, thresholds, guardrails) and to documentation artefacts (Model/Datasheets), while heavier model-internals may require additional governance mechanisms.

Design trade-offs. Retaining the ordered rule list as the authoritative artefact avoided precedence regressions and unstable diffs, while a compiled tree/DAG served analysis and visualisation. Risk tiers simplify policy but can be tuned too coarsely; over-broad tiers increase reviewer load, while over-narrow tiers risk under-testing. Similarly, Reviewer agents help surface issues early, yet their prompts and guardrails must be versioned and audited like code.

Limitations and future work. Our demonstration targets a single domain and evaluates agents offline on a small set of change requests; end-to-end deployment within national maintenance systems and longitudinal monitoring were out of scope. The evaluation focuses on an explainable, rule-set-based decision model; applying CCL to opaque statistical models such as neural networks would require separate studies and potentially additional governance mechanisms. Many aspects of NORDDRG development also require statistical cost analysis on real patient data; while such analyses could in principle be performed by CCL contributor agents, they are outside the scope of this study. Future work includes live integration with

production pipelines, development of additional contributor agents trained on different data sets, broader user studies with clinicians and coders, stronger equivalence and red-team testing across risk tiers, deeper role specialisation (multi-agent patterns for subdomains), and replication in other governed rule systems (e.g., finance and public policy). A fuller assessment requires collaboration with maintainers and access to operational workflows. NordDRG stakeholders are invited to co-design stronger evaluations and will revise this work to correct any mischaracterizations.

Regulatory and ethical considerations. CCL’s human-in-command constraint, immutable audit trail, and risk-based gates align with common governance expectations for high-risk AI systems. Nonetheless, role boundaries must be explicit (agents cannot merge), provenance for all sources must be maintained, and post-release monitoring and rollback should be configured for any deployment that affects patients, finances, or public services.

8 CONCLUSIONS

This chapter discusses conclusions from all the material presented in this Thesis.

DSRP flow, see Table 3 (DSRP process sequence linked to Phases and Structure of this Thesis).

8.1 Phase I: The History of Semantic Web and Software Agents

Main RQ I.1 (Table 4):

What are the historical origins of the Web and of software agents?

The Historical Convergence of the Web and Software Agents

This thesis traces the parallel yet interconnected evolution of the Web and software agents, revealing how these technologies emerged from distinct intellectual traditions before converging in contemporary AI systems.

The Web’s Evolution The Web’s conceptual foundations extend back to Vannevar Bush’s 1945 Memex vision of associative information trails, through Ted Nelson’s Xanadu hypertext project (1960s) and Douglas Engelbart’s NLS system demonstrating collaborative knowledge work. Tim Berners-Lee’s 1989 synthesis at CERN combined hypertext with TCP/IP networking through the trinity of URLs, HTML, and HTTP—creating not merely a document system but the foundation for humanity’s largest collaborative information space. The Web became the Internet’s “killer application,” assembling an unprecedented distributed document corpus and validating principles of decentralized, hyperlinked knowledge representation.

Software Agents’ Evolution Software agents emerged from a distinct research tradition rooted in classical AI, particularly work on autonomous systems, distributed artificial intelligence, and adaptive human-computer interaction. From early reactive agents and Brooks’s subsumption architecture through the BDI (Belief-Desire-Intention) cognitive models of the 1990s, agent research pursued computational entities capable of autonomous action, experiential learning, and dynamic environmental adaptation. The agent community’s vision of intelligent, interconnected systems—articulated in foundational works like Wooldridge and Jennings (1995) [149]—aimed to instantiate systems that could perceive their environment, pursue delegated goals, and coordinate with other agents in distributed settings.

The Convergence Point Critically, this research revealed that the Semantic Web vision articulated by Berners-Lee *et al.* (2001) [41] essentially reimagined earlier agent research goals within Web architecture. The promised “web of data” where machines could understand and process information autonomously directly echoed the multi-agent systems community’s aspirations from the previous decade, and anticipates themes from even earlier AI research on knowledge representation and automated reasoning. The Semantic Web did not introduce a completely new vision, but rather re-articulated long-standing agent and AI aspirations in Web-native form, representing a convergence point where the Web’s success as a universal information substrate met the agent community’s desire for machine-understandable content and automated action.

While the original Semantic Web’s formal ontology approach achieved limited adoption, its core vision - machine-processable knowledge enabling autonomous action - has been realized through different means in modern LLMs and transformer-based agents. Today’s intelligent software agents, exemplified by LLM-based systems, represent the fulfillment of these parallel traditions: the Web’s global information infrastructure meeting the agent community’s autonomy and intelligence goals, realized through *Deep Learned Algorithms* and the *Function Graph* formalism rather than symbolic reasoning alone.

8.2 Phase I: Software Agents and Semantic Web Integration

Main RQ I.2 (Table 4):

To what extent can Semantic Web formalisms and software-agent protocols be integrated to deliver runtime interoperability and adaptation on the Web, and what irreducible gaps prevent practical convergence?

Phase I approached this question through three increasingly sophisticated implementations (2001–2006), each probing a different segment of the Semantic Web–agent integration space.

Runtime protocol adaptation (SRQ.I.1.1–SRQ.I.1.4, Section 1.2.1.3). The first prototype investigated whether agents can ingest RDF-serialized FIPA interaction protocols at run time and execute them reliably in a construction expediting scenario. The work confirmed that RDF enabled surface integration—protocols could be parsed into executable state machines and the contractor agent successfully interacted with previously unknown partners. However, all “understanding” resided in pre-programmed automata rather than semantic reasoning over RDF/OWL, highlighting the gap between syntactic interoperability and the Semantic Web vision. Moreover, adaptability remained confined to a single Java-based platform and controlled ontology, with no straightforward path to Web-scale, open-world interoperability. The system could extract parameters and reports but did not achieve deeper semantic integration or composition across heterogeneous services.

B2B negotiation systems (SRQ.I.2.1–SRQ.I.2.3, Section 1.2.2.2). The second prototype explored ontology-driven B2B purchasing, where suppliers and buyers exchange machine-interpretable requests and offers. Experiments demonstrated that product and process knowledge can be formalized in domain ontologies, enabling agents to interpret and compare bids automatically, though the ontologies remained deliberately simplified. FIPA-style agent communication proved adequate for autonomous matching and basic negotiation under tightly aligned vocabularies, but the cost and fragility of ontology engineering limited portability across heterogeneous enterprise systems.

Context-aware mobile services (DYNAMOS, SRQ.I.3.1–SRQ.I.3.4, Section 1.2.3.2). The third prototype combined location-, time-, and activity-aware personalization with community annotations to recommend mobile services to recreational sailors. End-to-end performance measurements revealed that remote method invocation was the primary bottleneck: while local operations sustained thousands of requests per second, cross-network calls reduced throughput by 100 - 1000×, implying that filtering should be pushed server-side. The field trial in harsh sailing conditions confirmed both usefulness and fragility: users appreciated proactive information, but pre-iPhone devices suffered from JVM memory leaks, intermittent Bluetooth connectivity, and crashes under multi-sensor loads, forcing the interface to be reduced to single-action interactions.

8.2.1 Phase I: Runtime Protocol Adaptation

The construction expediting prototype demonstrated agents dynamically interpreting RDF-serialized FIPA protocols for supply-chain coordination. The system achieved machine-processable protocol intake and functioning FIPA-ACL conversations on the JADE platform, establishing that conversation structure, when explicitly modeled in RDF/OWL and consumed at run time, can lift integration beyond brittle endpoint-centric glue code.

However, the supporting research questions (SRQ.I.1.1–SRQ.I.1.4, Section 1.2.1.3) exposed fundamental limitations:

SRQ.I.1.1 Surface integration versus semantic understanding. While RDF successfully serialized FIPA protocols for runtime parsing, the system’s reliance on preprogrammed finite-state machines rather than semantic reasoning revealed a critical gap: the contractor agent could parse ontology-conformant protocol descriptions but conflated “following a script” with genuine adaptation. True semantic understanding would require goal-directed transition functions or policies over goals, context, and constraints—capabilities absent from the implementation. This demonstrates that syntactic data exchange, though machine-processable, falls short of the Semantic Web vision of autonomous machine understanding.

SRQ.I.1.2 Platform-specific versus Web-scale interoperability. The prototype succeeded within a controlled Java/JADE environment but demonstrated no path to true Web-based interoperability. The platform dependency was absolute: there was no demonstration of reliability under contention, recovery from failures, or cross-platform communication beyond JADE’s internal messaging. The gap between agent-platform protocols and Web-native standards (HTTP, REST) remained unbridged, with agent technologies showing no convergence toward mainstream Web infrastructure. This platform lock-in contradicted the Web’s principle of universal, standards-based interoperability.

SRQ.I.1.3 Constrained adaptation versus open-world flexibility. The expeditor adapted only within strictly conforming protocols and a fixed ontology, with flexibility confined to one role and bound to pre-enumerated FSM states. When protocols deviated from the ontology, exhibited version skew, or employed alternative modeling idioms, behavior was undefined—no graceful degradation, schema mediation, or negotiation capabilities were demonstrated. This exposed the practical convergence barrier: multi-party Web ecosystems concentrate variance precisely at ontology

boundaries (partial adoption, divergent exception patterns), yet the system created a new lock-in at the ontology layer itself. The controlled demonstration remained far from the open-world heterogeneity characteristic of Web-scale services.

SRQ.I.1.4 Data extraction versus semantic integration. Basic report parsing and cross-validation mechanisms functioned within the scenario, establishing an auditable mechanism for lightweight status comparison. However, the absence of inference capabilities, context reasoning, sophisticated reconciliation policies for conflicting claims, or systematic data-quality assessment meant the system remained at the level of simple data extraction. Autonomous semantic service composition—requiring deep integration of reasoning, discovery, negotiation, and exception handling—was not evidenced. The comparison mechanism existed but lacked evaluation depth, leaving effectiveness suggestive rather than quantified.

These findings demonstrate that while ontology-grounded protocol descriptions enable basic interoperability, the integration remained better characterized as *ontology-constrained interoperability with scoped flexibility* rather than the open-ended runtime adaptation envisioned by the Semantic Web. The system required substantial infrastructure (9,000+ lines of code) for basic scenarios, contradicting the Web’s design principle of simplicity and suggesting that the “layer cake” Semantic Web architecture was incompatible with the principles that made the original Web successful.

8.2.2 Phase I: B2B Negotiation Systems.

Multi-agent purchasing automation employed shared domain ontologies for supplier discovery and quote evaluation, demonstrating that ontologies could make commercial artefacts machine-processable and enable basic acceptance reasoning. Buyer and supplier agents successfully conversed on JADE with ontology-mediated interactions.

The supporting research questions (SRQ.I.2.1–SRQ.I.2.3, Section 1.2.2.2) revealed critical gaps between proof-of-concept and production readiness:

SRQ.I.2.1 Formalizing product and process knowledge. Domain ontologies successfully formalized simplified purchasing scenarios, enabling machine-interpretable requests and offers for basic supplier matching. However, the commercial surface remained deliberately narrow: payment and delivery semantics were simplified, with no modeling of penalties, rebates, Incoterms nuances, volume breaks, or exception clauses. The ontologies were adequate for illustration but far from the operational breadth required for real B2B procurement. This exposed *the ontology engineering bottleneck*: creating ontologies comprehensive enough for production use proved prohibitively difficult, and the challenge of combining or aligning heterogeneous ontologies—where two programs must reconcile different ontologies describing the same domain—resisted automation entirely. The demonstration thus revealed that ontology engineering scaled poorly beyond controlled scenarios.

SRQ.I.2.2 Agent communication and coordination mechanisms. FIPA-ACL conversations on JADE provided functioning message exchange and basic registry-mediated discovery. However, the coordination remained scripted rather than adaptive: typical B2B constructs like multi-round negotiation, counter-offers, expiry handling, delivery-slot conflicts, and exception workflows were absent. The “automation” amounted to single-shot acceptance decisions rather than end-to-end task delegation. Without escalation triggers, approval gates, or audit trails, human oversight—essential for production systems—was not demonstrated. The agent-based realization worked within platform boundaries but provided no evidence of reliability under contention or cross-platform interoperability beyond JADE.

SRQ.I.2.3 Ontology-based reasoning for interoperability. Shared ontological vocabularies enabled basic semantic matching between buyer requirements and supplier capabilities within the controlled demonstration. However, true interoperability across heterogeneous enterprise systems remained undemonstrated: with XML simulators standing in for ERPs and PIMs, integration challenges around data freshness, latency, and failure modes were not addressed. The narrow ontology subset and absence of reasoning over complex constraints (volume pricing, conditional terms, relationship history) meant the system stayed at the level of *semantic enablement* rather than operational automation. The structural tension between semantic rigor and operational completeness became evident: ontology-based reasoning contributed to interoperability within

controlled boundaries but could not bridge the heterogeneity of real enterprise ecosystems without substantial additional engineering.

The B2B work demonstrated that while ontology-driven agents could reason over well-scoped purchasing artefacts, advancing toward routine task automation would require: (i) broadening commercial coverage in ontologies to include real-world terms and exceptions, (ii) embedding oversight primitives as first-class protocol elements, and (iii) hardening discovery and integration with live ERP/PIM links and reliability patterns. The demonstration remained closer to decision support than production-grade delegation, reinforcing the finding that ontology complexity was a fundamental scaling barrier.

8.2.3 Phase I: Matching Service Descriptions in Context-Aware Environment

Main RQ I.3 (Table 4):

Which end-to-end architecture best supports an intelligent model that fuses context-aware personalisation with community-shared annotations of services, so it can operate on resource-limited smartphones and still provide proactive recommendations?

Synthesis. The Phase I experiments indicate that, under the technological conditions studied, the most suitable end-to-end architecture is a *server-centric design with a thin, activity-aware client*. In this arrangement, the mobile device focuses on capturing context and offering very simple interactions, while the back-end performs the heavy filtering and matching over both services and community annotations. The supporting research questions (SRQ.I.3.1–SRQ.I.3.4, Section 1.2.3.2) clarify how performance constraints, field usability, context modelling, and collaborative content each shape this architecture.

SRQ.I.3.1 End-to-end bottlenecks. Performance measurements revealed that remote method invocation (RMI) between components dominated latency: local operations achieved thousands of requests per second, whereas cross-network calls reduced throughput by two to three orders of magnitude (10^2 – 10^3). This finding directly mandated a server-centric architecture where matching and ranking occurred server-side, with only compact, pre-filtered result sets transmitted to clients. Architectures attempting full matching on mobile devices were not viable on 2005-era networks and hardware, establishing that integration plumbing—not algorithmic sophistication—was the primary performance determinant.

SRQ.I.3.2 Field usability and resilience. The sailing trial under harsh conditions (15 m/s winds, high workload, divided attention) demonstrated that architectural sophistication was worthless without extreme UI simplicity. Users valued proactive delivery of competition standings and safety information but had minimal attention for complex menus or multi-step workflows, requiring single-action interactions with clear defaults. Pre-iPhone mobile platform fragility further constrained the design: Nokia 66x0 devices suffered from Java Virtual Machine memory leaks, unstable Bluetooth connectivity, crashes under combined GPS/GPRS/camera use, and battery life under half a day. These findings mandated defensive programming, graceful degradation, aggressive sensor scheduling, and minimal on-device complexity—a thin-client architecture focused on context capture rather than local processing.

SRQ.I.3.3 Lightweight context ontology. A simple, activity-centric context model linking user activities to interests and locations proved sufficient for meaningful personalisation without exceeding device capabilities. This lightweight ontology enabled effective filtering while keeping computation and data transfer modest. However, the simplicity constrained fine-grained activity transitions (e.g., racing to docking required coarse state changes), and cross-activity granularity remained limited. The finding confirmed that for resource-constrained mobile scenarios, architectural trade-offs favoured minimal context models with semantic richness concentrated server-side, rather than expressive but computationally heavy on-device reasoning.

SRQ.I.3.4 Collaborative annotations as first-class content. Treating user-generated notes and ratings as matchable content alongside commercial services successfully broadened the recommendation space and enabled peer-to-peer information sharing via event-based distribution. However, performance degraded when result sets grew large without appropriate indexing, and the lack of moderation and provenance controls left noise and redundancy unmanaged. This demonstrated that community annotations must be integrated into the same server-side indexing and optimisation pipeline as provider-managed services, with proper curation mechanisms, rather than

treated as an afterthought. Scalability and signal quality at larger result sets required hardening beyond the proof-of-concept implementation.

Taken together, SRQ.I.3.1–SRQ.I.3.4 indicate that an effective architecture for RQ I.3 combines (i) server-side filtering and matching over both services and community annotations, (ii) a thin mobile client focused on context capture and low-cognitive-load interactions, and (iii) a lightweight context ontology that respects device and network constraints while still enabling proactive, context-aware recommendations. However, success came at the cost of substantial complexity (context modelling, distributed coordination, platform-specific workarounds), revealing that pre-iPhone mobile hardware imposed fundamental constraints independent of algorithmic or semantic sophistication.

8.2.4 Phase I: Irreducible Integration Gaps

Across the three Phase I implementations, five interconnected barriers prevented practical Semantic Web–agent convergence:

Complexity explosion versus Web simplicity. Agent systems required extensive infrastructure—9,000+ lines of code for basic scenarios, sophisticated middleware stacks, platform-specific integration layers—contradicting the Web’s architectural minimalism that enabled its success. The “layer cake” Semantic Web architecture, with its vertical stack of standards (RDF, RDFS, OWL, SPARQL, rules, proof, trust), proved fundamentally incompatible with the design principles of simplicity, loose coupling, and graceful degradation that made the original Web universally adoptable. The complexity burden fell on implementers rather than being absorbed by standards and infrastructure.

The ontology engineering bottleneck. Creating shared ontologies for real-world domains remained prohibitively difficult. Simple, demonstration-scale ontologies worked within controlled boundaries but lacked the coverage, precision, and exception handling required for production systems. Comprehensive ontologies demanded domain expertise, formal modeling skills, and ongoing maintenance—effort that scaled poorly beyond initial demonstrations. More critically, the problem of ontology integration proved intractable: two programs could not reliably combine or align different ontologies describing the same domain without extensive manual intervention. This challenge resisted automation and created fragmentation rather than interoperability, as each community or organization developed incompatible semantic models. The vision of universal semantic agreement clashed with the reality of distributed, heterogeneous knowledge creation.

Technology divergence. Throughout Phase I (2001–2006), industry consistently favored lightweight, pragmatic solutions over Semantic Web standards. While we pursued RDF, OWL, and FIPA protocols, mainstream Web development converged on HTTP, REST, JSON, and simple XML—technologies that traded semantic expressiveness for operational simplicity and universal tool support. Agent technologies (FIPA-ACL, JADE) showed no convergence with emerging Web standards (WSDL, SOAP, and later REST), remaining confined to specialized research platforms. The gap widened rather than closed: by 2006, Web 2.0’s success with simple, data-centric APIs demonstrated that loose coupling and pragmatic conventions outperformed formal semantics for achieving interoperability at scale. The Semantic Web remained aspirational while simpler approaches achieved market dominance.

Absence of genuine learning. All Phase I implementations relied on pattern matching, rule execution, and lookup within rigid frameworks. Adaptation occurred only within pre-enumerated options—the contractor agent selected from predefined protocol states, the B2B agents matched against fixed ontological categories, and DYNAMOS filtered using static preference rules. None exhibited learning from experience, adaptation to novel situations, or improvement from feedback. The systems could not discover new patterns, refine their models, or handle truly unanticipated scenarios. This fundamental limitation—the inability to learn—pointed toward a need for data-driven approaches that could extract patterns from examples rather than requiring complete a priori specification. The symbolic, knowledge-engineering paradigm had reached its practical limits.

Mobile platform constraints. Pre-iPhone mobile hardware imposed severe operational barriers that no amount of semantic sophistication could overcome. DYNAMOS faced Java Virtual Machine memory leaks, half-day battery life under normal operation, unstable Bluetooth and GPS connectivity, and application crashes when accessing multiple device features simultaneously (GPS + GPRS + camera). The sailing trial’s demanding conditions revealed that even well-architected systems became unusable when cognitive

load, interaction complexity, or resource consumption exceeded mobile thresholds. While these constraints were temporary—the 2007 iPhone revolution would eventually provide capable mobile platforms—they demonstrated that semantic technologies alone could not deliver practical mobile services without appropriate hardware and software infrastructure.

8.2.5 Phase I Transition: Toward Data-Driven Approaches

Phase I revealed two qualitatively different categories of limitations that demanded different responses.

Transient Hardware Constraints

The mobile platform limitations—device instability, battery life, network latency, sensor unreliability—were temporary problems that would be addressed by technological evolution. These contingent limitations, rooted in the state of mobile and network infrastructure in the early 2000s, made it difficult to deploy sophisticated context-aware services even when their usefulness was evident. The emergence of the iPhone in 2007 and the subsequent smartphone revolution validated this expectation, eventually providing the hardware capabilities that DYNAMOS anticipated but could not rely upon. These findings suggested *patience*: waiting for inevitable mobile technology improvements while developing architectural patterns (server-centric design, thin clients, efficient synchronization) that would prove valuable once capable platforms arrived.

Structural Limitations of Symbolic AI

The ontology bottleneck, absence of learning, architectural complexity, and technology divergence represented fundamental limitations that no amount of hardware improvement or tooling refinement could address. These structural limitations were intrinsic to the Semantic Web and classical agent paradigms as instantiated in the prototypes.

Knowledge engineering does not scale. Manual ontology creation and rule specification required exponentially growing effort as domain coverage expanded. Automated ontology learning, matching, and integration remained unsolved research problems. **Brittleness under deviation.** Systems failed when encountering inputs outside predefined ontological boundaries, with no graceful degradation or ability to reason about novel situations. **No learning from experience.** Pattern matching against fixed knowledge bases could not improve through feedback, discover latent regularities, or adapt to distributional shift. **Incompatible with Web culture.** The Semantic Web’s emphasis on formal precision, global standards, and complex tooling clashed with the Web’s embrace of pragmatic conventions, loose coupling, and evolutionary design.

The Paradigm Shift

These structural limitations justified a fundamental pivot toward ML and information retrieval methods that offered different trade-offs.

Learn patterns from data rather than requiring explicit semantic encoding, enabling systems to discover regularities without complete a priori specification. **Scale gracefully** without exponential growth in manual knowledge engineering effort, as model capacity grows with data rather than human annotation labor. **Handle ambiguity and incompleteness** naturally through probabilistic reasoning and statistical inference rather than brittle logical rules. **Operate without global agreement** on semantic standards, as statistical models can bridge heterogeneous representations through learned mappings. **Eventually leverage improved hardware** (mobile platforms, cloud infrastructure, accelerators) when available, while remaining viable on existing infrastructure through algorithmic efficiency.

The transition from Phase I to Phase II thus represented not an incremental refinement but a paradigm shift: from knowledge engineering to ML, from symbolic reasoning to statistical inference, from ontological agreement to data-driven adaptation. The subsequent success of ML-based approaches in Phase II—achieving practical clinical decision support with sparse, heterogeneous healthcare data—and the contemporary triumph of LLMs in Phase III would retrospectively validate this pivot.

Enduring Lessons

However, the lessons from Phase I were not entirely negative. Phase I provided both a proof of concept for semantic and agent-based integration *within* constrained settings and a set of arguments for why data-

driven approaches would be necessary to reach robust, Web-scale interoperability and adaptation. The architectural insights—server-centric design, thin clients, careful attention to integration plumbing, lightweight semantic models, event-driven updates—would prove durable across all three phases. The failed attempt at Semantic Web–agent convergence clarified what AI systems required to achieve practical utility: not universal semantic agreement, but robust learning from diverse data; not comprehensive formal specification, but graceful handling of incompleteness; not complex standardization, but pragmatic integration with existing infrastructure. Phase I’s careful documentation of failure provided the foundation for Phase II’s data-centric success and Phase III’s governable, auditable LLM agents.

8.3 Phase II: Assisting Clinical Decision Support with ML and Software Agents

Answers to Sub-Research Questions

Main RQ II.1 (Table 4):

How can software agents perform ad-hoc, flexible analysis over large healthcare item sets on commodity hardware under real-world data-quality defects while meeting explicit bounds on latency, memory, and accuracy to support clinical decision-making and hospital funding decisions (DRG coding)?

To address this main research question, Phase II investigated four focused sub-research questions through the design, implementation, and evaluation of a multi-agent system augmented with ML-based information retrieval techniques for clinical decision support.

SRQ.II.1.1 Robust ML architectures. What ML/IR model characteristics are essential for agent-based systems operating on large, imperfect healthcare datasets with limited hardware resources?

Answer: Vector-space information retrieval using TF–IDF weighting and cosine similarity proved essential for handling extreme sparsity and high dimensionality in clinical coding datasets. The mathematical analysis revealed 2.8×10^{230} possible clinical code combinations, justifying the selection of IR techniques over dense neural network approaches. Key characteristics included: (i) bounded similarity scores yielding interpretable rankings for coder workflows, (ii) hierarchical code relationships augmenting the base IR model to capture semantic similarities across clinical classification systems (ICD-10, NCSP), and (iii) demand-driven agent coordination (Crawler/Integrator/Commander/Aggregator) that decomposed retrieval and scoring tasks while localizing failures and managing computational costs. The architecture favored explainable, controllable retrieval over opaque end-to-end models, achieving governance and latency control at the cost of some semantic expressivity.

SRQ.II.1.2 Recommendation accuracy thresholds. What recommendation accuracy levels are achievable with real healthcare data using vector space models in multi-agent architectures?

Answer: The system achieved 66% first-proposal accuracy and 90% accuracy within eight recommendations when evaluated on real-world Finnish hospital datasets containing hundreds of thousands of patient episodes. These results exceeded the stated thresholds of >50% first-proposal and >90% top-10 accuracy, demonstrating clinically viable performance for coder-assist triage. The SHOULD-occur fetching strategy consistently outperformed MUST-occur across all accuracy metrics. However, accuracy was computed against historical documentation assumed as ground truth despite reported coding error rates of 20–40%, and metrics emphasized common cases potentially under-capturing rare-but-critical conditions. The results indicate practical viability for supporting clinical decision-making and DRG coding workflows on commodity infrastructure.

SRQ.II.1.3 Robustness to data-quality defects. How do realistic documentation defects affect recommendation accuracy, and which retrieval constraints mitigate degradation?

Answer: Comparative tests on higher- versus lower-quality datasets and deliberate perturbations (removed/incorrect/novel codes) revealed that the MUST-occur fetching strategy degraded rapidly under data defects, while the SHOULD-occur strategy maintained acceptable performance with graceful degradation aligned with real-world data quality issues. The flexible SHOULD-occur approach demonstrated superior robustness by accommodating incomplete or inconsistent documentation through redundancy and approximate matching. This finding validates the architectural choice of flexible matching over strict constraint satisfaction for

production deployments across heterogeneous hospital data estates. Limitations include that experiments predominantly removed single facts, whereas real episodes may exhibit multiple concurrent defects and temporal uncertainty; robustness beyond diagnoses/procedures (e.g., noisy laboratory results) remains untested.

SRQ.II.1.4 Real-time scalability on commodity hardware. What maximum healthcare data volumes can MAS architecture handle while maintaining sub-second query response times on standard hardware?

Answer: The system maintained sub-250 ms median query times for datasets up to 10^8 patient episodes with near-linear scaling characteristics when operating on standard desktop hardware. The demand-driven multi-agent coordination reduced polling overhead and enabled dynamic task distribution while preserving real-time responsiveness. These results demonstrate that agent-based architectures with adapted IR techniques can meet explicit latency bounds critical for interactive clinical workflows without requiring specialized infrastructure—a key requirement for healthcare organizations with limited IT budgets. However, large-scale tests leveraged synthetic extrapolations, and failure modes (agent crashes, network partitions), security hardening, and contention under production loads were not fully characterized, indicating that operational resilience testing remains necessary for productionization.

Synthesis: Addressing the Main Research Question

Collectively, the answers to the four sub-research questions demonstrate that combining agent-based architectures with domain-adapted information retrieval techniques can address the main research question: software agents *can* perform ad-hoc, flexible analysis over large healthcare item sets on commodity hardware under real-world data-quality defects while meeting explicit performance bounds. Specifically: (i) domain-aware IR with hierarchical relationships provides an architecturally appropriate foundation for sparse, fragmented clinical data (SRQ.II.1.1), (ii) the system delivers clinically viable top- k recommendations exceeding stated accuracy thresholds (SRQ.II.1.2), (iii) flexible fetching strategies enable graceful degradation under real-world documentation defects (SRQ.II.1.3), and (iv) demand-driven agent coordination sustains sub-second responsiveness at 10^8 -scale on standard hardware (SRQ.II.1.4). These findings close a practical gap between theoretical ML capabilities and the operational constraints of real healthcare information systems.

Future Work

As discussed in Section 1.3.2, this research was conducted some time ago. Future research should address current limitations through: (i) temporal modeling of patient episodes to capture disease progression patterns, (ii) uncertainty quantification providing confidence intervals for recommendations to support risk-aware clinical use, (iii) federated learning approaches enabling cross-institutional model improvement while preserving patient privacy, (iv) expansion beyond diagnosis and procedure codes to incorporate laboratory results, medications, and vital signs through enhanced vector space formulations, and (v) clinical validation through controlled trials in operational hospital environments to assess real-world impact on decision-making quality and efficiency. Near-term refinements include calibrated uncertainty estimates, brief explanatory rationales (“why/why-no”), light hybridization with LLMs for clustering and justification (not final decisions), and reliability testing to harden multi-agent coordination for production deployment.

8.4 Phase III: Human-AI Agent Decision Alignment (HADA) Architecture

Main RQ III.1 (Table 4):

How could we leverage LLM agents to facilitate human-AI decision alignment in large corporate settings?

Large enterprises can achieve human-AI decision alignment by adopting **HADA**—a governance layer that connects *strategy, values, responsibility, and workflow* directly to agent behaviour. Concretely, HADA (i) binds every agent action to a human owner and escalation path (RACI), (ii) translates strategy and KPIs (OKRs) into machine-interpretable policies and reward signals, (iii) encodes organisational values as enforceable, testable constraints, and (iv) preserves auditability via version-controlled artefacts and agent-level logs; all while remaining protocol-agnostic for tool integration (MCP/A2A).

Results

HADA fulfilled all six design objectives (Section 1.4.1.4). The reference architecture:

- enables natural-language interaction among humans, LLM agents, and connected tools across roles and systems;
- provides modular, protocol-agnostic agent–tool integration (MCP/A2A) with transparent audit trails;
- maintains value alignment through explicit policies, guardrails, and runtime conformance checks; and
- supports role-grounded accountability through embedded RACI mappings and escalation paths.

A proof-of-concept in retail banking demonstrated explainable, auditable decision-making through conversational interfaces. Stakeholders—customers, managers, auditors, and ethics officers—interacted via the *HADA Controller*, streamlining governance and aligning automated outcomes with human judgment. By making model reasoning explicit and traceable, the prototype reduced “black-box” risk and addressed practical facets of the broader alignment problem.

Observed benefits included improved decision consistency, interpretability, and stakeholder trust, alongside increased organisational adaptability. Future evaluations will extend to additional domains and decision types, and explore scale and behavioural diversity under production-like load. Overall, HADA offers a practical foundation for AI-augmented governance and ethically grounded enterprise automation.

Summary of the contribution

HADA embeds governance and accountability into agent workflows by tying strategy (OKRs), values, responsibility (RACI), and workflow (ITSM) to runtime behaviour.

Supporting research questions (Section 1.4.1.2).

What LLM agent protocol-level gaps hinder human–AI decision alignment in enterprise settings?

Existing protocols such as MCP and A2A provide interoperability—the ability to share data and tasks—but not alignment: they lack mechanisms for target/KPI traceability, value conformance, and stakeholder–intent mapping. Four additional gaps are critical in enterprise use:

- **Strategic alignment gap.** Current agent protocols do not natively connect agent behavior to company strategy. In practice, objectives and key results (OKRs) must be translated into verifiable constraints, rewards, and guardrails for agents; without this OKR-to-policy bridge, agents can be effective yet strategically misaligned.
- **Value alignment gap.** Protocols lack first-class representations of organizational values (ethics codes, fairness constraints, risk appetites) and automated *conformance checks*. There is no standard way to (i) bind value catalogs to specific tools/models, (ii) trace OKR→algorithm connections, or (iii) continuously test for proxy harms (e.g., ZIP/postcode effects) and value drift. Today these detections are left to human oversight rather than being machine-checkable at runtime.
- **Accountability gap.** AI systems cannot bear responsibility; decisions must be traceably assigned to humans. Protocols lack a standard way to bind agent actions to organizational roles, reporting lines, and escalation paths. A RACI mapping (Responsible, Accountable, Consulted, Informed) tied to the firm’s org chart is required to preserve clear lines of accountability.
- **Workflow gap.** Protocols do not specify *where and how* humans are connected into the operational flow. They omit runtime control points (approvals, four-eyes checks), routing to role-based queues, change/incident lifecycles, segregation-of-duties constraints, and evidence capture. Without explicit hooks into enterprise workflows (e.g., ITSM ticketing), alignment and accountability cannot be enforced in production.

What integration issues does large-scale LLM-agent integration reveal for human-AI decision alignment?

The HADA architecture addresses the gaps identified above by layering governance tool constructs atop existing agent protocols. In large-scale LLM-agent decision automation, not all organizations have the necessary systems in place; the following integration issues commonly arise:

- **Strategic alignment gap (OKR integration).** OKRs may live in spreadsheets or slide decks rather than machine-readable systems. Integration requires tooling to extract, normalize, and version OKRs, and to expose them as constraints/rewards/guardrails that agents can consume.
- **Value alignment gap (policy codification).** Ethics codes, fairness principles, and risk appetites are often in documents or training videos with no system-of-record. Integration requires parsing and codifying value catalogs, binding them to specific tools/models, and enabling runtime conformance checks (including tests for proxy harms and value drift).
- **Accountability gap (role mapping).** Responsibility definitions are fragmented across HRIS, org charts, and project tools. Integration requires a unifying service that maps RACI roles to authoritative identities and reporting lines, so agent actions can be traceably assigned and escalated.
- **Workflow gap (human-in-the-loop control points).** Operational processes may be only partly digitized. Integration requires ITSM ticket templates, approval and four-eyes checkpoints, segregation-of-duties rules, evidence capture, and role-based routing so agents can connect to enforceable human control in production.

What human-in-the-loop control mechanisms are required in practice to keep decision responsibilities clear?

Enterprise alignment requires that automated actions remain anchored to human ownership and standard operational controls. Controls should mirror the four gap areas:

- **Strategic alignment controls.** Human oversight is required at the OKR→policy bridge: (i) approve OKR-to-constraint/reward mappings, (ii) review impact analyses for KPI trade-offs, (iii) periodically revalidate objectives and guardrails, (iv) trigger rollback or pause when strategic thresholds are breached.
- **Value alignment controls.** Governance over the Value tool and its enforcement: (i) designate human stewards for value policies and fairness constraints, (ii) require four-eyes review for changes to sensitive-attribute handling, (iii) mandate human triage for detected proxy harms or value drift, (iv) log rationale for overrides and document mitigations for audit.
- **Accountability controls.** Binding every agent action to a responsible human: (i) embed RACI (Responsible, Accountable, Consulted, Informed) tied to reporting lines, (ii) enforce segregation of duties for approval vs. execution, (iii) maintain signer identity, timestamps, and justification on all decisions, (iv) define explicit escalation paths and ownership at each risk tier.
- **Workflow controls.** Operationalize human-in-the-loop via existing service management: (i) connect agents to ITSM workflows (*as Tools via MCP*) for ticketing, change, and incident lifecycles, (ii) require pre-approval and four-eyes checkpoints at defined gates, (iii) route items to role-based queues with time-bound SLAs and automatic escalation, (iv) capture end-to-end evidence (who, what, when, why, artefacts) in the ITSM record for traceability and auditability.

Limitations.

- Validation currently limited to a single domain (banking / credit scoring).
- Evaluation based on scripted user stories and one automatic decision-making algorithm.
- Scalability, stress testing, and behavioural diversity remain to be demonstrated in real-world contexts.

Future Work and Research Directions

The HADA reference architecture establishes a baseline for human–AI decision alignment in the banking domain, yet the design space remains rich with unanswered questions. Future research should therefore pursue the following directions, each aimed at strengthening generalisability, robustness, and regulatory fitness:

- **Automated business-objective propagation.** Automate the translation of business objectives (e.g., KPIs or OKRs) into machine-readable policies and reward signals, allowing AI agents to propagate intent to connected AI tools through natural-language policies.
- **Self-aligning agents.** Develop adaptive agents that autonomously adjust models and reasoning parameters in response to changing KPIs, ethical values, or contextual constraints, while preserving traceability.

- **Cross-domain replication and scalability.** Extend evaluation beyond retail banking to additional domains—such as healthcare triage, energy-trading optimisation, and logistics—to test external validity, scalability, and behavioural diversity under production-level loads.
- **Alignment metadata and protocol evolution.** Formalise metadata schemas for alignment, provenance, and value mapping to be included in future versions of MCP and A2A protocols, ensuring consistent treatment of accountability signals across frameworks.
- **Continuous values alignment.** Extend the Values Catalogue so that amendments (e.g., newly regulated sensitive attributes) automatically trigger model retraining or redeployment, achieving near-real-time policy compliance.
- **Human-in-the-loop checkpoints.** Embed configurable approval gates that escalate disputed or high-risk decisions to human reviewers or committees, quantifying trade-offs among latency, cost, and accountability.
- **Usability and user-experience studies.** Conduct longitudinal trials with non-technical stakeholders—such as executives, ethics officers, and customers—to evaluate whether conversational interfaces enable effective participation in decision governance.
- **Benchmarking and interoperability.** Compare HADA against alternative stacks (e.g., rule-engine wrappers, RAG-centric pipelines, LangGraph or CrewAI architectures) to measure alignment speed, transparency, and maintenance effort. Prototype orchestration bridges to validate framework-agnostic deployment.
- **Multi-modal and multi-agent extensions.** Augment the agent layer with vision and speech models to handle multi-modal decision contexts, and explore coordination mechanisms for heterogeneous multi-agent ecosystems.
- **Regulatory-grade compliance and auditability.** Evaluate how HADA’s audit ledger, alignment triggers, and traceability mechanisms satisfy emerging EU AI Act and GDPR requirements for explainability, risk-tiering, and data minimisation.

8.5 Phase III: NordDRG AI Benchmark

Main RQ III.2 (Table 4):

To what extent can LLMs emulate, explain, and apply DRG-based hospital payment rules?

This question is operationalised by releasing and evaluating *NordDRG-AI-Benchmark*, a *rule-complete*, governance-grade testbed that turns the NordDRG definition tables and manuals into machine-readable artefacts and exact-match tasks. The benchmark comprises two complementary suites: (i) a 13-task *Logic* benchmark that probes rule-level skills (cross-table joins, properties, CC/MCC validity, multilingual grounding); and (ii) a 13-task *Grouper* benchmark that requires *full specification emulation* with strict exact-match on `drg_nat` and the triggering `drg_logic.id`.

The design objectives from this dissertation were achieved (Section 1.4.2.4):

- O.III.2.1 Artefact construction (coverage & fidelity).** A versioned, machine-readable bundle is released (tables ~20 sheets, governance PDFs, curated FI/FI-EN excerpts) with schema-stable identifiers for drop-in yearly updates.
- O.III.2.2 Benchmark operationalisation.** Two suites with governance-grade scoring are instantiated: order-invariant set equality for *Logic* and strict pairwise exact match for *Grouper*.
- O.III.2.3 Baseline evaluation.** Under an artefact-only (no-web) setting, top-tier models *fully* solve *Logic* (GPT-5 Thinking, Opus 4.1: 13/13; o3: 12/13), while *partial* emulation is achieved on *Grouper* (GPT-5 Thinking: 7/13; o3: 6/13; o4-mini: 3/13; GPT-5 Thinking Mini: 1/13; others: 0/13).

Taken together, the results show that state-of-the-art LLMs can reliably *read and reason over* the NordDRG rule base, but exact reproduction of the governed control flow—especially priority order (ORD) and CC/MCC exclusions—remains the limiting step. To available knowledge, this is the first public evidence of an LLM *partially emulating* a complete DRG grouper with traceable rule-row identification.

Practical contribution

This phase delivers a reusable *infrastructure for teaching and auditing* AI on hospital funding logic:

- **Open artefacts.** Rule-complete tables, governance manuals, curated FI and FI-EN subsets, prompts, and gold keys with schema-stable IDs.
- **Governance-grade scoring.** Exact-match outputs on codes (Logic) and on (`drg_nat`, `drg_logic.id`) pairs (Grouper) align evaluation with production grouper semantics.
- **Reference agents.** Lightweight *LogicAgent* and *GrouperAgent* enable artefact-only evaluation with auditable traces, lowering the barrier for coders, clinicians, and policy analysts.
- **Reproducibility.** A public repository with versioned releases, folder-per-release workflow, and drop-in annual updates supports longitudinal teaching and comparative research.

Limitations

- **Scope of evaluation.** Experiments isolate specification-faithful reasoning on clean artefacts; they do not assess extraction from noisy EHR narratives or proprietary claims data.
- **Model coverage.** Baselines prioritise widely used proprietary endpoints; systematic evaluation of open-weight models and fine-tuned variants is out of scope.
- **National and language breadth.** The benchmark centres on NordDRG (with FI and FI-EN excerpts); other Nordic languages and non-Nordic DRG families are not yet included.
- **Operational constraints.** Consumer subscription quotas affected throughput for some providers; while accuracy is unaffected, this impacts practicality for classroom and large-batch settings.

Future work

1. **Cross-system generalisation.** Port the artefact and scoring protocol to additional DRG families (e.g., AR-DRG, MS-DRG/APR-DRG) and to ICD-11-based groupers.
2. **Open-weight and fine-tuned baselines.** Benchmark instruction-tuned and task-specific fine-tunes; compare against retrieval-augmented and tool-augmented patterns under identical artefact constraints.
3. **Minority-language expansion.** Extend curated excerpts beyond Finnish to Swedish, Norwegian, Icelandic, and Sámi; measure zero-shot and bilingual transfer.
4. **Governance workflow automation.** Prototype agents that draft, validate, and merge annual technical-change templates against the released tables with audit-ready diffs.
5. **End-to-end realism.** Add optional noisy-document tracks (de-identified notes/claims) that funnel into the same governed scoring, enabling study of robustness without weakening auditability.
6. **Evaluation harness.** Release a standardised runner (prompt packs, context assembly, version pinning, exact-match metrics) and dataset/model cards for transparent reporting.

Closing remark

This work demonstrates that carefully packaged *governed artefacts* plus *exact-match protocols* make hospital funding logic teachable to LLMs and auditable by humans. The strongest models master rule-level reasoning and show *partial*, traceable emulation of the full NordDRG grouper. The benchmark thus serves a dual role in this dissertation: a practical tool for regulators, payers, and educators, and a durable yardstick for research on trustworthy automation in healthcare finance.

8.6 Phase III: Collaborative Curated Learning (CCL)

Main RQ III.3 (Table 4):

What end-to-end governance requirements and workflow properties enable AI agents to contribute to decision-model development without eroding human expert accountability?

Safety-critical and high-stakes decision-making systems must evolve while preserving auditability. The answer to RQ provided by this study is that AI agents can contribute to a version-control-centric development process in which every update is a typed, auditable transaction and *only humans* can approve merges to the decision-making model (*human-in-command*). Building on this principle, this study introduces *CCL*, a governance-first, pull-request-centric process that operationalises the loop in practice.

Concretely, this study delivered **C.III.3.1–C.III.3.3** and **C.III.3.6** (see Section 6.3): (i) **C.III.3.1** — the governed CCL process and gate policy that treat each update as a typed, auditable pull request transaction; (ii) **C.III.3.2** — an *agent-operable* NORDDRG representation enabling safe atomic diffs and reviewer-legible slices; (iii) **C.III.3.3** — an agentic workflow with role-constrained LLM agents (Contributor, Reviewer) that participate in proposer–reviewer loops under human-in-command control; and (iv) **C.III.3.6** — a reusable suite (converter, prompts, SVG fragments, and reproducibility scripts) to support replication. These contributions are operationalised in the CCL Platform, a full-stack web application comprising approximately 33 000 lines of code (Section 6.3.2.1), which served as the execution environment for all demonstrations and evaluations.

Using NORDDRG as a large-scale decision-making model testbed, this study instantiated **C.III.3.4** by parsing the official specification, normalising it for PR-style maintenance, and exercising realistic change scenarios. Empirical findings in this study (**C.III.3.5**) indicate that CCL scales oversight to rule sets on the order of $\approx 10^7$ decision points, with the potential to shorten validation cycles and to preserve certification-grade behaviour on regression checks, and improves explanation quality relative to current practice; introduction of the Reviewer agent role builds another layer of safety net to ensure that the decision model stays coherent. With the help of Reviewer agents, human reviewers can focus on evaluating systemic impacts, as the agents have already corrected the most obvious faults in the change sets. Merges to the final decision model remain strictly under human control.

Answers to the supporting research questions

SRQ.III.3.1 — How should a million-rule decision model (e.g., NordDRG) be represented to support safe, atomic diffs?

As outlined in the research gap (Section 1.4.3.3), standard ML decision-tree formats are unsuitable for *collaborative development* of regulated decision logic. The NORDDRG format, by contrast, supports expert review, approval workflows, and regulatory transparency through established tooling and human-readable documentation. Accordingly, this study treats the ordered rule list as the authoritative artefact under version control, while the compiled tree/DAG representation serves analytical, visualisation, and agent-facing purposes. The auditable specification functions as the *model of record*: a line-addressable, version-controlled table with stable identifiers and a concise interpretation guide, enabling diffable, reviewable edits while preserving rule precedence.

SRQ.III.3.2 — Under which constraints do LLM agents produce reviewer-acceptable proposals at $\approx 10^7$ decision-point scale?

LLM agents are reliable first-line *proposers/reviewers* when constrained by (i) grounding in versioned specification and guidelines, (ii) role restrictions that prohibit merges or unilateral changes, and (iii) fail-closed gate policy (the four CI gates above). Within these bounds, agents reduce reviewer burden by drafting well-typed diffs and explanation-rich briefs; adding a dedicated *Reviewer* agent improves acceptance versus single-agent workflows. Residual failure modes include scope creep, incorrect code-set references, and hallucinated justifications; these are mitigated by locality/budget checks, red-team/adversarial tests, provenance requirements, and mandatory human sign-off.

8.7 Summary

This dissertation traces a twenty-year design-science journey on intelligent software agents across three technological waves—Semantic-Web semantics, data-centric ML, and today’s LLM-powered generative agents—culminating in a blueprint for auditable, value-aligned AI systems.

Phase I (2001-2006).

- **Problem.** Early-2000s Web content was almost entirely human-readable; autonomous service composition required machine-interpretable semantics as well as awareness of user context (location, time, device).
- **Artefacts.** Three proof-of-concept multi-agent prototypes were produced: (a) *run-time interaction-protocol adaptation*: a semantically adaptive agents removing the need for hard-coded workflows. (b) *dynamic multi-party service negotiation* in supply-chain scenarios; and (c) an *intelligent service-matching engine* that delivered context-aware recommendations to mobile devices on the move.
- **Findings.** Ontology-backed agents could discover and orchestrate Web services once contextual cues were available, but performance was limited by immature tooling and fragmented technologies and standards. More importantly, after years of experimentation, software-agent tooling and mainstream Web standards showed little sign of converging—the Semantic Web “layer-cake” vision of seamless integration remained largely aspirational.
- **Contribution.** The studies demonstrated how machine-readable descriptions plus lightweight context models can elevate the Web from a document repository to an *interaction substrate*. The persistent tooling gap—and the observed lack of convergence between agent technologies and emerging Web standards—motivated the pivot to data-centric learning explored in Phase II.
- **Transition.** In light of the Phase I findings, the research pivoted towards flexible ML algorithms and large datasets in Phase II.

Phase II (2007–2013).

- **Problem.** Hospitals (and retail) generate millions of sparsely coded records whose meaning resides in hierarchical taxonomies (ICD-10, NCSP, product catalogues). Exact-match SQL pipelines ignore hierarchy, degrade under missing/partial codes, and struggle to deliver ad-hoc similarity search within strict latency and memory budgets.
- **Artefacts.** A modular multi-agent platform that orchestrates heterogeneous data flows and ML-backed retrieval—vector-space IR (TF-IDF/cosine), kernel methods, and k -NN—via crawler, integrator, commander, and aggregator agents running on commodity hardware.
- **Findings.** High-dimensional code embeddings enabled sub-second similarity queries on desktop-class machines, remained robust to documentation defects (“SHOULD” vs. “MUST” occur fetching), achieved $\sim 66\%$ top-1 and $\sim 90\%$ top-8 recommendation accuracy, imputed missing codes, and exposed DRG granularity effects without modifying the underlying taxonomies.
- **Contribution.** Coupling symbolic hierarchies with statistical retrieval inside an agentic architecture yielded scalable, flexible clinical decision support and DRG analytics under real-world constraints, and established an evaluation playbook on Finnish hospital (and retail) data that informed Phase III.

Phase III (2023-2025).

- **Problem.** Modern organisations must align rapidly evolving LLM agents with objectives, regulations and values while preserving auditability and containment.
- **Artefacts.** (a) *HADA* (Human-AI Decision Alignment) reference architecture: a role-segmented cohort of LLM agents supervised by a policy controller. (b) *NordDRG-AI-Benchmark*: a rule-complete, governance-grade benchmark that operationalises the full NordDRG rule graph. It bundles ~ 20 interlinked definition sheets and expert manuals, and exposes two suites: a 13-task *Logic* benchmark (rule-level reasoning with order-invariant exact-match sets) and a 13-task *Grouper* benchmark requiring full specification emulation with strict exact match on both `drg_nat` and the triggering `drg_logic.id`. Curated FI / FI-EN subsets and lightweight reference agents support artefact-only evaluation and teaching.

- (c) *CCL* (Collaborative Curated Learning): a governance-first, pull-request-centric workflow where agent-proposed updates arrive as typed atomic diffs with Impact Briefs and pass risk-tiered CI gates to an immutable audit ledger; merges remain strictly human-in-command.
- **Findings.** In a banking sandbox HADA met six predefined objectives (natural-language control, KPI & values alignment, traceability, scalability, framework agnosticism). Under an artefact-only (no-web) setting, NordDRG baselines show top-tier models fully mastering rule-level tasks (GPT-5 Thinking, Opus 4.1: 13/13; o3: 12/13), while exact, traceable grouper emulation remains challenging (GPT-5 Thinking: 7/13; o3: 6/13; o4-mini: 3/13; others $\leq 1/13$), with errors concentrated in priority order (ORD) and CC/MCC exclusion control flow.
- **Contribution.** The phase positions conversational, auditable agent cohorts as a viable pattern for responsible AI governance and provides a living test-bed for research on domain-specific LLM reasoning.

In sum, this thesis offers longitudinal evidence that *agent architectures can remain practical and governable across successive AI paradigms*, providing scholars and practitioners a tested pathway from symbolic reasoning to trustworthy generative systems.

Future Work

As the evidence in this thesis suggests, **AI alignment is no longer only a scientific mystery but increasingly an engineering challenge**—one that demands reproducible methods, rigorous tooling, and cross-disciplinary collaboration. Building on these insights, the following avenues are organised under three lenses—*methodological*, *technological*, and *socio-technical*—to highlight their interdependence.

1. Methodological: Towards Verified Alignment. While HADA demonstrates policy-controlled cohorts of agents, a formally verified alignment layer remains an open goal. Future work should explore the integration of symbolic proof systems (e.g. temporal-logic model checkers) with probabilistic guarantees from uncertainty-aware LLMs, enabling *machine-checkable certificates* for agent behaviour in safety-critical settings.

2. Technological: Cross-Domain Generalisation. The agent architectures evaluated here were tested primarily in healthcare and retail. Extending them to finance, public administration, and cyber-physical systems will stress new aspects such as real-time constraints and adversarial robustness. Transfer-learning studies across domains—and across languages—will reveal how much of the current ontology layer and retrieval machinery can be reused without retraining.

3. Technological: Energy & Sustainability. Large-scale generative agents incur non-trivial carbon footprints. Replicating the performance-per-watt analyses of Phase II for Phase III stacks, and designing *adaptive sparsity* or *on-device distillation* strategies, could ensure that alignment does not come at unsustainable energy cost.

4. Socio-Technical: Human-Agent Co-Governance. Empirical studies are needed to test how domain experts interact with audit trails and policy dashboards. Mixed-methods research—combining controlled experiments with ethnographic observation—can surface usability bottlenecks and inform the design of *explanation interfaces* that remain intelligible as agent autonomy increases.

5. Socio-Technical: Open Benchmarks and Shared Tasks. The NordDRG-AI-BENCHMARK proved the value of a public yardstick for hospital funding. Next steps include extending the Logic suite to remaining corners of the NordDRG specification and providing idiomatic loaders and exporters to ease integration into common toolchains. A reproducible evaluation harness will standardise prompt packs, context assembly, version pinning, and exact-match scoring, alongside dataset/model cards that document governance, traceability, and known risks.

6. Long-Term Vision: Continuous, Trusted Autonomy. Ultimately, the field must converge on **continuous assurance**: mechanisms that monitor agents post-deployment, detect distributional shift, and trigger automated realignment without service interruption. Embedding these capabilities natively into the

HADA stack—and exposing them through standardised APIs—would close the loop from design to operation, realising a truly *trustworthy, self-regulating agent ecosystem*.

By pursuing these directions, the research community can transform the conceptual blueprint offered in this thesis into a resilient, domain-spanning infrastructure for auditable and value-aligned artificial agents.

9 EPILOGUE

9.1 Agents are (finally) useful

During the course of this thesis research, intelligent software agents have evolved from theoretical DAI concepts to become integral components of practical, everyday applications through the emergence of *LLM Agents*. This is illustrated by the grammar review process employed for this 150+ page document, which was conducted within a couple of hours using a small MAS including two agents:

Agent 1: Prompt for Grammatical Error Detection on Sentence Level

- 1 Your task will be to receive LaTeX-formatted sections of text. What you should do:
- 2 i) Find obvious grammatical mistakes from the text
- 3 ii) List all those sentences that have grammatical mistakes as they appear in the
↪ original text
- 4 iii) Do not list sentences that are grammatically correct but could be formulated in
↪ another way

Agent 2: Prompt for Grammatical Fixes on Sentence Level

- 1 Your task will be to receive LaTeX-formatted individual sentences of text. What you
↪ should do:
- 2 i) Find obvious grammatical mistakes from the text
- 3 ii) Rephrase the sentence so that it is grammatically correct
- 4 iii) Make sure that the content or message in the sentence does not change when you
↪ fix the grammar

Setting up these LLM agents can be done in seconds with modern tools. More importantly, this foregrounds the question of what role humans should choose to play in such workflows. In the CCL approach introduced as part of this Thesis, humans retain the *Authority to Define Workflows*. For this thesis, I adopted the left-hand, human-in-the-loop process shown in Figure 90, where the human retains full control and dispatches tasks directly to the agents rather than enabling an *Agentic Workflow*. The trade-off is coordination overhead: given A sections, each with B sentences flagged for review, I had to select a review budget of D sentences and decide which changes to accept. Even under this “inefficient” regime, two LLM agents identified and corrected the primary grammatical issues across the manuscript within hours, demonstrating the practical efficacy of a minimal LLM MAS.

By contrast, the right-hand, agentic workflow in Figure 90 would have reduced human touches and my own actions substantially, but it would not have shifted accountability: the same review burden would remain, only in larger batches. In the end, I chose to contain the AI, since I did not fully trust that it aligned with my objectives or that it was able to perform exactly as I intended.

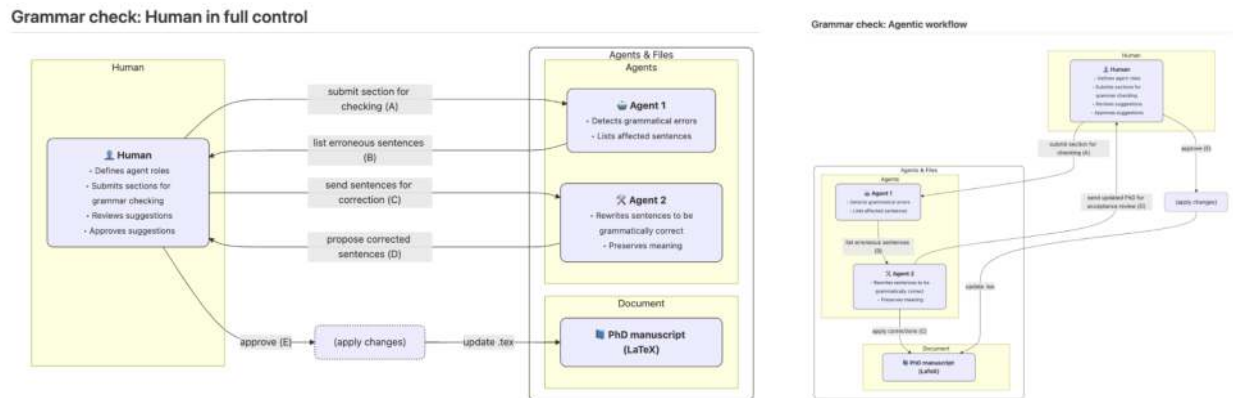


Figure 90: Human in control or agentic workflow

References

- [1] T. Pitkäranta, “Software Agent in Semantic Web Environment, Helsinki University of Technology, Masters Thesis, URL = <https://aaltodoc.aalto.fi/handle/123456789/92001>,” 2004.
- [2] T. Pitkäranta, “Semantic Service Matching in Context-Aware Environment, Helsinki University of Technology, Licentiate Thesis, URL = <https://aaltodoc.aalto.fi/handle/123456789/106536>,” 2006.
- [3] T. Pitkäranta, O. Riva, and S. Toivonen, “Designing and implementing a system for the provision of proactive context-aware services,” in *Proceedings of the Workshop on Context Awareness for Proactive Systems CAPS 2005, June 16-17, Helsinki, Finland*, vol. 1, pp. 21–30, 2005.
- [4] O. Riva, T. Pitkäranta, and S. Toivonen, “Proactive context-aware service provisioning for a marine community,” *CAPS 2005*, p. 175, 2005.
- [5] S. Toivonen, T. Pitkäranta, and J. Min, “On Describing B2B Processes with Semantic Web Technologies,” *ERCIM News*, pp. 48–49, 2004.
URL: http://www.ercim.org/publication/Ercim_News/enw56/toivonen.html.
- [6] S. Toivonen, H. Helin, M. Laukkanen, and T. Pitkäranta, “Context-sensitive conversation patterns for agents in wireless environments,” in *IWUC*, pp. 11–17, 2004.
- [7] S. Toivonen, T. Pitkäranta, H. Helin, and J. U. Min, “Using interaction protocols in distributed construction processes,” in *ICEIS (4)*, pp. 344–349, 2004.
- [8] S. Toivonen, T. Pitkäranta, and O. Riva, “Determining information usefulness in the semantic web: A distributed cognition approach,” in *SemAnnot@ ISWC*, 2005.
- [9] T. P. Matti Halinen, Heli Koukkunen, “Diagnoosien kirjaamiskäytännöissä kehittämisen varaa,” *Lääkärilehti*, no. 9, 2008.
- [10] T. Pitkäranta, “The impact of diagnosis coding granularity on the casemix,” in *PCSI 2008: Proceedings of the 24th Patient Classification Systems International Working Conference, October 2008, Lisbon, Portugal*, p. 10, 2008.
- [11] T. Pitkäranta, “Kernel based imputation of coded data sets,” in *European Symposium on Time Series Prediction ESTSTP 2008, Proceedings, European Symposium on Time Series Prediction, September 2008, Porvoo, 2008*, 2008.
- [12] T. Pitkäranta, “Affinity analysis of coded data sets,” in *In Proceedings of 12th International Conference on Extending Database Technology, PhD Workop, March 24-26 2009, Saint Petersburg, Russia*, p. 10, 2009. ISBN 978-1-60558-650-2.
- [13] T. Pitkäranta, “Applying information retrieval for market basket recommender systems,” in *ICEIS 2009: Proceedings of the 11th International Conference on Enterprise Information Systems, May 2009, Milan, Italy*, pp. 137–144, 2009. ISBN: 978-989-8111-84-5.
- [14] T. Pitkäranta, “Applying machine learning for supporting clinical decision making and documentation,” in *PCSI 2009: Proceedings of the 25th International Conference on Patient Classification Systems, November 2009, Fukuoka, Japan*, p. 10, 2009.
- [15] T. Pitkäranta, “Applying software change management tools for clinical classification (ontology) development,” in *PCSI 2009: Poster presentation in 25th International Conference on Patient Classification Systems, November 2009, Fukuoka, Japan*, p. 5, 2009.
- [16] M.-L. S. Tapio Pitkäranta, Martti Virtanen, “National drg cost weights in finland,” in *PCSI 2011: Proceedings of the 27th International Conference on Patient Classification Systems, November 2011, Montreal, Canada*, p. 10, 2011.
- [17] M.-L. S. Tapio Pitkäranta, Petra Kokko, “Hospital affinity analysis in drg spectrum,” in *PCSI 2012: Proceedings of the 28th International Conference on Patient Classification Systems, October 2012, Avignon, France*, p. 10, 2012.
- [18] L. Pitkäranta, T. Pitkäranta, and N. Vuori, “Towards a software platform of algorithmic strategic decision-making: How to orchestrate algorithms to sing along with the company’s strategy?,” (Cyprus), 2024. Proceedings of the PROS 2024 Conference.
- [19] L. Pitkäranta, T. Pitkäranta, and N. Vuori, “Towards a software platform of strategic decision-making: Orchestrating algorithms to sing along with company’s strategy,” 2024. Conference presentation at the 44th Annual Conference of the Strategic Management Society (SMS), Istanbul, Türkiye, 2024.

- [20] T. Pitkäranta, “Discussing with your casemix specification with natural language: how to make large language models (llm) understand norddrg logic?,” in *PCSI 2024: Proceedings of the 36th International Conference on Patient Classification Systems, May 2024, Bled, Slovenia, 2024*.
- [21] T. Pitkäranta, “Teaching llms the nuances of hospital funding instruments,” in *Proceedings of the International Conference on Computer-Human Interaction Research and Applications (CHIRA 2024)*, SCITEPRESS, 2024.
- [22] T. Pitkäranta, “Forecasting surprises in machine-learning-driven interaction systems: Lessons from the transformer breakthrough,” in *Proceedings of the 9th International Conference on Computer-Human Interaction Research and Applications (CHIRA 2025)*, (Porto, Portugal), Nov. 2025. To be presented at CHIRA 2025, held in conjunction with ICINCO, WEBIST, icSPORTS, and CoopIS 2025.
- [23] T. Pitkäranta, “The norddrg ai benchmark for large language models.” <https://arxiv.org/abs/2506.13790>, 2025. Presented at the 9th International Conference on Computer-Human Interaction Research and Applications (CHIRA 2025), Porto, Portugal, November 18–20, 2025. Also available as arXiv:2506.13790 [cs.AI].
- [24] T. Pitkäranta and E. Hyvönen, “Semantic web – a forgotten wave of artificial intelligence?,” in *Proceedings of the 14th International Joint Conference on Knowledge Graphs (IJCKG 2025)*, (Heraklion, Crete, Greece), IJCKG, October 2025. <https://arxiv.org/abs/2506.13790> arXiv: 2503.20793, cs.SI.
- [25] T. Pitkäranta and L. Pitkäranta, “Bridging human and ai decision-making with llms: The ragada approach,” in *Proceedings of the 26th International Conference on Enterprise Information Systems (ICEIS), Query date*, vol. 6, p. 45, 2024.
- [26] T. Pitkäranta and L. Pitkäranta, “Aadar: Using agentic llm architecture to align human and ai decisions,” in *Lecture Notes in Business Information Processing*, vol. 566 of *LNBP*, Springer, 2025.
- [27] T. Pitkäranta and L. Pitkäranta, “Hada: Human-ai agent decision alignment architecture.” <https://arxiv.org/abs/2506.04253>, 2025. Presented at AGENTICS 2025, part of the 17th International Joint Conference on Computational Intelligence (IJCCI 2025), Porto, Portugal, November 18–20, 2025. Also available as arXiv:2506.04253 [cs.AI].
- [28] T. Pitkäranta and L. Pitkäranta, “Towards curated learning: Analyzing casemix norddrg system expert system using machine learning tools,” in *Proceedings of the 37th Patient Classification Systems International (PCSI) Conference*, (Quebec City, Canada), Sept. 2025. Conference theme: Patient Grouping as a Driver of Value in Health: The Patient at the Heart of Our Decisions.
- [29] T. Pitkäranta, “Why are we living the age of ai applications right now? the long innovation path from ai’s birth to a child’s bedtime magic,” 2025.
- [30] C. R. Jones and B. K. Bergen, “Large language models pass the turing test,” 2025.
- [31] B. Christian, *The alignment problem: Machine learning and human values*. WW Norton & Company, 2020. ISBN: 9781786494306.
- [32] B. Nick, “Superintelligence: Paths, dangers, strategies,” 2014.
- [33] K. Peffers, T. Tuunanen, C. E. Gengler, M. Rossi, W. Hui, V. Virtanen, and J. Bragge, “Design science research process: A model for producing and presenting information systems research,” 2020.
- [34] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [35] S. Gregor and D. Jones, “The anatomy of a design theory,” *Journal of the Association for Information Systems*, vol. 8, no. 5, pp. 312–335, 2007.
- [36] T. Berners-Lee, “Information Management: A Proposal,” March 1989. URL: <http://www.w3.org/History/1989/proposal.html>.
- [37] W. Theilmann and K. Rothermel, “Domain Experts for Information Retrieval in the World Wide Web,” in *Proceedings of the 2nd International Workshop on Cooperative Informative Agents (CIA ’98)* (G. W. M. Klusch, ed.), LNAI 1435, pp. 216–227, 1998.
- [38] T. Berners-Lee and M. Fischetti, *Weaving the Web*. HarperBusiness, 2000. ISBN: 0-06-251587-X.
- [39] D. Fensel, J. Hendler, H. Lieberman, and W. Wahlster, *Spinning the Semantic Web*. The MIT Press, 2003. ISBN: 0-262-06232-1.
- [40] D. Fensel and C. Bussler, “The Web Service Modeling Framework WSMF,” tech. rep., Vrije Universiteit Amsterdam, Oracle Corporation, 2003. URL: <http://www.swsi.org/resources/wsmf-paper.pdf>.

- [41] T. Berners-Lee, J. Hendler, and O. Lassila, “The Semantic Web,” *The Scientific American*, May 2001. URL: <http://www.scientificamerican.com/>.
- [42] K. Kim, B. Paulson, C. Petrie, and V. Lesser, “Compensatory Negotiation for Agent-Based Project Schedule Coordination,” 1999.
- [43] T. Mikolov, “Efficient estimation of word representations in vector space,” *arXiv preprint arXiv:1301.3781*, vol. 3781, 2013.
- [44] P.-S. Huang, X. He, J. Gao, L. Deng, A. Acero, and L. Heck, “Learning deep structured semantic models for web search using clickthrough data,” in *Proceedings of the 22nd ACM International Conference on Information and Knowledge Management (CIKM)*, 2013.
- [45] P. Covington, J. Adams, and E. Sargin, “Deep neural networks for youtube recommendations,” in *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys)*, 2016.
- [46] C. Rudin, “Stop explaining black box machine learning models for high-stakes decisions and use interpretable models instead,” *Nature Machine Intelligence*, vol. 1, pp. 206–215, 2019.
- [47] M. T. Ribeiro, S. Singh, and C. Guestrin, ““why should i trust you?”: Explaining the predictions of any classifier,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [48] A. Adadi and M. Berrada, “Peeking inside the black-box: A survey on explainable artificial intelligence (xai),” *IEEE Access*, vol. 6, pp. 52138–52160, 2018.
- [49] D. Kreuzberger, N. K  hl, and M. Hirschl, “Mlops: A systematic literature review and technology classification,” *arXiv preprint arXiv:2303.09040*, 2023.
- [50] H. A. Kautz, “The third ai summer: Aaai robert s. engelmore memorial lecture,” *AI Magazine*, 2022.
- [51] P. McCorduck, *Machines Who Think: A Personal Inquiry into the History and Prospects of Artificial Intelligence*. AK Peters Ltd, 2004.
- [52] M. Minsky and S. Papert, *Perceptrons: An Introduction to Computational Geometry*. MIT Press, 1969.
- [53] W. Mettrey, “An assessment of tools for building large knowledge-based systems,” *AI Magazine*, vol. 8, no. 4, 1987.
- [54] J. D. Kelleher, *Deep learning*. The MIT Press essential knowledge series, Cambridge, Mass. London: The MIT Press, 2019.
- [55] P. Hitzler and M. K. Sarker, eds., *Neuro-Symbolic Artificial Intelligence: The State of the Art*. IOS Press, 2022.
- [56] A. Bhuyan, Bikram Pratim nsd Ramdane-Cherif and R. Tomar, “Neuro-symbolic artificial intelligence: a survey,” *Neural Computing and Applications*, vol. 36, pp. 12809–12844, 2024.
- [57] T. Berners-Lee, “The semantic web wave,” 2003. Accessed: 2025-02-12.
- [58] P. Hitzler, “A review of the semantic web field,” *Communications of the ACM*, vol. 64, no. 2, pp. 76–83, 2021.
- [59] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [60] S. Reed, K. Zolna, *et al.*, “A generalist agent,” *DeepMind*, 2022.
- [61] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” 2023.
- [62] M. Steinert and L. Leifer, “Scrutinizing gartner’s hype cycle approach,” in *Picmet 2010 technology management for global economic growth*, pp. 1–13, IEEE, 2010.
- [63] T. Menzies, “21st-century ai: proud, not smug,” *IEEE Intelligent Systems*, vol. 18, no. 3, pp. 18–24, 2003.
- [64] T. Noguchi, Y. Hashizume, H. Moriyama, L. Gauthier, Y. Ishikawa, T. Matsuno, and A. Suganuma, “A practical use of expert system “ai-q” focused on creating training data,” in *2018 5th international conference on business and industrial research (ICBIR)*, pp. 73–76, IEEE, 2018.
- [65] A. Toosi, A. G. Bottino, B. Saboury, E. Siegel, and A. Rahmim, “A brief history of ai: how to prevent another winter (a critical review),” *PET clinics*, vol. 16, no. 4, pp. 449–469, 2021.

- [66] E. Francesconi, “The winter, the summer and the summer dream of artificial intelligence in law: Presidential address to the 18th international conference on artificial intelligence and law,” *Artificial intelligence and law*, vol. 30, no. 2, pp. 147–161, 2022.
- [67] O. Colliot, “A non-technical introduction to machine learning,” *Machine Learning for Brain Disorders*, pp. 3–23, 2023.
- [68] J. Harguess and C. M. Ward, “Is the next winter coming for ai? elements of making secure and robust ai,” in *2022 IEEE Applied Imagery Pattern Recognition Workshop (AIPR)*, pp. 1–7, IEEE, 2022.
- [69] A. Perspectives, “Ai perspectives.” <https://www.aiperspectives.com/introduction/>, 2020. Accessed: 2025-01-12.
- [70] A. Lu, “Ai summer and winter: The ups and downs of ai.” <https://www.labelf.ai/blog/four-seasons-of-ai-the-brief-history-of-artificial-intelligence-i>, 2022. Accessed: 2025-01-12.
- [71] B. Research and Analysis, “The rise of ai.” <https://www.brightworkresearch.com/why-did-ai-rise-for-a-third-time-after-the-1st-and-2nd-ai-winters/>, 2020. Accessed: 2025-01-12.
- [72] S. Winiger, “Ai winters = hcl summers.” <https://samim.io/p/2018-02-19-ai-winters-hci-summers/>, 2017. Accessed: 2025-01-12.
- [73] Y. Altmann, “Small history of ai,” June 2023. Accessed: December 28, 2024, <https://omq.ai/blog/history-of-ai/>.
- [74] J. Latta, “History of ai – from winter to winter,” 2021. Accessed: 2024-12-29, <https://jaylatta.net/history-of-ai-from-winter-to-winter/>.
- [75] S. Schuchmann, “History of the first ai winter,” 2019. Accessed: 2024-12-29, <https://towardsdatascience.com/history-of-the-first-ai-winter-6f8c2186f80b>.
- [76] M. Lim, “History of ai winters,” 2018.
- [77] R. Solé, “Tweet on artificial intelligence history,” 2022. Includes reference to Google Books Ngram Viewer: https://books.google.com/ngrams/graph?content=perceptron&year_start=1930&year_end=2019&corpus=26&smoothing=3, Accessed: 2024-12-29.
- [78] G. Hinton, “Full interview: “godfather of artificial intelligence” talks impact and potential of ai.” <https://youtu.be/qpoR0378qRY?si=7MLdIW9W0sVWn4Pc>, 2023. Interview.
- [79] V. Bush, *As we may think*, pp. 85–110. ISBN: 0-12-523270-5, From Memex to hypertext: Vannevar Bush and the mind’s machine, Academic Press Professional, Inc., 1945. URL: <http://www.theatlantic.com/unbound/flashbks/computer/bushf.htm>.
- [80] Livinginternet.com, “Living Internet - The Internet History.” URL: http://livinginternet.com/i/ii_bush.htm.
- [81] T. H. Nelson, “A file structure for the complex, the changing, and the indeterminate,” in *Proceedings of the 1965 20th National Conference*, (New York, NY, USA), pp. 84–100, Association for Computing Machinery, 1965.
- [82] T. Nelson, “Getting It Out of Our System: Critique of Information Retrieval,” *Thompson Books*, pp. 191–210, 1967.
- [83] G. Wolf, “The Curse of Xanadu,” *Wired Magazine*, Jun 1995. URL: http://www.wired.com/wired/archive/3.06/xanadu_pr.html.
- [84] D. C. Engelbart and W. K. English, *A Research Center For Augmenting Human Intellect*, vol. 33, pp. 395–410. Thompson Book Co, afips conference proceedings ed., 1968.
- [85] T. Berners-Lee, “WWW: Past, Present, and Future,” *IEEE Computer*, October 1996.
- [86] T. Berners-Lee, L. Masinter, and M. McCahill, “Uniform Resource Locators (URL),” RFC 1738, Internet Engineering Task Force, Dec. 1994. URL: <http://www.ietf.org/rfc/rfc1738.txt>.
- [87] T. Berners-Lee, R. Fielding, and L. Masinter, “Uniform Resource Identifiers (URI): Generic Syntax,” RFC 2396, Aug. 1998. URL: <http://www.ietf.org/rfc/rfc2396.txt>.
- [88] W3C, “World Wide Web Consortium Recommendation: Naming and Addressing: URIs and URLs.” URL: <http://www.w3.org/Addressing/>.

- [89] T. Berners-Lee, R. Fielding, L. Masinter, H. Frystyk, P. Leach, J. Gettys, and J. Mogul, “Hypertext Transfer Protocol – HTTP/1.1,” RFC 2616, June 1999.
URL: <http://www.ietf.org/rfc/rfc2616.txt>.
- [90] W3C, “World Wide Web Consortium Recommendation: HyperText Markup Language (HTML).” URL: <http://www.w3.org/MarkUp/>.
- [91] W. W. W. Consortium, “A Little History of the World Wide Web.”
URL: <http://www.w3.org/History.html>.
- [92] A. Heimbürger and T. Kuhanen, *Hypermedia*, vol. Tekoälyn ensyklopedia, pp. 327–334. Gaudeamus, 1993. ISBN: 951-662-559-2.
- [93] I. S. Consortium, “Internet Domain Survey Host Count,” 2004.
URL: <http://www.isc.org/>.
- [94] T. Johnson, “Spinning the World Wide Web,” *SLAC Beamline*, 1994.
URL: <http://www.slac.stanford.edu/pubs/beamline/pastissues.html>.
- [95] A. Weaver and T. Hall, “The Internet and the World Wide Web,” *IEEE Industrial Electronics, Control and Instrumentation*, vol. 4, pp. 1529 – 1540, 1997.
- [96] L. Paulson, “Building rich web applications with ajax,” *Computer*, vol. 38, no. 10, pp. 14–17, 2005.
- [97] T. O’Reilly, “What is web 2.0? design patterns and business models for the next generation of software,” September 2005. Accessed 17 Jun 2025.
- [98] A. Russell, “Progressive web apps: Escaping tabs without losing our soul,” 2016. Accessed 17 Jun 2025.
- [99] A. Haas, A. Rossberg, D. Schuff, *et al.*, “Bringing the web up to speed with webassembly,” in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI ’17)*, pp. 185–200, 2017.
- [100] C. Hewitt, “Viewing control structures as patterns of passing messages,” *Artificial intelligence*, vol. 8, no. 3, pp. 323–364, 1977.
- [101] A. H. Bond and L. Gasser, *Readings in distributed artificial intelligence*. Morgan Kaufmann, 2014.
- [102] R. Gasser and M. N. Huhns, *Distributed Artificial Intelligence: Volume II*, vol. 2. Morgan Kaufmann, 2014.
- [103] M. Wooldridge, *An introduction to multiagent systems*. John wiley & sons, 2009.
- [104] H. S. Nwana, “Software agents: An overview,” *Knowledge Engineering Review*, vol. 11, no. 3, pp. 205–244, 1996.
- [105] M. Wooldridge and N. R. Jennings, “Intelligent agents: Theory and practice,” *Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, 1995.
- [106] P. Maes, “Agents that reduce work and information overload,” *Communications of the ACM*, vol. 37, no. 7, pp. 30–40, 1994.
- [107] G. Weiss, *Multiagent systems: A modern approach to distributed artificial intelligence*. MIT press, 1999.
- [108] S. Franklin and A. Graesser, “Is it an agent, or just a program?: A taxonomy for autonomous agents,” in *Third International Workshop on Agent Theories, Architectures, and Languages (ATAL-96)*, pp. 21–35, Springer, 1996.
- [109] Foundation for Intelligent Physical Agents (FIPA), “FIPA ’97 Specification, Version 1.0 — Part 2: Agent Communication Language,” standard specification, FIPA, Geneva, Switzerland, 1997.
- [110] D. L. McGuinness and F. Van Harmelen, “Owl web ontology language overview,” *W3C recommendation*, vol. 10, no. 10, p. 2004, 2004.
- [111] N. Shadbolt, W. Hall, and T. Berners-Lee, “The semantic web revisited,” *IEEE intelligent systems*, vol. 21, no. 3, pp. 96–101, 2006.
- [112] J. Hendler, “Agents and the semantic web,” *IEEE Intelligent Systems*, vol. 16, no. 2, pp. 30–37, 2001.
- [113] S. A. McIlraith, T. C. Son, and H. Zeng, “Semantic web services,” *IEEE Intelligent Systems*, vol. 16, no. 2, pp. 46–53, 2001.
- [114] O. Lassila and R. R. Swick, “Enabling the web of data with RDF,” *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, vol. 24, no. 4, pp. 6–10, 2001.

- [115] Foundation for Intelligent Physical Agents (FIPA), “FIPA ACL Message Structure Specification,” Standard Specification SC00061G, FIPA, Geneva, Switzerland, Oct. 2002.
- [116] G. A. Vreeswijk, “Liberalizing protocols for argumentation in multi-agent systems,” in *Proceedings of the fourth international joint conference on Autonomous agents and multiagent systems*, pp. 1259–1260, 2005.
- [117] O. Lassila and R. R. Swick, “Resource description framework (rdf) model and syntax specification,” *W3C Proposed Recommendation PR-rdf-syntax-19981008*, 1998.
- [118] I. Horrocks, P. F. Patel-Schneider, and F. Van Harmelen, “From shiq and rdf to owl: The making of a web ontology language,” in *International Semantic Web Conference*, pp. 7–26, Springer, 2003.
- [119] T. R. Gruber, “Toward principles for the design of ontologies used for knowledge sharing,” *International journal of human-computer studies*, vol. 43, no. 5-6, pp. 907–928, 1993.
- [120] FIPA, “FIPA communicative act library specification.” Foundation for Intelligent Physical Agents. <http://www.fipa.org/specs/fipa00037/>, 2002. Accessed: 2024-01-26.
- [121] D. Martin, M. Burstein, J. Hobbs, O. Lassila, D. McDermott, S. McIlraith, S. Narayanan, M. Paolucci, B. Parsia, T. Payne, *et al.*, “Owl-s: Semantic markup for web services,” in *W3C member submission*, vol. 22, p. 2004, 2004.
- [122] M. Sah, “Semantic linking and personalization in context,” 06 2009. Illustration by Tim Berners-Lee 2006.
- [123] D. Brickley and R. V. Guha, “Rdf vocabulary description language 1.0: Rdf schema,” *W3C recommendation*, vol. 10, no. 1, pp. 1–107, 2004.
- [124] I. Horrocks, P. F. Patel-Schneider, H. Boley, S. Tabet, B. Grosz, and M. Dean, “SWRL: A semantic web rule language combining OWL and RuleML,” *W3C Member Submission*, 2004. Accessed: 2024-01-26.
- [125] B. N. Grosz, I. Horrocks, R. Volz, and S. Decker, “Description logic programs: Combining logic programs with description logic,” in *Proceedings of the 12th International World Wide Web Conference (WWW 2003)*, (Budapest, Hungary), pp. 48–57, ACM, 2003.
- [126] B. Motik, U. Sattler, and R. Studer, “Query answering for OWL-dl with rules,” *Journal of Web Semantics: Science, Services and Agents on the World Wide Web*, vol. 3, no. 1, pp. 41–60, 2005.
- [127] D. Longley, G. Kellogg, D. Yamamoto, and M. Sporny, “Rdf dataset canonicalization 1.0,” W3C Recommendation REC-rdf-canon-20240521, World Wide Web Consortium (W3C), May 2024.
- [128] M. Sporny, T. Thibodeau Jr., I. Herman, D. Longley, and G. Bernstein, “Verifiable credential data integrity 1.0,” W3C Recommendation REC-vc-data-integrity-20250515, World Wide Web Consortium (W3C), May 2025.
- [129] M. Sporny, T. Thibodeau Jr., I. Herman, G. Cohen, and M. B. Jones, “Verifiable credentials data model v2.0,” W3C Recommendation REC-vc-data-model-2.0-20250515, World Wide Web Consortium (W3C), May 2025.
- [130] M. Sporny, A. Guy, M. Sabadello, and D. Reed, “Decentralized identifiers (DIDs) v1.0,” W3C Recommendation REC-did-core-20220719, World Wide Web Consortium (W3C), July 2022.
- [131] M. Nickel, K. Murphy, V. Tresp, and E. Gabrilovich, “A review of relational machine learning for knowledge graphs,” *Proceedings of the IEEE*, vol. 104, no. 1, pp. 11–33, 2016.
- [132] R. Confalonieri and G. Guizzardi, “On the multiple roles of ontologies in explainable ai,” 2023.
- [133] J. Amara, B. König-Ries, and S. Samuel, “Enhancing explainability in multimodal large language models using ontological context,” 2024.
- [134] M. Li, S. Miao, and P. Li, “Simple is effective: The roles of graphs and large language models in knowledge-graph-based retrieval-augmented generation,” 2025.
- [135] A. Singhal, “Introducing the knowledge graph: Things, not strings,” May 2012. Accessed 17 Jun 2025.
- [136] S. Lassen, M. Curtiss, and S. Sankar, “Under the hood: Building out the infrastructure for graph search,” Mar. 2013. Accessed 17 Jun 2025.
- [137] ACM, “Acm computing classification system.” <https://dl.acm.org/ccs>, 2024.
- [138] H. Nwana, “Software Agents: An Overview,” *Knowledge Engineering Review*, 1996. AA&T, BT Laboratories, Ipswich, United Kingdom.

- [139] H. S. Nwana and D. T. Ndumu, “A Perspective on Software Agents Research,” vol. 14, no. 2, pp. 1–18, 1999.
- [140] S. Nadella, “Bg2 pod series: January 2025 episode.” YouTube, 2024. Available at: https://youtu.be/9NtsnzRFJ_o?t=2811, Accessed: March 2025.
- [141] O. Etzioni and D. Weld, “Intelligent agents on the Internet: Fact, fiction, and forecast,” *IEEE Intelligent Internet Services*, August 1995.
- [142] S. Franklin and A. Graesser, “Is it an Agent, or just a Program?: A Taxonomy for Autonomous Agents,” in *Intelligent Agents III. Agent Theories, Architectures and Languages (ATAL’96)*, vol. 1193, (Berlin, Germany), Springer-Verlag, 1996.
- [143] J. Hendler, “Making Sense out of Agents,” *IEEE Intelligent Systems*, march/april 1999.
- [144] N. Jennings, *On Agent-Based Software Engineering*, vol. 117 of *Artificial Intelligence*, pp. 277–296. Elsevier Science, 2000.
URL: <http://www.ecs.soton.ac.uk/nrj/download-files/aij2000.ps>.
- [145] S. J. Russell and P. Norvig, *Artificial Intelligence - A Modern Approach*. Prentice Hall, second ed., 2003. ISBN: 0-13-080302-2.
- [146] C. J. Petrie, “Agent-Based Engineering, the Web, and Intelligence,” *IEEE Expert*, Dec 1996. URL: <http://www-cdr.stanford.edu/NextLink/Expert.html>.
- [147] K. Sycara, “The Many Faces of Agents,” *American Association for Artificial Intelligence*, 1998. URL: <http://www.aaai.org/Resources/Papers/AIMag19-02-002.pdf>.
- [148] S. Toivonen, “Ohjelmistoagentin Määrittelemien,” Master’s thesis, University of Helsinki, Department of Psychology, 1999.
- [149] M. Wooldridge and N. R. Jennings, “Intelligent agents: Theory and practice,” *Knowledge engineering review*, pp. 115–192, 1995. URL: <http://agents.umbc.edu/introduction/>.
- [150] F. Gandon, *Distributed Artificial Intelligence and Knowledge Management: Ontologies and Multi-Agent Systems for a Corporate Semantic Web*. PhD thesis, INRIA and University of Nice - Sophia Antipolis - Doctoral School of Sciences and Technologies of Information and Communication, 2002.
URL: <http://www-sop.inria.fr/acacia/personnel/Fabien.Gandon/research/PhD2002/>.
- [151] M. Wooldridge, *Reasoning about Rational Agents*. The MIT Press, 2000.
- [152] H. Helin, *Supporting Nomadic Agent-based Applications in the FIPA Agent Architecture*. PhD thesis, University of Helsinki Finland, 2003.
- [153] M. Wooldridge, N. R. Jennings, and D. Kinny, “A Methodology for Agent-Oriented Analysis and Design,” pp. 69–76, 1999.
- [154] T. Finin and C. Nicholas, “Agent-based Information Retrieval,” 1998.
URL: <http://www.csee.umbc.edu/abir/>.
- [155] J. Searle, *Speech acts: an essay in the philosophy of language*. Cambridge University Press, Cambridge, 1969.
- [156] M. E. Bratman, *Intentions, Plans, and Practical Reason*. Cambridge, USA: Harvard, 1987. ISBN: 0-674-45818.
- [157] FIPA, “FIPA Agent Management Specification,” Tech. Rep. SC00023J, Foundation for Intelligent Physical Agents, 2002. URL: <http://www.fipa.org/specs/fipa00023/SC00023J.html>.
- [158] R. Brooks, “A robust layered control system for a mobile robot,” *IEEE journal on robotics and automation*, vol. 2, no. 1, pp. 14–23, 1986.
- [159] R. A. Brooks, “Intelligence without representation,” *Artificial intelligence*, vol. 47, no. 1-3, pp. 139–159, 1991.
- [160] P. Maes, *Designing autonomous agents: Theory and practice from biology to engineering and back*. MIT press, 1990.
- [161] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, W. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, “The rise and potential of large language model based agents: A survey,” 2023.

- [162] L. P. Kaelbling, M. L. Littman, and A. W. Moore, “Reinforcement learning: A survey,” *Journal of artificial intelligence research*, vol. 4, pp. 237–285, 1996.
- [163] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, *et al.*, “Mastering the game of go with deep neural networks and tree search,” *nature*, vol. 529, no. 7587, pp. 484–489, 2016.
- [164] Y. Labrou, T. Finin, and Y. Peng, “Agent Communication Languages: The Current Landscape,” *IEEE Intelligent Systems*, march/april 1999.
- [165] Foundation for Intelligent Physical Agents (FIPA), “Mission.” <http://www.fipa.org/about/mission.html>, 2003. Accessed: 2025-08-25.
- [166] FIPA, “Publicly Available FIPA Agent Platform Implementations,” 2004. URL: <http://www.fipa.org/resources/livesystems.html>.
- [167] M. Laukkanen, “Evaluation of FIPA-Compliant Agent Platforms,” Master’s thesis, Lappeenranta University of Technology, 2002.
- [168] TILAB, “Java Agent DEvelopment Framework (JADE),” 2004. Open Source, URL: <http://jade.cse.it/>.
- [169] O. Source, “Lightweight Extensible Agent Platform (LEAP),” 2004. URL: <http://leap.crm-paris.com/>.
- [170] R. T. Fielding, *Architectural Styles and the Design of Network-Based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [171] W. Butcher *et al.*, “grpc: A high-performance, open-source universal rpc framework,” 2016. Google white paper.
- [172] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A distributed messaging system for log processing,” in *Proceedings of the NetDB Workshop*, 2011.
- [173] A. Videla and J. Williams, *RabbitMQ in Action*. Manning, 2012.
- [174] G. Alonso, F. Casati, H. Kuno, and V. Machiraju, “Web services: Concepts, architectures, and applications,” *Data & Knowledge Engineering*, vol. 54, no. 1, pp. 183–276, 2005.
- [175] A. Wollrath, R. Riggs, and J. Waldo, “A distributed object model for the java system,” *USENIX Computing Systems*, vol. 9, no. 4, pp. 471–497, 1996.
- [176] O. M. Group, “Common object request broker architecture (corba) specification version 2.1,” tech. rep., OMG, 1997.
- [177] Microsoft, “MS-DCOM: Overview.” https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-dcom/86b9cf84-df2e-4f0b-ac22-1b957627e1ca, 2024.
- [178] T. Finin, R. Fritzson, D. McKay, and R. McEntire, “KQML as an Agent Communication Language,” in *In Proceedings of the Third International Conference on Information and Knowledge Management (CIKM94)*, pp. 456–463, New York: Association of Computing Machinery, 1994. URL: <http://www.cs.umbc.edu/kqml/papers/kqmlacl.pdf>.
- [179] F. for Intelligent Physical Agents, “FIPA ACL Message Structure Specification,” Tech. Rep. XC00061, Geneva, Switzerland, 2000. URL: <http://www.fipa.org/specs/fipa00061/>.
- [180] F. for Intelligent Physical Agents, “FIPA Communicative Act Library Specification,” Tech. Rep. XC00037, Geneva, Switzerland, 2002. URL: <http://www.fipa.org/specs/fipa00037/>.
- [181] F. for Intelligent Physical Agents, “FIPA ACL Message Representation in Bit-Efficient Encoding Specification,” Tech. Rep. SC00069, Geneva, Switzerland, 2002. URL: <http://www.fipa.org/specs/fipa00069/>.
- [182] F. for Intelligent Physical Agents, “FIPA ACL Message Representation in String Specification,” Tech. Rep. SC00070, Geneva, Switzerland, 2002. URL: <http://www.fipa.org/specs/fipa00070/>.
- [183] F. for Intelligent Physical Agents, “FIPA ACL Message Representation in XML Specification,” Tech. Rep. SC00071, Geneva, Switzerland, 2002. URL: <http://www.fipa.org/specs/fipa00071/>.

- [184] F. for Intelligent Physical Agents, “FIPA Content Language Library Specification,” Tech. Rep. SC00007, Geneva, Switzerland, 2002.
URL: <http://www.fipa.org/specs/fipa00007/>.
- [185] FIPA, “FIPA Interaction Protocol Library,” tech. rep., Foundation for Intelligent Physical Agents, 2002. URL: <http://www.fipa.org/repository/ips.html>.
- [186] M. Purvis, S. Cranefield, G. Bush, D. Carter, and B. McKinlay, “An Agent-Based Distributed Information Systems Architecture,” in *Proceedings of the Hawai’i International Conference On System Sciences*, (Maui, Hawaii), IEEE, January 7 2000.
URL: http://nzdis.otago.ac.nz/download/papers/nzdis-project_1-00.pdf.
- [187] F. for Intelligent Physical Agents, “FIPA Contract Net Interaction Protocol Specification,” Tech. Rep. XC00029, Geneva, Switzerland, 2000.
URL: <http://www.fipa.org/specs/fipa00029/>.
- [188] F. for Intelligent Physical Agents, “FIPA SL Content Language Specification,” Tech. Rep. SC00008I, Geneva, Switzerland, 2002.
URL: <http://www.fipa.org/specs/fipa00008/>.
- [189] M. R. Genesereth and R. E. Fikes, “Knowledge Interchange Format, Version 3.0 Reference Manual,” Tech. Rep. Logic-92-1, Computer Science Department, Stanford University, Stanford, CA, USA, June 1992.
- [190] S. J. Russell and P. Norvig, *Artificial intelligence: a modern approach*. Prentice hall, 3rd ed., 2010.
- [191] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [192] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, *et al.*, “Training language models to follow instructions with human feedback,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 27730–27744, 2022.
- [193] J. Wei, X. Wang, D. Schuurmans, M. Bosma, E. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837, 2022.
- [194] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, “Why do multi-agent llm systems fail?,” 2025.
- [195] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian, “A comprehensive overview of large language models,” 2024.
- [196] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, W. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, “The rise and potential of large language model based agents: A survey,” 2023.
- [197] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, Mar. 2024.
- [198] S. Yang, O. Nachum, Y. Du, J. Wei, P. Abbeel, and D. Schuurmans, “Foundation models for decision making: Problems, methods, and opportunities,” 2023.
- [199] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” 2023.
- [200] H. Chase, “Langchain.” <https://github.com/langchain-ai/langchain>, 2022. Accessed: 2024-01-26.
- [201] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “Autogen: Enabling next-gen llm applications via multi-agent conversation,” 2023.
- [202] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, p. 1–38, Mar. 2023.
- [203] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, V. Kulkarni, M. Lewis, N. Reimers, S. Riedel, and L. Zettlemoyer, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

- [204] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, “On the dangers of stochastic parrots: Can language models be too big?,” in *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*, pp. 610–623, 2021.
- [205] L. Weidinger, J. Mellor, M. Rauh, C. Griffin, J. Uesato, P.-S. Huang, M. Cheng, M. Glaese, B. Balle, A. Kasirzadeh, Z. Kenton, S. Brown, W. Hawkins, T. Stepleton, C. Biles, A. Birhane, J. Haas, L. Rimell, L. A. Hendricks, W. Isaac, S. Legassick, G. Irving, and I. Gabriel, “Ethical and social risks of harm from language models,” 2021.
- [206] F. Doshi-Velez and B. Kim, “Towards a rigorous science of interpretable machine learning,” 2017.
- [207] R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim, C. Hesse, S. Jain, V. Kosaraju, W. Saunders, X. Jiang, K. Cobbe, T. Eloundou, G. Krueger, K. Button, M. Knight, B. Chess, and J. Schulman, “Webgpt: Browser-assisted question-answering with human feedback,” 2022.
- [208] M. Laskin, L. Wang, J. Oh, E. Parisotto, S. Spencer, R. Steigerwald, D. Strouse, S. Hansen, A. Filos, E. Brooks, M. Gazeau, H. Sahni, S. Singh, and V. Mnih, “In-context reinforcement learning with algorithm distillation,” 2022.
- [209] Y. Yang, H. Chai, Y. Song, S. Qi, M. Wen, N. Li, J. Liao, H. Hu, J. Lin, G. Chang, W. Liu, Y. Wen, Y. Yu, and W. Zhang, “A survey of ai agent protocols,” 2025.
- [210] X. Zhu, Y. Chen, H. Tian, C. Tao, W. Su, C. Yang, G. Huang, B. Li, L. Lu, X. Wang, Y. Qiao, Z. Zhang, and J. Dai, “Ghost in the minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory,” *arXiv preprint arXiv:2305.17144*, 2023.
- [211] E. Karpas, O. Abend, Y. Belinkov, B. Lenz, O. Lieber, N. Ratner, Y. Shoham, H. Bata, Y. Levine, K. Leyton-Brown, D. Muhlgay, N. Rozen, E. Schwartz, G. Shachaf, S. Shalev-Shwartz, A. Shashua, and M. Tenenholz, “Mrkl systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning,” *arXiv preprint arXiv:2205.00445*, 2022.
- [212] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” *arXiv preprint arXiv:2302.04761*, 2023.
- [213] OpenAI, “Function calling and other api updates.” <https://openai.com/index/function-calling-and-other-api-updates/>, 2023.
- [214] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, “Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face,” *arXiv preprint arXiv:2303.17580*, 2023.
- [215] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, Mar. 2024. Code and resources available at <https://github.com/Paitesanshi/LLM-Agent-Survey>.
- [216] E. Meyerson, G. Paolo, R. Dailey, H. Shahrzad, O. Francon, C. F. Hayes, X. Qiu, B. Hodjat, and R. Miikkulainen, “Solving a million-step llm task with zero errors,” 2025.
- [217] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” 2023.
- [218] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” 2023.
- [219] Wildcard AI, K. Mahorker, and Y. Patel, “agents-json: Translate openapi into llm tools.” <https://github.com/wild-card-ai/agents-json>, 2025. GitHub repository, release 0.1.3 (2025-02-09).
- [220] S. Marro, E. L. Malfa, J. Wright, G. Li, N. Shadbolt, M. Wooldridge, and P. Torr, “A scalable communication protocol for networks of large language models,” 2024.
- [221] “Mcp general architecture.” <https://modelcontextprotocol.io/introduction>, 2025. Accessed: 2025-08-21.
- [222] G. Developers, “A2a: A new era of agent interoperability.” <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>, 2024. Accessed: 2024-11-30.
- [223] Google, “Agent2agent (a2a) protocol repository.” <https://github.com/google/A2A>, 2024. GitHub Repository.

- [224] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “Autogen: Enabling next-gen llm applications via multi-agent conversation,” 2023.
- [225] LangChain Inc., “Langchain documentation.” <https://python.langchain.com/>, 2025. Accessed 2025-08-21.
- [226] LangChain Inc., “Langgraph: Building stateful, multi-actor applications with llms.” <https://langchain-ai.github.io/langgraph/>, 2025. Accessed 2025-08-21.
- [227] Y. Qin *et al.*, “Autogen: Enabling next-gen llm applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [228] Microsoft Research, “Autogen (ag2) documentation.” <https://microsoft.github.io/autogen/>, 2025. Accessed 2025-08-21.
- [229] LlamaIndexAI, Inc., “Llamaindex documentation.” <https://docs.llamaindex.ai/>, 2025. Accessed 2025-08-21.
- [230] CrewAI, “Crewai documentation.” <https://docs.crewai.com/>, 2025. Accessed 2025-08-21.
- [231] Microsoft, “Semantic kernel documentation.” <https://learn.microsoft.com/semantic-kernel/>, 2025. Accessed 2025-08-21.
- [232] deepset AI, “Haystack documentation.” <https://docs.haystack.deepset.ai/docs/intro>, 2025. Accessed 2025-08-21.
- [233] Embedchain, “Embedchain documentation.” <https://docs.embedchain.ai/>, 2025. Accessed 2025-08-21.
- [234] SuperAGI, “Introduction to superagi.” <https://superagi.com/docs/>, 2025. Accessed 2025-08-21.
- [235] LangGenius, Inc., “Dify documentation.” <https://docs.dify.ai/>, 2025. Accessed 2025-08-21.
- [236] T. Xie, F. Zhou, Z. Cheng, P. Shi, L. Weng, Y. Liu, T. J. Hua, J. Zhao, Q. Liu, C. Liu, L. Z. Liu, Y. Xu, H. Su, D. Shin, C. Xiong, and T. Yu, “Openagents: An open platform for language agents in the wild,” *arXiv*, 2023.
- [237] Microsoft, “Agent development kit (adk) / ag2.” <https://microsoft.github.io/autogen/stable/user-guide/adk/>, 2025. Accessed 2025-08-21.
- [238] Google Firebase, “Firebase genkit.” <https://firebase.google.com/docs/genkit>, 2025. Accessed 2025-08-21.
- [239] Prefect, Inc., “Marvin: Ai engineering framework for python.” <https://github.com/PrefectHQ/marvin>, 2025. Accessed 2025-08-21.
- [240] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [241] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [242] J. Hoffmann, A. Botev, G. Penedo, A. N. Alvi, *et al.*, “Training compute-optimal large language models,” *arXiv preprint arXiv:2203.15556*, 2022.
- [243] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, *et al.*, “Palm: Scaling language modeling with pathways,” *arXiv preprint arXiv:2204.02311*, 2022.
- [244] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, *et al.*, “Opt: Open pre-trained transformer language models,” *arXiv preprint arXiv:2205.01068*, 2022.
- [245] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [246] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian, “A comprehensive overview of large language models,” *arXiv preprint arXiv:2307.06435*, 2023.
- [247] Y. Cao, S. Li, Y. Liu, Z. Yan, Y. Dai, P. S. Yu, and L. Sun, “A comprehensive survey of ai-generated content (aigc): A history of generative ai from gan to chatgpt,” *arXiv preprint arXiv:2303.04226*, 2023.
- [248] C. O’neil, *Weapons of math destruction: How big data increases inequality and threatens democracy*. Crown, 2017.

- [249] E. Hubinger, C. Van Merwijk, V. Mikulik, J. Skalse, and S. Garrabrant, “Risks from learned optimization in advanced machine learning systems,” *arXiv preprint arXiv:1906.01820*, 2019.
- [250] R. Shah, L. L. F. Langosco, C. Wild, V. Krakovna, P. Eckersley, Z. Kenton, D. Krueger, and S. Russell, “Goal misgeneralization in deep reinforcement learning,” *arXiv preprint arXiv:2210.01790*, 2022.
- [251] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, “Concrete problems in AI safety,” *arXiv preprint arXiv:1606.06565*, 2016.
- [252] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, “Deep reinforcement learning from human preferences,” *arXiv preprint arXiv:1706.03741*, 2017.
- [253] J. Leike, D. Krueger, T. Everitt, M. Martic, V. Maini, and S. Legg, “Scalable agent alignment via reward modeling: A research direction,” *arXiv preprint arXiv:1811.07871*, 2018.
- [254] Y. Bai, S. Kadavath, S. Kundu, *et al.*, “Constitutional ai: Harmlessness from ai feedback,” *arXiv preprint arXiv:2212.08073*, 2022.
- [255] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, “Glue: A multi-task benchmark and analysis platform for natural language understanding,” in *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*, pp. 353–355, 2018.
- [256] A. Wang, Y. Pruksachatkun, N. Nangia, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, “Superglue: A stickier benchmark for general-purpose language understanding systems,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [257] B. bench Collaboration, “Beyond the imitation game: Quantifying and extrapolating the capabilities of language models,” *arXiv preprint arXiv:2206.04615*, 2022.
- [258] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, *et al.*, “Measuring massive multitask language understanding,” *arXiv preprint arXiv:2009.03300*, 2021.
- [259] A. Warstadt, A. Liu, C. Kirov, and *et al.*, “The babylm challenge: Sample-efficient pretraining on 10m tokens,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [260] P. Liang, R. Bommasani, T. Lee, D. Tsipras, *et al.*, “Holistic evaluation of language models,” *arXiv preprint arXiv:2211.09110*, 2022.
- [261] LMSYS, “Lmarena (chatbot arena) leaderboard.” <https://lmarena.ai/leaderboard>, 2025. Accessed: 2025-08-19.
- [262] S. Zheng, Z. Zhao, E. Li, and *et al.*, “Judging llm-as-a-judge with mt-bench and chatbot arena,” *arXiv preprint arXiv:2306.05685*, 2023.
- [263] Vellum.ai, “Llm leaderboard.” <https://www.vellum.ai/llm-leaderboard>, 2025. Accessed: 2025-08-18.
- [264] S. Chacon and B. Straub, *Pro Git*. Berkeley, CA: Apress, 2nd ed., 2014.
- [265] Mercurial Project, “Mercurial: Guide,” 2025.
- [266] Fossil Project, “Fossil scm,” 2025.
- [267] D. Roundy, “The theory of patches,” 2005. Unpublished manuscript.
- [268] Pijul Project, “The Pijul manual: Theory,” 2025.
- [269] Jujutsu Project, “Jujutsu (jj) documentation,” 2025.
- [270] Jujutsu Project, “Jujutsu (jj): A Git-compatible VCS,” 2025.
- [271] D. Kreuzberger, N. Köhl, and S. Hirschl, “Machine learning operations (mlops): Overview, definition, and architecture,” *arXiv preprint arXiv:2205.02302*, 2022.
- [272] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, “Software engineering for machine learning: A case study,” in *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pp. 291–300, IEEE, 2019.
- [273] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, “Hidden technical debt in machine learning systems,” in *Advances in Neural Information Processing Systems*, 2015.

- [274] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, “The ml test score: A rubric for ml production readiness and technical debt reduction,” in *2017 IEEE International Conference on Big Data (Big Data)*, IEEE, 2017.
- [275] D. Baylor, E. Breck, H.-T. Cheng, N. Fiedel, C. Foo, Z. Haque, S. Haykal, M. Ispir, V. Jain, L. Koc, *et al.*, “Tfx: A tensorflow-based production-scale machine learning platform,” in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 1387–1395, ACM, 2017.
- [276] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, F. Xie, and C. Zumar, “Accelerating the machine learning lifecycle with mlflow,” *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, vol. 41, no. 4, pp. 39–45, 2018.
- [277] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, “Model cards for model reporting,” *arXiv preprint arXiv:1810.03993*, 2018.
- [278] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, “Datasheets for datasets,” *arXiv preprint arXiv:1803.09010*, 2021.
- [279] NordicCasemixCentre, “Norddrg specification, url: <https://nordcase.org/>,” 2025.
- [280] X. Wang, L. Zhang, M. Yan, *et al.*, “Drg-llama: An open-source model for diagnosis-related group coding,” *arXiv preprint arXiv:2310.01234*, 2023.
- [281] K. Kwan, “Large language models are good medical coders, if provided with tools,” *arXiv preprint arXiv:2407.12849*, 2024.
- [282] P. Renc, Y. Jia, A. E. Samir, J. Was, Q. Li, D. W. Bates, and A. Sitek, “Zero shot health trajectory prediction using transformer,” *arXiv preprint arXiv:2407.21124*, 2024.
- [283] H. Wang, Z. Wu, G. Kolar, H. Korsapati, B. Bartlett, B. Hull, and J. Sun, “Reinforcement learning for out-of-distribution reasoning in llms: An empirical study on diagnosis-related group coding,” *arXiv preprint arXiv:2505.21908*, 2025.
- [284] D. Hajialigol, D. Kaknes, T. Barbour, D. Yao, C. North, J. Sun, D. Liem, and X. Wang, “Drgcoder: Explainable clinical coding for the early prediction of diagnostic-related groups,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 373–380, Association for Computational Linguistics, 2023.
- [285] Y. He, C. Wang, S. Zhang, N. Li, Z. Li, and Z. Zeng, “Kg-mtt-bert: Knowledge graph enhanced BERT for multi-type medical text classification,” *arXiv preprint arXiv:2210.03970*, 2022.
- [286] J. S. Boyle, A. Kascenas, P. Lok, M. Liakata, and A. Q. O’Neil, “Automated clinical coding using off-the-shelf large language models,” *arXiv preprint arXiv:2310.06552*, 2023.
- [287] Z. Boukhers, A. Khan, Q. Ramadan, and C. Yang, “Large language model in medical informatics: Direct classification and enhanced text representations for automatic icd coding,” *arXiv preprint arXiv:2411.06823*, 2024.
- [288] R. Busse, A. Geissler, W. Quentin, and M. Wiley, *Diagnosis-Related Groups in Europe: Moving Towards Transparency, Efficiency and Quality in Hospitals*. Copenhagen: European Observatory on Health Systems and Policies, 2011.
- [289] Centers for Medicare & Medicaid Services, “Icd-10 ms-drugs version 42.1, effective april 1, 2025,” Apr. 2025. MS-DRG Grouper overview and definitions.
- [290] Independent Hospital and Aged Care Pricing Authority (IHACPA), “Ar-drg version 11.0 technical specifications,” 2022.
- [291] 3M Health Information Systems, “3m™ all patient refined drg (apr drg) classification system: Overview,” 2021.
- [292] Nordic Casemix Centre, “How to read the norddrg definition tables,” 2025.
- [293] S. Toivonen and H. Helin, “Representing interaction protocols in DAML,” in *Agent-Mediated Knowledge Management. Papers from 2003 AAAI Spring Symposium*, pp. 145–147, AAAI Press, 2003.
- [294] F. for Intelligent Physical Agents, “FIPA Request Interaction Protocol Specification,” Tech. Rep. XC00026, Geneva, Switzerland, 2000.
URL: <http://www.fipa.org/specs/fipa00026/>.

- [295] W3C, “World Wide Web Consortium Recommendation: Resource Description Framework (RDF) 1.0.” URL: <http://www.w3.org/RDF/>.
- [296] F. for Intelligent Physical Agents, “FIPA Query Interaction Protocol Specification,” Tech. Rep. XC00027, Geneva, Switzerland, 2000. URL: <http://www.fipa.org/specs/fipa00027/>.
- [297] W3C, “World Wide Web Consortium Member Submission: RDQL - A Query Language for RDF.” URL: <http://www.w3.org/Submission/2004/SUBM-RDQL-20040109/>.
- [298] H. P. S. W. R. Lab, “Jena: A Semantic Web Framework for Java,” 2004. Open Source, URL: <http://www.hpl.hp.com/semweb/index.html>.
- [299] F. for Intelligent Physical Agents, “FIPA Propose Interaction Protocol Specification,” Tech. Rep. XC00036, Geneva, Switzerland, 2000. URL: <http://www.fipa.org/specs/fipa00036/>.
- [300] S. Toivonen, “Hybrid service provision model for mobile users: Prospects for the DYNAMOS project,” in *Proceedings of the 11th Finnish Artificial Intelligence Conference STeP 2004, September 1-3, Vantaa, Finland*, vol. 2, pp. 183–192, 2004.
- [301] R. Järvensivu, R. Pitkänen, and T. Mikkonen, “Object-oriented middleware for location-aware systems,” in *SAC ’04: Proceedings of the 2004 ACM symposium on Applied computing*, pp. 1184–1190, ACM Press, 2004.
- [302] D. Davis and M. P. Parashar, “Latency performance of soap implementations,” in *CCGRID ’02: Proceedings of the 2nd IEEE/ACM International Symposium on Cluster Computing and the Grid*, p. 407, IEEE Computer Society, 2002.
- [303] Fuego Core Project, “Url: <http://www.hiit.fi/fuego/fc/>,” 2005.
- [304] D. J. Tufts-Conrad, A. N. Zincir-Heywood, and D. Zitner, “Som - feature extraction from patient discharge summaries,” in *Symposium on Applied Computing*, pp. 263–267, ACM, 2003. ISBN: 1-58113-624-2.
- [305] C. Servais, “Computer/assisted coding for inpatients / a case study,” in *Perspectives in Health Information Management, Computer Assisted Coding Conference*, American Health Information Management Association, 2006.
- [306] P. Resnik, M. Niv, M. Nossal, G. Schnitzer, J. Stoner, A. Kapit, and R. Toren, “Using intrinsic and extrinsic metrics to evaluate accuracy and facilitation in computer-assisted coding,” in *Perspectives in Health Information Management, Computer Assisted Coding Conference*, American Health Information Management Association, 2006.
- [307] J. Stausberg, N. Lehmann, D. Kaczmarek, and M. Stein, “Reliability of diagnoses coding with icd-10,” *International Journal of Medical Informatics* (77), pp. 50–57, 2006.
- [308] J. Stausberg, P.-D. Med, D. Koch, J. Ingenerf, R. Nat, and Betzler, “Comparing paper-based with electronic patient records: Lessons learned during a study on diagnosis and procedure codes,” *Journal of the American Medical Informatics Association*, no. 10, pp. 470–477, 2003.
- [309] G. Surjan, “Questions on validity of international classification of diseases-coded diagnoses,” *International Journal of Medical Informatics* (54), pp. 77–95, 1999.
- [310] C. Colin, R. Ecochard, F. Delahaye, G. Landrivon, P. Messy, E. Morgon, and Y. Matillon, “Data quality in a drg-based information system,” *International Journal for Quality in Health Care*, no. 6, pp. 275–280, 1994.
- [311] Socialstyrelsen, *Kodningskvalitet i patientregistret, Slutenvård 2005*. No. 2007-125-13, 2007. ISBN 978-91-7164-281-3, Available (in Swedish) at URL: <http://www.socialstyrelsen.se/>.
- [312] Socialstyrelsen, *Kvalitet och innehåll i patientregistret, Utskrivningar från slutenvård 1964-2006 och besök i öppenvård (exklusive primärvårdsbesök) 1997-2006*. No. 2008-125-1, 2008. ISBN 978-91-7164-281-3, Available (in Swedish) at URL: <http://www.socialstyrelsen.se/>.
- [313] D. Sammon and P. Finnegan, “The ten commandments of data warehousing,” *SIGMIS Database*, vol. 31, no. 4, pp. 82–91, 2000.
- [314] J. Zobel and A. Moffat, “Inverted files for text search engines,” *ACM Comput. Surv.*, vol. 38, no. 2, p. 6, 2006.

- [315] Q. Gang, S. Sural, Y. Gu, and S. Pramanik, “Similarity between euclidean and cosine angle distance for nearest neighbor queries,” in *Proceedings of the 2004 ACM symposium on Applied computing*, pp. 1232–1237, Michigan State University, ACM, 2004. ISBN: 1-58113-812-1.
- [316] WHO, *International Statistical Classification of Diseases and Related Health Problems*, vol. 1, pp. 1–10. World Health Organization, 1992. ISBN: 92 4 154419 8.
- [317] WHO, *International Statistical Classification of Diseases and Related Health Problems, Instruction manual*, vol. 2. World Health Organization, 2004. ISBN: 92 4 154649 8.
- [318] N. C. Centre, *NOMESCO Classification of Surgical Procedures*. Nordic Medico-Statistical Committee (NOMESCO), 2006. ISBN: 87-89702-57-3.
- [319] P. R. Niven and B. Lamorte, *Objectives and Key Results: Driving Focus, Alignment, and Engagement with OKRs*. John Wiley & Sons, Inc., 2016.
- [320] L. H. Crawford and P. Bryce, “Project monitoring and evaluation: a method for enhancing the efficiency and effectiveness of aid project implementation,” *International Journal of Project Management*, vol. 21, no. 5, pp. 363–373, 2003.
- [321] R. Busse, A. Geissler, W. Quentin, and M. Wiley, eds., *Diagnosis-Related Groups in Europe: Moving Towards Transparency, Efficiency and Quality in Hospitals*. Copenhagen: European Observatory on Health Systems and Policies, 2011.
- [322] A. Klein, I. Mathauer, K. Stenberg, and T. Habicht, “Diagnosis-related groups (drg): A question & answer guide on case-based classification and payment systems,” Tech. Rep. WHO/UHC/HGF/Guidance/20.10, World Health Organization, 2020.
- [323] “Innovative provider payment models for promoting value-based health systems,” 2023.
- [324] Centers for Medicare & Medicaid Services, “Acute inpatient prospective payment system (ipps) and ms-drgs,” 2025. Accessed: 2025-08-18.
- [325] World Health Organization, “Global spending on health: Coping with the pandemic,” Jan. 2024. Published 31 January 2024.
- [326] M. T. Schneider, A. Y. Chang, A. Chapin, C. S. Chen, S. W. Crosby, A. C. Harle, G. Tsakalos, B. S. Zlavog, and J. L. Dieleman, “Health expenditures by services and providers for 195 countries, 2000–2017,” *BMJ Global Health*, vol. 6, no. 7, p. e005799, 2021.
- [327] OECD, “Health at a glance 2023: OECD indicators,” 2023. See chapter “Health expenditure by provider”.
- [328] NordicCasemixCentreForum, “Norddrg forum, url: <https://nordcaseforum.easyredmine.com>,” 2025.